INSTRUCTOR KIT · PHP AND WORDPRESS DEVELOPMENT · COMPLETE FACILITATOR GUIDE

PHP and WordPress Development

The full teaching manual for the 18-session PHP and WordPress Development kit (Modules 1 to 9, in two parts; one 3-hour session, two lessons each). Timing, live-demo scripts, the mistakes to pre-empt, lab facilitation, and checks for understanding, for every session.

18 sessions · 31 lessons | 18 sessions × 3 h · 54 h | Instructor copy · not for students

CONTENTS

PART 1: THEMES · SESSIONS 1 TO 8

S01 **WordPress for Developers and Your Local Environment**

1.1 WordPress for Developers

1.2 Your Local Development Environment

S02 **PHP the WordPress Way and the Content Model**

1.3 PHP the WordPress Way

2.1 The Content Model

S03 **Core Architecture, the Database and Hooks**

2.2 Core Architecture & the Database

2.3 The Hooks System

S04 **Theme Anatomy and Building a Block Theme · Checkpoint 1**

3.1 Theme Anatomy & the Template Hierarchy

3.2 Building a Block Theme

S05 **Patterns, Style Variations and Child Themes**

3.3 Patterns, Style Variations & the Site Editor

3.4 Child Themes & Customizing Existing Themes

S06 **The Loop and Custom Queries**

4.1 The Loop & Template Tags

4.2 Custom Queries

S07 **functions.php: Supports, Assets, Menus & Widgets**

4.3 functions.php: Supports, Assets, Menus & Widgets

S08 Part 1 Project and Assessment

PART 2: PLUGINS TO LAUNCH · SESSIONS 9 TO 18

S09 Your First Plugin and Custom Post Types

5.1 Your First Plugin

5.2 Custom Post Types, Taxonomies & Meta

S10 Security and the Settings API

5.3 Security in WordPress

5.4 Admin Pages, Settings API & Options

S11 Custom Tables, Shortcodes and Block Setup

5.5 Shortcodes, $wpdb & Custom Tables

6.1 Block Development Setup

S12 Static and Dynamic Blocks

6.2 Static Blocks: Attributes, Controls & Supports

6.3 Dynamic Blocks & Block Bindings

S13 The Interactivity API and Using the REST API · Checkpoint 2

6.4 The Interactivity API

7.1 Using the REST API

S14 Custom Endpoints, External APIs and Cron

7.2 Custom REST Endpoints

7.3 External APIs, Transients & WP-Cron

S15 Debugging, Testing and Performance

8.1 Debugging & Testing

8.2 Performance & Caching

S16 Distribution and Deployment

8.3 Internationalization, Accessibility & Distribution

8.4 Deployment & Maintenance

S17 Final Project Kickoff and Studio I

9.1 Final Project: Launch the Pages & Co. Website

S18 Final Project Studio II, Showcase and Next Steps

9.1 Final Project: Launch the Pages & Co. Website

9.2 Next Steps: Headless, AI in Core & Your Career

PHP AND WORDPRESS DEVELOPMENT · FACILITATOR GUIDE

Part 1: Themes

Sessions 1 to 8. Modules 1 to 4: set up a WordPress 7.1 development site with Docker, WP-CLI, debugging and Mailpit, write PHP the WordPress way, learn the content model, the database and hooks, then build the Pages & Co. block theme with theme.json, templates, patterns and style variations, a child theme, and a classic/hybrid theme with the Loop, custom queries and functions.php. Runs on its own as a short course and ends with the Part 1 project and assessment.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 1: WordPress for Developers and Your Local Environment

How to teach the first session of PHP and WordPress Development. Welcome a room of PHP and MySQL developers, find out which machines run the course's Docker project and what the room remembers from the Foundations courses, then show WordPress as a developer sees it (one request through core, the folder that is yours, an edited core file and an old tutorial caught) and make every laptop ready: an install clinic, WP-CLI and debugging on the real site, a site-report lab, and a health check learners can rerun all course.

`2 lessons` `3 hrs session` `Lessons 1.1–1.2` `Session 1 of 18 · Part 1`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's ideas) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Type, do not paste. Every command below was run to produce the output printed under it, on WordPress 7.1.3, PHP 8.5.12, MySQL 9.7 and WP-CLI 2.12.0: a fresh course site, exactly what a learner has at the end of Lesson 1.2. Ask for a prediction before every run, and read every message from the left, out loud: the level, the message, the file, the line.
- The lessons are the reference. Keep `lesson_01.html` and `lesson_02.html` open in browser tabs for the request diagram and the per-platform Docker sections. Labs live in `labs/module-01/` (learners keep the folder in their project folder, next to compose.yaml): `_starter` files are handouts, `_solution` files and the `store_hours_solution/` folder are yours.
- Session 1 is different in two ways. It opens with 30 minutes of welcome, environment check and diagnostic instead of the usual 10-minute warm-up, and Lesson 1.2's room time is mostly an **install clinic**: setting up Docker and the course project is prework (the kit's setup guide follows Lesson 1.2), so the room fixes what went wrong instead of watching downloads.
- Part 1, "Themes" (Sessions 1 to 8), can run on its own as a short course; nothing in this session depends on Part 2. Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (1.1, 1.2) match the course site.

> **THE #1 RISK OF SESSION 1: THE SETUP EATS THE SESSION**
>
> Docker installs differently on every platform, and each path has its own trap: Windows needs WSL 2, Docker Desktop's WSL integration switched on, and the project in the Linux file system; Linux needs the docker group (and a new login) and the course's file-owner fix before the first start; every platform downloads about a gigabyte of images on the first `docker compose up -d`. Done together on venue Wi-Fi, setup fills three hours and nobody looks at WordPress. **Before:** send the setup guide a week ahead (Docker, the pagesco-dev project, the install commands, the labs/ folder in the project folder, then `check_environment.sh`) and ask every learner to reply with its last line. **In the room:** the check gets 12 minutes in the opening; a NOT READY machine gets a ready partner until the 25-minute install clinic at 1:40, not your attention during Lesson 1.1. **After:** whoever is still not ready leaves with a numbered plan and runs the check again before Session 2.

BEFORE THE SESSION

- ☐ **A week before:** email the setup guide (install Docker following Lesson 1.2 for your platform, unpack pagesco-dev in the home folder, on Linux and WSL do the file-owner setup BEFORE the first start, `docker compose up -d`, the two install commands, the Mailpit plugin, copy the kit's `labs/` folder into `pagesco-dev`, then `bash labs/module-01/l1_0-orientation/check_environment.sh`). Ask learners to reply with the last line it prints. No reply means a setup risk: chase it.
- ☐ **Files to hand out** (shared folder or USB stick, in case the network fails): the whole `labs/` folder **without** the `*_solution.*` files and the `store_hours_solution/` folder; `pagesco-dev.zip` from the course site; the workbook. If you can, put the course images on the stick too: on your machine, `docker save -o pagesco-images.tar wordpress:7.1-php8.5-apache mysql:9.7 axllent/mailpit:latest wordpress:cli-php8.5`, and learners run `docker load -i pagesco-images.tar` (it saves the gigabyte download; the wpcli image is then built from the local base).
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-01` from this kit's folder (every line PASS; 15 results in about two minutes, a third of it the environment check's own throwaway project). With your own course project: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-01` (labs/README.md). Then run `check_environment.sh` on the projector machine so you can show what READY looks like.
- ☐ **Projector machine:** the course project running (`docker compose up -d`) with the `wp` shortcut defined, VS Code open on `pagesco-dev` at a large font, two terminals in the project folder at 18 pt or larger (one for commands, one for `tail -f wordpress/wp-content/debug.log`), a browser with http://localhost:8080, http://localhost:8080/wp-admin/ (admin / password) and http://localhost:8025 in tabs, plus `lesson_01.html` and `lesson_02.html`.
- ☐ **Board, before learners arrive:** the request path (index.php → wp-blog-header.php → wp-load.php → wp-config.php → wp-settings.php → wp() → template-loader.php → HTML); the triage line "docker not found = install · Cannot connect to the Docker daemon = start Docker Desktop · no configuration file = wrong folder · Permission denied in wordpress/ = the Linux fix"; and a box labeled **wp-content = yours**.

- ☐ **Print or share:** the workbook (its Start Here page has the environment checklist and the diagnostic), sticky notes for help flags and the exit ticket.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`docker: command not found`, or `Cannot connect to the Docker daemon`	Docker not installed, or Docker Desktop not started	Install it (Lesson 1.2) or start Docker Desktop and wait for "running"; Linux: `sudo systemctl start docker`.
Windows: `docker` works in PowerShell but not in Ubuntu	Docker Desktop's WSL integration is off for that distribution	Docker Desktop > Settings > Resources > WSL integration: switch on Ubuntu, then a new Ubuntu terminal.
Linux: `permission denied while trying to connect to the Docker daemon socket`	The user is not in the docker group yet, or has not logged in again	`sudo usermod -aG docker "$USER"`, then log out and back in (a new terminal is not enough).
`mkdir: cannot create directory ... Permission denied` in wordpress/ (Linux, WSL)	The containers created the files as uid 33 (www-data)	Lesson 1.2's repair box: `docker compose run --rm --user root wpcli chown ...`, the .env and compose.override.yaml, `docker compose up -d`.
`required variable PAGESCO_UID is missing a value`	compose.override.yaml copied, .env not made	The `printf ... > .env` line of Lesson 1.2, in the project folder.
`tar: ./index.php: Cannot open: Permission denied` in `docker compose logs wordpress`	Docker created wordpress/ as root before the first start	Stop, `docker compose run --rm --user root wpcli chown -R "$(id -u):$(id -g)" /var/www/html`, or delete the empty folder and `mkdir wordpress` as yourself, then `up -d`.
`Bind for 0.0.0.0:8080 failed: port is already allocated`	Another project, an old container or a local server uses 8080 (or 8025)	`docker ps` (PORTS column), stop the other one. Changing the course port means three edits and lessons that no longer match: free the port.
`no configuration file provided: not found`	The terminal is not in the project folder	`cd ~/pagesco-dev`; add the `wp` alias to ~/.bashrc or ~/.zshrc.

SYMPTOM	CAUSE	FIX
`Error establishing a database connection` after editing compose.yaml	The database password was changed after the first start	Put the old password back and `docker compose up -d` (Common Mistakes 4).
`wp_mail()` prints `bool(false)`; the check says FIX for Mailpit	mailpit-smtp.php is not in wordpress/wp-content/mu-plugins/	`cp config/mailpit-smtp.php wordpress/wp-content/mu-plugins/`.
Pages load in several seconds (Windows)	The project is on the Windows drive (/mnt/c/...)	Move it to the Linux home folder inside WSL (the check says CHECK for this).
The first `docker compose up -d` takes forever	About a gigabyte of images over venue Wi-Fi	`docker load -i pagesco-images.tar` from the USB stick, or pair today and finish at home.

SESSION 1 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	30	Welcome, environment check and diagnostic	Introductions, how the course runs, `check_environment.sh` on every machine, a six-question readiness diagnostic (not graded).
0:30	10	**1.1** Frame and mental model	The apartment building; extension points; WordPress.org and WordPress.com; one request through core.
0:40	15	**1.1** Live demo	The front door in core, the end of the trip with curl, where code lives, an edited core file caught, an old tutorial caught.
0:55	15	**1.1** Guided lab	`labs/module-01/l1_1-wordpress-for-developers/core_tour_starter.sh` in pairs: find the lines of core that do each step.
1:10	10	**1.1** Start the lesson project	Plan Pages & Co.: the site map and the first features (Steps 3 and 4); Playground Steps 1 and 2 go home.
1:20	10	Break	NOT READY machines line up for the install clinic. Leave the request drawing on the board.
1:30	10	**1.2** Frame and mental model	The food truck; four services; the bind mount; the wp shortcut.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:40	25	**1.2** Install clinic	Fix the machines that were NOT READY, by platform; ready learners help, then rerun the check.
2:05	15	**1.2** Live demo	WP-CLI on the site, the escaped title, a warning in the page and in debug.log, an email caught by Mailpit.
2:20	15	**1.2** Guided lab	`labs/module-01/l1_2-local-environment/site_report_starter.sh`: a one-screen site report, one wp command per line.
2:35	15	**1.2** Start the lesson project	The Dev Site Health Check, Steps 1 to 3; Step 4 (break it, clean up) goes home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 2.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 100 + 150 = 250 minutes for a learner working alone, and after the 30-minute opening, the break and the wrap-up the room has 130. Lesson 1.1 keeps its full shape (model, demo, lab, project start). Lesson 1.2's Docker installation sections are prework from the setup guide: the room uses its time for an install clinic, a WP-CLI-and-debugging demo, a short lab and the start of the health check. The lesson's alternatives section (wp-env, Playground) is reading at home.

Goes home (about 2 to 3 hours). The Pages & Co. plan (Playground Steps 1 and 2, and the rest of Step 4) and the health check Step 4 (break it, read it, clean up); each lesson's Hands-on Exercises; both Quick Quizzes; both journal entries; Lesson 1.2's "Before the Next Lesson" list. Anyone still NOT READY finishes the setup with the numbered plan from the clinic.

Protect, in this order: the environment check result for every learner (nobody leaves without knowing whether their machine is ready and what to do next); the install clinic; the 1.1 demo through Mistake 1 (core is not yours: the rule the whole course leans on); the 1.2 debug demo (every later lesson assumes learners read debug.log). **Cut first:** the 1.1 demo's last step (the Link header) and the 1.2 Mailpit step (learners see it in the health check), then the 1.1 project start (send the whole plan home), then the 1.2 project start (the demo already showed the pieces; learners type the script at home).

OPENING SEGMENT

0:00 Welcome, Environment Check and Diagnostic — 30 min

TIMING

5m	Welcome and introductions
5m	How the course runs: 18 sessions in two parts, one or two lessons a session, the room and home split, the journal
12m	Environment check: `check_environment.sh` ends with READY on every machine, or the machine gets a partner and a clinic slot
8m	Diagnostic: six readiness questions, not graded

SEQUENCE AND TALKING POINTS

- **Welcome (5m).** One minute on yourself, including a WordPress site you once had to fix (the plugin that broke on an update, the theme someone edited by hand). Then a quick round: name, what they want to build with WordPress (client sites, a plugin to sell, the site at work), and which PHP and MySQL they already know. Write three or four goals on the board; they come back in the 1.1 journal prompt. Housekeeping in one breath: break at 1:20, where the lab files are, and "if you are stuck, put a sticky note on your laptop lid and keep trying while I get to you."
- **How the course runs (5m).** Show the course home page. Say: "Eighteen sessions of three hours in two parts. Part 1, Themes, the first eight sessions, takes you from a local site to the Pages & Co. block theme, a child theme and a classic theme; Part 2, Plugins to Launch, builds the pagesco-books plugin, custom blocks, REST endpoints, tests and a deployment, and ends with the whole site launched." Then the deal: "Each lesson was written for someone working alone for 100 to 150 minutes, so two do not fit in three hours. In this room we do what is hard alone: the mental model, a live demo, a guided lab, and the first steps of each project. At home you finish the projects and do the exercises, quiz and journal: plan on two to three hours a week." Scroll once through `lesson_01.html`: objectives, sections with real output, common mistakes made on purpose, a project in steps, exercises with hidden solutions, a quiz, a journal prompt.
- **Environment check (12m).** Say: "Before any new ideas, we find out which machines are ready." On the projector, then theirs: **1** open a terminal in the project folder (`cd ~/pagesco-dev`, the folder with compose.yaml and labs/); **2** `docker compose up -d` if the site is not running; **3** `bash labs/module-01/l1_0-orientation/check_environment.sh`. Done = the last line says `READY for PHP and WordPress Development` (`READY WITH NOTES` on PHP 8.4 or a project on the Windows drive is fine for today). Anyone who saw READY at home runs it again and becomes a helper. NOT READY falls into three piles: Docker not running or not reachable (often one minute: start Docker

Desktop, fix the WSL integration), FIX lines on a running site (the site is not installed, the Mailpit plugin is missing, the Linux owner fix: two to five minutes, fix now), no Docker or no project at all (do not start a download now: pair with a ready neighbor; install clinic at 1:40). Write the third pile's names on a sticky note.

- **Diagnostic (8m).** Say: "Six quick questions on what WordPress development builds on: PHP arrays, escaping, SQL, the terminal, HTTP and callbacks. Not a test, nobody collects it. It tells you what to brush up and tells me what to slow down for." Learners answer D1 to D6 on the workbook's Start Here page (about four minutes), then show fingers (1 to 4 for A to D) as you reveal each. Keep a tally of wrong answers per question on a sticky note.

DIAGNOSTIC ANSWERS AND HOW TO READ THE TALLY

Answers: **D1** A) 3 Dune; **D2** B) `Hello, <b>Maya</b>`; **D3** C) `SELECT title FROM books WHERE price < 10 ORDER BY price;`; **D4** D) `cd pagesco-dev`, then `docker compose up -d`; **D5** A) 404 Not Found; **D6** B) DUNE! EMMA!.

- **D1 (arrays) and D6 (callbacks)** predict the next two sessions: Lesson 1.3 is arrays of arguments and callbacks in every form, Lesson 2.3 hooks your functions into core by name. Two or more misses on D6: say "core calls YOUR function" every time a callback appears in Sessions 2 and 3.
- **D2 (escaping)** comes back in Lesson 1.3 (esc_html() and friends) and Lesson 5.3 (security). A miss here is the most important one to follow up: escaping is non-negotiable in WordPress code.
- **D3 (SQL)** comes back in Lesson 2.2 (WordPress's own tables) and 5.5 ($wpdb). Learners who miss it are usually rusty, not lost: point them to MySQL Foundations' SELECT lessons before Session 3.
- **D4 (folders)** predicts trouble in the next hour: `no configuration file provided` is almost always it. Learners who missed it get the typing role in the lab pair, with you nearby for the first command.
- **D5 (HTTP)** matters from Module 3 on (the 404 template, redirects). Misses are rare.
- **Three or more wrong:** not a problem to announce. Pair them with a strong neighbor today, and at the wrap-up hand them `labs/module-01/l1_0-orientation/diagnostic_starter.php`: D1, D2 and D6 done by PHP on their own site, one small gap each (`diagnostic_solution.php` is your reference). The workbook's Start Here page says where each skill comes back; PHP Foundations and MySQL Foundations are the review.

WATCH-OUTS

- **Do not debug one machine in front of the room.** Two minutes per machine during the check, then pair and move on: the install clinic at 1:40 exists so that you can say "we will fix that at 1:40" with a straight face.
- **Mixed backgrounds.** Some learners have built WordPress sites in the dashboard and never written a plugin; some write PHP daily and have never used WordPress. The diagnostic and the check are for both groups; say so.

- **"I use Local / XAMPP / MAMP."** Those work for WordPress, but the course's outputs, paths, port, mail catcher and WP-CLI fixes are tested on the Docker project (Lesson 1.2 FAQ). Ask them to run the course project alongside for the labs; nothing needs uninstalling.
- **"I already know WordPress."** Welcome them and give them a job: helper during the check and the clinic, and the stretch tasks in each lab. Old habits (editing a theme's functions.php from the dashboard, FTP to the live site, tutorials from before blocks) are exactly what Module 1 replaces.

LESSON PLANS

1.1 WordPress for Developers

50 min

OBJECTIVES (FROM THE LESSON PAGE)

- Describe WordPress 7.1 as a developer sees it: a PHP and MySQL application with themes, plugins, blocks and APIs as extension points
- Explain the difference between WordPress.org (the software) and WordPress.com (a hosting service), and pick the right one for a project
- Trace a front-end request from index.php to the finished HTML, naming the core files on the way
- Say where your code belongs (themes, plugins, must-use plugins in wp-content/) and why you never edit core files
- Map what you know from PHP Foundations and MySQL Foundations to the way WordPress does the same jobs
- Describe the Pages & Co. project and plan how you will work through this course

TIMING IN THE ROOM

10m	Frame: the apartment building; the extension points; WordPress.org and WordPress.com; load, query, template
15m	Live demo: the front door in core, curl, where code lives, two mistakes caught
15m	Guided lab: `core_tour_starter.sh` in pairs
10m	Start the lesson project: the Pages & Co. plan, Steps 3 and 4

MENTAL MODEL TO SAY

Tell it while you draw it: "WordPress core is an apartment building. The owner, the WordPress project, maintains the structure, the wiring and the plumbing, and renovates a few times a year. You are a tenant. You paint and furnish your rooms (themes), plug appliances into the sockets the building provides (plugins and hooks), and you can add a doorbell that rings in the lobby (the REST API). You never knock down a wall of the building: the next renovation would put it back."

Then the extension points as a list on the board: **hooks** (Lesson 2.3), **themes** (Modules 3 and 4), **plugins** (Module 5), **blocks** (Module 6), **APIs** (Module 7). "Almost everything a WordPress developer does is one of these five. Each has its module." Then WordPress.org and WordPress.com in two sentences: "WordPress.org is the free software and its community; WordPress.com is one company's hosting of it, where your own plugins depend on the plan. This course is self-hosted: your own site from Lesson 1.2."

Then the trip, one box at a time, on the drawing from the board: "Every front-end address goes to index.php. WordPress loads itself, settings first, then core, must-use plugins, plugins and the theme, and fires init. Then wp() turns the address into a query and runs it. Then the template loader picks the theme's template and draws the page. **Load, then query, then template.**" Write **inversion of control: core runs the program and calls you** under it.

◆ LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's own commands, typed live on the projector in the WP-CLI container shell (`docker compose run --rm wpcli sh`; the prompt changes to `/var/www/html $`). Ready learners type along; the others watch and get their turn after the clinic. Ask for a prediction before every run.

1. **The front door (3m).** "In PHP Foundations one file answered one address. Here almost every address goes to ONE file. Let us read it." Open a shell in the WP-CLI container (`docker compose run --rm wpcli sh`) and grep the two front-door files. Ask first: how many lines of real code does index.php have?

```
grep -n "WP_USE_THEMES\|require" index.php
grep -n "require\|wp();" wp-blog-header.php
```

```
14:define( 'WP_USE_THEMES', true );
17:require __DIR__ . '/wp-blog-header.php';
13: require_once __DIR__ . '/wp-load.php';
16: wp();
19: require_once ABSPATH . WPINC . '/template-loader.php';
```

   Two: one constant and one `require`. Then wp-blog-header.php does three things in order: load WordPress, `wp()`, the template loader. Draw them on the board as three boxes: **load, query, template**.

2. **The end of the trip (2m).** "Now from the outside, the way a browser asks." curl runs inside the container, where http://localhost:8080 is the site.

```
curl -s http://localhost:8080/hello-world/ | grep -o '<title>.*</title>'
```

```
<title>Hello world! &#8211; Pages &amp; Co.</title>
```

   "Hello world!" came from the main query, "Pages & Co." from the options table, the dash is an HTML entity. None of it is in a file: it is the database plus the theme.

3. **Where code lives (3m).** "Everything in the root belongs to WordPress, except two things." List the root, then the one folder that is yours.

```
ls
ls -d wp-content/*/
ls wp-content/themes wp-content/plugins
```

```
index.php             wp-activate.php       wp-comments-post.php  wp-config.ph
license.txt           wp-admin              wp-config-docker.php  wp-content
readme.html           wp-blog-header.php    wp-config-sample.php  wp-cron.php
wp-content/mu-plugins/  wp-content/plugins/     wp-content/themes/      wp-co
wp-content/plugins:
akismet    hello.php  index.php

wp-content/themes:
index.php          twentytwentyfive   twentytwentyfour   twentytwentythree
```

Point at `wp-config.php` (your settings) and `wp-content/` (your themes, plugins, must-use plugins, uploads). Ask: "who updates each folder?" Core: WordPress. Twenty Twenty-Five: its own updates. pagesco: you.

4 **Mistake 1: a quick fix in core (3m).** "A tutorial says: add a line to wp-includes/functions.php. Watch WordPress notice." The lesson's block makes a backup, appends a harmless comment, then compares every core file with the official checksums.

```
cp wp-includes/functions.php wp-content/functions.php.bak
echo '// my quick fix' >> wp-includes/functions.php
wp core verify-checksums
```

```
Warning: File doesn't verify against checksum: wp-includes/functions.php
Warning: File should not exist: wp-config-docker.php
Error: WordPress installation doesn't verify against checksums.
```

Read it from the top: the edited file fails; the second warning is the Docker image's own extra file (expected on every Docker site); the verdict is an Error. Restore and check again:

```
mv wp-content/functions.php.bak wp-includes/functions.php
wp core verify-checksums
```

```
Warning: File should not exist: wp-config-docker.php
Success: WordPress installation verifies against checksums.
```

"The next update would have removed your line without asking, at night, and a security scanner would have reported a hack."

5 **Mistake 2: an old tutorial (2m).** "This one still works. Listen to what WordPress says about it."

```
wp eval 'get_currentuserinfo();'
```

```
Deprecated: Function get_currentuserinfo is <strong>deprecated</strong> since
```

The message names the version that deprecated it (4.5.0, years ago) and the replacement. You only see it because debugging is on: that is why the course site always runs with WP_DEBUG (Lesson 1.2).

6. **What gives WordPress away (2m).** "Every WordPress site announces itself." Ask for the response headers only:

```
curl -sI http://localhost:8080/ | grep -i '^link'
```

```
Link: <http://localhost:8080/wp-json/>; rel="https://api.w.org/"
```

The REST API's address in a header (Module 7). Learners try the Bonus Challenge on a site they admire at home; hiding these clues is not security (Lesson 2.3). Type `exit` to leave the container shell.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **"Just edit the file."** A learner finds the line in core (or in Twenty Twenty-Five) that does what they want and changes it. Coach with the drawing and one question: "Who updates this folder?" If the answer is not "me", the change goes in a plugin or theme, on a hook. (Self-check Q1 and Q4.)
- **Thinking in pages, not in requests.** "Where is the file for /about/?" Coach: "There is none. index.php answers, wp() finds the page in the database, the theme draws it." (Self-check Q5, the lesson's Exercise 2.)
- **Trusting an old tutorial.** Classic-only code, `get_currentuserinfo()`, `query_posts()`, editing functions.php from the dashboard. Coach: "Which version was it written for? What does debug.log say when you run it?" (Mistake 2, the debug handout.)

GUIDED LAB

File: learners copy `labs/module-01/l1_1-wordpress-for-developers/core_tour_starter.sh` into wordpress/ and run it in the WP-CLI container; your reference is `labs/module-01/l1_1-wordpress-for-developers/core_tour_solution.sh`. It practices the lesson's central picture on core itself: each TODO is one grep that finds the line that does a step of the trip, so learners leave knowing the trip is real code they can read.

- **Run it as shipped first (3m).** On the projector, from the project folder: `cp labs/module-01/l1_1-wordpress-for-developers/core_tour_starter.sh wordpress/` then `docker compose run --rm wpcli sh core_tour_starter.sh`. Step 1 prints its line (`50:` of wp-load.php); every other step prints `TODO`.

- **TODOs 2 to 5 in pairs (10m).** Each is one grep (or a find and a count). Circulate for quoting: a pattern with `'` inside needs double quotes around it, and `do_action( 'init' )` has spaces inside the parentheses (core's coding standard, Lesson 1.3). TODO 4's plain `grep template_include` also finds a comment: ask what makes a pattern exact.
- **Expected (WordPress 7.1.3):** must-use plugins at line 506 of wp-settings.php, `plugins_loaded` at 630, `init` at 779, 1059 PHP files in wp-includes (the lesson says "more than a thousand").
- **Debrief (2m):** "Must-use plugins load at line 506, plugins after them, init at 779: what does that tell you about where a plugin can register a post type?" (On init or later: the lesson's Exercise 2.)
- **Fast finishers:** `labs/module-01/l1_1-wordpress-for-developers/wp_paths_starter.php` (where code lives, asked of WordPress itself with constants and two functions), then `labs/module-01/l1_1-wordpress-for-developers/debug_old_tutorial.php` (three deprecated calls from an old tutorial, fixed one at a time).

START THE LESSON PROJECT

The lesson project is **Meet WordPress and Plan Pages & Co.**: a test drive in WordPress Playground (Steps 1 and 2) and the plan the course builds from (Steps 3 and 4, `pagesco-plan.txt`). In the room, start the plan: it is the part that benefits from discussion (10 minutes).

- Step 3: learners create `pagesco-plan.txt` in a course folder and copy the lesson's seven addresses, then add three of their own (a staff picks page? a gift card page?).
- Step 4: the feature list, each feature marked content, theme or plugin with a reason. In the room: the first four of the lesson's eight required features. Use the lesson's test aloud: "What should happen if Pages & Co. switches to a different design next year?"
- **Done in the room** = ten addresses and four features with a reason. Take two answers for "the store notice bar" from the room: theme or plugin? (The course puts it in a must-use plugin in Lesson 2.3, and the arguments for both are good.)
- **Goes home:** Playground Steps 1 and 2 (their own dev site works for Step 2 too: the Code editor shows the same block markup) and the rest of Step 4. Keep the file: Lesson 5.1 asks for it again.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold-call gently, or think-pair-share):

- Which folder of the WordPress root is yours? **Answer:** `wp-content/` (and your settings in `wp-config.php`); everything else is core.
- In which order: init, the main query, the template? **Answer:** init (end of loading), then `wp()` runs the main query, then the template loader draws the page.
- A client is on WordPress.com and wants your plugin. What do you ask first? **Answer:** which plan they are on: only some allow your own plugins.

- Why does `get_currentuserinfo()` still work if it is deprecated? **Answer:** WordPress keeps old functions so old sites keep running; the message says what replaced it and since which version.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** C) `wp-includes/` ; **Q2** `<title>Pages & Co.</title>` ; **Q3** C) the init action fires, after plugins and the theme are loaded; **Q4** D) Someone edited a core file: restore it and put the change in a plugin or theme, on a hook; **Q5** B) `index.php` in the WordPress root; **Q6** `esc_html()` . Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners whose machines are not ready do the lab with a ready neighbor (one keyboard, two heads, the NOT READY learner reads the core lines aloud and says what each does). For learners new to grep, write `grep -n "text" file` on the board and leave it there; Step 1 of the starter is the model.

Stretch: `wp_paths_starter.php` and `debug_old_tutorial.php` ; then the lesson's Exercise 4 (read the front door yourself, including where `wp()` is defined) and the Bonus Challenge (spot WordPress on a site they admire).

1.2 Your Local Development Environment

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Install Docker and start the course's Docker Compose project: WordPress, MySQL, WP-CLI and Mailpit
- Install WordPress from the command line with WP-CLI and use the wp shortcut for everyday tasks
- Fix the Linux and WSL file-ownership problem so your editor can save into wp-content/
- Find and read PHP errors on the page and in wp-content/debug.log, and catch WordPress email in Mailpit
- Start, stop and reset the site safely, and know when wp-env or WordPress Playground is the better tool

TIMING IN THE ROOM

10m	Frame: the food truck; four services; the bind mount; the wp shortcut
25m	Install clinic: the machines that were NOT READY, by platform
15m	Live demo: versions, options, cron, debugging, a warning in the page and the log, Mailpit
15m	Guided lab: `site_report_starter.sh`
15m	Start the lesson project: the Dev Site Health Check, Steps 1 to 3

MENTAL MODEL TO SAY

Back to the board: "Installing a server stack on your laptop is building a restaurant kitchen into your home. Docker is a food truck parked next to the house: everything the kitchen needs comes with the truck, you can drive it away with `docker compose down` and park an identical one tomorrow. The one thing that stays in the house is your work: the `wordpress/` folder, which the truck reaches through a window, the bind mount."

Then the four services in a row: **wordpress** (Apache and PHP 8.5, the site on port 8080), **db** (MySQL 9.7, its data in a volume), **wpcli** (WP-CLI, started only when you call it, removed afterwards), **mailpit** (a fake mail server with an inbox on port 8025). "Everyone in this room runs the same images, so your site behaves like the one every lesson was tested on."

Then the two habits that save the most time: "Every command runs in the project folder, and `wp` is a shortcut for `docker compose run --rm wpcli wp`. When something breaks, the answer is in `wordpress/wp-content/debug.log`." Triage line on the board: **docker not found = install · daemon = start Docker · no configuration file = wrong folder · Permission denied = the Linux fix · critical error = debug.log**.

INSTALL CLINIC

Learners from the opening's NOT READY pile sit together by platform; ready learners pair with them as helpers. You move from table to table; each fix ends with `check_environment.sh` run again. Twenty-five minutes is enough to start Docker, fix an owner or a missing plugin, and to start a download, not to download a gigabyte on slow Wi-Fi: use the USB stick.

PLATFORM	MOST LIKELY STATE	CLINIC FIX (IN THIS ORDER)
Windows (WSL 2)	Docker Desktop installed, Ubuntu cannot see it; or the project under /mnt/c	WSL integration on for Ubuntu, a new Ubuntu terminal; the project in `~` inside WSL; then the Linux file-owner steps (WSL counts as Linux).
Windows (no WSL yet)	`wsl --install` not run, or not restarted	`wsl --install` in an administrator PowerShell, restart, create the Ubuntu user; finish Docker at home if the restart takes too long.
macOS	Docker Desktop installed but not started; or the Intel build on Apple silicon	Start it from Applications and wait for "running"; the right build for the chip. No file-owner steps on macOS.
Linux	docker group not active; the distribution's old docker-compose	`sudo usermod -aG docker "$USER"` and log in again; Docker's own repository for Engine and the Compose plugin (docs.docker.com/engine/install).
Linux and WSL, started without the fix	Files owned by uid 33; VS Code cannot save	Lesson 1.2's repair box: chown through a root wpcli container, the .env, compose.override.yaml, `docker compose up -d`.
Site up, not installed	The two install commands were skipped	`wp core install ...` and `wp rewrite structure '/%postname%/'` exactly as in the lesson; then the Mailpit plugin copy.
Nothing installed	Setup guide not done	Start the Docker install now (and `docker load` from the USB stick); pair for the rest of today; finish at home with the setup guide.

Done for each machine = `check_environment.sh` prints READY (or READY WITH NOTES). Anyone not done by 2:05 leaves with a numbered plan written in their workbook's Start Here box, and you check it at the start of Session 2.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

From the project folder on the projector, with the `wp` shortcut. The outputs below come from a fresh course site; on yours the time stamps differ and everything else matches.

1. **Which versions? (2m).** "First question on any site: which WordPress, which PHP?" From the project folder, with the `wp` shortcut:

```
wp core version --extra
wp eval 'echo "PHP ", PHP_VERSION, "\n";'
```

```
WordPress version: 7.1.3
Database revision: 61833
TinyMCE version:   4.9110 (49110-20250317)
Package language:  en_US
PHP 8.5.12
```

WordPress 7.1.3 on PHP 8.5.12: the versions every output in the course was made with. On PHP 8.4 the last line says 8.4 and the lessons say where that matters.

2. **Options, and an escaped title (2m).** "Settings live in the options table. Read three."

```
wp option get siteurl
wp option get blogname
wp option get permalink_structure
```

```
http://localhost:8080
Pages &amp;amp; Co.
/%postname%/
```

Stop on `Pages & Co.`: WordPress stored the title HTML-escaped. Ask: "where would that bite you?" (comparing strings, the subject line of an email).

3. **WordPress can call itself (1m).** "The reason for the port-8080 setup, tested in one command."

```
wp cron test
```

```
Success: WP-Cron spawning is working as expected.
```

4. **Debugging is on (2m).** "Where do these come from?" The WORDPRESS_* lines of compose.yaml, read by wp-config.php.

```
wp eval 'var_dump( WP_DEBUG, WP_DEBUG_LOG, WP_DEBUG_DISPLAY, SCRIPT_DEBUG, wp_
```

```
bool(true)
bool(true)
bool(true)
bool(true)
string(5) "local"
```

Open wordpress/wp-config.php in VS Code for ten seconds and point at the `getenv_docker()` lines (the workbook's worked example shows them).

5 **A warning in the page and in the log (5m).** "A real mistake, caught twice." Create the must-use plugins folder in the container shell, save the lesson's buggy demo in VS Code as wordpress/wp-content/mu-plugins/pagesco-debug-demo.php, reload the home page. Have learners tail the log in a second terminal: `tail -f wordpress/wp-content/debug.log`.

```php
<?php
/**
 * Plugin Name: Pages & Co. Debug Demo
 * Description: Lesson 1.2 demo: a deliberate bug in the page footer.
 */

function pagesco_debug_demo_footer() {
    echo '<p class="pagesco-demo">Hello, ' . $pagesco_name . '!</p>';
}
add_action( 'wp_footer', 'pagesco_debug_demo_footer' );
```

The new line in debug.log (the time stamp is the time of your reload):

```
[26-Oct-2026 11:58:58 UTC] PHP Warning:  Undefined variable $pagesco_name in /
```

And in the page itself, just above the footer greeting (here with curl from the container):

```
curl -s http://localhost:8080/ | grep -o 'Warning: [^<]*\|<p class="pagesco-de
```

```
Warning: Undefined variable $pagesco_name in /var/www/html/wp-content/mu-plug:
<p class="pagesco-demo">Hello, !</p>
```

Read it from the left: level, message, file inside the container (`/var/www/html/...` is your `wordpress/...`), line. Fix it with the lesson's second version (a value, and `esc_html()`), reload:

```
<p class="pagesco-demo">Hello, reader!</p>
```

Delete the demo file (`rm wp-content/mu-plugins/pagesco-debug-demo.php`): real must-use plugins start in Lesson 2.3.

6 **Email that never leaves the laptop (3m).** "WordPress sends email all the time; on a development site, Mailpit catches it." Show the must-use plugin, send one message, open http://localhost:8025.

```
wp plugin list --status=must-use
wp eval 'var_dump( wp_mail( "admin@example.com", "Hello from Pages & Co.", "If
```

```
+--------------+----------+--------+---------+----------------+-------------+
| name         | status   | update | version | update_version | auto_update |
+--------------+----------+--------+---------+----------------+-------------+
| mailpit-smtp | must-use |        |         |                | off         |
+--------------+----------+--------+---------+----------------+-------------+
bool(true)
```

Then open the message in Mailpit's inbox. What Mailpit caught:

```
From: WordPress <wordpress@example.com>
To: admin@example.com
Subject: Hello from Pages & Co.

If you can read this in Mailpit, email works.
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Typing commands in the wrong folder.** `no configuration file provided`, or a `wp eval-file` that cannot find a file. Coach: "Where is compose.yaml? And `wp eval-file x.php` means wordpress/x.php: where did you save it?" (Self-check Q10.)
- **Reaching for sudo or chmod 777** on Linux and WSL when the editor cannot save. Coach: "Who owns the files? `ls -ln wordpress`. The fix is to run the containers as you, once." (Lesson quiz Q2.)
- **Ignoring the log.** "The page is blank" or "There has been a critical error" with debug.log never opened. Coach: "What does the last line of debug.log say? Read it from the left." Make `tail -f wordpress/wp-content/debug.log` in a second terminal a habit from today.

GUIDED LAB

File: `labs/module-01/l1_2-local-environment/site_report_starter.sh`; your reference is `labs/module-01/l1_2-local-environment/site_report_solution.sh`. It practices WP-CLI on the learner's own site: each TODO is one `wp` command (read an option, list a theme or plugin with `--status` and `--field`, list pages as a table), with `wp help` as the reference. A script in the container needs no shortcut: `wp` is installed there.

- **Run it as shipped (3m).** `cp labs/module-01/l1_2-local-environment/site_report_starter.sh wordpress/` then `docker compose run --rm wpcli`

`sh site_report_starter.sh`: the WordPress line works, the rest say TODO.

- **TODOs 1 to 5 in pairs (9m).** Encourage trying each command alone first (`wp option get admin_email`), then putting it in the script. Circulate for `--status=active` spelled `--active` (WP-CLI says which options exist), and for TODO 5 without `--orderby=ID --order=asc` (the two install pages share a date, so the order is not guaranteed).
- **Expected when done** (a fresh course site):

```
== Pages & Co. site report ==
WordPress: 7.1.3
Title: Pages &amp;amp; Co.
Admin email: wordpress@example.com
Active theme: twentytwentyfive
Inactive plugins:
akismet
hello
Pages:
+----+----------------+-------------+
| ID | post_title     | post_status |
+----+----------------+-------------+
| 2  | Sample Page    | publish     |
| 3  | Privacy Policy | draft       |
+----+----------------+-------------+
```

- **Debrief (3m):** "Why does the title say `&`?" (stored escaped, the demo). "Which of these could you have done in the admin, and which not?" (All could, but none in one second, and none from a script you can rerun on any site.)
- **Fast finishers:** the folder lab `labs/module-01/l1_2-local-environment/store_hours_starter/` (a must-use plugin like the debug demo: Saturday's hours in the footer and one line in debug.log; copy `pagesco-hours.php` into `wordpress/wp-content/mu-plugins/`, test with `labs/module-01/l1_2-local-environment/test_store_hours.sh`), then `labs/module-01/l1_2-local-environment/debug_health_check.php` (two loud bugs and a silent one).

START THE LESSON PROJECT

The lesson project is the **Dev Site Health Check**: a PHP script that WP-CLI runs with WordPress loaded and that checks every piece the lesson set up. In the room: Steps 1 to 3 (15 minutes); the environment check already did most of Step 1.

- Step 1: `docker compose ps`, the site, the admin login, Mailpit, `wp core version` in a new terminal (two minutes after the check).
- Step 2: type `wordpress/pagesco-check.php` from the lesson (ten minutes). Type, do not paste: every line is PHP they know plus one WordPress function explained in its comment.

- Step 3: `wp eval-file pagesco-check.php` until all nine lines say OK, then find the health-check email in Mailpit.
- **Done in the room** = "All checks passed." on the screen. **Goes home:** Step 4 (stop Mailpit, watch one check fail, start it again, delete the script) and the checklist.

CHECKS FOR UNDERSTANDING

Ask these out loud during the clinic, the demo and the lab:

- Where does `wp option get blogname` actually run? **Answer:** in a new wpcli container that is removed when the command ends (Lesson quiz Q1).
- Your posts after `docker compose down` and `up -d`? And after `down -v`? **Answer:** still there; gone (the volume holds the database).
- A page shows "There has been a critical error on this website." First move? **Answer:** read the end of `wordpress/wp-content/debug.log`.
- Why is there a Mailpit at all? **Answer:** so test email from a development site never reaches real people; the mu-plugin sends it there, and fixes the sender address WordPress would build from localhost.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** C) `docker compose down -v`; **Q8** `Pages & Co.`; **Q9** B) `wordpress/wp-content/mu-plugins/pagesco-debug-demo.php` in your project folder; **Q10** A) You are not in the project folder: Docker Compose found no compose.yaml here or above; **Q11** D) WordPress calls its own address, http://localhost:8080, from inside the container (WP-Cron, Site Health), and that must reach Apache; **Q12** `local`: for example, Mailpit's mu-plugin only reroutes email when the environment is local; application passwords work over plain HTTP; WordPress 7.1 turns pings off. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners who spent the clinic installing do the lab at home with the README's expected output; for anyone overwhelmed by containers, one card with three lines: `cd ~/pagesco-dev`, `wp ...` for WordPress commands, `docker compose run --rm wpcli sh` for a shell with ls, grep and curl.

Stretch: the store-hours folder lab and `debug_health_check.php`; the lesson's Exercise 3 (the `wplog` and `wpclean` shortcuts) and Exercise 4 (wp-env alongside, with the right package name); the Bonus Challenge (Site Health against what they configured).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note and leave it on the way out. Sort the notes into "got it" and "not yet" before Session 2: question 1's pile tells you how much of

Session 2's warm-up goes back to "core is not yours".

1. Where does your own code go in WordPress, and why never in core? **Answer:** in wp-content/: your theme, your plugin, a must-use plugin; core files are replaced by every update (and an edited core file looks like a hack).
2. Name the four services of the course project and the job of each. **Answer:** wordpress (Apache + PHP, the site on 8080), db (MySQL, data in a volume), wpcli (WP-CLI on demand), mailpit (catches email, inbox on 8025).
3. A page says "There has been a critical error on this website." Where do you look first, and with which command? **Answer:** wordpress/wp-content/debug.log, for example `tail -n 20 wordpress/wp-content/debug.log` (or `tail -f` while you reload).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** The Pages & Co. plan (Playground Steps 1 and 2, the rest of the feature list) and the health check Step 4 (break it, read it, clean up: `mu-plugins/` holds only mailpit-smtp.php afterwards).
- **Lesson exercises.** 1.1: Exercises 1 to 3 (theme, plugin or content; follow /books/dune/; choose the hosting); Exercise 4 is a stretch. 1.2: Exercises 1 to 3 (a restart, your MySQL skills with `wp db query`, the `wplog` shortcut); Exercise 4 (wp-env) is a stretch.
- **Both Quick Quizzes** on the lesson pages (five questions each; answers are in the Answer Key).
- **Journal:** both entries. The 1.2 prompt asks for a cheat card of the development site; it is the page learners will open most in the next weeks.
- **Before the Next Lesson (from 1.2):** the health check passes and `wp` works in a brand-new terminal; mailpit-smtp.php stays in mu-plugins/; `docker compose stop` at the end of a session, `up -d` at the start.
- **If the environment check was not READY:** follow the numbered plan from the clinic, run the check again and email you its last line.
- **Optional:** `labs/module-01/l1_0-orientation/diagnostic_starter.php` for anyone who missed two or more diagnostic questions; `wp_paths_starter.php`, the store-hours lab and `debug_health_check.php` for anyone who did not reach them.

BRING LIST FOR SESSION 2

- A laptop on which `check_environment.sh` ends with READY (or READY WITH NOTES), and VS Code with a PHP extension.
- The project folder with the `labs/` folder in it. Session 2 (Lessons 1.3 and 2.1) installs a translation and runs Composer from the `composer:2` image: run `docker pull composer:2` at home, on a good connection.
- `pagesco-plan.txt`, the journal, and the workbook.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the check result for every learner; the install clinic; the 1.1 demo through Mistake 1; the 1.2 debug demo.
- **Cut first (saves 4m):** the 1.1 demo's Link header and the 1.2 Mailpit step (the health check shows Mailpit at home).
- **Cut second (saves 10m):** the 1.1 project start. Show the lesson's site map, send Steps 1 to 4 home.
- **Cut third (saves 15m):** the 1.2 project start: learners type the health check at home; the demo already showed every piece it checks.

RUNNING SHORT: WHAT TO ADD

- Run `debug_old_tutorial.php` for everyone and fix it together, one call per volunteer, reading the replacement from each message.
- Lesson 1.2's Exercise 2 live: `wp db query "SHOW TABLES"` and a SELECT on wp_posts. Ask who can predict the table prefix and which page is a draft.
- Start the 1.1 Bonus Challenge together on a real site: View Source, /wp-content/ paths, the generator tag, /wp-json/.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 2: PHP the WordPress Way and the Content Model

How to teach the session where learners start writing WordPress code and stock the site with content: the house rules of WordPress PHP (coding standards checked by PHPCS, prefixes, sanitize early and escape late, translatable strings, callbacks, arrays of arguments, WP_Error), then the content model seen from WP-CLI and PHP, ending with the Pages & Co. staff, pages and News posts that every later lesson starts from.

`2 lessons` `3 hrs session` `Lessons 1.3-2.1` `Session 2 of 18 · Part 1`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's ideas) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Type, do not paste (the block-markup files of the 2.1 project are the one exception: learners copy them from the lesson page). Every command below was run to produce the output printed under it, on WordPress 7.1.3, PHP 8.5, MySQL 9.7 and WP-CLI 2.12: a fresh course site, exactly what a learner has at the end of Session 1. Ask for a prediction before every run.
- The lessons are the reference. Keep `lesson_03.html` and `lesson_04.html` open in browser tabs for the bridge table, the WPCS rules table, the escaping table and the Settings-to-options table. Labs live in `labs/module-02/` (learners keep the folder in their project folder): `_starter` files are handouts, `_solution` files are yours.
- Session 2 is where the course site starts to hold real content. Lesson 1.3 leaves the site as it found it; Lesson 2.1's project creates the staff accounts, pages and News posts that **every later lesson starts from**, with fixed IDs (users 2 and 3, pages 12 to 14, posts 15 to 17). Protect that project's start above everything else in the session.
- Part 1, "Themes" (Sessions 1 to 8), can run on its own; nothing here depends on Part 2. Teal blocks are hands-on, amber blocks are watch-outs.

> **THE #1 RISK OF SESSION 2: THE IDS DRIFT**
>
> Lesson 2.1's project prints IDs 12 to 17 because every earlier block of the lesson used one up: attachment 5, the revision demo's post 6 and its revisions 7 and 8, the Gift Cards draft 9, the duplicate Sample Page 10 and its trash revision 11. A learner who watched the demo instead of typing it along gets pages 5 to 7 and posts 8 to 10, and every later lesson that names an ID (2.2 adds meta to post 15, Session 3's site check lists 12 to 17) stops matching. **So:** in the 2.1 demo everybody types along, and before the project every machine runs the next-ID check below. It must print 12. If it prints less, the catch-up line uses up the missing IDs on drafts it deletes again. Run the check AFTER the guided lab: the lab only reads.

BEFORE THE SESSION

- ☐ **Read Session 1's exit tickets** and the list of NOT READY machines: anyone who is still not ready pairs with a neighbor today and gets the install clinic's plan checked in the warm-up.
- ☐ **Network:** Lesson 1.3 downloads the WordPress coding standards with Composer (the `composer:2` image, about 70 MB once, which Session 1's bring list asked learners to pull at home, and about 3 MB of packages) and a Spanish translation (about 1 MB). Test the venue Wi-Fi with both the day before. Without it: show the lesson's output cards for the PHPCS and es_ES steps, and PHPCS moves to homework.
- ☐ **Files to hand out:** the `labs/` folder **without** the `*_solution.*` files (learners who copied the whole folder in Session 1 already have it); the workbook.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-02` from this kit's folder (every line PASS; 10 results in about a minute). With your own course project at the end of Session 1: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-02`.
- ☐ **Projector machine:** the course site exactly as Session 1 left it (only mailpit-smtp.php in mu-plugins, no content beyond the install: run the warm-up's site check), the `composer:2` image pulled, VS Code on `pagesco-dev` at a large font, two terminals in the project folder (one for commands, one for `tail -f wordpress/wp-content/debug.log`), `lesson_03.html` and `lesson_04.html` in tabs, http://localhost:8080/wp-admin/ logged in as admin.
- ☐ **Board, before learners arrive:** the escaping table (text → `esc_html()`, attribute → `esc_attr()`, href/src → `esc_url()`, post HTML → `wp_kses_post()`); "sanitize early, escape late"; the next-ID check command; a box labeled **one filing cabinet: wp_posts**.

KNOWN TRAPS TODAY

SYMPTOM	CAUSE	FIX
Composer shell: `Permission denied` writing composer.json (Linux, WSL)	The practice folder was created by a container (root) or `--user` was left out	`mkdir wordpress/pagesco-practice` as yourself first, then the lesson's `docker run --rm -it --user "$(id -u):$(id -g)" ...` line exactly.

SYMPTOM	CAUSE	FIX
PowerShell: `docker run ... $PWD/wordpress` mounts nothing	PowerShell writes the variable differently	Lesson 1.3's note: leave `--user` out and write `${PWD}`; better, use the Ubuntu terminal like every other command.
`Cannot create cache directory /composer-cache/...`	Composer's cache is not writable for your user in the container	Harmless: Composer proceeds without a cache.
`vendor/bin/phpcs: not found`	PHPCS runs outside the Composer shell, or in the wrong folder	In the Composer shell, in `/var/www/html/pagesco-practice` (the prompt shows it).
`wp eval-file pagesco-practice/escape.php`: `Error: 'pagesco-practice/escape.php' does not exist.`	The file was saved in the project folder, not in `wordpress/pagesco-practice/`	Save it in VS Code under `wordpress/pagesco-practice/`; `wp` runs in the WordPress root.
The es_ES install hangs or fails	No internet or translate.wordpress.org blocked	Show the lesson's output card; nothing later depends on it (the language is uninstalled again).
Project 2.1: `Error: Invalid category 'news'` or the posts land in Uncategorized	Step 3's `wp term create category News --slug=news` skipped	Create the term, then `wp post term set <ID> category news` for each post (or delete and recreate the posts).
Project 2.1: About Us is page 5 (or 8), not 12	The learner did not type along the 2.1 demo	Delete the new content (`wp post delete <IDs> --force`, `wp user delete maria devon --yes`... and recreate after the catch-up line). Better: run the next-ID check before Step 2.
`wp post create pagesco-content/about.html`: `Error: Unable to read content from 'pagesco-content/about.html'`	The folder is in the project folder, not in `wordpress/`	`mkdir wordpress/pagesco-content`; the files go there.

SESSION 2 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 1, the site check, homework check (the health check, the plan, the composer:2 image).
0:10	10	**1.3** Frame and mental model	The shared house; the bridge table from PHP Foundations; core still runs on PHP 7.4, your code above its own minimum.
0:20	25	**1.3** Live demo	PHPCS on PHP Foundations code and on the WordPress version, escaping per context, the added slashes, core in Spanish, seven callbacks, WP_Error, the silent _e().
0:45	20	**1.3** Guided lab	`labs/module-02/l1_3-php-the-wordpress-way/signup_form_starter.php` in pairs: sanitize a slashed form, then escape each value for its place.
1:05	15	**1.3** Start the lesson project	The Book Card Helper, Step 1 typed and run (Step 2 if time); the break test, PHPCS and the clean-up go home.
1:20	10	Break	Write "one filing cabinet, many labels" on the board for after the break, and the next-ID check.
1:30	10	**2.1** Frame and mental model	One filing cabinet; posts and pages; terms; roles bundle capabilities; settings are options; block markup is text.
1:40	20	**2.1** Live demo	The lesson's blocks in order, typed along: post types, WP_Post, statuses, terms, an attachment, revisions, capabilities, options, parse_blocks(), two silent mistakes, then the next-ID check (12).
2:00	15	**2.1** Guided lab	`labs/module-02/l2_1-content-model/content_census_starter.sh`: the site counted the way the lesson sees it, one WP-CLI command per line.
2:15	35	**2.1** Start the lesson project	Stock the Pages & Co. site, Steps 1 to 3: Maria and Devon, the three pages, the News category and three posts. Step 4 goes home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 3 (the site at the end of Lesson 2.1).
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 130 + 120 = 250 minutes for a learner working alone; after the warm-up, the break and the wrap-up the room has 150 (70 for 1.3, 80 for 2.1). Lesson 1.3 keeps model, demo and lab; its project starts with Step 1 (typing the helper is where the habits meet). Lesson 2.1's demo runs the lesson's blocks in order, typed along, and its project gets the longest segment of the day because Session 3 starts from it. **Not in the room:** 1.3's i18n.php, args.php, price.php and globals.php examples and its Common Mistakes 1, 2 and 4 (the escape.php

demo covers 1; 2 and 4 take the whole site down and are best read with the lesson's output cards); the strict-types box and the FAQ; 2.1's image sizes and srcset (`wp_get_attachment_metadata`, `wp_get_attachment_image`), the roles table, the user meta behind roles, Mistakes 3 and 4 (a role checked instead of a capability: self-check Q9 and the debug handout; HTML without block comments) and the version note.

Goes home (about 2 to 3 hours). 1.3: project Steps 2 to 4 (render three books, the break test, PHPCS until it is silent, then delete the practice folder); 2.1: project Step 4 (Maria's edit and its revision, Devon refused, `rm -r pagesco-content`, the final list); each lesson's Hands-on Exercises; both Quick Quizzes; both journal entries; both "Before the Next Lesson" lists.

Protect, in this order: the 2.1 project Steps 1 to 3 on every machine, with the right IDs; the 2.1 demo's ID-using steps (the attachment, the revisions, the two mistakes); the 1.3 PHPCS and escaping steps; the 1.3 guided lab. **Cut first:** the 1.3 es_ES and callbacks steps (6 minutes; both are in the lesson), then the 1.3 project start (send Step 1 home: 15 minutes), then the 2.1 guided lab (15 minutes: at home its counts are those of the stocked site, which the README lists).

OPENING SEGMENT

0:00 Warm-up: Session 1 Revisited — 10 min

TIMING

5m	Five retrieval questions, no notes (workbook Warm-up box), then answers by show of hands
3m	Site check: two WP-CLI commands on every machine, the same answer as the projector
2m	Homework check: the health check, the plan, the composer:2 image

RETRIEVAL QUESTIONS AND ANSWERS

1. Which folder of the WordPress root is yours, and why is nothing else? **Answer:** `wp-content/` (themes, plugins, mu-plugins, uploads), plus your settings in `wp-config.php`. Everything else is core and every update replaces it.
2. Put them in order: the main query, `init`, the template. **Answer:** `init` (the end of loading: core, mu-plugins, plugins, the theme), then `wp()` runs the main query, then the template loader draws the page: load, query, template.
3. `docker compose down` and `docker compose down -v`: what survives each? **Answer:** `down` keeps the files and the database (the volume); `-v` deletes the database volume too.
4. A page says "There has been a critical error on this website." Where do you look first? **Answer:** The end of `wordpress/wp-content/debug.log` (`tail -n 20 wordpress/wp-content/debug.log`, or `tail -f` while reloading).
5. What does `wp option get blogname` print on the course site, and why? **Answer:** `Pages & Co.`: WordPress stored the title HTML-escaped when it was saved. Today's first lesson explains when to escape yourself.

SITE CHECK (ON THE PROJECTOR, THEN EVERY MACHINE)

Say: "Both lessons today start from the site Session 1 left. Check yours is the same." From the project folder:

```
wp core version
wp plugin list --status=must-use --fields=name,status
```

```
7.1.3
+--------------+----------+
| name         | status   |
+--------------+----------+
| mailpit-smtp | must-use |
+--------------+----------+
```

- **Same answer:** ready. A learner who already did Lesson 2.1 at home (users maria and devon exist) is ahead: they repeat nothing today and help in the 2.1 project.
- **Another must-use plugin listed** (pagesco-hours, pagesco-debug-demo): a Session 1 lab or demo was not removed. `rm wordpress/wp-content/mu-plugins/<file>` now.
- `wp` **not found or** `no configuration file provided` **:** the shortcut or the folder (Session 1). Still NOT READY from Session 1: pair with a neighbor, check the clinic plan in the break.
- **Homework check (two minutes):** hands up for "All checks passed." from the health check and for `pagesco-plan.txt` (Lesson 2.1's journal reopens it). `docker image ls composer:2` on every machine: the image Lesson 1.3 needs in 15 minutes. Missing: `docker pull composer:2` now, it downloads during the frame.

1.3 PHP the WordPress Way

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Write PHP that follows the WordPress coding standards, and check it with PHPCS and the WordPress ruleset
- Choose the right escaping function for each output context (esc_html, esc_attr, esc_url, wp_kses_post) and the right sanitizing function for each input
- Make strings translatable with __(), esc_html__(), _n() and sprintf(), with a text domain and translator comments
- Pass callbacks in every form WordPress accepts, and write functions that take an array of arguments merged with wp_parse_args()
- Handle WP_Error results, and use core's globals ($wpdb, $post, $wp_query) and classes without fighting them

TIMING IN THE ROOM

10m	Frame: the shared house; the bridge table; core on PHP 7.4, your code above its own minimum
25m	Live demo: PHPCS before and after, escaping per context, slashes, core in Spanish, callbacks, WP_Error, the silent _e()
20m	Guided lab: `signup_form_starter.php` in pairs
15m	Start the lesson project: the Book Card Helper, Step 1

MENTAL MODEL TO SAY

"In PHP Foundations your code was the whole application. In WordPress it is a guest. Moving from your own apartment into a shared house: you label your food (prefixes), you use the shared kitchen tools the way everyone does (core's functions instead of your own helpers), you leave notes others can read (coding standards and DocBlocks), and you do not rearrange the living room (globals). None of the rules is hard. Ignoring them is what causes the fights."

Then the lesson's bridge table, four rows on the board: **htmlspecialchars() → esc_html() and friends · filter_input() → wp_unslash() + sanitize_*() · exceptions → WP_Error · PDO → $wpdb**. And one sentence on versions: "WordPress 7.1 still runs on PHP 7.4, so core looks old-fashioned. Your plugin declares its own minimum (Lesson 5.1) and may use modern PHP above it. You are learning conventions, not going back in time."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

From the project folder on the projector, with the `wp` shortcut; the Composer shell is a second terminal. Files are saved in VS Code under `wordpress/pagesco-practice/`, exactly as the lesson names them (the lesson page has each one).

1. **PHPCS on PHP Foundations code (6m).** "You already write good PHP. Let a tool tell us what WordPress thinks of it." Create the practice folder, save the lesson's two versions of one function (`style-before.php` and `style-after.php`) in VS Code, and open the Composer shell from the project folder:

   ```
   mkdir wordpress/pagesco-practice
   docker run --rm -it --user "$(id -u):$(id -g)" -v "$PWD/wordpress:/var/www/htm
   ```

   In that shell, the three lines that install the WordPress rules (about 20 seconds on good Wi-Fi; the download lines differ on every machine):

   ```
   composer init --name=pagesco/practice --no-interaction
   composer config allow-plugins.dealerdirect/phpcodesniffer-composer-installer t
   composer require --dev wp-coding-standards/wpcs:^3
   ```

   ```
   Writing ./composer.json
   ...
   Lock file operations: 5 installs, 0 updates, 0 removals
     - Locking dealerdirect/phpcodesniffer-composer-installer (v1.2.1)
     - Locking phpcsstandards/phpcsextra (1.5.1)
     - Locking phpcsstandards/phpcsutils (1.2.3)
     - Locking squizlabs/php_codesniffer (3.13.6)
     - Locking wp-coding-standards/wpcs (3.4.1)
   PHP CodeSniffer Config installed_paths set to ../../phpcsstandards/phpcsextra,
   No security vulnerability advisories found.
   ```

   Then the PHP Foundations version:

   ```
   vendor/bin/phpcs --standard=WordPress style-before.php
   ```

```
FILE: /var/www/html/pagesco-practice/style-before.php
----------------------------------------------------------------------
FOUND 20 ERRORS AND 1 WARNING AFFECTING 9 LINES
----------------------------------------------------------------------
 1 | ERROR   | [ ] Missing file doc comment
 2 | ERROR   | [ ] Function name "bookTitle" is not in snake case format, try
   |         |     "book_title"
 2 | ERROR   | [ ] Missing doc comment for function bookTitle()
 2 | ERROR   | [x] Expected 1 spaces after opening parenthesis; 0 found
 2 | ERROR   | [x] Expected 1 spaces before closing parenthesis; 0 found
 3 | ERROR   | [x] Opening brace should be on the same line as the declaratio
 4 | ERROR   | [x] Tabs must be used to indent lines; spaces are not allowed
 4 | ERROR   | [x] No space after opening parenthesis is prohibited
 4 | WARNING | [x] Loose comparisons are not allowed. Expected: "==="; Found:
   |         |     "=="
 4 | ERROR   | [ ] Use Yoda Condition checks, you must.
 4 | ERROR   | [x] String "" does not require double quotes; use single quote
   |         |     instead
 4 | ERROR   | [x] No space before closing parenthesis is prohibited
 5 | ERROR   | [x] Tabs must be used to indent lines; spaces are not allowed
 5 | ERROR   | [x] String "Untitled" does not require double quotes; use sing
   |         |     quotes instead
 6 | ERROR   | [x] Tabs must be used to indent lines; spaces are not allowed
 7 | ERROR   | [x] Tabs must be used to indent lines; spaces are not allowed
 7 | ERROR   | [x] String "<h2>" does not require double quotes; use single
   |         |     quotes instead
 7 | ERROR   | [x] String "</h2>" does not require double quotes; use single
   |         |     quotes instead
 8 | ERROR   | [x] Tabs must be used to indent lines; spaces are not allowed
 8 | ERROR   | [ ] All output should be run through an escaping function (see
   |         |     the Security sections in the WordPress Developer Handbooks)
   |         |     found '$out'.
 9 | ERROR   | [x] Tabs must be used to indent lines; spaces are not allowed
----------------------------------------------------------------------
PHPCBF CAN FIX THE 16 MARKED SNIFF VIOLATIONS AUTOMATICALLY
----------------------------------------------------------------------
```

Read three lines from the left: line, ERROR or WARNING, `[x]` when phpcbf can fix it. 20 errors and 1 warning, 16 of them mechanical. Stop on line 8: "All output should be run through an escaping function". PHPCS just found a cross-site scripting hole. That is the real reason professionals run it. Then the WordPress version:

```
vendor/bin/phpcs --standard=WordPress style-after.php && echo "No problems fou
```

```
No problems found.
```

Show `style-after.php` side by side for one minute: tabs, spaces inside parentheses, the Yoda condition, single quotes, the DocBlocks, `esc_html()` at the echo, and the one `// phpcs:ignore` with a reason. Learners type `exit` to leave the Composer shell; keep it in the terminal history for the project's Step 4.

2. **One value, four contexts (4m).** "Escape late, for the exact place the value goes." Save the lesson's `escape.php` in the practice folder and run it from the project folder:

```
wp eval-file pagesco-practice/escape.php
```

```
esc_html:    Dune &lt;script&gt;alert(&quot;hi&quot;)&lt;/script&gt; &amp; &qu
esc_attr:    Pages &amp; Co. &quot;staff pick&quot;
esc_url:     https://example.com/search?q=dune&#038;page=2
esc_url:     (empty)
wp_kses_post: <p>A <strong>classic</strong>alert(1) <a href="https://example.
twice:       Pages &amp; Co.
```

Line by line: text between tags (`esc_html()`), an attribute (`esc_attr()`), a URL whose `&` becomes `&` and whose `javascript:` disappears (`esc_url()`), user HTML kept but defanged (`wp_kses_post()`), and the kindness on the last line: no double-encoding.

3. **WordPress adds slashes to your input (3m).** "One surprise for PHP developers." Save the lesson's `pagesco-slashes.php` in `wordpress/wp-content/mu-plugins/`, open the address in the browser (View Source, search for pagesco-q), or with curl in the container:

```
curl -s 'http://localhost:8080/?pagesco_q=O%27Reilly%20%3Cb%3Ebooks%3C%2Fb%3E'
```

```
<p class="pagesco-q">raw: O\&#039;Reilly &lt;b&gt;books&lt;/b&gt; | clean: O&
```

Raw: `O\'Reilly` (the backslash from `wp_magic_quotes()`, the quote escaped for printing). Clean: unslashed, then sanitized: quote kept, tags gone. "Save a raw value and every apostrophe in your database grows a backslash." Delete the demo: `rm wp-content/mu-plugins/pagesco-slashes.php` in the container.

4. **Core speaks Spanish (3m).** "Every visible string goes through a translation function." Install Spanish, ask core for two of its strings, uninstall again (it needs the internet; if the venue blocks it, show the lesson's output card):

```
wp language core install es_ES
wp eval 'switch_to_locale( "es_ES" ); echo __( "Posts" ), " / ", _n( "%s comme
wp language core uninstall es_ES
```

```
Downloading translation from https://downloads.wordpress.org/translation/core/
Unpacking the update...
Installing the latest version...
Removing the old version of the translation...
Translation updated successfully.
Language 'es_ES' installed.
Success: Installed 1 of 1 languages.
Entradas / %s comentarios
Posts
Success: Language uninstalled.
```

`__( 'Posts' )` found "Entradas" in Spanish and returns the English string otherwise. Your theme adds a text domain, its folder slug: `pagesco`. On the board: `__()` **returns,** `_e()` **echoes,** `esc_html__()` **is the everyday choice,** `_n()` **for plurals,** `sprintf()` **with a translators comment for variables**.

5 **Seven shapes of a callback (3m).** "WordPress asks for callbacks everywhere: every hook, shortcode, REST route." Save `callbacks.php` from the lesson and run it:

```
wp eval-file pagesco-practice/callbacks.php
```

```
function name   DUNE!
closure         Dune
arrow function  enud
object method   dune (staff pick)
static method   "dune"
static, string  "dune"
first-class     DUNE!
```

Point at the column of names. Ask: "which of these can another plugin unhook later?" (The named function and the methods; never the closure or the arrow function: Lesson 2.3.)

6 **Errors come back as values (3m).** "Core predates exceptions." Save `errors.php` and run it:

```
wp eval-file pagesco-practice/errors.php
```

```
Code:    existing_user_login
Message: Sorry, that username already exists!
int(0)
Content, title, and excerpt are empty.
```

`wp_insert_user()` returns a WP_Error; `wp_insert_post()` returns 0 unless you pass `true`. "Read the Return line of the Code Reference before you trust a result." Nothing was created: both calls failed before the database.

7 **Mistake 3: _e() in an expression (silent) (2m).** "One last trap, and it makes no noise." Save `mistake-e.php` and run it:

```
wp eval-file pagesco-practice/mistake-e.php
```

```
New arrivals
<h2></h2>
New arrivalsNULL
```

The text printed before the heading was even built, and the heading is empty. `_e` echoes, the others return. Leave the practice folder in place: the project uses it (and its PHPCS) next; Mistakes 2 and 4 (a name already taken, strict types) take the whole site down and go home, with the lesson's output cards.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **The wrong escaper, or one for everything.** `esc_html()` around an href, or `esc_attr()` everywhere. Coach with the board's table and one question: "Where does this value land: between tags, in an attribute, in a URL?" (Self-check Q1, lesson quiz Q1, the guided lab's TODOs 6 and 7.)
- **Sanitizing on output, escaping on input.** `$nick = esc_html( $_POST['nickname'] )`. Coach: "Sanitize is for the way in, escape for the way out; the database should hold the clean value, not HTML entities." (Lesson quiz Q5.)
- **Forgetting `wp_unslash()`.** The apostrophe in O'Neil grows a backslash. Coach: "WordPress added it on the way in; take it off before you clean." (The slashes demo, the guided lab's TODO 1.)

GUIDED LAB

File: learners copy `labs/module-02/l1_3-php-the-wordpress-way/signup_form_starter.php` into wordpress/ and run it with `wp eval-file --use-include signup_form_starter.php`; your reference is `labs/module-02/l1_3-php-the-wordpress-way/signup_form_solution.php`. It practices both halves of the rule on data nobody has seen: a book club sign-up with the backslashes WordPress adds, a name with tags, a shouting email address, a negative seat count, a shelf that is not on the list, and a `javascript:` website.

- **Run it as shipped (2m).** Every value prints `TODO`, and the two HTML lines show where escaping is missing.
- **TODOs 1 to 4 in pairs (10m): sanitize early.** Circulate for `wp_unslash()` missing (the name keeps `O\'Neil`), and for TODO 4 written as a sanitize call: the shelf must be VALIDATED against the three

allowed values with `in_array( ..., true )`.

- **TODOs 5 to 7 (5m): escape late.** `esc_attr()` in the value attribute, `esc_url()` around the unslashed website (it comes back empty), `esc_html()` for the link text.
- **Expected when done:**

```
name:  string(18) "Siobhan O'Neil VIP"
email: Siobhan.ONeil@Example.COM (valid: yes)
seats: 3
shelf: fiction
field: <input name="name" value="Siobhan O&#039;Neil VIP">
link:  <a href="">Siobhan O&#039;Neil VIP</a>
```

- **Debrief (3m):** "Why did the name keep VIP?" (`sanitize_text_field()` strips the tags, not the text between them.) "Seats: -3 became 3. Is that right?" (No: `absint()` flips the sign; a real form should REJECT a negative, the lesson's point about validating.) "Why is the href empty?" (`esc_url()` drops a `javascript:` URL.)
- **Fast finishers:** `labs/module-02/l1_3-php-the-wordpress-way/event_line_starter.php` (an events list with an array of arguments, `_n()` and `sprintf()` with translator comments, a sort callback and a WP_Error), then `labs/module-02/l1_3-php-the-wordpress-way/debug_book_badge.php` (two loud bugs and a silent one in a badge helper).

START THE LESSON PROJECT

The lesson project is **The Book Card Helper**: `pagesco_book_card( $book, $args )` in `wordpress/pagesco-practice/book-card.php`, using every habit of the lesson, with a clean PHPCS report at the end. In the room: Step 1 (15 minutes).

- Step 1: type `book-card.php` from the lesson, reading each comment aloud with a neighbor: it names the rule the line follows (the DocBlock with `@type` keys, `wp_parse_args()`, the heading level validated, `esc_attr()` / `esc_url()` / `esc_html()`, `__()` and `_n()` with translators comments).
- If there is time: Step 2's `render-cards.php` and `wp eval-file pagesco-practice/render-cards.php`. Three cards, "1,178 pages".
- **Done in the room** = book-card.php saved and loading without a message (Step 2 prints three cards when it runs). **Goes home:** Steps 2 to 4: the break test (a `javascript:` link, quotes in the class, a heading level of 99), PHPCS until "no coding standards problems", then `rm -r pagesco-practice` and `ls wp-content/mu-plugins` (only mailpit-smtp.php). Keep a copy of book-card.php in the course notes: Lesson 6.2 compares it with a block, and Session 3's warm-up asks whether it passed PHPCS.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold-call gently, or think-pair-share):

- Which escaper for a link's href, for its title attribute, and for its text? **Answer:** `esc_url()`, `esc_attr()`, `esc_html()`.
- Why does every function in the course theme start with `pagesco_`? **Answer:** one global namespace per request, shared by core and every plugin; two functions with one name are a fatal error.
- What do `__()` and `_e()` do differently? **Answer:** `__()` returns the translation, `_e()` echoes it (and returns null).
- A WordPress function failed. Exception, false, or something else? **Answer:** usually a WP_Error (check with `is_wp_error()`); some return 0 or false unless you ask for the error: read the Return line.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** B) `esc_attr()`; **Q2** `Tom & Jerry`; **Q3** C) "Staff picks" first, then an empty `<h2></h2>`; **Q4** A) A class inside the namespace `PagesCo\Books`, such as `PagesCo\Books\Plugin`; **Q5**

```
int(7)
string(10) "staffpicks"
```

; **Q6** D) `0`: pass `true` as the second argument to get a WP_Error with the message. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: learners who are slow in the Composer shell watch the PHPCS step on the projector and set it up at home (the project's Step 4 needs it). For the lab, a card with the four sanitizers and the four escapers, each with one example input and output, from the demo.

Stretch: `event_line_starter.php` and `debug_book_badge.php`; the lesson's Exercise 1 (make PHPCS happy with `phpcbf`), Exercise 4 (`wp_list_filter()` and friends) and the Bonus Challenge (PHPCS on Hello Dolly and Twenty Twenty-Five, with the three rule sets).

2.1 The Content Model

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain why posts, pages, media, revisions, templates and navigation menus are all posts with a different post_type, and read a WP_Post object
- Choose between posts and pages, use post statuses, and organize content with categories, tags and terms
- Describe how roles, capabilities and users fit together, and check capabilities rather than roles in code
- Find the option behind a Settings screen, and read the block markup the editor stores in post_content with parse_blocks()
- Create users, pages and posts from the command line, with authors, categories, dates and revisions

TIMING IN THE ROOM

10m	Frame: one filing cabinet; posts and pages; terms; roles and capabilities; options; block markup
20m	Live demo: the lesson's blocks in order, typed along, then the next-ID check
15m	Guided lab: `content_census_starter.sh`
35m	Start the lesson project: Stock the Pages & Co. site, Steps 1 to 3

MENTAL MODEL TO SAY

Draw a filing cabinet with one drawer labeled **wp_posts**: "Every document goes into the same kind of folder with the same fields: title, content, author, date, status. A colored label says what it is: blog post, page, image, old version, template, menu. Code that works on 'a folder' (saving, revisions, permissions, the REST API) works on every label at once. That is why a custom post type, Lesson 5.2's book, gets all of it for free."

Then the rest of the picture around the cabinet: **terms** (colored tabs: categories, tags, later genre), **users** (who may open which drawer: roles are bundles of capabilities, and code checks the capability), **options** (the settings on the wall: every Settings field is one), and **block markup** (what is written on the paper: HTML wrapped in comments). "Every feature you meet later, in core or in a plugin, is one of these in disguise."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's own commands, in the lesson's order, from the project folder with the `wp` shortcut. **Everyone types along**: the attachment, the revision demo and the two mistakes each use up post IDs that the project's IDs (12 to 17) count on. Keep the commands on the projector for the slow typists; skip nothing that creates a post.

1. **Sixteen types, one table (2m).** "Everything in the admin is a post with a label." Everyone types along from here on, in the project folder: the lesson's IDs depend on it (the box below).

```
wp post-type list --fields=name,label,hierarchical,public
```

```
+--------------------+----------------------+--------------+--------+
| name               | label                | hierarchical | public |
+--------------------+----------------------+--------------+--------+
| post               | Posts                |              | 1      |
| page               | Pages                | 1            | 1      |
| attachment         | Media                |              | 1      |
| revision           | Revisions            |              |        |
| nav_menu_item      | Navigation Menu Items |              |        |
| custom_css         | Custom CSS           |              |        |
| customize_changeset | Changesets           |              |        |
| oembed_cache       | oEmbed Responses     |              |        |
| user_request       | User Requests        |              |        |
| wp_block           | Patterns             |              |        |
| wp_template        | Templates            |              |        |
| wp_template_part   | Template Parts       |              |        |
| wp_global_styles   | Global Styles        |              |        |
| wp_navigation      | Navigation Menus     |              |        |
| wp_font_family     | Font Families        |              |        |
| wp_font_face       | Font Faces           |              |        |
+--------------------+----------------------+--------------+--------+
```

16 types; five of them are what an editor calls content. Point at `wp_template`, `wp_navigation` and `wp_global_styles`: "block themes keep their templates, menus and styles in this same table. Module 3 lives here."

2. **A post as PHP sees it (2m).** "One row, one WP_Post object."

```
wp post get 1 --fields=ID,post_type,post_status,post_name,post_title,post_auth
```

```
+----------------+--------------+
| Field          | Value        |
+----------------+--------------+
| ID             | 1            |
| post_author    | 1            |
| post_title     | Hello world! |
| post_status    | publish      |
| comment_status | open         |
| post_name      | hello-world  |
| post_parent    | 0            |
| post_type      | post         |
+----------------+--------------+
```

`post_name` is the slug, `post_author` a user ID, `post_parent` a post ID. A book's price will NOT be a column: it goes to post meta (Lessons 2.2 and 5.2).

3. **Posts, pages and statuses (2m).** "Two tools, two defaults."

```
wp post list --post_type=page --orderby=ID --order=ASC --fields=ID,post_title,
wp eval 'foreach ( get_posts( array( "post_type" => "page" ) ) as $page ) { ec
```

```
+----+----------------+----------------+-------------+
| ID | post_title     | post_name      | post_status |
+----+----------------+----------------+-------------+
| 2  | Sample Page    | sample-page    | publish     |
| 3  | Privacy Policy | privacy-policy | draft       |
+----+----------------+----------------+-------------+
2 Sample Page publish
```

WP-CLI lists every status; `get_posts()` and WP_Query default to publish, like the front end. That one line explains half of all "my page does not show" questions.

4. **Terms (1m).** "Categories and tags are two taxonomies of seven."

```
wp taxonomy list --fields=name,label,hierarchical,public
wp term list category --fields=term_id,name,slug,count
```

```
+----------------------+---------------------+--------------+--------+
| name                 | label               | hierarchical | public |
+----------------------+---------------------+--------------+--------+
| category             | Categories          | 1            | 1      |
| post_tag             | Tags                |              | 1      |
| nav_menu             | Navigation Menus    |              |        |
| post_format          | Formats             |              | 1      |
| wp_theme             | Themes              |              |        |
| wp_template_part_area | Template Part Areas |              |        |
| wp_pattern_category  | Pattern Categories  |              |        |
+----------------------+---------------------+--------------+--------+
+---------+---------------+---------------+-------+
| term_id | name          | slug          | count |
+---------+---------------+---------------+-------+
| 1       | Uncategorized | uncategorized | 1     |
+---------+---------------+---------------+-------+
```

Every post must have a category: the `default_category` option names Uncategorized. The News category arrives in the project.

5 **An attachment is a post too (3m).** "An upload is a file AND a post." The lesson switches the year/month folders off so the paths match:

```
wp option update uploads_use_yearmonth_folders 0
wp media import wp-content/themes/twentytwentyfive/assets/images/book-image.we
wp post get 5 --fields=ID,post_type,post_status,post_title,post_mime_type,post
wp post meta get 5 _wp_attachment_image_alt
find wp-content/uploads -name '*.webp' | sort
```

```
Success: Updated 'uploads_use_yearmonth_folders' option.
Imported file 'wp-content/themes/twentytwentyfive/assets/images/book-image.wel
Success: Imported 1 of 1 images.
+----------------+------------------------+
| Field          | Value                  |
+----------------+------------------------+
| ID             | 5                      |
| post_title     | The Stories Book cover |
| post_status    | inherit                |
| post_parent    | 0                      |
| post_type      | attachment             |
| post_mime_type | image/webp             |
+----------------+------------------------+
Cover of The Stories Book: a plaster bust on a black background
wp-content/uploads/book-image-150x150.webp
wp-content/uploads/book-image-214x300.webp
wp-content/uploads/book-image-730x1024.webp
wp-content/uploads/book-image-768x1077.webp
wp-content/uploads/book-image.webp
```

Status `inherit`, a MIME type, the alt text in post meta, four generated sizes on disk. Delete it again (with `--force` WordPress deletes the files too):

```
wp post delete 5 --force
wp option update uploads_use_yearmonth_folders 1
find wp-content/uploads -name '*.webp'
```

```
Success: Deleted post 5.
Success: Updated 'uploads_use_yearmonth_folders' option.
```

6 **Every update is a revision (3m).** "Predict: three versions, how many revisions?"

```
wp post create --post_title="Revision demo" --post_content="Version one." --po
wp post update 6 --post_content="Version two."
wp post update 6 --post_content="Version three."
wp post list --post_type=revision --post_parent=6 --orderby=ID --order=ASC --f
```

```
6
Success: Updated post 6.
Success: Updated post 6.
+----+---------------+----------------+-------------+
| ID | post_name     | post_content   | post_status |
+----+---------------+----------------+-------------+
| 7  | 6-revision-v1 | Version two.   | inherit     |
| 8  | 6-revision-v1 | Version three. | inherit     |
+----+---------------+----------------+-------------+
```

Two, and the newest equals the current post: "every update is a revision, and the most recent revision always matches the current post". Then the constants and the clean-up:

```
wp eval 'var_dump( WP_POST_REVISIONS, wp_revisions_to_keep( get_post( 6 ) ), A
```

```
bool(true)
int(-1)
int(60)
int(30)
```

```
wp post delete 6 --force
wp post list --post_type=revision --format=count
```

```
Success: Deleted post 6.
0
```

7 **Roles bundle capabilities (2m).** "Code asks: may this user do X? Never: is this user an editor?"

```
wp eval 'var_dump( user_can( 1, "manage_options" ), user_can( 1, "edit_post",
wp eval 'var_dump( current_user_can( "edit_posts" ) );' --user=admin
```

```
bool(true)
bool(true)
bool(false)
bool(true)
```

The last two lines differ only by `--user=admin`: WP-CLI runs as nobody unless you name a user, and so does code that runs too early or from cron.

8 **Settings are options, all strings (1m).** "Every Settings field is a row in wp_options."

```
wp eval 'foreach ( array( "show_on_front", "posts_per_page", "date_format", "t
```

```
show_on_front               'posts'
posts_per_page              '10'
date_format                 'F j, Y'
timezone_string             ''
gmt_offset                  '0'
default_role                'subscriber'
default_category            '1'
wp_page_for_privacy_policy  '3'
```

`'10'` with quotes: the strict-types trap of Lesson 1.3, from the other side.

9 **Block markup is text (2m).** "What the editor stores in post_content."

```
wp post get 1 --field=post_content
```

```
<!-- wp:paragraph -->
<p>Welcome to WordPress. This is your first post. Edit or delete it, then sta
<!-- /wp:paragraph -->
```

Then the lesson's `blocks-demo.php` (saved in `wordpress/`), which parses and renders three blocks:

```
wp eval-file blocks-demo.php
rm blocks-demo.php
```

```
core/heading       attrs={"level":3} inner=0
core/list          attrs=[] inner=1
core/latest-posts  attrs={"postsToShow":1} inner=0


<h3 class="wp-block-heading">Opening hours</h3>



<ul class="wp-block-list">
<li>Thursday: 9 a.m. to 9 p.m.</li>
</ul>


<ul class="wp-block-latest-posts__list wp-block-latest-posts is-layout-flow wp
</ul>
```

The heading's level is in the comment, not only the HTML; the dynamic Latest Posts block has no saved HTML, and PHP rendered a link that was never in the markup. On the front end that is the `the_content` filter at work (Lesson 2.3).

10 **Two silent mistakes (2m).** "No PHP error in either: WordPress does exactly what you asked." Mistake 1, a page created as a draft:

```
wp post create --post_type=page --post_title="Gift Cards" --porcelain
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/gift-cards/
wp post get 9 --fields=post_status,post_name
```

```
9
404
+-------------+-------+
| Field       | Value |
+-------------+-------+
| post_status | draft |
| post_name   |       |
+-------------+-------+
```

A draft has no slug and no URL: 404. Mistake 2, a slug that is taken:

```
wp post create --post_type=page --post_title="Sample Page" --post_status=publi
wp post list --post_type=page --orderby=ID --order=ASC --fields=ID,post_title,
wp post delete 10
wp post list --post_type=page --post_status=trash --fields=ID,post_name
```

```
10
+----+----------------+----------------+
| ID | post_title     | post_name      |
+----+----------------+----------------+
| 2  | Sample Page    | sample-page    |
| 3  | Privacy Policy | privacy-policy |
| 10 | Sample Page    | sample-page-2  |
+----+----------------+----------------+
Success: Trashed post 10.
+----+------------------------+
| ID | post_name              |
+----+------------------------+
| 10 | sample-page-2__trashed |
+----+------------------------+
```

`sample-page-2`, then `__trashed` in the trash. Delete both for good (`wp post delete 9 --force`, `wp post delete 10 --force`).

BEFORE THE PROJECT: THE NEXT-ID CHECK (EVERYONE)

One read-only query: the ID the next post will get. It must print 12.

```
wp db query "SET SESSION information_schema_stats_expiry = 0; SELECT AUTO_INCREMEN
```

```
next_id
12
```

- **12:** start Step 1.
- **Less than 12** (a learner skipped demo steps: a site that ran none of them says 5): run the catch-up line in the WP-CLI container shell (`docker compose run --rm wpcli sh`), with as many numbers in the list as IDs are missing (seven from 5), then the check again. It creates drafts and deletes each at once, so only the IDs are used up:

```
for i in 1 2 3 4 5 6 7; do wp post delete $(wp post create --post_title=filler --p
```

```
Success: Deleted post 5.
Success: Deleted post 6.
Success: Deleted post 7.
Success: Deleted post 8.
Success: Deleted post 9.
Success: Deleted post 10.
Success: Deleted post 11.
(the check again) next_id
12
```

More than 12: the learner experimented (fine): their IDs are higher throughout, and later lessons that name an ID need their own number. Note it on a sticky note on their laptop lid.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **"My page is not there."** A draft (the default of `wp post create` and `wp_insert_post()`) or a PHP list that only shows published content. Coach: "Which status? Which default does this tool have?" (Mistake 1, self-check Q8.)
- **Checking a role.** `in_array( 'editor', $user->roles )` for "may edit others' posts" locks out the administrator. Coach: "Which capability does the feature need?" (Lesson Mistake 3, self-check Q9, the debug handout.)
- **Copying block markup by hand and changing it.** A missing `wp-block-heading` class: fine on the front end, "invalid block" in the editor. Coach: "Copy it from the Code editor or the lesson, never retype the HTML." (The lesson's warning box; in the project, learners copy the files from the page.)

GUIDED LAB

File: `labs/module-02/l2_1-content-model/content_census_starter.sh`; your reference is `labs/module-02/l2_1-content-model/content_census_solution.sh`. Each TODO is one WP-CLI command inside `$( )` that counts or names part of the content model (posts by status, pages, terms, an option, a role's capabilities, the blocks of a post). It only reads, so it is safe right before the project, and its counts are those of the site Session 1 left (the demo deleted everything it created).

- **Run it as shipped (2m).** `cp labs/module-02/l2_1-content-model/content_census_starter.sh wordpress/` then `docker compose run --rm wpcli sh content_census_starter.sh`: the post-type count works, the rest say TODO.
- **TODOs 1 to 7 in pairs (10m).** Try each command alone first. Circulate for `--format=count` (WP-CLI counts for you), TODO 2 with a `--post_status` (any status means leaving it out), and TODO 7's pipe (`--field=post_content`, then grep).
- **Expected when done** (the course site before the project):

```
== Pages & Co. content census ==
Post types: 16
Published posts: 1
Pages, any status: 2
Draft pages: 1
Categories: Uncategorized
Home page shows: posts
Editor capabilities: 34
Blocks in post 1:
      1 <!-- wp:paragraph
```

- **Debrief (3m):** "Two pages, one of them a draft: which?" (the Privacy Policy WordPress made for you to finish). "34 capabilities for an editor: why do roles exist at all?" (So nobody hands out 34 permissions one by one; code still checks one capability at a time.)
- **Fast finishers:** `labs/module-02/l2_1-content-model/post_report_starter.php` (one row of wp_posts seen through the whole model: the type object, the author's capability, terms, revisions, blocks, the address, for five IDs including the navigation menu), then `labs/module-02/l2_1-content-model/debug_page_report.php` (two loud bugs and a silent one about what posts and pages have). Both only read; run them before the project too, or accept different counts after it.

START THE LESSON PROJECT

The lesson project is **Stock the Pages & Co. Site**: two staff accounts, three pages and a News category with three posts, created with WP-CLI from files of block markup. It is the content every later lesson starts from: names, slugs, dates and IDs exactly as the lesson prints them. In the room: Steps 1 to 3 (35 minutes).

- Step 1 (8m): `wp user create maria ...` and `wp user create devon ...` exactly as in the lesson (the course passwords `maria-pass`, `devon-pass` matter: later lessons log in as them). Maria is user 2, Devon user 3.

- Step 2 (12m): `mkdir wordpress/pagesco-content`, the three page files COPIED from the lesson page into it (typing block markup invites invalid blocks), then the three `wp post create ... --user=maria` lines. They must print 12, 13 and 14; open http://localhost:8080/about/ and watch the header menu list the new pages.
- Step 3 (12m): `wp term create category News --slug=news ...` (prints 2), the three post files, the three `wp post create ... --post_date=...` lines (15, 16, 17), and http://localhost:8080/category/news/ lists them newest first.
- **Done in the room** = pages 12 to 14 and posts 15 to 17 exist, by the right authors. Check two machines with the last command of the lesson's Step 4 (`wp post list --post_type=page,post --orderby=ID --order=ASC ...`).
- **Goes home:** Step 4: Maria's edit of About Us (revision 18), Devon refused on the Pages screen, `rm -r pagesco-content`, the final list. Session 3 starts from exactly that site.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo, the lab and the project:

- Where are a page, an uploaded image and an old version of a post stored? **Answer:** all three in wp_posts, with post_type page, attachment and revision.
- `wp post list` shows the Privacy Policy; `get_posts()` does not. Why? **Answer:** different defaults: WP-CLI lists every status, `get_posts()` only published content.
- Code must let "people who can edit everyone's posts" see a tool. What does it check? **Answer:** `current_user_can( 'edit_others_posts' )`, never the role.
- Where is "Blog pages show at most" stored? **Answer:** the option `posts_per_page` (a string, '10').

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** D) `wp_navigation`; **Q8** `1 1`: one published page for `get_posts()` (it defaults to publish), and one draft page counted by `wp_count_posts()` (the Privacy Policy); **Q9** A) false: `edit_post` is a meta capability that maps to `edit_others_posts` here, which authors lack; **Q10** `core/separator`; **Q11** B) Two: the newest is identical to the current post; **Q12** C) `home`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: in the project, pair a fast typist with a learner who struggles; the second reads each command aloud before the first runs it, and both check the printed ID against the lesson. A learner who falls behind in the demo copies its commands from the lesson page: the IDs only need the commands to RUN, in order.

Stretch: `post_report_starter.php` and `debug_page_report.php` (before the project); after it, the lesson's Exercise 1 (who may do what) and Exercise 2 (the block outline of Staff picks); the Bonus Challenge (a child page of Events, the revisions screen, the wp_navigation post) at home.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note and leave it on the way out. Sort the notes before Session 3: question 1's pile tells you how much of Session 3's warm-up goes back to escaping.

1. Which escaping function for text between tags, for an attribute value, and for an href? **Answer:** `esc_html()`, `esc_attr()`, `esc_url()` (and `wp_kses_post()` for HTML a post may contain).
2. Why does every global name in your theme and plugin start with `pagesco_`? **Answer:** WordPress, the theme and every plugin share one PHP process (and one options table): two functions with the same name are a fatal error, and you cannot know which plugins the client installs next year.
3. Posts, pages, uploaded images, revisions and block-theme menus: where are they stored, and what tells them apart? **Answer:** all are rows of wp_posts; the post_type column (post, page, attachment, revision, wp_navigation).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 1.3: Steps 2 to 4 (render three cards, the break test, PHPCS with the WordPress standard until it says "no coding standards problems", then `rm -r pagesco-practice`; keep a copy of book-card.php in your notes). 2.1: Step 4 (Maria's edit and revision 18, Devon refused, `rm -r pagesco-content`) and the final list, which must match the lesson: users 2 and 3, pages 12 to 14, posts 15 to 17.
- **Lesson exercises.** 1.3: Exercises 1 to 3 (PHPCS on a sale badge, one link in three contexts, a sale option for the price); Exercise 4 (list helpers) is a stretch. 2.1: Exercises 1 to 3 (who may do what, the block outline, a static front page, put back afterwards); Exercise 4 (deleting a user safely) is a stretch. Each exercise undoes itself.
- **Both Quick Quizzes** on the lesson pages (five questions each; answers in the Answer Key).
- **Journal:** both entries (1.3: habits you keep and habits you change; 2.1: your `pagesco-plan.txt` sorted into the content model).
- **Before the Next Lesson** (both lists): pagesco-practice/ deleted and mu-plugins/ holding only mailpit-smtp.php; the final list of 2.1 checked; no user sam, Settings > Reading back on "Your latest posts"; log in as Maria once.
- **Optional:** `labs/module-02/l1_3-php-the-wordpress-way/event_line_starter.php`, `labs/module-02/l1_3-php-the-wordpress-way/debug_book_badge.php`, `labs/module-02/l2_1-content-model/post_report_starter.php` and `labs/module-02/l2_1-content-model/debug_page_report.php` for anyone who did not reach them (the two 2.1 files print other counts once the content exists: their README says which).

BRING LIST FOR SESSION 3

- The course site at the end of Lesson 2.1 (Session 3's two lessons start from it): users maria (2) and devon (3), pages About Us, Contact and Events (12 to 14), the News posts 15 to 17, revision 18 of About Us, `pagesco-`

`content/` deleted. Check: `wp post list --post_type=page,post --orderby=ID --order=asc --fields=ID,post_type,post_name,post_author`.

- Your copy of book-card.php, PHPCS-clean (Session 3's warm-up asks), and `pagesco-plan.txt` sorted into the content model.
- The project folder with `labs/`; VS Code; the journal and the workbook. Session 3 reads the database with SQL (`wp db query`, `wp db cli`): your MySQL Foundations notes help.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the 2.1 project Steps 1 to 3 with the right IDs; the ID-using steps of the 2.1 demo; the 1.3 PHPCS and escaping steps; the 1.3 guided lab.
- **Cut first (saves 6m):** the 1.3 es_ES and callbacks steps (the lesson has both outputs).
- **Cut second (saves 15m):** the 1.3 project start: show book-card.php on the projector, send Steps 1 to 4 home, give the time to the 2.1 project.
- **Cut third (saves 15m):** the 2.1 guided lab: learners do it at home, after the project, so their counts are those of the stocked site (the README lists them). Never cut the 2.1 demo's ID-using steps: run them without discussion instead.

RUNNING SHORT: WHAT TO ADD

- Lesson 1.3's Exercise 1 live: PHPCS on the sale badge, `phpcbf` for the mechanical fixes, the rest by hand. Ask which problems a tool cannot fix, and why.
- Lesson 2.1's Exercise 1 together after Step 3 (who may do what: admin, maria, devon), predicted first on the board.
- `debug_book_badge.php` for everyone, one bug per volunteer, starting from the messages and ending with the silent one.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 3: Core Architecture, the Database and Hooks

How to teach the session where WordPress opens up: the files and the load order, the twelve tables and their key-value pattern read with SQL from the course site, then the hooks system that every theme and plugin is built on, ending with the first file of the course that stays, the Pages & Co. store notice as a must-use plugin with hooks of its own.

2 lessons | 3 hrs session | Lessons 2.2-2.3 | Session 3 of 18 · Part 1

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's ideas) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Type, do not paste (the long grep of wp-settings.php is the exception). Every command below was run to produce the output printed under it, on WordPress 7.1.3, PHP 8.5 and MySQL 9.7, on the site exactly as Lesson 2.1 leaves it: the users maria and devon, the pages About Us, Contact and Events, three News posts. Ask for a prediction before every run.
- The lessons are the reference. Keep `lesson_05.html` and `lesson_06.html` open in browser tabs (the table diagram, the action table, the full stack trace of Mistake 2). Labs live in `labs/module-03/`: `_starter` files are handouts, `_solution` files and the `title_parts_solution/` folder are yours.
- Session 3 is the hinge of Part 1. Lesson 2.2 is read-only: the site ends exactly as it started. Lesson 2.3's project, `mu-plugins/pagesco-site.php` with the store notice, is the first file of the course that STAYS: Session 4 and every lesson after it assume it. Protect its start in the room.
- Lesson 2.2 is SQL-heavy. Learners came through MySQL Foundations, so the joins are review; the new ideas are the schema (no foreign keys, the key-value pattern, serialized arrays, autoload) and the rule that follows from them: change WordPress data through WordPress.

THE #1 RISK OF SESSION 3: THE DATABASE HOUR EATS THE HOOKS HOUR

Lesson 2.2 is the longest lesson so far (140 minutes for a learner alone) and full of queries learners enjoy running; every one invites "what if I also ...". Lesson 2.3 is shorter but it is the lesson the rest of the course stands on, and its project must exist on every machine before Session 4. **Hold the line at the break (1:15):** the 2.2 demo is 20 minutes and has a cut list (below), the inventory starts with two SQL steps and its PHP half goes home. After the break, the 2.3 project start gets 30 minutes, the longest segment of the day: learners who leave with `pagesco-site.php` printing the notice are ready for Session 4; Steps 3 and 4 can be finished at home in 20 minutes.

BEFORE THE SESSION

- ☐ **Learners' sites:** Session 2's homework finished Lesson 2.1's project. Remind them two days before: `wp user list` shows maria and devon, `wp post list --post_type=page` shows About Us, Contact and Events. A learner who skipped it can build the state in 15 minutes at the start of the break with Lesson 2.1's project steps; until then the 2.2 SQL shows fewer rows (fine) and the labs find no Maria (less fine).
- ☐ **Files to hand out:** `labs/module-03/` without the `*_solution.*` files and the `title_parts_solution/` folder; the workbook.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-03` from this kit's folder (every line PASS; 14 results in about a minute). With your own course project at the end of Lesson 2.1: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-03` (labs/README.md).
- ☐ **Projector machine:** the course site at the end of Lesson 2.1 (the instructor's site follows the lessons too), VS Code open on `pagesco-dev` at a large font with `wordpress/wp-content/mu-plugins/` expanded, two terminals in the project folder (commands; `tail -f wordpress/wp-content/debug.log`), a third for `wp db cli`, a browser with http://localhost:8080/hello-world/ and View Source, plus `lesson_05.html` and `lesson_06.html`.
- ☐ **Board, before learners arrive:** the load order as a column (database · mu-plugins · plugins · pluggable · theme · user · init · wp() · template); the five table groups (content, taxonomies, users, comments, settings) with the ID arrows; and a two-column box **action = a moment, does something · filter = a value, returns it**.
- ☐ **A safety net for Mistake 3:** the demo skips the typo'd callback (it takes the site and WP-CLI down). If a learner's own typo does it in the lab, the fix is `rm` of the file in `wordpress/wp-content/mu-plugins/` from the host terminal, not WP-CLI (which is down too).

KNOWN TRAPS IN THIS SESSION

SYMPTOM	CAUSE	FIX
`wp db cli` greets "Welcome to the MariaDB monitor" and	The WP-CLI image's client program is MariaDB's; the server is MySQL 9.7	Nothing: every query works the same. Leave with `exit`.

SYMPTOM	CAUSE	FIX
the prompt says `MySQL [wordpress]>`		
The lesson's counts differ from a learner's (more revisions, `_edit_lock` rows)	The learner opened posts in the editor after Lesson 2.1	Expected; the queries are the answer, not the numbers (the lesson says so).
A query at the prompt prints nothing and waits	The semicolon is missing; the prompt shows `->`	Type `;` and Enter.
`wp db query` output is tab-separated, no table	wp db query never draws tables; `wp db cli` does	Use the prompt for reading, `wp db query` in scripts.
`wp eval-file query-news.php`: `File not found`	Saved in the project folder instead of `wordpress/`	WP-CLI's current folder is the WordPress root: `wordpress/query-news.php`.
A must-use plugin edit takes down every page, the admin AND WP-CLI	A fatal error in mu-plugins/ (Lesson 2.3, Mistake 3); WordPress cannot pause a must-use plugin	From the host: `rm wordpress/wp-content/mu-plugins/<file>`, read the end of debug.log, fix, copy back.
The notice does not appear; no message	The file is in `plugins/` instead of `mu-plugins/`, or named `.php.txt`, or the hook name is misspelled	`wp plugin list --status=must-use`; `wp eval 'var_dump( has_action( "wp_body_open", "pagesco_render_store_notice" ) );'`.
`curl ... | grep -c` exits with an error in a script	grep -c exits 1 when it counts zero	Expected (the lesson's Step 4 says so).

SESSION 3 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 2, and the site check: users 2 and 3, pages 12 to 14, posts 15 to 17.
0:10	10	**2.2** Frame and mental model	Opening the shop in the morning; twelve tables in five groups; no foreign keys; the key-value pattern.
0:20	20	**2.2** Live demo	wp-config.php and wp-settings.php read live, the tables and a join, the SQL WP_Query writes, meta sorted as text, serialized data broken, what autoload saves.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:40	20	**2.2** Guided lab	`labs/module-03/l2_2-core-database/meta_pivot_starter.sh` in pairs: key-value rows back into columns, in SQL and with WP-CLI.
1:00	15	**2.2** Start the lesson project	The database inventory, Steps 1 and 2 at the wp db cli prompt; Steps 3 and 4 go home.
1:15	10	Break	Write the load order on the board for after the break: mu-plugins, plugins, pluggable, theme, user, init.
1:25	10	**2.3** Frame and mental model	The radio show with call-ins; actions and filters; the prefix rule; must-use plugins.
1:35	25	**2.3** Live demo	A first hook, the order of a request, filters in core and yours, priorities and accepted arguments, removing a hook, asking the site, three mistakes.
2:00	20	**2.3** Guided lab	`labs/module-03/l2_3-hooks/filter_chain_starter.php`: three callbacks on one filter, in the right order.
2:20	30	**2.3** Start the lesson project	The store notice in mu-plugins/pagesco-site.php, Steps 1 to 3 (the file stays for the rest of the course); Step 4 if time.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, Checkpoint 1 announced for Session 4, bring list.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 140 + 120 = 260 minutes for a learner working alone; after the warm-up, the break and the wrap-up the room has 150. Lesson 2.2 keeps model, demo and lab, and its project starts with the two SQL steps (the prompt is where discussion helps); the PHP half of the inventory and the settings check go home. Lesson 2.3 keeps its full shape and gives its project the longest segment, because the file stays for the rest of the course. Not in the room: Lesson 2.2's drop-ins section and constant table, the indexes and EXPLAIN (the workbook's worked example covers EXPLAIN), Mistakes 2 to 4 of 2.2 (orphans, a typed table name, the third argument: self-check Q4 and the debug handout practice two of them), and 2.3's "Finding the Hook You Need" beyond has_filter() (the second 2.3 lab, `hook_finder_starter.sh`).

Goes home (about 2 to 3 hours). The inventory Steps 3 and 4 (the same report from PHP with $wpdb, the settings check, the clean-up); the store notice Step 4 (and Step 3 if it did not fit); each lesson's Hands-on Exercises; both Quick Quizzes; both journal entries; both "Before the Next Lesson" lists; and the review for **Checkpoint 1 in Session 4** (Modules 1 and 2: Lessons 1.1 to 2.3).

Protect, in this order: the 2.3 project start (pagesco-site.php printing the notice on every machine); the 2.3 demo through removing a hook; the 2.2 demo through "every value is a string"; the 2.2 guided lab. **Cut first:** the 2.2

autoload step and the 2.3 Mistake 4 step (4 minutes; both are in the lesson and the handouts), then the 2.2 project start (send both SQL steps home: 15 minutes), then the 2.2 lab's Steps 4 and 5 (5 minutes).

OPENING SEGMENT

0:00 Warm-up: Session 2 Revisited — 10 min

TIMING

6m	Five retrieval questions, no notes (workbook Warm-up box), then answers by show of hands
4m	Site check: two WP-CLI commands on every machine, the same IDs as the projector

RETRIEVAL QUESTIONS AND ANSWERS

1. Which escaping function goes around a URL in an href attribute? **Answer:** `esc_url()` (text between tags: `esc_html()`, other attribute values: `esc_attr()`).
2. Name the four shapes a WordPress callback can take. Which one is hard for anyone else to unhook? **Answer:** A function name, a closure, `array( $object, 'method' )`, a static method (`'Class::method'` or `array( 'Class', 'method' )`). The closure: nobody else has a handle on it (Lesson 2.3 comes back to this).
3. A page and a post: which table holds each, and which column tells them apart? **Answer:** Both are rows in `wp_posts`; `post_type` is `page` or `post` (Lesson 2.1's filing cabinet).
4. Code must decide whether the current user may publish pages. What does it check, and why not the role? **Answer:** `current_user_can( 'publish_pages' )`: a capability. Roles are bundles that sites change; capabilities are what WordPress checks.
5. What does `wp_parse_args( $args, $defaults )` return? **Answer:** One array: the defaults, overridden by whatever the caller passed in `$args`.

SITE CHECK (ON THE PROJECTOR, THEN EVERY MACHINE)

Say: "Today both lessons start from the site Lesson 2.1 left. Check yours is the same." From the project folder:

```
wp post list --post_type=page,post --orderby=ID --order=asc --fields=ID,post_type,
wp user list --orderby=ID --fields=ID,user_login,roles
```

```
+----+-----------+----------------+-------------+
| ID | post_type | post_name      | post_author |
+----+-----------+----------------+-------------+
| 1  | post      | hello-world    | 1           |
| 2  | page      | sample-page    | 1           |
| 3  | page      | privacy-policy | 1           |
| 12 | page      | about          | 2           |
| 13 | page      | contact        | 2           |
| 14 | page      | events         | 2           |
| 15 | post      | welcome        | 2           |
| 16 | post      | staff-picks    | 3           |
| 17 | post      | book-club-dune | 2           |
+----+-----------+----------------+-------------+
+----+------------+---------------+
| ID | user_login | roles         |
+----+------------+---------------+
| 1  | admin      | administrator |
| 2  | maria      | editor        |
| 3  | devon      | author        |
+----+------------+---------------+
```

- **Same IDs:** ready. Higher IDs or extra revisions are fine (the learner experimented).
- **No maria, devon or pages 12 to 14:** Lesson 2.1's project was not finished. The learner follows today with a neighbor and builds the content in the break (Lesson 2.1, project Steps 1 to 4, about 15 minutes); the 2.2 queries then match.
- **Homework check (one minute):** hands up for the 1.3 Book Card Helper passing PHPCS and the 2.1 content in place. Questions from the journals go on a sticky note for the 2.2 frame.

2.2 Core Architecture & the Database

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Find your way around a WordPress install: what lives in wp-admin/, wp-includes/ and wp-content/, and what a drop-in is
- Read wp-config.php and explain its settings, including the table prefix and the security keys
- Trace how wp-settings.php loads WordPress, and say what is available at each stage
- Map the twelve wp_* tables, their keys and indexes, and join them with SQL the way WordPress's own queries do
- Explain the meta (key-value) pattern, serialized values and options autoloading, with their trade-offs
- Query the Pages & Co. site with wp db cli, wp db query and WP-CLI, and choose the API over raw SQL in your code

TIMING IN THE ROOM

10m	Frame: opening the shop in the morning; twelve tables; no foreign keys; the key-value pattern
20m	Live demo: wp-config.php, wp-settings.php, tables, a join, WP_Query's SQL, meta as text, serialized data, autoload
20m	Guided lab: `meta_pivot_starter.sh` in pairs
15m	Start the lesson project: the database inventory, Steps 1 and 2

MENTAL MODEL TO SAY

Draw on the load-order column from the board: "Think of a bookseller opening Pages & Co. in the morning. First the lights and the till: the database connection and the cache. Then the things that are always there, whatever happens: must-use plugins. Then the departments open today: active plugins. Then the window display: the theme. Only when the staff member on duty has signed in, the current user, does the shop open its doors, init, and serve the first customer, the main query. Code that tries to serve a customer while the lights are still off is the 'too early' bug of the next lesson."

Then the database in three sentences: "Twelve tables, the same on every WordPress site, linked by ID columns but with **no foreign keys**: integrity lives in PHP, so you change data through WordPress. Extra fields are **rows, not columns**: post_id, meta_key, meta_value, the key-value pattern, and every value is text. Arrays are stored **serialized**, with their lengths inside, so you never edit them as text." Write the rule under the drawing: **read with SQL if you like; write through the API.**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's own commands on the projector: grep in the WP-CLI container shell, SQL at the `wp db cli` prompt, PHP files in `wordpress/` run with `wp eval-file`. Learners type along on their own site (which is the same one). The site ends the demo exactly as it started.

1. **wp-config.php, read as a developer (3m).** "Lesson 1.2 said this file reads your compose settings. Now read its skeleton." In the WP-CLI container shell (`docker compose run --rm wpcli sh`), the lesson's grep leaves out the secret keys:

```
grep -n "define(\|table_prefix =\|stop editing\|require_once" wp-config.php |
```

```
44:define( 'DB_NAME', getenv_docker('WORDPRESS_DB_NAME', 'wordpress') );
47:define( 'DB_USER', getenv_docker('WORDPRESS_DB_USER', 'example username') )
50:define( 'DB_PASSWORD', getenv_docker('WORDPRESS_DB_PASSWORD', 'example pass
59:define( 'DB_HOST', getenv_docker('WORDPRESS_DB_HOST', 'mysql') );
62:define( 'DB_CHARSET', getenv_docker('WORDPRESS_DB_CHARSET', 'utf8mb4') );
65:define( 'DB_COLLATE', getenv_docker('WORDPRESS_DB_COLLATE', '') );
102:$table_prefix = getenv_docker('WORDPRESS_TABLE_PREFIX', 'wp_');
116:define( 'WP_DEBUG', !!getenv_docker('WORDPRESS_DEBUG', '') );
118:/* Add any custom values between this line and the "stop editing" line. */
131:/* That's all, stop editing! Happy publishing. */
135:    define( 'ABSPATH', __DIR__ . '/' );
139:require_once ABSPATH . 'wp-settings.php';
```

   Point at three lines: `$table_prefix` (a variable, not a constant: changing it on a running site makes WordPress think it is not installed), the "stop editing" comment, and line 139, the hand-over to the loader. Ask: "what happens to a define() pasted after line 139?" (Too late: self-check Q1.)

2. **wp-settings.php, the order of the morning (4m).** "Lesson 1.1 grepped three lines of this file. Here are its milestones." Type the lesson's grep (or paste it from the page: it is long and the output is the point):

```
grep -n "^require_wp_db\|^wp_start_object_cache\|^foreach ( wp_get_mu_plugins\
```

```
136:require_wp_db();
151:wp_start_object_cache();
506:foreach ( wp_get_mu_plugins() as $mu_plugin ) {
548:do_action( 'muplugins_loaded' );
582:foreach ( wp_get_active_and_valid_plugins() as $plugin ) {
612:require ABSPATH . WPINC . '/pluggable.php';
630:do_action( 'plugins_loaded' );
643:do_action( 'sanitize_comment_cookies' );
705:do_action( 'setup_theme' );
741:foreach ( wp_get_active_and_valid_themes() as $theme ) {
757:do_action( 'after_setup_theme' );
766:$GLOBALS['wp']->init();
779:do_action( 'init' );
801:do_action( 'wp_loaded' );
```

Walk it with the shop-opening picture: lights and till (database 136, cache), the things always there (must-use plugins 506), today's departments (plugins 582), then pluggable.php at 612, the window display (theme 741), the staff member signs in (766), doors open (init 779). Ask: "why is pluggable.php after the plugins?" (So a plugin can replace wp_mail() by defining it first.)

3 **Twelve tables (3m).** "Time for SQL." `wp db cli` opens the `wp db cli` prompt (`MySQL [wordpress]>`) with the credentials from wp-config.php (it greets you as the MariaDB monitor: the client is MariaDB's, the server is MySQL 9.7). Type:

```
SELECT TABLE_NAME, ENGINE, TABLE_COLLATION
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE()
ORDER BY TABLE_NAME;
```

```
+-----------------------+--------+-----------------------+
| TABLE_NAME            | ENGINE | TABLE_COLLATION       |
+-----------------------+--------+-----------------------+
| wp_commentmeta        | InnoDB | utf8mb4_unicode_520_ci |
| wp_comments           | InnoDB | utf8mb4_unicode_520_ci |
| wp_links              | InnoDB | utf8mb4_unicode_520_ci |
| wp_options            | InnoDB | utf8mb4_unicode_520_ci |
| wp_postmeta           | InnoDB | utf8mb4_unicode_520_ci |
| wp_posts              | InnoDB | utf8mb4_unicode_520_ci |
| wp_term_relationships | InnoDB | utf8mb4_unicode_520_ci |
| wp_term_taxonomy      | InnoDB | utf8mb4_unicode_520_ci |
| wp_termmeta           | InnoDB | utf8mb4_unicode_520_ci |
| wp_terms              | InnoDB | utf8mb4_unicode_520_ci |
| wp_usermeta           | InnoDB | utf8mb4_unicode_520_ci |
| wp_users              | InnoDB | utf8mb4_unicode_520_ci |
+-----------------------+--------+-----------------------+
```

Draw the five groups beside the drawing from Lesson 2.1: content, taxonomies, users, comments, settings. Then leave the prompt (`exit`) and show the rule for code: the prefix comes from $wpdb.

```
wp eval 'global $wpdb; echo $wpdb->prefix, "\n", $wpdb->posts, "\n", $wpdb->po
```

```
wp_
wp_posts
wp_postmeta
wp_options
```

4 **A join, then the SQL WordPress writes (4m).** Back at the prompt, "who wrote each published post and page?" is the join from MySQL Foundations:

```
SELECT p.ID, p.post_type, p.post_title, u.display_name AS author
FROM wp_posts AS p
JOIN wp_users AS u ON u.ID = p.post_author
WHERE p.post_type IN ('post', 'page') AND p.post_status = 'publish'
ORDER BY p.ID;
```

```
+----+-----------+----------------------------------------+-------------+
| ID | post_type | post_title                             | author      |
+----+-----------+----------------------------------------+-------------+
|  1 | post      | Hello world!                           | admin       |
|  2 | page      | Sample Page                            | admin       |
| 12 | page      | About Us                               | Maria Lopez |
| 13 | page      | Contact                                | Maria Lopez |
| 14 | page      | Events                                 | Maria Lopez |
| 15 | post      | Welcome to the new Pages & Co. website | Maria Lopez |
| 16 | post      | Staff picks for the season             | Devon Park  |
| 17 | post      | Book club: Dune                        | Maria Lopez |
+----+-----------+----------------------------------------+-------------+
```

Then save the lesson's `wordpress/query-news.php` and run it with `wp eval-file query-news.php`: WP_Query shows the SQL it sent.

```php
<?php
$pagesco_query = new WP_Query(
    array(
        'category_name' => 'news',
        'fields'        => 'ids',
    )
);

echo trim( preg_replace( '/\n\s+/', "\n", $pagesco_query->request ) ), "\n\n";
echo 'Found: ', implode( ', ', $pagesco_query->posts ), "\n";
```

```
SELECT SQL_CALC_FOUND_ROWS  wp_posts.ID
FROM wp_posts  LEFT JOIN wp_term_relationships ON (wp_posts.ID = wp_term_relat
WHERE 1=1  AND (
wp_term_relationships.term_taxonomy_id IN (2)
) AND ((wp_posts.post_type = 'post' AND (wp_posts.post_status = 'publish')))
GROUP BY wp_posts.ID
ORDER BY wp_posts.post_date DESC
LIMIT 0, 10

Found: 17, 16, 15
```

Read it like any query: one join instead of three (WordPress looked up the slug first), the defaults nobody asked for (post type, status, newest first, 10 per page), SQL_CALC_FOUND_ROWS for pagination. "Core APIs before SQL: the API knows the schema, the defaults and the caches." `rm query-news.php`.

5 **Every value is a string (3m).** "Give the News posts a reading time, as a plugin might, and sort by it."

```
wp post meta add 15 pagesco_reading_minutes 2
wp post meta add 16 pagesco_reading_minutes 12
wp post meta add 17 pagesco_reading_minutes 4

SELECT post_id, meta_value
FROM wp_postmeta
WHERE meta_key = 'pagesco_reading_minutes'
ORDER BY meta_value;
```

```
Success: Added custom field.
Success: Added custom field.
Success: Added custom field.
+---------+------------+
| post_id | meta_value |
+---------+------------+
|      16 | 12         |
|      15 | 2          |
|      17 | 4          |
+---------+------------+
```

Twelve minutes "before" two: meta_value is LONGTEXT, compared character by character. In SQL a CAST, in WP_Query `'orderby' => 'meta_value_num'`. Keep the three rows until the last step.

6 **Mistake 1: serialized data edited with SQL (2m).** "The shop's details as one array option, and a quick rename with REPLACE()." Ask for a prediction first:

```
wp option add pagesco_demo_shop '{"city":"Springfield","phone":"555-0142"}' --
wp db query "UPDATE wp_options SET option_value = REPLACE(option_value, 'Sprin
wp db query "SELECT option_value FROM wp_options WHERE option_name = 'pagesco_
wp option get pagesco_demo_shop
```

```
Success: Added 'pagesco_demo_shop' option.
Success: Query succeeded. Rows affected: 1
option_value
a:2:{s:4:"city";s:11:"Portland";s:5:"phone";s:8:"555-0142";}

Error: Could not get 'pagesco_demo_shop' option. Does it exist?
```

`s:11:"Portland"` says eleven characters; Portland has eight. unserialize() fails, WordPress returns false, WP-CLI says the option does not exist. In a theme the address just vanishes. Fix: `wp option patch`, `update_option()`, and `wp search-replace` for whole sites (Lesson 8.4). `wp option delete pagesco_demo_shop`.

7 **What autoload saves (1m).** Save the lesson's `options-cost.php` and run it: `$wpdb->num_queries` counts queries.

```php
<?php
global $wpdb;

foreach ( array( 'blogname', 'fresh_site', 'fresh_site', 'pagesco_no_such_opti
    $pagesco_before = $wpdb->num_queries;
    $pagesco_value  = get_option( $pagesco_name );
    printf( "%-24s %-18s queries: %d\n", $pagesco_name, var_export( $pagesco_v
}
```

```
blogname                 'Pages &amp;amp; Co.'  queries: 0
fresh_site               '0'                    queries: 1
fresh_site               '0'                    queries: 0
pagesco_no_such_option   false                  queries: 1
```

Autoloaded: free. Not autoloaded: one query, then cached. Missing: still one query. Then clean up: `rm options-cost.php` and the lesson's three `wp post meta delete` lines.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Typing `wp_posts` in PHP.** It works on the course site and breaks on the first client site with the prefix `pc_`. Coach: "Where does the prefix come from?" `{$wpdb->posts}` in code, wp_posts only at the prompt. (The lesson's Mistake 3; `wpdb_report_starter.php`.)
- **Fixing data with UPDATE or DELETE** because it is quicker. Coach with the two broken demos: "Which PHP would have run if you had used wp post delete?" (wp_delete_post()'s four cleanup lines.) The prompt is for reading. (Self-check Q5.)
- **Comparing meta as numbers in SQL.** `ORDER BY meta_value` and `meta_value > 10` on text. Coach: "What type is meta_value?" (LONGTEXT.) CAST in SQL, `meta_value_num` in WP_Query. Forgetting `true` in get_post_meta() is the PHP version of the same surprise (self-check Q4, `debug_meta_report.php`).

GUIDED LAB

File: `labs/module-03/l2_2-core-database/meta_pivot_starter.sh`; your reference is `labs/module-03/l2_2-core-database/meta_pivot_solution.sh`. It practices the key-value pattern from the reading side: wp_usermeta's rows turned back into one row per user with a column per field (the "pivot" every report on meta needs), then the same through WP-CLI, which reads the meta for you.

- **Run it as shipped first (3m).** `cp labs/module-03/l2_2-core-database/meta_pivot_starter.sh wordpress/` then `docker compose run --rm wpcli sh meta_pivot_starter.sh`. Step 1 prints the users; the rest say TODO. Learners develop each query at the `wp db cli` prompt (tables are easier to read there), then paste it into the script inside `wp db query "..."`.

- **Steps 2 and 3 in pairs (12m).** Step 2 is the lesson's "key condition in ON" join; Step 3 is new: one join, GROUP BY the user, `MAX(CASE WHEN um.meta_key = 'first_name' THEN um.meta_value END)`. Circulate for the key condition in WHERE in Step 3 (it filters to one key and the second column comes out empty), and for single quotes inside the double-quoted `wp db query` argument (fine) versus double quotes (they end the argument).
- **Steps 4 and 5 (3m).** `wp user list` with the two meta fields does the pivot for you; the count per user shows how many rows one user costs.
- **Expected when done** (the site after Lesson 2.1):

```
== Step 1: the users ==
ID  user_login  display_name
1   admin   admin
2   maria   Maria Lopez
3   devon   Devon Park

== Step 2: first names, one join ==
ID  user_login  first_name
1   admin
2   maria   Maria
3   devon   Devon

== Step 3: one row per user, two fields ==
ID  user_login  first_name  last_name
1   admin
2   maria   Maria   Lopez
3   devon   Devon   Park

== Step 4: the same through WordPress ==
+----+------------+------------+-----------+
| ID | user_login | first_name | last_name |
+----+------------+------------+-----------+
| 1  | admin      |            |           |
| 2  | maria      | Maria      | Lopez     |
| 3  | devon      | Devon      | Park      |
+----+------------+------------+-----------+
== Step 5: meta rows per user ==
user_id meta_rows
1   16
2   15
3   16
```

- **Debrief (2m):** "Each extra field in a report is one more CASE, or one more join. What would twelve book fields cost?" (The price of the key-value pattern; Lesson 5.2 stores book fields as post meta anyway, and

5.5 builds a real table for events.) "Why is admin's first name empty, not NULL?" (The row exists with an empty value: WordPress writes it at install.)

- **Fast finishers:** `labs/module-03/l2_2-core-database/wpdb_report_starter.php` ($wpdb's get_var, get_col and get_row with prepare(), each checked against the API), then `labs/module-03/l2_2-core-database/debug_meta_report.php` (two Warnings and a silent wrong answer).

START THE LESSON PROJECT

The lesson project is **A Database Inventory of Pages & Co.**: the same report twice, at the SQL prompt as an auditor would and in PHP with $wpdb as a plugin would, plus a settings check; nothing on the site changes. In the room: Steps 1 and 2 at the prompt (15 minutes).

- Step 1 (5m): content by type and status with GROUP BY, then the same numbers from `wp_count_posts()`. Ask before running: "how many published pages?" (Four: the Sample Page counts.)
- Step 2 (10m): who wrote what and where it is filed, with LEFT JOINs to the term tables and GROUP_CONCAT(). The teaching moment is the condition `tt.taxonomy = 'category'` in ON rather than WHERE: ask a pair to move it to WHERE and watch the pages disappear.
- **Done in the room** = both queries return the lesson's rows and are saved in `pagesco-inventory.sql` in the learner's notes.
- **Goes home:** Step 3 (the PHP report with `$wpdb->prepare()` and `get_the_category()`) and Step 4 (the largest autoloaded options, the clean-up, the final check).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Why can a plugin call `get_option()` at the top of its file, but not `wp_get_current_user()`? **Answer:** the database and the options are ready at line 136; pluggable.php (and the current user) come after the plugins.
- A plugin needs a `featured` flag per post. Column, meta, taxonomy or table? **Answer:** post meta (one value per post; Lesson 2.2 quiz Q1). A taxonomy if it were a shared label to filter by.
- Where does `$wpdb->posts` get its prefix? **Answer:** `$table_prefix` in wp-config.php.
- Why does WP_Query's SQL join only one term table? **Answer:** it looked up the term's ID first and filters on term_taxonomy_id directly.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** C) It is defined too late: wp-settings.php has already loaded WordPress, so the file editors stay; **Q2** `db`: the name of the database service in compose.yaml (on a typical host it is localhost); **Q3** B) Must-use plugins, active plugins, pluggable.php, the theme's functions.php; **Q4**

```
array(1) {
  [0]=>
  string(5) "Devon"
}
```

; **Q5** B) It works, but only because both words have 6 characters: the s:6 length still matches. Change roles through WordPress anyway; **Q6** D) Nobody chose, and WordPress decided not to autoload it, usually because the value is large (over 150,000 bytes). Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: learners rusty on joins get the Step 2 query of the lab written on a card with blanks for the ON clause; pair them with someone who did MySQL Foundations recently. For anyone lost in the tables, the workbook's table-groups diagram is the map: point at it before every query.

Stretch: `wpdb_report_starter.php` and `debug_meta_report.php`; the lesson's Exercise 4 (SAVEQUERIES in a must-use plugin, and why the autoloaded options are missing from it) and the Bonus Challenge (SHORTINIT).

2.3 The Hooks System

85 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain the difference between an action and a filter, and choose the right one for a task
- Attach callbacks with add_action() and add_filter(), using priorities and the accepted-arguments count correctly
- Remove a hook someone else added with remove_action() / remove_filter(), and say why a removal silently fails
- Create your own action and filter hooks with do_action() and apply_filters() so other code can extend yours
- Find the hook you need: in the core source, with has_filter() and $wp_filter, and from the command line with WP-CLI
- Build the Pages & Co. store notice as a must-use plugin that others can customize through its own hooks

TIMING IN THE ROOM

10m	Frame: the radio show with call-ins; actions and filters; must-use plugins; the prefix rule
25m	Live demo: a first hook, the order of a request, filters, priorities and accepted arguments, removing, has_filter(), three mistakes
20m	Guided lab: `filter_chain_starter.php`
30m	Start the lesson project: the store notice, Steps 1 to 3 (Step 4 if time)

MENTAL MODEL TO SAY

"WordPress is a radio host working through a running order. An **action** is the host saying 'we are now at the weather, any callers?': callers may talk, and the show goes on the same way afterwards. A **filter** is the host saying 'here is today's headline, any corrections?': each caller gets the headline, may rewrite it, and hands it to the next caller, and the host reads out whatever comes back from the last one. If a caller keeps the headline and hands back nothing, the host reads out nothing." (That is Mistake 1, the filter that forgets to return: promise it.)

Then the box on the board: **action = a moment, your function does something · filter = a value, your function returns it.** Then where hooks live until Module 5: "a must-use plugin, any PHP file in wp-content/mu-plugins/, loaded on every request without activation, before all plugins. You wrote a throwaway one in Lesson 1.2; today you write one that stays." And the prefix rule in one breath: everything you name, `pagesco_`.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Files in VS Code (`wordpress/wp-content/mu-plugins/` for must-use plugins, `wordpress/` for `wp eval-file` scripts), checks in the WP-CLI container or the browser's View Source. Outputs below come from the site at the end of Lesson 2.1; your debug.log time stamps differ.

1. **Your first hook (3m).** "The question for every WordPress task from today: which hook, and is it an action or a filter?" Save the lesson's file in VS Code as `wordpress/wp-content/mu-plugins/pagesco-hello.php` (must-use plugins: Lesson 2.2's line 506), reload http://localhost:8080/hello-world/, View Source:

```php
<?php
/**
 * Plugin Name: Pages & Co. Hello Hooks
 * Description: Lesson 2.3 demo: prints a line at the bottom of every page.
 */

function pagesco_hello_footer() {
    echo '<p class="pagesco-hello">Hello from a hook!</p>';
}
add_action( 'wp_footer', 'pagesco_hello_footer' );
```

```
<p class="pagesco-hello">Hello from a hook!</p>
```

Two lines do the work: the function says what, `add_action()` says when. The name is a string without parentheses: you are not calling it, you are telling WordPress what to call later. Ask: "why the pagesco_ prefix?" (One shared PHP program: Lesson 1.3.)

2. **The order of one request (3m).** "The load order from before the break, logged by a request." Save the lesson's `pagesco-hook-order.php`, reload the post once, and read the new lines of debug.log (a second terminal: `tail -f wordpress/wp-content/debug.log`):

```php
<?php
/**
 * Plugin Name: Pages & Co. Hook Order
 * Description: Lesson 2.3 demo: logs the order of the main actions of a front
 */

$pagesco_hooks = array( 'plugins_loaded', 'after_setup_theme', 'init', 'wp_loa

foreach ( $pagesco_hooks as $pagesco_hook ) {
    add_action(
        $pagesco_hook,
        function () {
            if ( ! is_admin() && ! wp_doing_cron() && ! defined( 'WP_CLI' ) )
                error_log( 'hook order: ' . current_action() );
            }
        }
    );
}
```

```
[26-Oct-2026 14:08:26 UTC] hook order: plugins_loaded
[26-Oct-2026 14:08:26 UTC] hook order: after_setup_theme
[26-Oct-2026 14:08:26 UTC] hook order: init
[26-Oct-2026 14:08:26 UTC] hook order: wp_loaded
[26-Oct-2026 14:08:26 UTC] hook order: template_redirect
[26-Oct-2026 14:08:26 UTC] hook order: wp_head
[26-Oct-2026 14:08:26 UTC] hook order: wp_footer
[26-Oct-2026 14:08:26 UTC] hook order: shutdown
```

Same closure eight times; `current_action()` names the hook. Point at template_redirect: the query is done, nothing printed yet, the place for redirects (self-check Q8). Delete the file (`rm wp-content/mu-plugins/pagesco-hook-order.php`): a log line per request gets noisy fast.

3 **A filter, from both sides (4m).** "A filter in core: the last line of get_the_title()."

```
grep -n "apply_filters( 'the_title'" wp-includes/post-template.php
```

```
174:    return apply_filters( 'the_title', $post_title, $post_id );
```

Then append the lesson's separator filter to pagesco-hello.php and reload: the `<title>` in View Source.

```php
function pagesco_title_separator( $separator ) {
    return '|';
}
add_filter( 'document_title_separator', 'pagesco_title_separator' );
```

```
<title>Hello world! | Pages &amp;amp; Co.</title>
```

Then the signature on the_content (append it too); the post's content area, as text:

```php
function pagesco_content_signature( $content ) {
    return $content . '<p class="pagesco-signature">Thanks for reading, from t
}
add_filter( 'the_content', 'pagesco_content_signature' );
```

```
Welcome to WordPress. This is your first post. Edit or delete it, then start w
Thanks for reading, from the Pages & Co. team.
```

"A filter takes the value and returns the value." Under the hood add_action() is one line: `return add_filter( ... );`.

4 **Priorities and accepted arguments (4m).** "Who goes first?" Save the lesson's `wordpress/priorities.php`, ask for a prediction, `wp eval-file priorities.php`:

```php
<?php
add_filter( 'pagesco_demo', fn( $text ) => $text . ' [priority 20]', 20 );
add_filter( 'pagesco_demo', fn( $text ) => $text . ' [priority 5]', 5 );
add_filter( 'pagesco_demo', fn( $text ) => $text . ' [default 10, added first]
add_filter( 'pagesco_demo', fn( $text ) => $text . ' [default 10, added second

echo apply_filters( 'pagesco_demo', 'Start:' ), "\n";
```

```
Start: [priority 5] [default 10, added first] [default 10, added second] [pri
```

Lower first; ties in the order added. Then `title-ids.php`: the_title passes two values, and `10, 2` asks for both.

```php
<?php
function pagesco_title_with_id( $title, $post_id ) {
    return "{$title} (#{$post_id})";
}
add_filter( 'the_title', 'pagesco_title_with_id', 10, 2 );

echo get_the_title( 1 ), "\n";
echo get_the_title( 2 ), "\n";
```

```
Hello world! (#1)
Sample Page (#2)
```

`rm priorities.php title-ids.php`.

5 **Removing a hook, and asking the site (4m).** "Every page tells the world your exact version." Append the lesson's one line to pagesco-hello.php and count the generator tags from the container:

```
remove_action( 'wp_head', 'wp_generator' );

curl -s http://localhost:8080/ | grep -c 'name="generator"'
```

```
0
```

(grep -c exits with 1 when it counts zero: expected.) The three rules on the board: **same priority, after it was added, a handle on the callback**. Then ask the running site what is attached:

```
wp eval 'var_dump( has_action( "wp_head", "_wp_render_title_tag" ), has_filter
```

```
int(1)
int(10)
bool(false)
```

A number is a priority, false is nothing attached; 0 is a valid priority, so compare with `false !==`.

6 **Mistake 1: a filter that forgets to return (3m).** Save the lesson's `mistake-no-return.php` in mu-plugins/ and reload /hello-world/: the title is gone. From the container:

```
<?php
add_filter( 'the_title', function ( $title ) {
    $title = strtoupper( $title );
} );

curl -s http://localhost:8080/hello-world/ | head -n 3
curl -s http://localhost:8080/hello-world/ | grep -c 'wp-block-post-title'
```

```
Deprecated: str_replace(): Passing null to parameter #3 ($subject) of type arr
<!DOCTYPE html>
0
```

The only message blames core (formatting.php, line 5777): your closure handed null to the next callback on the_title, core's capital_P_dangit() at 11. It sits above <!DOCTYPE html> because a block theme renders the template before the head. "When a notice blames core, look at what your code handed to it." `rm` the file.

7 **Mistake 2: two arguments asked, one registered (2m).** Save `mistake-args.php` and reload /hello-world/. The page is replaced by the fatal error; read the first line from the left:

```
<?php
add_filter( 'the_title', function ( $title, $post_id ) {
    return "{$title} (#{$post_id})";
} );
```

```
Fatal error: Uncaught ArgumentCountError: Too few arguments to function {closu
```

Closure named after its file and line (PHP 8.4 and later; PHP 8.3 says only `{closure}`), 1 passed, exactly 2 expected. The stack trace below it (in the lesson) is the way back: WP_Hook, apply_filters('the_title'), get_the_title(), the Post Title block. Fix: `10, 2`. `rm` the file.

8 **Mistake 4: removing with the wrong priority (2m).** "The silent one." Save `mistake-remove.php` in the WordPress root and run it:

```
<?php
var_dump( remove_action( 'wp_head', '_wp_render_title_tag' ) );
var_dump( remove_action( 'wp_head', '_wp_render_title_tag', 1 ) );
```

```
bool(false)
bool(true)
```

False, then true: `_wp_render_title_tag` was added at 1. If nobody looks at the return value, the first line "just does not work". `rm mistake-remove.php` and the demo plugin pagesco-hello.php: the project starts clean. (Mistake 3, the typo in a callback name, is in the deck and the lesson: it takes the site and WP-CLI down, which is too much drama in front of the room right before the project.)

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Echoing in a filter** or returning nothing. Coach with the radio host: "What does the host read out if you hand back nothing?" Every path of a filter ends in `return`. (Self-check Q11, `debug_price_tag.php`.)
- **Calling instead of naming:** `add_action( 'wp_footer', pagesco_hello_footer() )`. The function runs immediately, while plugins load, and WordPress gets its return value (null) as the callback. Coach: "Are you calling it now, or telling WordPress what to call later?" Strings, no parentheses.
- **"My removal does nothing."** Wrong priority, too early, or a closure. Coach: `var_dump( remove_action( ... ) )` first, then the three rules on the board. (Self-check Q9, the lab's TODO 3.)

GUIDED LAB

File: `labs/module-03/l2_3-hooks/filter_chain_starter.php`; your reference is `labs/module-03/l2_3-hooks/filter_chain_solution.php`. It practices the three rules of a hook with several callbacks on a filter of the bookstore's own: the order (priorities), the values a callback receives (accepted arguments), and taking someone else's callback off (removal with the right priority), checked with has_filter().

- **Run it as shipped (2m).** `cp labs/module-03/l2_3-hooks/filter_chain_starter.php wordpress/` then `wp eval-file --use-include filter_chain_starter.php`:

```
Removed the sale badge: TODO
pagesco_lab_capitalize: TODO
Line: fahrenheit 451 [SALE]
```

- **TODOs 1 and 2 (8m).** Capitalize at 5, the author at the default priority with `10, 2`. Circulate for the fatal ArgumentCountError (the demo's Mistake 2, now in their own file: read it from the left together) and for `ucwords` at a later priority than the author ("Fahrenheit 451 By Ray Bradbury": ask why By is capitalized).
- **TODOs 3 and 4 (6m).** `remove_filter()` without the 30 returns false and the badge stays: let them find it with var_export(). has_filter() after the removal returns false for the badge.
- **Expected when done:**

```
Removed the sale badge: true
pagesco_lab_capitalize: 5
pagesco_lab_add_author: 10
pagesco_lab_sale_badge: false
Line: Fahrenheit 451 by Ray Bradbury
```

- **Debrief (4m):** "Which line would you change if the theme had to add [SALE] after the author?" (Nothing in the theme: its priority 30 already runs after 10.) "What changes if TODO 1 is hooked at 20 instead of 5?" (ucwords then runs after the author was added and capitalizes it too: Fahrenheit 451 By Ray Bradbury.)
- **Fast finishers:** `labs/module-03/l2_3-hooks/hook_finder_starter.sh` (find hooks in core with exact grep patterns, then ask the site), the folder lab `labs/module-03/l2_3-hooks/title_parts_starter/` (a must-use plugin on document_title_parts; test with `labs/module-03/l2_3-hooks/test_title_parts.sh`), then `labs/module-03/l2_3-hooks/debug_price_tag.php`.

START THE LESSON PROJECT

The lesson project is **The Pages & Co. Store Notice**: `wordpress/wp-content/mu-plugins/pagesco-site.php`, a notice bar printed from wp_body_open that offers its own hooks (a filter for the text, two actions) and is extended from the outside. **This file stays for the rest of the course**: Session 4 and every later lesson start from a site that has it. In the room: Steps 1 to 3 (30 minutes).

- Step 1 (8m): the plugin header and the notice on wp_body_open, escaped with `esc_html()`. Check: View Source, the `<div class="pagesco-notice">` is the first thing inside `<body>`.
- Step 2 (10m): replace the function with the version that offers `pagesco_store_notice`, `pagesco_before_store_notice` and `pagesco_store_notice_end`, each with its DocBlock. Nothing visible changes: "you are building the doors other code will use." Insist on typing the DocBlocks: hooks are a public API.
- Step 3 (10m): be "another developer": the Thursday hours at priority 20, the events link inside the bar, and the generator removal moved in from the demo file.
- **Done in the room** = every page starts with `Free shipping on orders over $35. Open until 9 p.m. on Thursdays. See events`. Check two or three screens yourself; the rest check with `curl -s http://localhost:8080/ | grep -o '<div class="pagesco-notice">.*</div>'` in the container.
- **Goes home:** Step 4 (delete pagesco-hello.php, the has_action and grep checks) and the checklist, including "returning '' hides the bar" (try it, then remove the test). Anyone who did not reach Step 3 finishes it first: Session 4 checks for the full notice.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Banner at the top of every page, or excerpts shortened to 20 words: action or filter? **Answer:** an action (wp_body_open) for the banner, a filter (excerpt_length) for the number.
- What does `add_action()` do with a function name that does not exist? **Answer:** nothing, until the hook fires; then a fatal TypeError. It stores a string.
- Two callbacks at priority 10 on one hook: which runs first? **Answer:** the one added first.
- Why does the project's filter for the Thursday hours use priority 20? **Answer:** to add to whatever earlier callbacks produced; a later one (99) could still change it, which is how a "quiet day" filter switches the bar off (the lesson's Exercise 1).

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** `XAb`; **Q8** C) `template_redirect`; **Q9** A) The plugin ran before the theme added the hook, so there was nothing to remove: remove it on init; **Q10** `int(9)`: do_blocks is attached to the_content at priority 9, before wpautop at 10; **Q11** A) Your callback does not return the title, so the next callback (core's, at priority 11) received null; **Q12** D) `do_action( 'pagesco_before_gift_form' );` with a DocBlock above it. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners who freeze at an empty file get a card with the two-line pattern (`function pagesco_x() { ... }` / `add_action( 'hook', 'pagesco_x' );`) and the box from the board. In the project, pair a learner who struggles with the guided lab with one who finished it; the typing stays with the first.

Stretch: `hook_finder_starter.sh`, the title-parts folder lab and `debug_price_tag.php`; the lesson's Exercise 3 (every callback on the_content, in order) and Exercise 4 (the `all` hook counter); the Bonus Challenge (the body_class filter).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note. Sort the notes before Session 4: question 3's pile tells you how much of the Checkpoint review to give to removing hooks.

1. Why should code change WordPress data through WordPress (functions or WP-CLI) and not with UPDATE or DELETE? **Answer:** there are no foreign keys, so WordPress's PHP does the cascading (meta, terms, revisions, comments); serialized values carry their lengths; and the API keeps caches and hooks in step.
2. Action or filter, and which hook: a banner at the very top of every page; the length of every excerpt. **Answer:** an action on wp_body_open; a filter on excerpt_length (return the number).

3. `remove_action()` returns false and the hook keeps running. Name two possible reasons. **Answer:** a different priority than the one it was added with; called before the hook was added (too early); a different callback (a closure or another object).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** The inventory: Step 3 (the PHP report) and Step 4 (the largest autoloaded options, `rm pagesco-inventory.php`, the final check). The store notice: Step 3 if it did not fit, Step 4 and the checklist. Afterwards `mu-plugins/` holds exactly `mailpit-smtp.php` and `pagesco-site.php`, and every page starts with the full notice.
- **Lesson exercises.** 2.2: Exercises 1 to 3 (the only comment, roles from the database, content without meta); Exercise 4 (SAVEQUERIES) is a stretch. 2.3: Exercises 1 to 3 (the quiet day, the login branding, who is on the_content); Exercise 4 (the `all` counter) is a stretch. Each exercise file is deleted afterwards.
- **Both Quick Quizzes** on the lesson pages (five questions each; answers in the Answer Key).
- **Journal:** both entries (2.2: your MySQL Foundations bookstore schema next to WordPress's tables; 2.3: a website you use, described as hooks).
- **Before the Next Lesson** (both lists): no demo or exercise files left in the WordPress root or mu-plugins/; the orphan query returns no rows; skim wp-includes/default-filters.php; read mailpit-smtp.php again and explain its two hooks.
- **Prepare for Checkpoint 1** (Session 4, 30 minutes, Modules 1 and 2: Lessons 1.1 to 2.3): reread the Key Takeaways of the six lessons and redo the self-checks of Sessions 1 to 3.
- **Optional:** `labs/module-03/l2_2-core-database/wpdb_report_starter.php`, `labs/module-03/l2_2-core-database/debug_meta_report.php`, `labs/module-03/l2_3-hooks/hook_finder_starter.sh`, the title-parts lab and `labs/module-03/l2_3-hooks/debug_price_tag.php` for anyone who did not reach them.

BRING LIST FOR SESSION 4

- The course site at the end of Lesson 2.3 (the state Session 4's lessons start from): `wordpress/wp-content/mu-plugins/pagesco-site.php` in place, the full notice on every page, no generator tag, `mu-plugins/` holding only it and mailpit-smtp.php. Check: `curl -s http://localhost:8080/ | grep -o '<div class="pagesco-notice">.*</div>'` in the WP-CLI container.
- The project folder with `labs/`; VS Code. Session 4 builds the pagesco block theme in `wordpress/wp-content/themes/` (Lessons 3.1 and 3.2) and holds Checkpoint 1.
- Both journals, `pagesco-inventory.sql`, and the workbook.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the 2.3 project start; the 2.3 demo through removing a hook; the 2.2 demo through "every value is a string"; the 2.2 guided lab.
- **Cut first (saves 4m):** the 2.2 autoload step and the 2.3 Mistake 4 step.
- **Cut second (saves 15m):** the 2.2 project start: show the Step 1 query, send Steps 1 to 4 home; give the time to the 2.3 project.
- **Cut third (saves 5m):** the 2.2 lab's Steps 4 and 5 (the README's expected output shows them).

RUNNING SHORT: WHAT TO ADD

- Lesson 2.2's Mistake 2 live (orphans): the DELETE, the LEFT JOIN that finds them, the multi-table DELETE that cleans up. It makes "write through the API" stick.
- Lesson 2.3's Exercise 1 together (the quiet day at priority 99): it uses the hooks the project just built, from the outside.
- `debug_price_tag.php` for everyone, one bug per volunteer.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 4: Theme Anatomy and Building a Block Theme

How to teach Session 4: a ten-minute warm-up and Checkpoint 1 on Modules 1 and 2 (a 30-minute individual must-use plugin task with a five-row rubric), then Twenty Twenty-Five taken apart and the template hierarchy logged on a running site, and the birth of pagesco: the smallest block theme, a theme.json design system and the CSS it generates, a header part, two quiet mistakes, a book-fair theme lab and the start of the pagesco project.

2 lessons | 3 hrs session | Lessons 3.1–3.2 | Checkpoint 1 | Session 4 of 18 · Part 1

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice) → start the lesson's own project. Exercises, quizzes, journals and the rest of each project are homework.
- Session 4 is a **checkpoint session**: the first 40 minutes are a short warm-up and Checkpoint 1 on Modules 1 and 2, so both lessons get less room time than usual. The cuts are planned below; do not let the checkpoint run over.
- Every command below was run to produce the output printed under it, on WordPress 7.1.3, PHP 8.5, MySQL 9.7 and WP-CLI 2.12, on the site exactly as the lessons leave it: the checkpoint and Lesson 3.1 on the site of Lesson 2.3, Lesson 3.2 on the site of Lesson 3.1. Time stamps in debug.log lines differ on every run.
- Keep `lesson_07.html` and `lesson_08.html` open: the hierarchy table, the full theme.json and the five templates are long, and learners paste them from the page in the room. Labs live in `labs/module-04/`: `_starter` files and folders are handouts, `_solution` ones and `checkpoint_solution/` are yours.
- Session 4 is where pagesco is born: learners leave with the theme active (finished at home). Sessions 5 to 7 and Part 1's project build on it.

> **THE #1 RISK OF SESSION 4: TWO LONG LESSONS AFTER A CHECKPOINT**
>
> The lesson pages estimate 120 + 140 = 260 minutes; after the warm-up, the checkpoint, the break and the wrap-up the room has 120. **Before:** in Session 3's wrap-up, tell learners the checkpoint opens Session 4 (30 minutes, open site and closed notes, Modules 1 and 2, not a grade that follows them; Session 3's homework says so too) and that pagesco-site.php must be in mu-plugins/. **In the room:** start the checkpoint at 0:10 sharp, call time at 0:33 and collect at 0:40 whatever state it is in; in 3.2, learners PASTE theme.json and the templates from the lesson page and you spend the time on reading them. **After:** mark with the rubric; anyone under 5 gets a short conversation before Session 5.

BEFORE THE SESSION

- ☐ **In Session 3's wrap-up:** announce Checkpoint 1 and its rules; ask learners to finish Lesson 2.3's project (pagesco-site.php in mu-plugins/), because the checkpoint builds on its hooks.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-04` from this kit's folder (15 results, about a minute and a half). With your own project: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-04` (labs/README.md; your site must be at the state each file names).
- ☐ **Projector machine:** the course site at the end of Lesson 2.3 (Twenty Twenty-Five active, pagesco-site.php in mu-plugins/), two terminals in the project folder at 18 pt or larger (one for `tail -f wordpress/wp-content/debug.log`), VS Code open on `wordpress/wp-content/themes/`, the browser with http://localhost:8080 and `lesson_07.html` / `lesson_08.html` in tabs. Delete any pagesco folder from a rehearsal first: learners watch you create it.
- ☐ **Files to hand out:** the checkpoint starter folder `labs/module-04/checkpoint-1/checkpoint_starter/` with `test_checkpoint.sh` (on every laptop before 0:10), the `labs/module-04/` starters, the workbook (its Checkpoint 1 page holds the task and the rubric). Keep `checkpoint_solution/` and the `*_solution*` files back.
- ☐ **Board, before learners arrive:** the three answers of wp_is_block_theme() / wp_theme_has_theme_json() (block true/true, hybrid false/true, classic false/false); a request arrow → "candidate list, most specific first" → "first one the theme has" → index; and a box **theme.json = style guide · templates = floor plans · parts = signs**.
- ☐ **Print:** five copies of the rubric for your own marking, sticky notes for the exit ticket.

KNOWN TRAPS IN THIS SESSION

SYMPTOM	CAUSE	FIX
`Error: Template is missing. Standalone themes need to have a templates/index.html ...`	A theme folder with only style.css (or index.html outside templates/)	Create `templates/index.html` (Lesson 3.1, Mistake 1).
A Notice `wp_json_file_decode(): ... Syntax error` above `<!DOCTYPE html>`; the design is plain	A trailing comma, a single quote or a comment in theme.json	The `$schema` line in VS Code underlines it; or `php -r` with `json_last_error_msg()` (Lesson 3.2).
The header or footer is missing, no message	A template-part slug that matches no file in parts/	Look logged in: "Template part has been deleted or is unavailable: heder".
A file change has no effect	A Site Editor save made a wp_template copy that wins	`wp post list --post_type=wp_template,wp_template_part`; Reset in the Site Editor or `wp post delete ID --force`.

SYMPTOM	CAUSE	FIX
The store notice vanished on a classic test theme	No `wp_body_open()` after `<body>`	Lesson 3.1, Mistake 3: every classic theme calls wp_head(), wp_body_open(), wp_footer().
`Success: Switched to 'Pages & Co.' theme.`	Not a bug: theme names are stored escaped, like the site title	Say it once; it comes back in every theme switch.
Checkpoint: the notice text is gone	The filter echoes instead of returning (or returns nothing)	Do not fix it during the checkpoint: rubric row 1 measures it.
The book-fair or café lab left the site on the wrong theme	The lab switched themes	`wp theme activate pagesco` (or twentytwentyfive before 3.2), then `wp theme delete pagesco-fair`.

SESSION 4 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Three retrieval questions on Session 3 (a filter's priority, the must-use plugins, the generator tag), and a 30-second check: Twenty Twenty-Five active, pagesco-site.php in mu-plugins/.
0:10	30	Checkpoint 1 (Modules 1–2)	A 23-minute individual task, open site and closed notes: `pagesco-checkpoint.php`, a must-use plugin on Lesson 2.3's own hooks, then a self-check against the test and hand-in. Marked with the five-row rubric after the session.
0:40	10	**3.1** Frame and mental model	The shop fit-out; block, classic and hybrid themes; the information desk (most specific first, index last).
0:50	15	**3.1** Live demo	Twenty Twenty-Five taken apart (kind, files, theme.json outline, templates, functions.php), the template-map logger on a post and a category, the home.html mistake.
1:05	15	**3.1** Guided lab	`labs/module-04/l3_1-theme-anatomy/which_template_starter.php`: predict, then let get_template_hierarchy() and get_block_templates() check you.
1:20	10	Break	Leave the request → hierarchy → template drawing on the board.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:30	10	**3.1** Start the lesson project	Step 1: map every kind of page with the logger and write the template map into pagesco-plan.txt. Steps 2 to 5 (the sketch theme) go home.
1:40	10	**3.2** Frame and mental model	Style guide, floor plans, signs; the file list from the template map; settings vs styles.
1:50	20	**3.2** Live demo	Type along: the smallest block theme, activated; theme.json and the CSS it generates; the header part and the real index; a trailing comma and a part slug typo caught.
2:10	20	**3.2** Guided lab	`labs/module-04/l3_2-block-theme/book_fair_starter/`: a small block theme of your own (a palette color, styles, a Query Loop, a footer part), tested with test_book_fair.sh.
2:30	20	**3.2** Start the lesson project	Step 1 (the full theme.json) and Step 2 (the footer part) of the pagesco project; Steps 3 and 4 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 5.
Total	180		Ends at 3:00.

In the room. The lessons estimate 120 + 140 = 260 minutes for a learner working alone; after the 10-minute warm-up, the 30-minute checkpoint, the break and the wrap-up the room has 120: 50 for 3.1, 70 for 3.2. Lesson 3.1 keeps its model, its demo of Twenty Twenty-Five and the hierarchy log, and its project's Step 1 (the template map); 3.2 keeps the build up to the finished index and Steps 1 and 2 of the project. Long JSON and markup are pasted from the lesson page in the room; reading them is the work.

What does not fit, and goes home (about 2 to 3 hours). 3.1: the anatomy table and style.css header (reading), Common Mistakes 1 and 3 (style.css only, the classic theme without wp_body_open) and project Steps 2 to 5 (the sketch theme turned classic, hybrid, block, then deleted). 3.2: Common Mistake 3 (the Site Editor copy), project Steps 3 and 4 (the five other templates and the per-URL verification). Both lessons' exercises, Quick Quizzes and journals; the labs not reached.

Protect, in this order: the checkpoint's 30 minutes (start on time, stop on time); the 3.2 build up to an active pagesco with theme.json (Session 5 assumes it, and learners finish faster at home once it works); the hierarchy log in 3.1; the template map. **Cut first:** the 3.1 functions.php step and the 3.2 part-slug mistake (both in the lessons), then the 3.1 guided lab (send which_template home), then the 3.2 guided lab (the book-fair theme goes home; go straight to the project start).

OPENING SEGMENT

0:00 Warm-up

10 min

SEQUENCE

- **Retrieval (6m).** Put W1 to W3 on the slide; learners write answers in the workbook, then run each command on their own site. Take answers from three different people; the answer key has the reasons.
- **State check (4m).** "Twenty Twenty-Five active, two must-use plugins: hands up when W2 matches." Anyone whose pagesco-site.php is missing saves it now from Lesson 2.3's project on the lesson page (the checkpoint builds on its hooks), and anyone whose theme is not Twenty Twenty-Five runs `wp theme activate twentytwentyfive`.

ANSWERS

- **W1** `20`: The priority: `has_filter()` returns the priority of that callback on that hook, or false. pagesco-site.php adds the late-opening text at 20, so it runs after a callback at the default 10.
- **W2** `mailpit-smtp` and `pagesco-site`: mailpit-smtp (Lesson 1.2) and pagesco-site (Lesson 2.3). Every PHP file in wp-content/mu-plugins/ runs on every request, before the normal plugins (the load order of Lesson 2.2).
- **W3** `0`: pagesco-site.php calls `remove_action( 'wp_head', 'wp_generator' )`, so the generator meta tag is never printed (and grep -c with no match exits 1).

0:10 Checkpoint 1 (Modules 1–2) 30 min

TIMING

2m	Brief: the workbook's Checkpoint 1 page; the starter is already in `wordpress/wp-content/mu-plugins/` and the test in `wordpress/`
23m	Work alone; open site, closed notes (their own course site, `wp help`, the starter's comments; no lesson pages, notes, neighbors or AI assistants)
5m	Self-check: run the test, compare with the workbook's "done" line, write one line "what I would fix with ten more minutes", hand in (file on a USB stick or the shared folder, or a photo of the test output)

WHAT IT CHECKS

One small must-use plugin that uses every Module 1 and 2 skill at once: callbacks hooked by name with priorities, a filter that returns (2.3), Lesson 2.3's own custom hooks, WordPress functions that take an arguments array (1.3), the content model (posts in a category, a user and her author archive, 2.1), a prepared `$wpdb` query on the posts table (2.2), escaping at output and the `pagesco_` prefix (1.3). Nothing from Module 3 is tested.

Say: "A readiness check for you and for me, not a grade that follows you. Work alone for 23 minutes. Partial work is fine. If (a) fights you, do (b) and (c) and come back."

The task as learners see it:

- **Start:** `cp labs/module-04/checkpoint-1/checkpoint_starter/pagesco-checkpoint.php wordpress/wp-content/mu-plugins/` and `cp labs/module-04/checkpoint-1/test_checkpoint.sh wordpress/`, then `docker compose run --rm wpcli sh test_checkpoint.sh` after each part.
- **(a) Latest news.** `pagesco_checkpoint_latest_news()` on the filter `pagesco_store_notice` (your hook from Lesson 2.3), priority 30: add " Latest news: <title>." with the title of the newest post in the category News (slug `news`), found with `get_posts()` and an arguments array. Return the text.
- **(b) Ask Maria.** `pagesco_checkpoint_ask_link()` on the action `pagesco_store_notice_end`, priority 20: print " · " and a link to the author archive of the user with the login `maria`, with the text "Ask " and her display name. Escape the URL and the text.
- **(c) Page count.** `pagesco_checkpoint_page_count()` on `wp_footer`: `<p class="pagesco-pages">Published pages: N</p>`, N counted with ONE prepared `$wpdb` query on the posts table.
- **Rules:** 23 minutes, alone, open site, closed notes (your own course site, WP-CLI with `wp help`, the starter's comments; no lesson pages, notes, neighbors or AI assistants). Partial work is fine. Done = the

test shows the newest post, the link and the count, with no message in the page or in debug.log.

◆ RUBRIC (2 / 1 / 0 PER ROW, 10 POINTS)

CRITERION	2 · MEETS	1 · PARTLY	0 · NOT YET
Hooks, names and priorities	`add_filter( 'pagesco_store_notice', ..., 30 )` and `add_action( 'pagesco_store_notice_end', ..., 20 )` with the callbacks named as strings; the filter RETURNS the text; the order in the bar is late opening, latest news, events link, Ask Maria.	Works, but a priority is off (Ask Maria before See events) or the filter is attached with add_action.	The filter echoes instead of returning (the notice text disappears), or a hook name is misspelled.
WordPress functions with arguments	`get_posts()` with an arguments array (`category_name` news, `numberposts` 1); `get_user_by( 'login', 'maria' )`; `get_author_posts_url()`; each result checked before use.	Right answer by a detour (all posts fetched and filtered in PHP, the URL typed by hand, the title or name hard-coded).	The newest post or Maria's link is wrong or missing.
Escaping at output	`esc_url()` on the link and `esc_html()` on its text; the filter returns PLAIN text, because the notice escapes it (Lesson 2.3).	One part unescaped, or the filter's text escaped a second time.	User data printed with no escaping at all.
A prepared query	ONE `$wpdb->get_var( $wpdb->prepare( ... ) )` on `{$wpdb->posts}` with `%s` for page and publish; the count printed as a number.	Correct count, but the values are written into the SQL string, or the table name is typed as wp_posts.	No query (a number typed in, or the count of the wrong post type).
Clean, standard code	The test prints the three expected lines and NO message in the page or in debug.log; every function has the pagesco_ prefix and a DocBlock; WordPress spacing and tabs.	One message, or the prefix or DocBlocks missing on some functions.	The site shows a warning or a critical error, or the work is unfinished.

Reading the scores. 8 to 10: ready for Module 3's pace. 5 to 7: name the row that cost the points and the lesson to reread (row 1 → 2.3, row 2 → 1.3 and 2.1, row 3 → 1.3, row 4 → 2.2). 0 to 4: a five-minute conversation before Session 5, and the checkpoint redone at home, this time with the lesson pages open.

WATCH-OUTS DURING THE CHECKPOINT

- **Do not help with the code.** Answer only questions about the task wording. A learner whose site is broken (a fatal error in the mu-plugin takes WP-CLI down too, Lesson 2.3) may `mv` the file out of mu-plugins/ to read the error: that is debugging, and allowed.
- **The filter that echoes.** The notice text vanishes. Do not point it out: rubric row 1 measures it.
- **Priority 20 forgotten in (b).** "Ask Maria" lands before "See events": mu-plugins load alphabetically, so pagesco-checkpoint.php adds its callback before pagesco-site.php does. A good debrief question for Session 5's warm-up.
- **Collect at 0:40 sharp,** whatever the state. Model answer and its test output are in the answer key.

3.1 Theme Anatomy & the Template Hierarchy

50 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what belongs in a theme and what belongs in a plugin, and where WordPress records which theme is active
- Tell block, classic and hybrid themes apart by their files, and confirm it with wp_is_block_theme() and wp_theme_has_theme_json()
- Find your way around a block theme: style.css, theme.json, templates/, parts/, patterns/, styles/ and functions.php in Twenty Twenty-Five
- Predict which template WordPress uses for a URL, for block themes and classic themes, from the template hierarchy
- Watch WordPress make that choice on a running site with the *_template_hierarchy filters and template_include

TIMING IN THE ROOM

10m	Frame: the shop fit-out; block, classic and hybrid; the information desk
15m	Live demo: Twenty Twenty-Five taken apart, the hierarchy logged, the home.html mistake
15m	Guided lab: `which_template_starter.php`
10m	Start the lesson project (after the break): Step 1, the template map

MENTAL MODEL TO SAY

"Pages & Co. moves into a newly decorated shop. The books (content) and the till system (plugins) move across unchanged; the shelving, the signs, the paint (the theme) are new. A good fit-out can be replaced next year without losing a single book. That is the test for where code belongs: would we keep this after a redesign? Then it is a plugin."

Then the three kinds, on the board: **block** = templates/*.html + theme.json (true, true); **classic** = PHP templates (false, false); **hybrid** = classic plus a theme.json (false, true; not an official name). "You will meet all three on client sites for years."

Then the information desk: "A customer asks for Dune. The bookseller checks the most specific place first: a display for this book? a shelf for Frank Herbert? the science fiction section? Every shop has general shelves: in WordPress that is index, and every theme must have it." Write **most specific first, the first one the theme has wins, index last.**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

On the projector, from the project folder (`wp` commands) and in the WP-CLI container shell (`ls`, `grep`, `curl`). The site is the end of Lesson 2.3. Ask for a prediction before every run.

1. **Which theme, which kind? (2m).** "Every page you have seen so far was drawn by a theme you did not write. Which one, and what kind?" From the project folder:

```
wp theme list
wp eval 'var_dump( wp_is_block_theme(), wp_theme_has_theme_json() );'
```

```
+-------------------+----------+--------+---------+----------------+---------
| name              | status   | update | version | update_version | auto_upda
+-------------------+----------+--------+---------+----------------+---------
| twentytwentyfive  | active   | none   | 1.5     |                | off
| twentytwentyfour  | inactive | none   | 1.6     |                | off
| twentytwentythree | inactive | none   | 1.7     |                | off
+-------------------+----------+--------+---------+----------------+---------
bool(true)
bool(true)
```

   Two options remember the active theme (`template` and `stylesheet`, the rows learners met in Lesson 2.2); they differ only for a child theme (Lesson 3.4). true, true: a block theme with a theme.json. Write the three answers on the board: block = true/true, hybrid = false/true, classic = false/false.

2. **Read a theme the WordPress team maintains (4m).** "The best way to learn block themes is to read one." In the WP-CLI container shell, list the folder, then ask for theme.json's outline:

```
ls -1 wp-content/themes/twentytwentyfive
wp eval '$json = wp_json_file_decode( get_theme_file_path( "theme.json" ), arr
```

```
assets
contributing.txt
functions.php
package-lock.json
package.json
parts
patterns
readme.txt
screenshot.png
style.css
style.min.css
styles
templates
theme.json
$schema, version, settings, styles, templateParts, customTemplates
appearanceTools, color, layout, spacing, typography, useRootPaddingAwareAlignm
color, spacing, typography, blocks, elements
```

Point at the four that matter today: `style.css` (the ID card), `theme.json` (settings = what editors may use, styles = what is applied), `templates/` and `parts/`. patterns/ and styles/ are Lesson 3.3, functions.php Lesson 4.3.

3 **Templates are a list WordPress keeps (3m).** "Every template has an ID theme//slug and a source."

```
wp eval 'foreach ( get_block_templates() as $t ) { echo $t->id, "  ", $t->sour
wp post list --post_type=wp_template,wp_template_part --fields=ID,post_type,po
```

```
twentytwentyfive//404  theme  Page: 404
twentytwentyfive//archive  theme  All Archives
twentytwentyfive//search  theme  Search Results
twentytwentyfive//single  theme  Single Posts
twentytwentyfive//page  theme  Pages
twentytwentyfive//index  theme  Index
twentytwentyfive//home  theme  Blog Home
twentytwentyfive//page-no-title  theme  Page No Title
+----+-----------+-----------+
| ID | post_type | post_name |
+----+-----------+-----------+
+----+-----------+-----------+
```

Eight templates, all `theme` (read from files). The empty list is the important one: a Site Editor save would add a row there, and that copy wins over the file (Lesson 3.2's Mistake 3).

4 **functions.php is hooks (1m).** "What does a block theme still need PHP for?"

```
grep -n "add_action\|add_filter" wp-content/themes/twentytwentyfive/functions.
```

```
24:add_action( 'after_setup_theme', 'twentytwentyfive_post_format_setup' );
38:add_action( 'after_setup_theme', 'twentytwentyfive_editor_style' );
65:add_action( 'wp_enqueue_scripts', 'twentytwentyfive_enqueue_styles' );
93:add_action( 'init', 'twentytwentyfive_block_styles' );
122:add_action( 'init', 'twentytwentyfive_pattern_categories' );
142:add_action( 'init', 'twentytwentyfive_register_block_bindings' );
```

Lesson 2.3's hooks, in someone else's code: theme supports on `after_setup_theme`, a stylesheet on `wp_enqueue_scripts`, block styles on `init`. Every function sits in `function_exists()` so a child theme can replace it.

5 **Watch WordPress choose (4m).** "Now the hierarchy, live." Save the lesson's logger as `wordpress/wp-content/mu-plugins/pagesco-template-map.php` (paste it from the lesson page: it is a tool, not today's skill), keep `tail -f wordpress/wp-content/debug.log` in the second terminal, and open a post, then the News category:

```
[26-Oct-2026 12:31:37 UTC] looked for: single-post-hello-world.php, single-pos
[26-Oct-2026 12:31:37 UTC] file: wp-includes/template-canvas.php
[26-Oct-2026 12:31:37 UTC] block template: twentytwentyfive//single
```

```
[26-Oct-2026 12:31:37 UTC] looked for: category-news.php, category-2.php, cate
[26-Oct-2026 12:31:37 UTC] looked for: archive.php
[26-Oct-2026 12:31:37 UTC] file: wp-includes/template-canvas.php
[26-Oct-2026 12:31:37 UTC] block template: twentytwentyfive//archive
```

Read the second one aloud: the category list found nothing, so template-loader.php asked its next question (is it an archive?) and archive.html answered. The file included is core's `template-canvas.php`: the block template is rendered first, then core prints the page around it, wp_body_open included. That is why the store notice needs no theme code.

6 **Mistake: home.html for the home page (1m).** "The shop wants About Us as its front page. A developer writes home.html. Watch."

```
wp option update show_on_front page
wp option update page_on_front 12
```

```
Success: Updated 'show_on_front' option.
Success: Updated 'page_on_front' option.
```

Reload http://localhost:8080/ and read the log:

```
[26-Oct-2026 12:31:38 UTC] looked for: front-page.php
[26-Oct-2026 12:31:38 UTC] looked for: page-about.php, page-12.php, page.php
[26-Oct-2026 12:31:38 UTC] file: wp-includes/template-canvas.php
[26-Oct-2026 12:31:38 UTC] block template: twentytwentyfive//page
```

No `home` in the list at all: with a static front page the front page is a page. front-page = "the front page, whatever it shows"; home = "the list of latest posts". Put it back (and leave the logger in place for the project start after the break):

```
wp option update show_on_front posts
wp option update page_on_front 0
```

```
Success: Updated 'show_on_front' option.
Success: Updated 'page_on_front' option.
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **"home means the home page."** Coach with the log from the demo and one question: "What does the front page show today, posts or a page?" home is the list of latest posts, wherever it lives. (Self-check Q5, lesson quiz Q3.)
- **Titles instead of slugs, or the type missing.** `page-About Us`, `single-hello-world`. Coach: "Where would WordPress get that name from?" The slug, and posts include their post type. (Self-check Q1.)
- **Editing Twenty Twenty-Five to try something.** Coach: "Who updates that folder?" Read it, never edit it; experiments go in a throwaway theme of their own, as in the project's Steps 2 to 5.

GUIDED LAB

File: `labs/module-04/l3_1-theme-anatomy/which_template_starter.php`; your reference is `labs/module-04/l3_1-theme-anatomy/which_template_solution.php`. It practices both halves of the hierarchy on the learner's site: the candidate list (`get_template_hierarchy()`, done for them) and the first template Twenty Twenty-Five has (`get_block_templates()`, their TODOs), with two rows they add themselves.

- **Predict first (3m).** Learners write "=> ?" for each URL on paper before touching code. `cp labs/module-04/l3_1-theme-anatomy/which_template_starter.php wordpress/` then `wp eval-file --use-include which_template_starter.php`: the candidate lists print, every answer says TODO.
- **TODOs 1 to 3 (9m).** `wp_list_pluck()` for the slugs, a `foreach` with `in_array()` and `break` for the first match, two new rows. Circulate for `in_array()` without the third argument (works, but say why `true` is the habit) and for a missing `break` (the LAST match wins: `index`).

- **Expected when done:**

```
/tag/staff-picks/   tag-staff-picks > tag > archive > index
                    => archive
/author/devon/      author-devon > author > archive > index
                    => archive
/hello-world/       single-post-hello-world > single-post > single > singular > i
                    => single
/?s=dune            search > index
                    => search
/                   front-page > home > index
                    => home
/contact/           page-contact > page > singular > index
                    => page
```

- **Debrief (3m):** "Why does `/` answer home, and what will it answer once pagesco is active?" (pagesco has no home template, so index: the template map's "index doubles as home".) "Which real names does this function leave out?" (The ID-based ones: author-3, page-13.)
- **Fast finishers:** `labs/module-04/l3_1-theme-anatomy/theme_inspector_starter.sh` (anatomy of all three default themes in one table), then `labs/module-04/l3_1-theme-anatomy/debug_theme_report.php` (two loud bugs and a silent one).

START THE LESSON PROJECT

The lesson project is **Map the Templates of Pages & Co.**: the template map (Step 1), then a throwaway theme that goes classic, hybrid, block (Steps 2 to 5). After the break, Step 1 only (10 minutes); the logger from the demo is still in mu-plugins/.

- Step 1: the lesson's one-line `for url in ...` loop in the WP-CLI container, then read the new debug.log lines together: which template draws each URL?
- Learners write the TEMPLATE MAP into `pagesco-plan.txt`, with the third column "pagesco will need". Ask: "How many templates does pagesco need?" (Six: index, single, page, archive, search, 404; index doubles as home.)
- **Done in the room** = the map with eight rows and the six-template conclusion. **Goes home:** Steps 2 to 5, which end by deleting the sketch theme AND the logger (`rm wp-content/mu-plugins/pagesco-template-map.php`). Anyone who will not do the steps at home deletes the logger now: Lesson 3.2 expects only the two must-use plugins.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- What makes a theme a block theme? **Answer:** a `templates/index.html` (wp_is_block_theme() is true); theme.json alone makes it hybrid.

- Which file does WordPress include for a block theme's page? **Answer:** core's `wp-includes/template-canvas.php`, which renders the block template and calls wp_head, wp_body_open and wp_footer.
- The News category gets its own design. Which file? **Answer:** `templates/category-news.html` (slug before ID); every other category still falls through to archive.
- A Site Editor save on a template: where does it go? **Answer:** a `wp_template` post with source custom, which wins over the file.

Workbook self-check, items Q1 to Q6 (learners do them at home).

Answers: **Q1** B) `page-about, page-12, page, singular, index`; **Q2** `author-devon, author, archive, index`; **Q3** D) It disappears, with no message anywhere; **Q4** A) So a child theme can define a function of the same name first and replace it; **Q5** C) front-page.html; **Q6** `category_template_hierarchy`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: pair learners who struggled in the checkpoint; for the lab, give them TODO 2's loop shape on the board (`foreach ( ... as $c ) { if ( in_array( $c, $have, true ) ) { $used = $c; break; } }`).

Stretch: the theme inspector and the debug handout; then the lesson's Exercise 4 (a custom page template assigned with `_wp_page_template`) and the Bonus Challenge (`register_block_template()`: who wins, theme or plugin?).

3.2 Building a Block Theme

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Write a theme header in style.css that WordPress reads and checks on activation
- Define a design system in theme.json: settings (palette, fonts, sizes, spacing, layout) and styles (global, elements, blocks), and read the CSS WordPress generates from it
- Write templates and template parts in block markup, combining static blocks with dynamic ones such as the Query Loop, Post Title and Post Content
- Activate and verify a block theme with WP-CLI and the Site Editor, with site features such as the store notice still working
- Diagnose an invalid theme.json, a missing template part and a Site Editor copy that overrides your file

TIMING IN THE ROOM

10m	Frame: style guide, floor plans, signs; the file list from the template map
20m	Live demo, typed along: the smallest theme, theme.json and its CSS, the header and index, two quiet mistakes
20m	Guided lab: the book-fair theme (`book_fair_starter/`)
20m	Start the lesson project: Step 1 (the full theme.json) and Step 2 (the footer part)

MENTAL MODEL TO SAY

"Designing a shop interior, you write a style guide first (the colors, the typefaces, the spacing), then floor plans for each room, then the signs you hang at every entrance. A block theme has the same three layers: **theme.json is the style guide**, **templates are the floor plans**, **template parts are the signs**. Change the style guide and every room follows."

Then the two halves of theme.json, side by side on the board: **settings = what is available** (the palette, font sizes, spacing steps, widths, which controls editors see); **styles = what is applied** (the page, elements such as links and headings, single block types), usually by pointing at a preset. And the file list from the template map: style.css, theme.json, two parts, six templates, no functions.php.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Learners type the short files and paste the long ones from `lesson_08.html`, on their own site; the outputs below come from the site at the end of Lesson 3.1. The theme they create is the real pagesco: it stays.

1. **The smallest block theme (4m).** "Lesson 3.1 said two files. Let us prove it." Learners type along: create `wordpress/wp-content/themes/pagesco/style.css` (the lesson's header) and the first

`templates/index.html`, then switch:

```
/*
Theme Name: Pages & Co.
Theme URI: https://example.com/pagesco/
Author: Pages & Co.
Author URI: https://example.com/
Description: The block theme of the Pages & Co. independent bookstore.
Requires at least: 6.6
Tested up to: 7.1
Requires PHP: 7.4
Version: 1.0.0
License: GNU General Public License v2 or later
License URI: http://www.gnu.org/licenses/gpl-2.0.html
Text Domain: pagesco
*/

<!-- wp:site-title {"level":0} /-->

<!-- wp:query {"queryId":1,"query":{"inherit":true}} -->
<div class="wp-block-query">
    <!-- wp:post-template -->
        <!-- wp:post-title {"isLink":true,"level":2} /-->
    <!-- /wp:post-template -->
</div>
<!-- /wp:query -->
```

```
wp theme activate pagesco
wp eval 'var_dump( wp_is_block_theme(), wp_theme_has_theme_json() );'
```

```
Success: Switched to 'Pages &amp;amp; Co.' theme.
bool(true)
bool(false)
```

Open http://localhost:8080/: ugly but correct. The store notice is the first thing in the body, from a theme that has never heard of it:

```
<div class="pagesco-notice">Free shipping on orders over $35. Open until 9 p.m
```

2 **theme.json, and the CSS it becomes (6m).** "Now the style guide." Learners paste the lesson's first theme.json (typing 50 lines of JSON is not the skill; reading it is). Walk through settings, then styles, then ask WordPress what it made of them:

```
curl -s http://localhost:8080/ | grep -o -- '--wp--preset--color--[a-z]*: #[0-
curl -s http://localhost:8080/ | grep -o 'body{[^}]*}' | head -n 1
```

```
--wp--preset--color--black: #000000
--wp--preset--color--white: #ffffff
--wp--preset--color--paper: #fbf8f1
--wp--preset--color--ink: #1f2a2e
--wp--preset--color--oxblood: #7a2e2e
--wp--preset--color--sage: #e3eadf
--wp--preset--color--slate: #56636a
body{background-color: var(--wp--preset--color--paper);color: var(--wp--preset
```

Each preset became a custom property (black and white stay even with `defaultPalette` false); `var:preset|color|paper` became `var(--wp--preset--color--paper)`. The rule to say twice: **templates use presets, never hex values**, so a style variation can change sage once.

3 **The header part and the real index (5m).** "Floor plans and signs." Paste the lesson's `parts/header.html` and the finished `templates/index.html`; read the header from the outside in (full-width sage band, wide flex row, Site Title, Navigation), then check the landmarks:

```
curl -s http://localhost:8080/ | grep -o '<header[^>]*>\|<main[^>]*>\|<h1[^>]*
```

```
<header class="wp-block-template-part">
<main id="wp--skip-link--target" class="wp-block-group is-layout-constrained w
<h1 class="wp-block-heading">News from the shop
<h2 class="wp-block-post-title"><a href="http://localhost:8080/hello-world/" t
```

`tagName` made the `<header>` and the `<main>`; WordPress added the layout classes and the skip-link id. Static blocks (Group, Heading) carry their HTML; dynamic ones (Site Title, Navigation, Query Loop) are self-closing.

4 **Mistake: a trailing comma (3m).** "JSON is stricter than PHP arrays." The lesson's sed adds one comma:

```
sed -i 's/"wideSize": "1180px"/"wideSize": "1180px",/' wp-content/themes/pages
curl -s http://localhost:8080/ | head -n 3
curl -s http://localhost:8080/ | grep -c -- '--wp--preset--color--oxblood'
```

```
Notice: wp_json_file_decode(): Error when decoding a JSON file at path /var/ww
<!DOCTYPE html>
0
```

The Notice prints above `<!DOCTYPE html>` (block themes render before the head, Lesson 2.3) and names the file, not the line; zero oxblood variables: the whole theme.json is ignored. On a live site there is no message at all. Remove the comma (`sed -i 's/"wideSize": "1180px",/"wideSize": "1180px"/' wp-content/themes/pagesco/theme.json`) and the count is back to 1.

5 **Mistake: a part slug typo (2m).** "A page template with `heder`." Save the lesson's three-line page.html, count site titles logged out, then look logged in:

```
<!-- wp:template-part {"slug":"heder","tagName":"header"} /-->

<!-- wp:post-title {"level":1} /-->
<!-- wp:post-content /-->

curl -s http://localhost:8080/about/ | grep -c 'wp-block-site-title'
```

```
0
```

Logged out: nothing, no message, nothing in debug.log. Logged in as admin, the first lines of the page:

```
Template part has been deleted or is unavailable: heder
About Us
...
```

Delete the test template (`rm wp-content/themes/pagesco/templates/page.html`): the real one comes in the project. Mistake 3 (a Site Editor copy that wins over the file) is reading at home; one sentence now: "If a file change does nothing, `wp post list --post_type=wp_template` first."

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Hex values in templates.** `"style":{"color":{"background":"#e3eadf"}}` copied from somewhere. Coach: "What happens when the shop rebrands?" Presets: `"backgroundColor":"sage"`. (Self-check Q8, the journal prompt.)
- **Hand-edited static markup that the editor calls invalid.** Coach: "Build it in the Site Editor, open the Code editor, copy the markup" (the lesson's tip), and never save there for real in this course: a save makes a copy.
- **"My change does nothing."** Three quiet causes, in this order: a JSON syntax error (debug.log), a slug that matches no file (look logged in), a Site Editor copy (`wp post list --post_type=wp_template,wp_template_part`). (Self-checks Q10 and Q11, lesson quiz Q5.)

GUIDED LAB

Folder: `labs/module-04/l3_2-block-theme/book_fair_starter/`, a small block theme `pagesco-fair` for the shop's book-fair weekend; your reference is `labs/module-04/l3_2-block-`

`theme/book_fair_solution/` and the test `labs/module-04/l3_2-block-theme/test_book_fair.sh`. It practices the lesson's three layers on a theme of the learner's own, so pagesco stays untouched: a palette color (settings), a styles section with presets, a Query Loop in the template and a footer part.

- **Install and run as shipped (4m).** `wp theme list --status=active --field=name` first (pagesco, after the demo). Then `cp -r labs/module-04/l3_2-block-theme/book_fair_starter wordpress/wp-content/themes/pagesco-fair`, `wp theme activate pagesco-fair`, `cp labs/module-04/l3_2-block-theme/test_book_fair.sh wordpress/`, `docker compose run --rm wpcli sh test_book_fair.sh`. The TODOs are in the theme's style.css.
- **TODOs 1 to 4 (13m), running the test after each.** Circulate for a comma after the last palette entry (the Notice from the demo, live), hex values in styles, and a footer saved as `parts/footer.html.html` or in templates/. TODO 1 also fixes the header: its `lantern` background class had no CSS until the preset existed. Ask why.
- **As shipped the test prints:**

```
Success: Switched to 'Pages &amp;amp; Co. Book Fair' theme.
active theme: pagesco-fair
palette:
--wp--preset--color--night: #1b1f3b
--wp--preset--color--cream: #fff8e7
lantern: (missing)
page colors: (none: theme.json has no styles.color)
posts listed: 0
header parts: 1
footer parts: 0
footer text: (none)
```

When done:

```
Success: Switched to 'Pages &amp;amp; Co. Book Fair' theme.
active theme: pagesco-fair
palette:
--wp--preset--color--night: #1b1f3b
--wp--preset--color--cream: #fff8e7
--wp--preset--color--lantern: #f2b134
page colors: body{background-color: var(--wp--preset--color--cream);color: var(--
posts listed: 4
header parts: 1
footer parts: 1
footer text: Book fair weekend · 12 Market Street
```

- **Debrief and clean up (3m):** "Which line of the test proves the styles section uses the preset and not the hex value?" (`var(--wp--preset--color--cream)`.) Then `wp theme activate pagesco` and

`wp theme delete pagesco-fair`: the project needs pagesco active.

- **Fast finishers:** `labs/module-04/l3_2-block-theme/theme_json_reader_starter.php` (settings and styles of Twenty Twenty-Five, read in PHP), then the café handout `labs/module-04/l3_2-block-theme/debug_cafe_theme/` with `labs/module-04/l3_2-block-theme/test_debug_cafe_theme.sh`.

START THE LESSON PROJECT

The lesson project is **Finish the pagesco Theme**: the full design system, the footer, the five other templates and a check of every URL. In the room: Steps 1 and 2 (20 minutes); pagesco must be active again first.

- Step 1: paste the full theme.json over the first one; read what it adds (appearanceTools, root padding, elements, blocks, the css string for the store notice, templateParts), then the lesson's grep for the link rule and the notice CSS. Stop on `a:where(:not(.wp-element-button))`: `:where()` adds no specificity, so content can override the theme.
- Step 2: `parts/footer.html`; reload the home page and read the footer text.
- **Done in the room** = a paper-colored site with the ink notice bar, sans headings and the dark footer. **Goes home:** Step 3 (single, page, archive, search, 404) and Step 4 (the per-URL loop: every page 200 with one notice, header, footer and h1; 404 for /nothing-here/).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo, the lab and the project start:

- settings or styles: the five shop colors? The page background? **Answer:** settings (available); styles (applied, pointing at paper).
- What does `"inherit":true` on the Query Loop mean? **Answer:** show what the main query found, so one block serves the blog home, every archive and the search page.
- Why does the store notice appear in a theme that never mentions it? **Answer:** template-canvas.php fires wp_body_open for block themes, and pagesco-site.php hooks it.
- A teammate's theme.json change has no effect and no message. First move? **Answer:** debug.log (a JSON syntax error is a Notice there), then the saved copies.

Workbook self-check, items Q7 to Q12 (learners do them at home).

Answers: **Q7** `1180px`; **Q8** C) `font-size: var(--wp--preset--font-size--large)`; **Q9** D) Post Title; **Q10** A) The site in core's default design: the theme's colors, fonts and sizes are gone, with no message; **Q11** `parts/footer.html`; it wraps the part in a `<footer>` element (the contentinfo landmark); **Q12** B) Hides WordPress's own palette from the editors, so they choose from the theme's colors. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners who fell behind in the demo copy the lesson's files in order (style.css, index.html, theme.json, header.html, index.html) and run the activate command: the build is the same, the reading happens at home with the lesson. Skip the book-fair lab for them and go straight to the project start.

Stretch: the theme.json reader and the café handout; the lesson's Exercise 4 (style every Quote block from theme.json) and the Bonus Challenge (the Navigation overlay colors, with contrast checked).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the questions on the slide; learners answer in the workbook's Exit Ticket box or on a sticky note. Question 2's pile tells you whether Session 5's warm-up goes back to settings and styles.

1. A block theme has index.html, page.html and archive.html. Which draws /author/maria/, and why? **Answer:** archive.html: author-maria, author-2 and author do not exist, archive is the next candidate.
2. In theme.json, where does the shop's palette go, and where the page's background color? **Answer:** settings.color.palette (what is available); styles.color.background as `var:preset|color|paper` (what is applied).
3. Your header vanished and there is no message. Name two quiet causes. **Answer:** a template-part slug that matches no file; a Site Editor copy of the template or part; (a theme.json syntax error loses the styles, not the header).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 3.1: Steps 2 to 5 (the sketch theme classic → hybrid → block, then Twenty Twenty-Five back and the sketch theme and the logger deleted; read Common Mistakes 1 and 3 first). 3.2: Step 3 (the five other templates) and Step 4 (the per-URL verification loop), then the completion checklist: pagesco active, six templates with source theme.
- **Lesson exercises.** 3.1: Exercises 1 to 3 (predict then ask core, the palette, style variations); Exercise 4 is a stretch. 3.2: Exercises 1 to 3 (a sixth color, a News template, the requirements check), each undone afterwards; Exercise 4 is a stretch.
- **Reading:** 3.2 Common Mistake 3 (the Site Editor copy) and the FAQ; 3.1's anatomy table.
- **Both Quick Quizzes** and **both journal entries** (the 3.2 prompt: which files would a rebrand touch?).
- **Checkpoint 1:** anyone who did not finish redoes it at home with the lesson pages open and runs the test; under 5 points: bring questions to Session 5.
- **Optional:** the labs not reached: `labs/module-04/l3_1-theme-anatomy/theme_inspector_starter.sh`, `labs/module-04/l3_1-theme-`

`anatomy/debug_theme_report.php`, `labs/module-04/l3_2-block-theme/theme_json_reader_starter.php`, the café handout.

BRING LIST FOR SESSION 5

- The site at the end of Lesson 3.2: **pagesco active**, the six templates and two parts, `wp post list --post_type=wp_template,wp_template_part` empty, mu-plugins/ = mailpit-smtp.php + pagesco-site.php.
- `pagesco-plan.txt` with the template map checked off, the journal, the workbook.
- One thing you would like the site to look different on a phone (Lesson 3.2, Before the Next Lesson): Session 5 adds patterns, a style variation and a child theme.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the checkpoint's 30 minutes; an active pagesco with theme.json and the header part; the hierarchy log; the template map.
- **Cut first (saves 5m):** the 3.1 functions.php step and the 3.2 part-slug mistake.
- **Cut second (saves 15m):** the 3.1 guided lab: show the solution's output, send which_template home.
- **Cut third (saves 20m):** the book-fair lab: go straight to the project start; the lab goes home.

RUNNING SHORT: WHAT TO ADD

- 3.1 Mistake 3 live: the bare classic theme without wp_body_open(), the notice gone (then delete the theme).
- 3.2 Exercise 2 together: `templates/category-news.html` with a welcome line, proved on the News archive and not on Maria's.
- Open the Site Editor on the projector (Appearance > Editor > Templates), show the six templates with live data, and close without saving: "a save makes a copy".

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 5: Patterns, Style Variations and Child Themes

How to teach the second half of Module 3. Give the Pages & Co. theme ready-made sections: pattern files that stay translatable, a front page built from them, a synced pattern shared by two pages, a section style, and the Site Editor round trip that keeps files and database copies apart. Then customize a theme you did not write: a child of Twenty Twenty-Five, the merge rules of theme.json, a stylesheet that must be enqueued, and functions.php's load order, with a Twenty Twenty-Four child as the lab.

`2 lessons` `3 hrs session` `Lessons 3.3–3.4` `Session 5 of 18 · Part 1`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's ideas) → start the lesson's own project. Exercises, the Quick Quiz and the journal are homework.
- Type, do not paste. Every command below was run to produce the output printed under it, on WordPress 7.1.3 and PHP 8.5: for 3.3 a course site exactly as Lesson 3.2 leaves it (pagesco active, no patterns yet), for 3.4 the site at the end of Lesson 3.3. Ask for a prediction before every run. IDs (the synced pattern's 20, the template copy's 24) can differ on learners' sites; say so when they appear.
- The lessons are the reference. Keep `lesson_09.html` and `lesson_10.html` open for the long file listings (hero.php, the about/contact markup, theme.json): paste-free typing of those is not the goal, reading them is. Labs live in `labs/module-05/` (learners keep the folder in their project folder): `_starter` files and folders are handouts, `_solution` ones are yours.
- Session 5 is Module 3's second half: the theme becomes something editors use (patterns, styles) and you learn to customize themes you did not write (child themes). It ends with pagesco active again, as Session 6 expects.

> **THE #1 RISK OF SESSION 5: THREE COPIES OF "THE DESIGN"**
>
> After today the design of Pages & Co. lives in theme files, in synced patterns, in user global styles and in Site Editor template copies. Learners who click around in the Site Editor at home end the week with database copies that make every later output differ, and they do not know it. **Before:** the warm-up's second command counts the copies; anyone above 0 resets them. **In the room:** spend the full six minutes on the Site Editor round trip and write the loop on the board. **After:** "Before the Next Lesson" in 3.3 asks for the list of template, template-part and global-styles posts to be empty; make it the first line of the homework.

BEFORE THE SESSION

- ☐ **Learner sites:** at the end of Lesson 3.2 (pagesco active with the ten files of the handoff, no Site Editor copies). Anyone who did not finish Session 4's project copies the lesson's files from `lesson_08.html` before class; the warm-up check shows who.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-05` from this kit's folder (17 results in about two minutes; every line PASS). On your own course project: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-05` (labs/README.md; your site must be at the states the module README lists).
- ☐ **Projector machine:** a course site at the end of Lesson 3.2 for the 3.3 demo (and ready to continue into 3.4: the demo's commands build on each other), VS Code on the theme folder at a large font, two terminals in the project folder (one with `docker compose run --rm wpcli sh` open), the browser on http://localhost:8080/ and on Appearance > Editor, `lesson_09.html` and `lesson_10.html` in tabs.
- ☐ **Files to hand out:** `labs/module-05/` without the `*_solution*` files and folders; the workbook. The pagesco-content/ files of the synced-pattern demo are on the lesson page (learners copy them from there).
- ☐ **Board, before learners arrive:** the three-tools table (theme pattern: a file, copied when inserted; synced pattern: a wp_block post, one original; template part: parts/, included by slug); the Site Editor loop (list copies → find the change → right file → delete the copy); and an empty inheritance table for 3.4 (style.css, theme.json, templates, patterns, styles/, functions.php) to fill during the demo.
- ☐ **Print or share:** the workbook; sticky notes for the exit ticket.

KNOWN TRAPS TODAY

SYMPTOM	CAUSE	FIX
A new pattern file is ignored, no message	The 30-minute cache of pattern files (no development mode yet)	`wp config set WP_DEVELOPMENT_MODE theme` (local only), or raise the theme's Version.
`Notice: Function get_block_patterns was called incorrectly ... ("Slug" field missing)`	A pattern header without `Slug:`	Add `Slug: pagesco/<name>` (letters, digits, `/`, `_`, `-`).
The hours vanished from About and Contact	The synced pattern was deleted (Patterns screen Delete is permanent)	Recreate it from a copy with `wp post create ... --import_id=<old ID>` (Lesson 3.3, Common Mistakes 3).
The page shows the Evening colors after a demo click	Someone saved a variation in the Site Editor: a `wp_global_styles` post	Styles > ⋮ > Reset styles, or delete the post (`wp post list --post_type=wp_global_styles`).

SYMPTOM	CAUSE	FIX
The font never loads, no message	A wrong path after `file:./` (or the folder name)	Network tab, filter Font: the 404 names the path.
`Error: The parent theme is missing. Please install the "twenty-twenty-five" parent theme.`	`Template:` has the display name instead of the folder	`Template: twentytwentyfive` (the folder name from `wp theme list`).
The child's style.css has no effect	A block child's style.css is not loaded until functions.php enqueues it	The lesson's `pagesco_tt5_child_enqueue_styles()` with the parent's handle as dependency.
`Cannot redeclare twentytwentyfive_...()` (in another parent)	A non-pluggable parent function defined again in the child	Only functions wrapped in `function_exists()` can be replaced; otherwise remove the parent's hook on `after_setup_theme`.
Learners stay on the child theme after class	Step 4 of the 3.4 project (switch back) went home	`wp theme activate pagesco`: Session 6 starts from pagesco active.

SESSION 5 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 4 (theme.json, the hierarchy, Site Editor copies), the two-command readiness check, homework questions.
0:10	10	**3.3** Frame and mental model	The rubber stamp and the hanging sign: theme pattern, synced pattern, template part.
0:20	25	**3.3** Live demo	A hero pattern file, the pattern cache and development mode, front-page.html, a synced pattern on two pages, a section style, a Site Editor save followed back into the file.
0:45	20	**3.3** Guided lab	`labs/module-05/l3_3-patterns-style-variations/hours_band_starter/` in pairs: finish a pattern file (header, translation, home_url).
1:05	15	**3.3** Start the lesson project	The featured-books pattern (Step 1) and the ink band (start of Step 2); the variation, the font and Steps 3 and 4 go home.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:20	10	Break	Leave the three-tools table on the board; machines without development mode get it now.
1:30	10	**3.4** Frame and mental model	Tracing paper over a floor plan; the inheritance table, row by row.
1:40	25	**3.4** Live demo	A one-file child, template and stylesheet, theme.json merged (and a partial palette broken), the stylesheet nobody loads, the load order, a parent hook removed too early.
2:05	20	**3.4** Guided lab	`labs/module-05/l3_4-child-themes/tt4_child_starter/`: a child of Twenty Twenty-Four, four inheritance rules, one test.
2:25	15	**3.4** Start the lesson project	The footer by pattern override (Step 1) and the complete functions.php (Step 2); the check and the switch back go home.
2:40	10	**3.4** Which tool for which change?	Five client requests sorted together: Site Editor, child theme, own theme or plugin.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 6.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 145 + 125 = 270 minutes for a learner working alone; after the warm-up, the break and the wrap-up the room has 150. Lesson 3.3 keeps its core (patterns, the cache, front-page, synced patterns, a section style, the Site Editor loop) and sends its second half home: the Evening style variation and what the Site Editor saves when someone chooses it, the web font with fontFace and its license, the Export zip, Common Mistakes, the project's Steps 2 to 4. Lesson 3.4 keeps the child theme, the merge rules, functions.php and its load order; the classic child theme (Twenty Twenty-One, which needs the internet), Common Mistake 1 and the project's Steps 3 and 4 go home.

Goes home (about 2 to 3 hours). 3.3: the style variation, the web font, Export, the three Common Mistakes, the project Steps 2 (rest) to 4 and the per-URL check; 3.4: the classic child theme section, Common Mistake 1, the project Steps 3 and 4 (the check and **the switch back to pagesco**); each lesson's Hands-on Exercises; both Quick Quizzes; both journal entries; the labs not reached in the room.

Protect, in this order: the Site Editor round trip in 3.3 (the hardest idea of the module); the theme.json merge rule with the partial palette; the stylesheet-nobody-loads moment and the load order; the synced pattern's one-edit-two-pages test. **Cut first:** the transient listing in the 3.3 demo (just say "a cache"), then the "Which tool" sorting (it becomes Exercise 3 at home), then the 3.4 project start (learners write the footer pattern at home; the lab practiced the same override).

OPENING SEGMENT

0:00 Warm-up and Readiness Check — 10 min

TIMING

6m	Five retrieval questions on Session 4 (W1 to W5, on the slide; think, pair, then cold-call)
4m	Homework questions from Session 4; who has Site Editor copies (W5)?

THE READINESS CHECK (W5): THE TWO COMMANDS

Learners run them in the project folder; the projector's site prints:

```
wp eval 'var_dump( wp_is_block_theme(), wp_theme_has_theme_json() );'
wp post list --post_type=wp_template,wp_template_part,wp_global_styles --format=co
```

```
bool(true)
bool(true)
0
```

Anything but `0` on the second line: the learner opened the Site Editor and saved. List the posts (`wp post list --post_type=wp_template,wp_template_part,wp_global_styles --fields=ID,post_type,post_name`) and delete them with `wp post delete <ID> --force` before the demo, or the front page they see will not be the one the lesson draws.

WARM-UP ANSWERS

1. **W1** In theme.json, what is the difference between settings and styles? **Answer:** settings say what editors may choose (the palette, font sizes, layout widths); styles say what is applied (the page's colors, fonts, element and block styles). Presets in settings become custom properties and classes (Lesson 3.2).
2. **W2** The front page shows the latest posts. In a block theme, which templates does WordPress look for, in order? **Answer:** front-page, then home, then index (Lesson 3.1). pagesco has only index.html so far; Lesson 3.3 adds front-page.html today.
3. **W3** The manager changed the Single Posts template in the Site Editor. Where is that change, and which version does a visitor see? **Answer:** In the database, a wp_template post whose source is custom; it wins over templates/single.html until someone resets it (Lesson 3.2, Common Mistakes 3).
4. **W4** A template part is included as {"slug":"heder"}. What do visitors see? And a logged-in administrator? **Answer:** Visitors: nothing, the header is just missing. Logged in: "Template part has been deleted or is unavailable: heder" (Lesson 3.2).

5 **W5** Run the two commands on the slide. What should a site ready for today print? **Answer:** bool(true) twice (pagesco is a block theme with a theme.json), then 0: no Site Editor copies. A learner with copies resets them first (wp post delete ID --force), or today's outputs will not match.

3.3 Patterns, Style Variations & the Site Editor

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Write PHP pattern files with the right header fields, translatable text and escaped theme URLs, and place them in templates with the Pattern block
- Explain the difference between theme patterns, synced patterns and template parts, and create a synced pattern that updates every page it is used on
- Define section styles and a style variation as JSON files in styles/ and read what the Site Editor saves when someone chooses one
- Bundle a web font with fontFace in theme.json, checking that its license allows redistribution
- Bring Site Editor changes back into the theme's files with Export and WP-CLI, then reset the database copies

TIMING IN THE ROOM

10m	Frame: the rubber stamp and the hanging sign; three tools in one table
25m	Live demo: hero.php, the cache and development mode, front-page.html, a synced pattern, a section style, the Site Editor loop
20m	Guided lab: `hours_band_starter/` in pairs
15m	Start the lesson project: Step 1 (featured books) and the start of Step 2

MENTAL MODEL TO SAY

"A theme pattern is a rubber stamp. You press it onto the page and get a perfect print; after that the print is just ink on paper, and changing the stamp does not change prints you already made. A synced pattern is a sign hung in several rooms: there is only one sign, so repainting it changes what every room shows. Template parts are the signs fixed to the building itself, at every entrance."

Then the table on the board, one row at a time, with the question that decides: **must every copy change when the original changes?** No: a theme pattern (a starting point). Yes, and it is content the owner runs: a synced pattern. Yes, and it is site structure: a template part.

Then the PHP argument in one breath: "Pattern files are PHP so they can be translated, build URLs that work on any site, and escape their output. Keep those three in mind; the Site Editor will take all three away from you at the end of the demo."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Run everything in the WP-CLI container shell (`docker compose run --rm wpcli sh`), where `wp`, `curl` and `grep` all work. Files are saved in VS Code from the lesson page. Ready learners type along on their own Lesson 3.2 site.

1. **Nothing registered yet (2m).** "The shop's staff face an empty canvas. Patterns are how you hand them finished sections." In the WP-CLI container shell, copy Twenty Twenty-Five's public-domain book image into the theme, then count the patterns whose name starts with `pagesco/`.

   ```
   mkdir -p wp-content/themes/pagesco/assets/images wp-content/themes/pagesco/pat
   cp wp-content/themes/twentytwentyfive/assets/images/book-image.webp wp-content
   ls wp-content/themes/pagesco/assets/images
   wp eval 'echo count( array_filter( WP_Block_Patterns_Registry::get_instance()-
   ```

   ```
   book-image.webp
   0
   ```

   Zero, as expected. Point at the image's license in Twenty Twenty-Five's readme.txt: CC0 and public domain, so it may travel with your theme.

2. **The hero pattern file (4m).** "A pattern is a PHP file in patterns/: a header, then block markup." Save the lesson's `hero.php` in VS Code. Read the header aloud (Title, Slug, Categories, Post Types: wp_template) and then the three PHP reasons: translatable strings, `get_theme_file_uri()` and `home_url()`, escaping late.

```php
<?php
/**
 * Title: Hero: welcome to the shop
 * Slug: pagesco/hero
 * Categories: banner, featured
 * Keywords: hero, welcome, intro
 * Description: A full-width welcome with the page heading, a short text, two
 * Viewport Width: 1200
 * Post Types: wp_template
 * Template Types: front-page
 *
 * @package pagesco
 */

?>
<!-- wp:group {"align":"full","className":"is-style-section-sage","style":{"sp
<div class="wp-block-group alignfull is-style-section-sage" style="padding-top
    <!-- wp:columns {"verticalAlignment":"center","align":"wide"} -->
    <div class="wp-block-columns alignwide are-vertically-aligned-center">
        <!-- wp:column {"verticalAlignment":"center","width":"60%"} -->
        <div class="wp-block-column is-vertically-aligned-center" style="flex-
            <!-- wp:heading {"level":1} -->
            <h1 class="wp-block-heading"><?php esc_html_e( 'Good books, good c
            <!-- /wp:heading -->

            <!-- wp:paragraph -->
            <p><?php esc_html_e( 'An independent bookstore with a big reading
            <!-- /wp:paragraph -->

            <!-- wp:buttons -->
            <div class="wp-block-buttons">
                <!-- wp:button -->
                <div class="wp-block-button"><a class="wp-block-button__link w
                <!-- /wp:button -->

                <!-- wp:button {"className":"is-style-outline"} -->
                <div class="wp-block-button is-style-outline"><a class="wp-blo
                <!-- /wp:button -->
            </div>
            <!-- /wp:buttons -->
        </div>
        <!-- /wp:column -->

        <!-- wp:column {"verticalAlignment":"center","width":"40%"} -->
        <div class="wp-block-column is-vertically-aligned-center" style="flex-
            <!-- wp:image {"width":"280px","sizeSlug":"full","linkDestination"
            <figure class="wp-block-image aligncenter size-full is-resized"><i
```

```
            <!-- /wp:image -->
        </div>
        <!-- /wp:column -->
    </div>
    <!-- /wp:columns -->
</div>
<!-- /wp:group -->
```

3 **Why is my pattern missing? (4m).** Ask the registry. Predict first: a correct file in the right folder, so true?

```
wp eval 'var_dump( WP_Block_Patterns_Registry::get_instance()->is_registered(
wp transient list --network --search="wp_theme_files*" --fields=name,value
```

```
bool(false)
+-------------------------------------------------------------+-----------------
| name                                                        | value
+-------------------------------------------------------------+-----------------
| wp_theme_files_patterns-ab0ac91d73cd40af5667134910f64f93 | a:2:{s:7:"versio
+-------------------------------------------------------------+-----------------
```

False, and nothing says why. The count you ran a minute ago built the cached list of pattern files (a transient, 30 minutes, keyed on the theme's Version) while patterns/ was empty: `patterns;a:0:{}`. On a live site that cache is what you want; on a development site, say so:

4 **Development mode (1m).**

```
wp config set WP_DEVELOPMENT_MODE theme
wp eval 'var_dump( wp_get_development_mode(), WP_Block_Patterns_Registry::get_
```

```
Success: Added the constant 'WP_DEVELOPMENT_MODE' to the 'wp-config.php' file
string(5) "theme"
bool(true)
```

Write on the board: **WP_DEVELOPMENT_MODE theme = local only**; on production, raise Version on every deploy (Lesson 8.4).

5 **A front page from patterns (3m).** "front-page.html sits above home and index for the front page." Save the lesson's first `templates/front-page.html` and reload the home page in the browser: the h1 now comes from the pattern.

```
<!-- wp:template-part {"slug":"header","tagName":"header"} /-->

<!-- wp:group {"tagName":"main","style":{"spacing":{"margin":{"top":"0"},"padd
<main class="wp-block-group" style="margin-top:0;padding-bottom:var(--wp--pres
    <!-- wp:pattern {"slug":"pagesco/hero"} /-->

    <!-- wp:heading -->
    <h2 class="wp-block-heading">News from the shop</h2>
    <!-- /wp:heading -->

    <!-- wp:query {"queryId":1,"query":{"inherit":true}} -->
    <div class="wp-block-query">
        <!-- wp:post-template -->
            <!-- wp:post-title {"isLink":true,"level":3} /-->
            <!-- wp:post-date /-->
        <!-- /wp:post-template -->
    </div>
    <!-- /wp:query -->
</main>
<!-- /wp:group -->

<!-- wp:template-part {"slug":"footer","tagName":"footer"} /-->
```

```
<h1 class="wp-block-heading">Good books, good company</h1>
```

Then View Source: no `wp:pattern` comment reaches the browser. WordPress included hero.php, ran its PHP and rendered the blocks it printed. In the file, the line is a reference: change hero.php and the page follows.

6 **One sign in two rooms: a synced pattern (5m).** "The opening hours are on About and the shop wants them on Contact too. Two copies disagree the first time the hours change." In the editor this is Create pattern with Synced on; here are the pieces it stores, with WP-CLI. Save the three files of the lesson into `wordpress/pagesco-content/` (the lesson page has them), then:

```
wp post create pagesco-content/opening-hours.html --post_type=wp_block --post_
wp post update 12 pagesco-content/about.html
wp post update 13 pagesco-content/contact.html
curl -s http://localhost:8080/contact/ | grep -o '<h2 class="wp-block-heading"
```

```
20
Success: Updated post 12.
Success: Updated post 13.
<h2 class="wp-block-heading">Opening hours
<li>Monday to Wednesday, Friday: 9 a.m. to 6 p.m.
<li>Thursday: 9 a.m. to 9 p.m.
<li>Saturday: 10 a.m. to 5 p.m.
```

`--porcelain` printed only the new ID, 20: on your site it may differ, which is the whole point of the next minute. Now the test that matters, Saturday's hours:

```
sed -i 's/Saturday: 10 a.m. to 5 p.m./Saturday: 10 a.m. to 6 p.m./' pagesco-co
wp post update 20 pagesco-content/opening-hours.html
curl -s http://localhost:8080/about/ | grep -o 'Saturday: [^<]*'
curl -s http://localhost:8080/contact/ | grep -o 'Saturday: [^<]*'
```

```
Success: Updated post 20.
Saturday: 10 a.m. to 6 p.m.
Saturday: 10 a.m. to 6 p.m.
```

One edit, two pages. Then the rule, out loud: a synced pattern is a `wp_block` post referenced by ID, `<!-- wp:block {"ref":20} /-->`; IDs differ per site, so a theme never ships one. Leave `pagesco-content/` for now (the lesson deletes it after Common Mistakes).

7 **A section style (3m).** "A JSON file in styles/ with blockTypes is a section style: a color band editors apply to one Group." Save the lesson's `styles/sections/section-sage.json`. The hero already carries the class; look at what WordPress generated:

```
curl -s http://localhost:8080/ | grep -o 'class="wp-block-group alignfull is-s
```

```
:root :where(.wp-block-group.is-style-section-sage--2){background-color: var(
class="wp-block-group alignfull is-style-section-sage has-global-padding is-la
```

A numbered class just for this Group (`--2` here, unique per page) and a rule with `:where()`, pointing at the sage preset. Without blockTypes the same file would be a variation for the whole site: Evening, which learners write at home.

8 **A Site Editor save, followed back into code (6m).** "The shop manager adds one sentence to the hero in the Site Editor and clicks Save." The lesson's script sends the same REST request the Site Editor sends; save it as `wordpress/pagesco-manager-edit.php` and run it as admin (`wp eval-file pagesco-manager-edit.php --user=admin`):

```php
<?php
// What the Site Editor saves: the template with every Pattern block expanded,
$template = get_block_template( 'pagesco//front-page' );
$blocks   = resolve_pattern_blocks( parse_blocks( $template->content ) );
$content  = str_replace( 'talk about books.', 'talk about books. Open late on

$request = new WP_REST_Request( 'POST', '/wp/v2/templates/pagesco//front-page'
$request->set_param( 'content', $content );
$response = rest_do_request( $request );

echo $response->get_status(), ' ', $response->get_data()['source'], "\n";
```

```
200 custom
```

`custom`: a database copy now wins over the file (Lesson 3.2). Look inside it:

```
wp post list --post_type=wp_template --fields=ID,post_name
wp post get $(wp post list --post_type=wp_template --field=ID) --field=post_co
```

```
+----+------------+
| ID | post_name  |
+----+------------+
| 24 | front-page |
+----+------------+
"patternName":"pagesco/hero"
Open late on Thursdays.
src="http://localhost:8080/wp-content/themes/pagesco/assets/images/book-image.
```

Stop here and read it with the room: no `wp:pattern` left, a `patternName` note, the sentence, and the image URL hard-coded to localhost: the PHP ran before the save, so translation and theme URLs are gone. Move the change to where it belongs and remove the copy:

```
sed -i "s/talk about books.', 'pagesco'/talk about books. Open late on Thursda
grep -o 'talk about books[^,]*' wp-content/themes/pagesco/patterns/hero.php
wp post delete $(wp post list --post_type=wp_template --field=ID) --force
rm -rf /tmp/pagesco-export pagesco-manager-edit.php
curl -s http://localhost:8080/ | grep -o 'talk about books[^<]*'
```

```
talk about books. Open late on Thursdays.'
Success: Deleted post 24.
talk about books. Open late on Thursdays.
```

Write the loop on the board: **list the copies → find the change → put it in the right file → delete the copy**, with the listing command under it:

```
wp post list --post_type=wp_template,wp_template_part,wp_global_styles
```

Export (the Site Editor's Options menu) is the same idea for the whole theme; learners read that section at home.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **"My pattern doesn't show up."** Either the cache (no development mode: `is_registered()` is false and nothing says why) or a header without `Slug:` (a notice at the top of every page). Coach: "Ask the registry first, then read the top of the page." (Self-check Q1; the handout `debug_pattern_files.sh`.)
- **Copying the Site Editor's markup back over the file.** It looks like the newest version. Coach: "Where is your `wp:pattern` line? What happened to `esc_html_e()`?" (Self-check Q4, the lesson quiz Q4.)
- **A synced pattern's reference in a theme file.** `<!-- wp:block {"ref":20} /-->` in front-page.html "because it worked". Coach: "On the live site, what is post 20?" (Self-check Q3.)

GUIDED LAB

Folder: `labs/module-05/l3_3-patterns-style-variations/hours_band_starter/` with `labs/module-05/l3_3-patterns-style-variations/test_hours_band.php`; your reference is `labs/module-05/l3_3-patterns-style-variations/hours_band_solution/`. A NEW pattern (the shop's address in a sage band, a button to Contact) that works but is not finished as a pattern: no categories, no keywords or description, strings that cannot be translated, a link hard-coded to localhost. The folder is a one-file child theme of pagesco, so pagesco itself stays untouched ("Lesson 3.4 explains how this works, in an hour").

- **Install and run as shipped (4m).** From the project folder: `cp -r labs/module-05/l3_3-patterns-style-variations/hours_band_starter wordpress/wp-content/themes/pagesco-lab-hours`, `wp theme activate pagesco-lab-hours`, copy the test into `wordpress/` and `wp eval-file --use-include test_hours_band.php`. Read the starter's output with the room:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Hours Band' theme.
Title:       Visit the shop: hours band
Categories:  (none)
Keywords:    (none)
Description: (none)
Heading:     Visit the shop
Text:        12 Market Street, open late on Thursdays.
Button:      Plan your visit -> http://localhost:8080/contact/
```

- **The test's trick (2m).** It renders the pattern as a Spanish visitor of the LIVE site would get it: a filter turns every string of the child's text domain into capitals (a stand-in for a translation), and `home_url()` pretends the site is pagesandco.example. Lower case = not translatable; localhost = broken on the live site.
- **TODOs 1 to 5 in pairs (10m).** Circulate for: the text domain `pagesco` instead of the child's `pagesco-lab-hours` (the heading stays lower case: "which theme owns this string?"); `esc_url()` forgotten around `home_url()`; a Categories line that names an unknown category (it registers anyway, and the inserter files it nowhere).
- **Done when** the test prints (the solution):

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Hours Band' theme.
Title:       Visit the shop: hours band
Categories:  contact
Keywords:    hours, address, visit
Description: A sage band with the shop's address and a button to the Contact page
Heading:     VISIT THE SHOP
Text:        12 MARKET STREET, OPEN LATE ON THURSDAYS.
Button:      PLAN YOUR VISIT -> https://pagesandco.example/contact/
```

- **Clean up (1m):** `wp theme activate pagesco` and remove the theme folder and the test (the README's last line).
- **Fast finishers:** `labs/module-05/l3_3-patterns-style-variations/debug_pattern_files.sh` (needs development mode from the demo: a pattern without a Slug, a broken JSON file, a style that turns out to be a section style). `labs/module-05/l3_3-patterns-style-variations/variation_contrast_starter.php` needs Evening, so it goes home.

START THE LESSON PROJECT

The lesson project is **The Pages & Co. Front Page**: featured books, an ink call to action with its own section style, the final front-page.html, a screenshot and a check of the whole site (about 45 minutes alone). In the room: Step 1 and the start of Step 2 (15 minutes).

- Step 1: `patterns/featured-books.php` from the lesson: the books are a PHP array (the Book post type arrives in 5.2) looped into columns; prefixed variables, `number_format_i18n()`, a translators comment. Check it with the lesson's two `wp eval` lines.
- Step 2, start: `styles/sections/section-ink.json`. Ask: "why do headings need their own color in a section style?" (theme.json's `styles.elements.heading` would not reach them.)
- **Done in the room** = featured-books registered and printing three books. **Goes home:** the call-to-action pattern, Step 3 (the final front-page.html), Step 4 (screenshot, the per-URL loop), and before all of it the lesson's Evening and web-font sections, which Step 4's check expects (`fonts:1`).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Theme pattern, synced pattern or template part: the shop's returns policy on six pages? **Answer:** a synced pattern (the lesson quiz Q2).
- Why are pattern files PHP and not .html? **Answer:** translatable strings, URLs built for the site they run on, escaping.
- What does WP_DEVELOPMENT_MODE theme switch off, and where must it never be set? **Answer:** the theme caches such as the pattern-file list; on production.
- Where does the Site Editor store a saved template, and what happens to the patterns in it? **Answer:** a wp_template post (source custom); the patterns are expanded into plain blocks.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** A) Raise the theme's Version in style.css; **Q2** `1 2`: one site variation (Evening) and two section styles (Sage band, Ink band); **Q3** C) `<!-- wp:block {"ref":20} /-->`; **Q4** D) The pattern was expanded before saving: its PHP already ran, so the URL is hard-coded and the strings are frozen in English; **Q5** B) Nothing: it is still a section style, because it has `blockTypes`; **Q6** The browser's developer tools, Network tab filtered by Font: the font request answers 404, so the src path after file:./ (or the folder name) is wrong. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: pair learners who are still unsure about block markup with someone who finished Session 4's project; they own the test runs, the partner edits. Point at the lesson's header-fields table while they do TODOs 1 and 2.

Stretch: the debug handout, then the lesson's Exercise 4 (pattern overrides) and the Bonus Challenge (a "Meet the staff" pattern offered on pages but not in templates: which header?).

3.4 Child Themes & Customizing Existing Themes

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Create and activate a child theme with the Template header, and explain the template and stylesheet options it sets
- Predict how a child's theme.json, templates, template parts, patterns and style variations combine with the parent's
- Use a child functions.php to enqueue the child's stylesheet, replace a pluggable parent function and remove a parent hook at the right time
- Build a classic child theme and explain how get_stylesheet_directory(), get_template_directory() and locate_template() choose files
- Choose between the Site Editor, a child theme, your own theme and a plugin for a requested change

TIMING IN THE ROOM

10m	Frame: tracing paper over a floor plan; the inheritance table
25m	Live demo: the one-file child, template and stylesheet, theme.json merged and broken, the stylesheet nobody loads, the load order, a hook removed too early
20m	Guided lab: `tt4_child_starter/`
15m	Start the lesson project: Steps 1 and 2
10m	Which tool for which change? (the lesson's decision table, as a sorting exercise)

MENTAL MODEL TO SAY

"Picture the parent theme as an architect's printed floor plan. You lay tracing paper on top and draw only what you want different. Anyone looking at the stack sees the combined plan. When the architect sends a revised plan, you slide your tracing paper onto the new print, and your changes are still there."

Then fill the table on the board during the demo, one row per step: **style.css** the child's header replaces it, its CSS loads only if enqueued · **theme.json** merged, but preset lists replaced · **templates, parts** same name replaces · **patterns** same slug wins · **styles/** same file name replaces · **functions.php** both run, the child's first.

And the warning that frames the end of the lesson: "A child theme is still a theme. Switch themes in three years and everything in it is gone. That is why the store notice is a plugin and the Book post type will be one too."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Starts from the site at the end of Lesson 3.3 (your projector site after the 3.3 demo plus learners' homework is close enough; the outputs below come from the saved end of Lesson 3.3). Files from `lesson_10.html`, saved in VS Code.

1. **The smallest child theme (3m).** "Most client sites run a theme you did not write. Edit it and the next update wipes your work." Save the lesson's `pagesco-tt5-child/style.css`: one file, and the line that matters is `Template:` with the parent's FOLDER name. Activate it and ask WordPress:

```
wp theme activate pagesco-tt5-child
wp option get template
wp option get stylesheet
wp theme list --fields=name,status,version
```

```
Success: Switched to 'Pages &amp;amp; Co. TT5 Child' theme.
twentytwentyfive
pagesco-tt5-child
+--------------------+----------+---------+
| name               | status   | version |
+--------------------+----------+---------+
| pagesco-tt5-child | active   | 1.0.0   |
| pagesco           | inactive | 1.0.0   |
| twentytwentyfive  | parent   | 1.5     |
| twentytwentyfour  | inactive | 1.6     |
| twentytwentythree | inactive | 1.7     |
+--------------------+----------+---------+
```

Two options, two jobs: `template` is the parent (templates and code), `stylesheet` the child (whose header names the site's theme); `wp theme list` calls Twenty Twenty-Five `parent`. In PHP each option has a directory function:

```
wp eval 'echo get_template_directory(), "\n", get_stylesheet_directory(), "\n"
```

```
/var/www/html/wp-content/themes/twentytwentyfive
/var/www/html/wp-content/themes/pagesco-tt5-child
bool(true)
bool(true)
```

2. **theme.json: merged, not replaced (5m).** Save the lesson's child `theme.json`: the shop's colors in Twenty Twenty-Five's role slots (base, contrast, accent-1 to accent-6). Predict: does the body font stay Manrope?

```
{
    "$schema": "https://schemas.wp.org/wp/7.1/theme.json",
    "version": 3,
    "settings": {
        "color": {
            "palette": [
                { "slug": "base", "name": "Paper", "color": "#fbf8f1" },
                { "slug": "contrast", "name": "Ink", "color": "#1f2a2e" },
                { "slug": "accent-1", "name": "Sage", "color": "#e3eadf" },
                { "slug": "accent-2", "name": "Sand", "color": "#ece3d4" },
                { "slug": "accent-3", "name": "Oxblood", "color": "#7a2e2e" },
                { "slug": "accent-4", "name": "Slate", "color": "#56636a" },
                { "slug": "accent-5", "name": "Cream", "color": "#f4efe3" },
                { "slug": "accent-6", "name": "Ink 20%", "color": "color-mix(i
            ]
        }
    },
    "styles": {
        "elements": {
            "link": {
                "color": { "text": "var:preset|color|accent-3" }
            }
        }
    }
}

curl -s http://localhost:8080/ | grep -o -- '--wp--preset--color--\(base\|cont
curl -s http://localhost:8080/ | grep -o 'a:where(:not(.wp-element-button)){[^
```

```
--wp--preset--color--base: #fbf8f1
--wp--preset--color--contrast: #1f2a2e
--wp--preset--color--accent-1: #e3eadf
--wp--preset--color--accent-2: #ece3d4
--wp--preset--color--accent-3: #7a2e2e
--wp--preset--color--accent-4: #56636a
--wp--preset--color--accent-5: #f4efe3
--wp--preset--color--accent-6: color-mix(in srgb, currentColor 20%, transparer
body{background-color: var(--wp--preset--color--base);color: var(--wp--preset-
a:where(:not(.wp-element-button)){color: var(--wp--preset--color--accent-3);}
```

Single values merge key by key (the link color is the child's, Manrope is the parent's); preset LISTS are replaced whole. Prove the second half with the lesson's Common Mistake 2, a two-color palette:

```
cp wp-content/themes/pagesco-tt5-child/theme.json theme.json.bak
sed -i '/"slug": "accent-/d; s/"color": "#1f2a2e" },/"color": "#1f2a2e" }/' wp
curl -s http://localhost:8080/ | grep -c -- '--wp--preset--color--accent-[0-9]
curl -s http://localhost:8080/ | grep -o 'var(--wp--preset--color--accent-[0-9
```

```
0
      1 var(--wp--preset--color--accent-3)
      3 var(--wp--preset--color--accent-4)
```

No accent variable is defined any more, while the parent's styles still use accent-3 once and accent-4 three times on this page. CSS ignores an undefined variable without a word. Restore the copy:

```
mv theme.json.bak wp-content/themes/pagesco-tt5-child/theme.json
curl -s http://localhost:8080/ | grep -c -- '--wp--preset--color--accent-[0-9]
```

```
1
```

3 **The stylesheet nobody loads (4m).** Append the lesson's notice rule to the child's style.css, then ask whether the page loads the file:

```
curl -s http://localhost:8080/ | grep -c 'pagesco-tt5-child/style.css'
```

```
0
```

Zero. A block theme's style.css is a header file first; CSS is loaded only when functions.php enqueues it, and Twenty Twenty-Five enqueues its own with `get_parent_theme_file_uri()`. Save the lesson's first `functions.php`:

```php
<?php
/**
 * Pages & Co. TT5 Child functions and definitions.
 *
 * @package pagesco-tt5-child
 */

/**
 * Loads the child theme's style.css after the parent's styles.
 */
function pagesco_tt5_child_enqueue_styles() {
    wp_enqueue_style(
        'pagesco-tt5-child-style',
        get_stylesheet_uri(),
        array( 'twentytwentyfive-style' ),
        wp_get_theme()->get( 'Version' )
    );
}
add_action( 'wp_enqueue_scripts', 'pagesco_tt5_child_enqueue_styles' );
```

```
curl -s http://localhost:8080/ | grep -o "<link rel='stylesheet' id='pagesco-t
```

```
<style id="twentytwentyfive-style-inline-css">
<link rel='stylesheet' id='pagesco-tt5-child-style-css' href='http://localhost
```

The parent's CSS inlined, the child's `<link>` after it, with the child's Version. The dependency on `twentytwentyfive-style` is what puts it second.

4. **Who loads first? (3m).** "Both functions.php files run. In which order?"

```
wp eval 'foreach ( get_included_files() as $f ) { if ( str_contains( $f, "/the
```

```
/var/www/html/wp-content/themes/pagesco-tt5-child/functions.php
/var/www/html/wp-content/themes/twentytwentyfive/functions.php
```

Child first, then parent. Two consequences, both used in the project: pluggable parent functions (wrapped in `function_exists()`) can be replaced by defining them in the child; parent hooks cannot be removed yet when the child's file runs.

5. **Mistake 3: removing a parent hook too early (5m).** Append the lesson's logging line to the end of the child's functions.php (it removes Twenty Twenty-Five's block styles at the top level and logs what `remove_action()` returns), then check:

```
error_log( 'remove_action returned ' . var_export( remove_action( 'init', 'twe

wp eval 'var_dump( WP_Block_Styles_Registry::get_instance()->is_registered( "c
grep 'remove_action returned' wp-content/debug.log | tail -n 1
```

```
bool(true)
[26-Oct-2026 13:24:18 UTC] remove_action returned false
```

Still registered, and the only clue is the return value you logged: false, nothing to remove yet. The fix is a callback on `after_setup_theme`, which the project's Step 2 writes. Then switch back so the site stays as the lesson expects:

```
wp theme activate pagesco
```

```
Success: Switched to 'Pages &amp;amp; Co.' theme.
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **The display name in `Template:`.** `Twenty Twenty-Five` or `twenty-twenty-five`. Coach: "Which column of `wp theme list` is the folder?" (Self-check Q7.)
- **Half a palette.** Only the colors they wanted to change. Coach with the demo's count: "How many accent variables does the page define now?" (Self-check Q9.)
- **Removing or replacing at the wrong moment.** A top-level `remove_action()` that returns false, or redefining a parent function that is not pluggable (`Cannot redeclare`). Coach: "Which file runs first? Is the parent's function wrapped in `function_exists()`?" (Self-check Q10; `debug_child_theme.sh`.)

GUIDED LAB

Folder: `labs/module-05/l3_4-child-themes/tt4_child_starter/` with `labs/module-05/l3_4-child-themes/test_tt4_child.sh`; your reference is `labs/module-05/l3_4-child-themes/tt4_child_solution/`. The same four inheritance rules as the lesson, on a DIFFERENT parent, Twenty Twenty-Four, so learners must read the parent instead of copying the lesson: its palette has ten slugs, it enqueues no stylesheet (no handle to depend on), its footer part shows the pattern `twentytwentyfour/footer`, and its block styles live in a pluggable `twentytwentyfour_block_styles()`.

- **Install and run as shipped (3m).** `cp -r labs/module-05/l3_4-child-themes/tt4_child_starter wordpress/wp-content/themes/pagesco-lab-tt4-child`, `wp theme activate pagesco-lab-tt4-child`, copy the test into `wordpress/`, `docker compose run --rm wpcli sh test_tt4_child.sh`:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: TT4 Child' theme.
template:    twentytwentyfour
stylesheet:  pagesco-lab-tt4-child
accent-3:    #d8613c
palette:     10 theme colors
child CSS:   0
footer:      (not the shop's: still the parent's footer pattern)
styles:      core/details arrow-icon-details, core/post-terms pill, core/list che
```

- **TODOs 1 to 4 in pairs (14m), running the test after each.** The first move is reading `wp-content/themes/twentytwentyfour/functions.php` and `parts/footer.html`. Circulate for: a palette with only accent-3 (the test's palette line drops to 1: Q9 live); `array( 'twentytwentyfour-style' )` copied from the lesson (the stylesheet then never prints, and debug.log says the dependency is not registered); the footer slug with the child's namespace.
- **Done when** the test prints (the solution):

```
Success: Switched to 'Pages &amp;amp; Co. Lab: TT4 Child' theme.
template:    twentytwentyfour
stylesheet:  pagesco-lab-tt4-child
accent-3:    #7a2e2e
palette:     10 theme colors
child CSS:   1
footer:      The reading corner at Pages &amp;amp; Co. · 12 Market Street
styles:      core/list checkmark-list
```

- **Debrief (2m):** "Which TODO needed you to read the parent first?" (All four; the handle most of all.) Then `wp theme activate pagesco` and remove the folder.
- **Fast finishers:** `labs/module-05/l3_4-child-themes/merge_layers_starter.php` (merge the lesson child's theme.json over Twenty Twenty-Five's with `WP_Theme_JSON`, the class WordPress uses), then `labs/module-05/l3_4-child-themes/debug_child_theme.sh`.

START THE LESSON PROJECT

The lesson project is **Finish pagesco-tt5-child** (about 40 minutes alone). In the room: Steps 1 and 2 (15 minutes).

- Step 1: `patterns/footer.php` with the parent's slug `twentytwentyfive/footer`, the parent's presets and the child's text domain. Check the footer text on the home page.
- Step 2: replace functions.php with the lesson's complete version: the enqueue, the pluggable post-format function, and the block-styles removal on `after_setup_theme`. The lesson's `wp eval` line prints the two formats and `bool(false)`.

- **Done in the room** = the shop's footer and `bool(false)`. **Goes home:** Step 3 (the check) and **Step 4,** `wp theme activate pagesco`: say it twice. Session 6 starts from pagesco active with the child installed.

WHICH TOOL FOR WHICH CHANGE? (10M)

The lesson's decision table as a quick sort: read five requests aloud, learners hold up S (Site Editor), C (child theme), T (own theme) or P (plugin), then one volunteer says who owns the change and whether it must survive a theme switch.

1. "Make the buttons a different red; we may change it again at Christmas." **S**: owner-changeable, database.
2. "Remove the comments from all posts on our Twenty Twenty-Five site, for good." **C**: copy single.html, delete one line, in Git.
3. "A booking form for author readings." **P**: a feature with data and security (Module 5).
4. "Our design has outgrown Twenty Twenty-Five: new templates everywhere." **T**: your own theme (Lessons 3.2 and 3.3).
5. "The store notice must survive any redesign." **P**: it already is a must-use plugin (Lesson 2.3).

Self-check Q12 and the lesson's Exercise 3 are the same skill; the journal prompt asks for the real-life version.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- With a child active, which option holds the parent? **Answer:** `template` (the lesson quiz Q1).
- The child has no `templates/index.html`. Which file draws the blog home? **Answer:** the parent's index.html; `single` comes from the child.
- Why does the child's stylesheet need `array( 'twentytwentyfive-style' )`? **Answer:** so it prints after the parent's and wins ties; the dependency must be a handle the parent really registers.
- Why can't a top-level `remove_action()` remove a parent hook? **Answer:** the child's functions.php runs first; the parent has not added the hook yet.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** B) It refuses: the parent theme is missing, and the theme is listed under Broken Themes; **Q8** `pagesco-tt5-child`, then `twentytwentyfive`; **Q9** A) They disappear: the child's palette replaces the parent's whole palette, and styles that use accent variables silently lose their colors; **Q10** D) The child's functions.php loads first, and the parent defines that function only `if ( ! function_exists( ... ) )`; **Q11** C) `array()`; **Q12** The Site Editor's Styles: a visual choice the owner wants to change again; it is saved in the database (the user global styles) and needs no developer. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners who find the TT24 lab too open do TODOs 1 and 2 only, with the lesson's child files open side by side; TODOs 3 and 4 go home. The workbook's worked example lists the lesson child's files and where each template comes from.

Stretch: `merge_layers_starter.php` and the debug handout; the lesson's Exercise 4 (a child file that replaces TWO parent variations: why the count drops to 22) and the Bonus Challenge (override only the hidden-written-by pattern).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note. Question 2's pile tells you whether Session 6's warm-up needs the three-copies picture again.

1. Theme pattern or synced pattern: the shop's opening hours on four pages? And a hero section for new landing pages? **Answer:** synced (one original, edit once); theme pattern (a starting point, copied and then edited).
2. Name two places where the Site Editor stores changes, and the command that lists them. **Answer:** wp_template / wp_template_part posts and the wp_global_styles post; `wp post list --post_type=wp_template,wp_template_part,wp_global_styles`.
3. In a child theme, which runs first, the child's functions.php or the parent's, and what does that mean for removing a parent hook? **Answer:** the child's; remove the hook in a callback on after_setup_theme.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Reset first:** `wp post list --post_type=wp_template,wp_template_part,wp_global_styles` lists nothing (the 3.3 "Before the Next Lesson" list).
- **Finish Lesson 3.3:** the Evening style variation section (including the Site Editor's save and its reset), the web font, Export, the three Common Mistakes, then the project Steps 2 to 4 and the per-URL check.
- **Finish Lesson 3.4:** the classic child theme section (it installs Twenty Twenty-One: needs the internet), Common Mistake 1, the project Steps 3 and 4. **Step 4 matters: pagesco must be active for Session 6.**
- **Lesson exercises.** 3.3: Exercises 1 to 3 (Exercise 4, pattern overrides, is a stretch). 3.4: Exercises 1 to 3 (Exercise 4 is a stretch). Each undoes itself; check that pagesco is active afterwards.
- **Both Quick Quizzes** on the lesson pages (answers in the Answer Key).
- **Journal:** both entries. The 3.3 prompt (who may change each copy of the design) and the 3.4 prompt (moving a predecessor's edits into a child theme) are the two client conversations of this module.
- **Labs not reached:** `labs/module-05/l3_3-patterns-style-variations/variation_contrast_starter.php` (after Evening exists), the two debug handouts, `labs/module-05/l3_4-child-themes/merge_layers_starter.php`.

BRING LIST FOR SESSION 6

- The course site at the end of Lesson 3.4: pagesco active, pagesco-tt5-child installed but inactive, Twenty Twenty-One deleted, no Site Editor copies.
- The project folder with `labs/`; `pagesco-plan.txt` (two new lines today: the front page checked off, "client sites on a ready-made theme: child theme").
- The journal and the workbook. Session 6 is classic themes (PHP templates, the Loop, custom queries): skim `wp-content/themes/twentytwentyone/` if you still have it, or any classic theme's index.php.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the Site Editor round trip; the merge rule with the partial palette; the stylesheet nobody loads and the load order.
- **Cut first (saves 3m):** the transient listing in the 3.3 demo; say "a cache keyed on the theme's version" and move on to development mode.
- **Cut second (saves 10m):** the "Which tool" sort; it is the lesson's Exercise 3 and self-check Q12 at home.
- **Cut third (saves 15m):** the 3.4 project start; the TT4 lab practiced the same pattern override and pluggable function.

RUNNING SHORT: WHAT TO ADD

- Do the Evening variation live: create `styles/evening.json`, choose it in Appearance > Editor > Styles, show the `wp_global_styles` post, reset it. Then the three-copies picture is complete.
- Run `debug_child_theme.sh` for everyone and fix it together, one bug per volunteer, reading debug.log after each run.
- The 3.3 Common Mistake 3 live: delete the synced pattern, watch the hours vanish from two pages, recreate it with `--import_id`.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 6: The Loop and Custom Queries

How to teach Module 4's first two lessons. Build pagesco-classic and show what a classic template really does: walk the posts the main query already found, with template tags printed or returned, parts and conditional tags. Then ask for lists of your own: secondary queries and their three numbers, the reset that keeps template tags honest, pre_get_posts with guards, and pagination that never ends in a 404.

2 lessons | 3 hrs session | Lessons 4.1–4.2 | Session 6 of 18 · Part 1

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (one mental model) → live demo (you type, they type along) → guided lab (new practice on the lesson's ideas) → start the lesson's own project. Exercises, the Quick Quiz and the journal are homework.
- Type, do not paste. Every command below was run to produce the output printed under it, on WordPress 7.1.3 and PHP 8.5: for 4.1 a course site exactly as Lesson 3.4 leaves it (pagesco active), for 4.2 the site at the end of Lesson 4.1 (pagesco-classic active). Ask for a prediction before every run.
- The lessons are the reference. Keep `lesson_11.html` and `lesson_12.html` open for the template listings: reading them aloud matters more than typing them fast. Labs live in `labs/module-06/` : `_starter` files and folders are handouts, `_solution` ones are yours.
- Session 6 turns to the classic theme world most existing sites live in: pagesco-classic is built today and stays active until Lesson 4.3 switches back. The Loop and WP_Query are also how plugins and dynamic blocks print posts later (Modules 5 and 6).

> **THE #1 RISK OF SESSION 6: THE BUGS ARE QUIET**
>
> Every mistake in these two lessons produces no message: a title before an empty heading, a template part that is not found, a page-2 link that 404s, navigation computed for the wrong post, a callback that changes every query on the site. Learners who only watch for errors will miss all of them. **In the room:** for every demo step ask "which query is this, and has it run yet?" and make learners compare what they see with what they expected. **After:** the self-check and the two debug handouts are built from exactly these quiet bugs.

BEFORE THE SESSION

- ☐ **Learner sites:** at the end of Lesson 3.4: pagesco active, pagesco-tt5-child installed, no Site Editor copies (the warm-up's two commands show it).
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-06` from this kit's folder (15 results in about a minute and a half; every line PASS). On your own project: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-06`.
- ☐ **Projector machine:** a course site at the end of Lesson 3.4 for the 4.1 demo (it continues into 4.2), VS Code on `wp-content/themes/pagesco-classic/`, two terminals (one with `docker compose run --rm wpcli sh`, one with `tail -f wordpress/wp-content/debug.log`), the browser on http://localhost:8080/ with View Source handy, `lesson_11.html` and `lesson_12.html` in tabs.
- ☐ **Files to hand out:** `labs/module-06/` without the `*_solution*` files and folders; the workbook.
- ☐ **Board, before learners arrive:** the Loop as a loop (`have_posts()?` → `the_post()` → template tags → back), the request order (URL → query vars → main query → template → the Loop), and the decision table of 4.2 (change the main list: pre_get_posts · an extra list: new WP_Query + wp_reset_postdata · just data: get_posts).
- ☐ **Print or share:** the workbook; sticky notes for the exit ticket.

KNOWN TRAPS TODAY

SYMPTOM	CAUSE	FIX
The page hangs, then a fatal "Maximum execution time of 30 seconds exceeded"	A Loop without the_post()	Every while (have_posts()) needs the_post() inside.
Each title appears BEFORE an empty heading	A the_ tag concatenated into a string	get_the_title() in strings, or the_title($before, $after).
Block comments in the page source, synced pattern or shortcode empty	get_the_content() printed directly	the_content(), or apply_filters('the_content', get_the_content()).
A template part prints nothing, no message	`.php` in the slug, or a typo	No extension; var_dump() the return value (false = not found).
`Notice: Function is_home was called incorrectly`	A conditional tag at the top level of a plugin	Ask inside a callback on template_redirect (or later).
Page 2 of a list answers 404	A custom WP_Query paged in the template	pre_get_posts on the main query; the_posts_pagination().
Previous/next links point at the wrong posts	A secondary Loop without wp_reset_postdata()	wp_reset_postdata() right after the while.

SYMPTOM	CAUSE	FIX
WP-CLI or the admin shows one post	A pre_get_posts callback without guards	is_admin(), $query->is_main_query(), the query's own conditionals.
`'cat' => 'news'` returns everything	cat takes IDs	'category_name' => 'news'.

SESSION 6 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 5 (patterns, styles/, child themes), the two-command readiness check, homework questions.
0:10	10	**4.1** Frame and mental model	The reading list at the till: the main query is done before any template runs; the Loop walks it.
0:20	25	**4.1** Live demo	A classic theme in two files, one index.php for every list, the Loop in core, template tags, header, footer and parts, conditional tags logged per URL, a tag that prints inside a string.
0:45	20	**4.1** Guided lab	`labs/module-06/l4_1-loop-template-tags/shelf_theme_starter/`: a one-template theme, five kinds of page.
1:05	15	**4.1** Start the lesson project	singular.php (Step 1); 404.php, theme.json and the per-URL check go home.
1:20	10	Break	Leave the Loop drawing (have_posts? the_post, tags, again) on the board.
1:30	10	**4.2** Frame and mental model	The customer's order and the shop's displays: one main query, any number of secondary ones, pre_get_posts before the order is written.
1:40	25	**4.2** Live demo	A secondary query and its three numbers, wp_reset_postdata(), pre_get_posts in core and in functions.php, pagination, a paged custom query that 404s, a callback without guards.
2:05	20	**4.2** Guided lab	`labs/module-06/l4_2-custom-queries/query_shelf_starter.php`: five questions, five argument arrays.
2:25	15	**4.2** Start the lesson project	front-page.php with the pages query and the news (Step 1); the related box and the check go home.
2:40	10	**4.2** Which query?	Six requests sorted: pre_get_posts, new WP_Query, get_posts().

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 7.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 125 + 130 = 255 minutes for a learner working alone; the room has 150. Lesson 4.1 keeps the Loop, template tags, the parts and conditional tags; Common Mistakes 2 to 4 (get_the_content(), a .php in get_template_part(), a tag asked too early), the project's Steps 2 to 4 (404.php, the hybrid theme.json, the check) go home. Lesson 4.2 keeps secondary queries, wp_reset_postdata(), pre_get_posts, pagination and Common Mistakes 1 and 3; the WP_Query argument tour (query-args.php), Common Mistakes 2 and 4, and the project's Steps 2 and 3 go home.

Goes home (about 2 to 3 hours). 4.1: the rest of the template-parts section if the room ran short, Common Mistakes 2 to 4, project Steps 2 to 4 (404.php, theme.json, the per-URL loop) and the pagesco-plan.txt line; 4.2: query-args.php, Common Mistakes 2 and 4, project Steps 2 (the related box with its reset) and 3 (the check); each lesson's Hands-on Exercises; both Quick Quizzes; both journal entries; the labs not reached.

Protect, in this order: the moment index.php draws every list (4.1); template tags printed vs returned and escaped; pre_get_posts with its guards; the paged custom query that 404s. **Cut first:** the parts step of the 4.1 demo (show the finished index.php only; the lesson has the files), then the "Which query" sort (self-check Q12 at home), then the 4.2 project start.

OPENING SEGMENT

0:00 Warm-up and Readiness Check — 10 min

TIMING

6m	Five retrieval questions on Session 5 (W1 to W5, on the slide; think, pair, then cold-call)
4m	Homework questions; who is still on the child theme (W5)?

THE READINESS CHECK (W5): THE TWO COMMANDS

On the projector's site:

```
wp theme list --fields=name,status
wp post list --post_type=wp_template,wp_template_part,wp_global_styles --format=co
```

```
+--------------------+----------+
| name               | status   |
+--------------------+----------+
| pagesco-tt5-child  | inactive |
| pagesco            | active   |
| twentytwentyfive   | inactive |
| twentytwentyfour   | inactive |
| twentytwentythree  | inactive |
+--------------------+----------+
0
```

A learner still on pagesco-tt5-child runs `wp theme activate pagesco` now (the 3.4 project's Step 4). Twenty Twenty-One still listed: `wp theme delete twentytwentyone`. Copies above 0: list and delete them as in Session 5.

WARM-UP ANSWERS

1. **W1** Theme pattern or synced pattern: which one is a post in the database, and why does a theme never ship one? **Answer:** The synced pattern: a wp_block post referenced by ID, and IDs differ on every site (Lesson 3.3).
2. **W2** Two JSON files in styles/: what makes one a section style and the other a style variation? **Answer:** The blockTypes key: with it, a section style for the listed blocks; without it, a look for the whole site.
3. **W3** A child theme is active. What do the template and stylesheet options hold? **Answer:** template = the parent's folder, stylesheet = the child's (Lesson 3.4).

4. **W4** Why did the child's style.css not load until functions.php enqueued it, and why must a parent hook be removed on after_setup_theme? **Answer:** A block theme's style.css is a header file; CSS loads only when enqueued. The child's functions.php runs first, so the parent's hooks do not exist yet when it runs.
5. **W5** Run the two commands on the slide. What should a site ready for today print? **Answer:** pagesco active, pagesco-tt5-child inactive (Twenty Twenty-One deleted), and 0 Site Editor copies. Today's demo starts by building pagesco-classic and switching to it.

LESSON PLANS

4.1 The Loop & Template Tags

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what the Loop does with the main query, step by step, and write it correctly with have_posts() and the_post()
- Use template tags and their the_ / get_ pairs, and choose between printing and returning (and escaping) a value
- Split a classic theme into header.php, footer.php and template parts with get_template_part(), including passing arguments
- Use conditional tags such as is_home(), is_singular() and is_search(), and say when they work and when they are called too early
- Build pagesco-classic with index.php, singular.php and 404.php, and turn it into a hybrid theme with theme.json

TIMING IN THE ROOM

10m	Frame: the reading list at the till
25m	Live demo: two files, every list, the Loop in core, template tags, parts, conditional tags, Mistake 1
20m	Guided lab: `shelf_theme_starter/`
15m	Start the lesson project: Step 1, singular.php

MENTAL MODEL TO SAY

"A customer hands the bookseller a list of books they ordered. The bookseller does not search the shop again; the list is done. They go down it one line at a time: is there another book on the list? (have_posts()) Take the next one off the pile and put it on the counter (the_post()). Read out its title, author and price (template tags). Start again. If they forget to take the next book off the pile, they read out the same title forever."

Then the request on the board: **URL → query vars → the main query → the template → the Loop**. "The template never asks which posts. It only walks the list." And the two-halves rule for template tags: the_ prints HTML-ready text, get_ returns a value you escape for its spot.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Run commands in the WP-CLI container shell (`docker compose run --rm wpcli sh`), where `wp` and `curl` both work; save files in VS Code from the lesson page.

1 **A classic theme in two files (4m).** "Before WordPress loads a single template, it has already done the hard part." Save the lesson's `pagesco-classic/style.css` and the first `index.php` (it calls `wp_head()`, `wp_body_open()` and `wp_footer()` itself; the Loop is the if/while in the middle). Activate it:

```
wp theme activate pagesco-classic
wp eval 'var_dump( wp_is_block_theme(), wp_theme_has_theme_json() );'
curl -s http://localhost:8080/ | grep -o '<h2>[^<]*</h2>'
```

```
Success: Switched to 'Pages &amp;amp; Co. Classic' theme.
bool(false)
bool(false)
<h2>Hello world!</h2>
<h2>Book club: Dune</h2>
<h2>Staff picks for the season</h2>
<h2>Welcome to the new Pages &#038; Co. website</h2>
```

false, false: a classic theme. Four headings: the four published posts, newest first. Now the same file on other addresses:

```
curl -s http://localhost:8080/category/news/ | grep -o '<h2>[^<]*</h2>'
curl -s "http://localhost:8080/?s=dune" | grep -o '<h2>[^<]*</h2>'
curl -s http://localhost:8080/about/ | grep -o '<h2>[^<]*</h2>'
```

```
<h2>Book club: Dune</h2>
<h2>Staff picks for the season</h2>
<h2>Welcome to the new Pages &#038; Co. website</h2>
<h2>Book club: Dune</h2>
<h2>Staff picks for the season</h2>
<h2>About Us</h2>
```

Three News posts, two search results, and a Loop of one for /about/. **The template does not fetch posts; it walks the list WordPress already found.**

2 **The four functions (2m).** "Thin wrappers around the main query."

```
grep -n "^function have_posts\|^function the_post\|^function rewind_posts\|^fu
```

```
961:function have_posts() {
984:function in_the_loop() {
1001:function rewind_posts() {
1018:function the_post() {
```

have_posts() asks `$wp_query`; the_post() moves on and calls setup_postdata(), which sets the global `$post` every template tag reads. Say the warning out loud: a Loop without the_post() never ends (30 seconds, then a fatal timeout).

3. **Template tags (4m).** Save the lesson's second index.php (an article per post). Predict: which tags print, which return?

```
<?php
/**
 * The main template file. For now it does everything itself.
 *
 * @package pagesco-classic
 */

?>
<!DOCTYPE html>
<html <?php language_attributes(); ?>>
<head>
    <meta charset="<?php bloginfo( 'charset' ); ?>">
    <title><?php echo esc_html( wp_get_document_title() ); ?></title>
    <?php wp_head(); ?>
</head>
<body <?php body_class(); ?>>
<?php wp_body_open(); ?>
<main>
    <?php
    if ( have_posts() ) {
        while ( have_posts() ) {
            the_post();
            ?>
            <article id="post-<?php the_ID(); ?>" <?php post_class(); ?>>
                <h2><a href="<?php the_permalink(); ?>"><?php the_title(); ?><
                <p><?php echo esc_html( get_the_date() ); ?> by <?php the_auth
                <?php the_excerpt(); ?>
            </article>
            <?php
        }
    } else {
        echo '<p>Nothing here yet.</p>';
    }
    ?>
</main>
<?php wp_footer(); ?>
</body>
</html>
```

```
<article id="post-16" class="post-16 post type-post status-publish format-star
                <h2><a href="http://localhost:8080/staff-picks/">Staff picks f
                <p>September 15, 2026 by Devon Park</p>
                <p>Three books we keep pressing into customers&#8217; hands th
            </article>
```

the_ID(), post_class(), the_permalink(), the_title(), the_author() and the_excerpt() print; get_the_date() returns, so it is escaped before echo. The excerpt skipped the list of three books: excerpt_remove_blocks() counts only some blocks as text. The get_ half works outside the Loop with a post ID:

```
wp eval 'echo get_the_title( 16 ), "\n", get_permalink( 16 ), "\n", get_the_da
```

```
Staff picks for the season
http://localhost:8080/staff-picks/
2026-09-15
<a href="http://localhost:8080/category/news/" rel="category tag">News</a>
```

4 **Header, footer and parts (5m).** Save the lesson's header.php, footer.php, template-parts/content.php (heading level from `$args`, absint()) and content-none.php, then the third index.php that loads them. The same article, now from the part:

```
<article id="post-16" class="post-16 post type-post status-publish format-star
    <h2 class="entry-title"><a href="http://localhost:8080/staff-picks/">Staff
            <time datetime="2026-09-15T10:00:00+00:00">September 15, 2026</tim
        <div class="entry-summary">
        <p>Three books we keep pressing into customers&#8217; hands this seaso
    </div>
</article>
```

Point at the `<time datetime>` and the one translatable sentence. All of these go through locate_template(): a child theme could replace any part (Lesson 3.4).

5 **Conditional tags, per URL (5m).** Save the lesson's conditions tool as a must-use plugin (it logs which tags are true, on template_redirect), request seven addresses and read the log:

```
for url in / /book-club-dune/ /about/ /category/news/ /author/maria/ "/?s=dune
```

```
[26-Oct-2026 13:24:00 UTC] / -> is_front_page, is_home
[26-Oct-2026 13:24:00 UTC] /book-club-dune/ -> is_singular, is_single
[26-Oct-2026 13:24:00 UTC] /about/ -> is_singular, is_page
[26-Oct-2026 13:24:00 UTC] /category/news/ -> is_archive, is_category
[26-Oct-2026 13:24:00 UTC] /author/maria/ -> is_archive, is_author
[26-Oct-2026 13:24:00 UTC] /?s=dune -> is_search
[26-Oct-2026 13:24:00 UTC] /nothing-here/ -> is_404
```

Home is both is_front_page and is_home; a post is singular and single, a page singular and page; archives are archive plus their kind. In index.php those answers choose the heading:

```
curl -s http://localhost:8080/ | grep -o '<h1>.*</h1>'
curl -s http://localhost:8080/category/news/ | grep -o '<h1>.*</h1>'
curl -s "http://localhost:8080/?s=dune" | grep -o '<h1>.*</h1>'
rm wp-content/mu-plugins/pagesco-conditions.php
```

```
<h1>News from the shop</h1>
<h1>Category: <span>News</span></h1>
<h1>Search results for: dune</h1>
```

6 **Mistake 1: a tag that prints, inside a string (4m).** "A scratch template that the hierarchy picks for one URL only" (category-news.php beats index.php for /category/news/). Save the lesson's first version:

```
<?php
get_header();

while ( have_posts() ) {
    the_post();
    echo '<h2 class="mistake">' . the_title() . '</h2>';
}

get_footer();
```

```
<main id="main" class="site-main">
Book club: Dune<h2 class="mistake"></h2>Staff picks for the season<h2 class="r
```

Each title BEFORE an empty heading: PHP evaluates the string first, the_title() prints on the spot and returns null. Fix: get_the_title() inside strings, or the_title($before, $after). Delete the scratch file (`rm wp-content/themes/pagesco-classic/category-news.php`).

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Concatenating a tag that prints.** Coach: "Does this function print or return? Then what does the string contain?" (Self-check Q3; `debug_loop_report.php`.)
- **Forgetting the_post().** The page hangs for 30 seconds. Coach: "Who moves the list on?" (Self-check Q6.)
- **Conditional tags too early.** In a must-use plugin's top level, or in functions.php outside a callback. Coach: "Has the main query run yet? Which hook comes after it?" (Self-check Q5.)

GUIDED LAB

Folder: `labs/module-06/l4_1-loop-template-tags/shelf_theme_starter/` with `labs/module-06/l4_1-loop-template-tags/test_shelf_theme.sh`; your reference is `labs/module-06/l4_1-loop-template-tags/shelf_theme_solution/`. A NEW classic theme with one template, index.php, that must draw five different addresses: the Loop, template tags (printed and returned, escaped), a title attribute, a heading picked by conditional tags, and an empty-list message. It works on any site from Lesson 3.4 on, whatever learners have of pagesco-classic yet.

- **Install and run as shipped (3m).** `cp -r labs/module-06/l4_1-loop-template-tags/shelf_theme_starter wordpress/wp-content/themes/pagesco-lab-shelf`, `wp theme activate pagesco-lab-shelf`, copy the test into `wordpress/`, `docker compose run --rm wpcli sh test_shelf_theme.sh`: every page says TODO.
- **TODOs 1 to 5 in pairs (14m), the test after each.** Circulate for: `echo get_the_date()` without esc_html(); the_title() inside the href string (the concatenation mistake again); the heading computed after the Loop (it must be decided before the HTML); single_cat_title() without its second argument false (it prints instead of returning).
- **Done when** the test prints (the solution):

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Shelf' theme.
/category/news/   Shelf: News
    Book club: Dune · 2026-10-01 · Maria Lopez
    Staff picks for the season · 2026-09-15 · Devon Park
    Welcome to the new Pages &amp;#038; Co. website · 2026-09-01 · Maria Lopez
/author/devon/    Archive
    Staff picks for the season · 2026-09-15 · Devon Park
/?s=dune          Found for: dune
    Book club: Dune · 2026-10-01 · Maria Lopez
    Staff picks for the season · 2026-09-15 · Devon Park
/?s=zzz           Found for: zzz
    Nothing on this shelf.
/book-club-dune/  Book club: Dune
    Book club: Dune · 2026-10-01 · Maria Lopez
the first link: <li class="shelf-item"><a href="http://localhost:8080/book-club-d
```

- **Debrief (2m):** "One file, five kinds of page: what did the work?" (The main query; the Loop only walked it.) "Why does /author/devon/ say Archive?" (No is_author() branch: the learners' own Exercise 1

material.) Then `wp theme activate pagesco` (or pagesco-classic) and remove the folder.

- **Fast finishers:** `labs/module-06/l4_1-loop-template-tags/template_tags_starter.php` (every the_/get_ pair on one post, outside the Loop), then `labs/module-06/l4_1-loop-template-tags/debug_loop_report.php` (a real main query with wp('s=dune'), three quiet and loud bugs).

START THE LESSON PROJECT

The lesson project is **Finish pagesco-classic** (about 40 minutes alone): singular.php, 404.php, a theme.json that makes it hybrid, and the per-URL check. In the room: Step 1 (15 minutes).

- Step 1: singular.php from the lesson: one template for posts and pages, `is_singular( 'post' )` around the post-only parts (date and author, categories, the_post_navigation()). Check /book-club-dune/ (Filed under: News, Staff picks before, Hello world! after) and /about/ (the synced opening hours render because the_content() runs the blocks).
- **Done in the room** = both pages correct. **Goes home:** Step 2 (404.php), Step 3 (theme.json: hybrid, and why the notice bar stays unstyled until Lesson 4.3), Step 4 (the per-URL loop) and the pagesco-plan.txt line. pagesco-classic stays active.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Who decides which posts a classic template shows? **Answer:** the main query, before the template runs.
- the_title() or get_the_title() inside `echo '<h2>' . ... . '</h2>'`? **Answer:** get_the_title(), escaped.
- Which file loads `get_template_part( 'template-parts/content', 'none' )` first? **Answer:** content-none.php (the lesson quiz Q5).
- Why can conditional tags not be used at the top of a must-use plugin? **Answer:** the main query has not run; they return false and WordPress says so.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** B) `index.php`, with a Loop over Maria's posts; **Q2** `bool(false)` and `string(0) ""`: WP-CLI loads WordPress but runs no main query, so is_home() has nothing to answer from and there is no current post for get_the_title(); **Q3** D) `title="<?php the_title_attribute(); ?>"` and `the_title( '<h2>', '</h2>' );`; **Q4** A) It looked for template-parts/content.php.php, found nothing and returned false; **Q5** C) `template_redirect`; **Q6** The main query holds exactly one post, and the_post() is what sets the global $post that every template tag reads (and fires loop_start / loop_end for plugins). Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners who are new to PHP templates work TODOs 1 and 2 of the Shelf lab only, with the lesson's second index.php open as a model; the heading can stay "TODO". The workbook's worked example shows

the body classes per URL.

Stretch: the debug handout; the lesson's Exercise 3 (category.php with found_posts and _n()) and Exercise 4 (two passes with rewind_posts()); the Bonus Challenge (the get_template_part_{$slug} action).

4.2 Custom Queries

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Tell the main query from secondary queries, and pick WP_Query, get_posts() or pre_get_posts for a task
- Write WP_Query arguments for post types, categories, authors, dates, searches, ordering and limits, and read found_posts and max_num_pages
- Run a secondary Loop on the same page as the main one and restore the current post with wp_reset_postdata()
- Change the main query with a pre_get_posts callback guarded by is_main_query() and is_admin()
- Paginate lists with the_posts_pagination(), and explain why paging a custom query in a template leads to a 404

TIMING IN THE ROOM

10m	Frame: the customer's order and the shop's displays
25m	Live demo: a secondary query, the reset, pre_get_posts, pagination, two mistakes
20m	Guided lab: `query_shelf_starter.php`
15m	Start the lesson project: Step 1, front-page.php
10m	Which query? (the decision table as a sort)

MENTAL MODEL TO SAY

"The main query is the customer's order at the counter: they asked for News, so they get News, and the whole page is organized around it. Secondary queries are the displays next to the counter, staff picks, you might also like; they never change the order. And when you want the order itself to be different, news in bundles of three, you do not grab the receipt after it is printed; you tell the person taking orders before they write it down. That is pre_get_posts."

Then the decision table on the board, with one question per row: **does it change the page's own list?** pre_get_posts. **Is it an extra list with template tags?** new WP_Query, then wp_reset_postdata(). **Do you only need the data?** get_posts(). query_posts() gets a line through it.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Starts from the site at the end of Lesson 4.1 (pagesco-classic active). Same shell as in 4.1.

1. **A secondary query and its three numbers (5m).** "For each request there is exactly one main query; every other list is a secondary query your code creates." Save the lesson's `query-news.php` and run it:

```php
<?php
$pagesco_query = new WP_Query(
    array(
        'category_name'  => 'news',
        'posts_per_page' => 2,
        'orderby'        => 'title',
        'order'          => 'ASC',
    )
);

printf(
    "found_posts: %d, max_num_pages: %d, post_count: %d\n",
    $pagesco_query->found_posts,
    $pagesco_query->max_num_pages,
    $pagesco_query->post_count
);

while ( $pagesco_query->have_posts() ) {
    $pagesco_query->the_post();
    echo get_the_ID(), '  ', get_the_title(), "\n";
}

wp_reset_postdata();
```

```
found_posts: 3, max_num_pages: 2, post_count: 2
17  Book club: Dune
16  Staff picks for the season
```

post_count (this page), found_posts (the total), max_num_pages: the three numbers pagination runs on. Every call goes to YOUR object (`$pagesco_query->have_posts()`); bare have_posts() would ask the main query. And the Loop ends with wp_reset_postdata():

```
grep -n "^function wp_reset_postdata" -A 7 wp-includes/query.php
```

```
127:function wp_reset_postdata() {
128-    global $wp_query;
129-
130-    if ( isset( $wp_query ) ) {
131-        $wp_query->reset_postdata();
132-    }
133-}
134-
```

2 **pre_get_posts: tell the order-taker, before the order is written (6m).** "The shop wants three posts per page on the home page. A WP_Query in the template looks right on page 1 and breaks on page 2." Where can you change the main query? Line 1916 of class-wp-query.php:

```
grep -n "do_action_ref_array( 'pre_get_posts'" wp-includes/class-wp-query.php
```

```
1916:       do_action_ref_array( 'pre_get_posts', array( &$this ) );
```

Save the lesson's first `pagesco-classic/functions.php` (the guarded callback) and count:

```
<?php
/**
 * Pages & Co. Classic functions and definitions.
 *
 * @package pagesco-classic
 */

/**
 * Shows three posts per page on the blog home.
 *
 * @param WP_Query $query The query WordPress is about to run.
 */
function pagesco_classic_adjust_main_query( $query ) {
    if ( is_admin() || ! $query->is_main_query() ) {
        return;
    }

    if ( $query->is_home() ) {
        $query->set( 'posts_per_page', 3 );
    }
}
add_action( 'pre_get_posts', 'pagesco_classic_adjust_main_query' );
```

```
curl -s http://localhost:8080/ | grep -o '<h2 class="entry-title"><a [^>]*>[^<
curl -s http://localhost:8080/category/news/ | grep -c '<h2 class="entry-title
```

```
Hello world!
Book club: Dune
Staff picks for the season
3
```

Three on the home page; the News archive still has its three (only `$query->is_home()` changed). Read the three guards aloud: is_admin(), is_main_query(), the query's own is_home(). It is an action:

the object is changed in place.

3. **Pagination (4m).** Add the_posts_pagination() to index.php after the Loop (the lesson's version) and look at what it printed:

```
<nav class="navigation pagination" aria-label="Posts pagination">
        <h2 class="screen-reader-text">Posts pagination</h2>
        <div class="nav-links"><span aria-current="page" class="page-numbers c
<a class="page-numbers" href="http://localhost:8080/page/2/">2</a>
<a class="next page-numbers" href="http://localhost:8080/page/2/">Next</a></d
    </nav>
```

aria-current on page 1, a visually hidden heading for screen readers (styled in Lesson 4.3). Follow it, and try one page too far:

```
curl -s http://localhost:8080/page/2/ | grep -o '<title>.*</title>\|<h2 class=
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/page/3/
```

```
<title>Pages &amp;amp; Co. &amp;#8211; Page 2</title>
<h2 class="entry-title"><a href="http://localhost:8080/welcome/">Welcome to th
404
```

4. **Mistake 1: paging a custom query in the template (5m).** Save the lesson's category-news.php, which ignores the main query and pages its own (two per page):

```
curl -s http://localhost:8080/category/news/ | grep -o '<h2 class="entry-title
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/category/news/p
```

```
<a href="http://localhost:8080/book-club-dune/">Book club: Dune
<a href="http://localhost:8080/staff-picks/">Staff picks for the season
<a class="page-numbers" href="http://localhost:8080/category/news/page/2/">2
<a class="next page-numbers" href="http://localhost:8080/category/news/page/2/
404
```

Page 1 looks perfect; its link answers 404. Before any template runs, WordPress runs the MAIN query for /category/news/page/2/: ten per page, three posts, no page 2, so 404.php. The fix is pre_get_posts (the lesson's Going Further). `rm` the file.

5. **Mistake 3: a callback without guards (4m).** Save the lesson's mistake-everywhere.php (no checks at all, one post per query) and ask WP-CLI and get_posts():

```
wp post list --post_type=page --orderby=ID --order=ASC --fields=ID,post_title
wp eval 'echo count( get_posts( array( "post_type" => "page", "numberposts" =>
```

```
+----+-------------+
| ID | post_title  |
+----+-------------+
| 2  | Sample Page |
+----+-------------+
1
```

Four published pages, one listed; get_posts() asked for all and got one. The callback changed every query on the site, the admin's included. Delete it and query-news.php.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A custom query where the main query should change.** "Pages of three" done in the template; page 2 404s. Coach: "Which query does WordPress use to decide that page 2 exists?" (Self-check Q8 and Q12; the lesson quiz Q1.)
- **No wp_reset_postdata().** The related box and the wrong navigation. Coach: "Which post is the global $post right now?" (Self-check Q10; `debug_related_box.php`.)
- **Argument names of the wrong kind.** cat with a slug, author with a login, p with a slug: no warning, too much data. Coach: "Slug or ID? Check the argument table." (The lesson's Common Mistakes 4; the guided lab's TODO 1.)

GUIDED LAB

File: `labs/module-06/l4_2-custom-queries/query_shelf_starter.php`; your reference is `labs/module-06/l4_2-custom-queries/query_shelf_solution.php`. Five NEW questions from the shop, each one argument array (the lesson's argument table): an author by login, a category with paging, a date_query with `before`, pages by slug in title order, and "how many" with `fields => ids`. The helper prints post_count, found_posts and max_num_pages for each.

- **Run it as shipped (2m).** `cp labs/module-06/l4_2-custom-queries/query_shelf_starter.php wordpress/`, then `wp eval-file --use-include query_shelf_starter.php`: five TODO lines.
- **TODOs 1 to 5 in pairs (15m).** Circulate for: `'author' => 'maria'` (an ID argument with a login: everything comes back), `'page' => 2` instead of `'paged'`, a date_query that is not an array of arrays, `post_name__in` with an orderby that forgets the order.
- **Done when** it prints (the solution):

```
Maria's posts:     Book club: Dune, Welcome to the new Pages &#038; Co. website
news, page 2:      Welcome to the new Pages &#038; Co. website  (post_count 1, fo
before October:    Welcome to the new Pages &#038; Co. website, Staff picks for t
pages, A to Z:     About Us, Contact, Events  (post_count 3, found_posts 3, max_n
how many news?     Book club: Dune  (post_count 1, found_posts 3, max_num_pages 3
```

- **Debrief (3m):** the last line. "It loaded one post. Why does found_posts say 3, and max_num_pages 3?" (The counting query ran; three pages of one.) "What would no_found_rows have changed?" (found_posts 0: self-check Q7.)
- **Fast finishers:** the folder lab `labs/module-06/l4_2-custom-queries/archive_rules_starter/` (a guarded pre_get_posts in a must-use plugin, tested with `labs/module-06/l4_2-custom-queries/test_archive_rules.sh`), then `labs/module-06/l4_2-custom-queries/debug_related_box.php`.

START THE LESSON PROJECT

The lesson project is **A Front Page with Several Loops** (about 40 minutes alone). In the room: Step 1 (15 minutes).

- Step 1: front-page.php from the lesson: the hero, a secondary WP_Query for the three pages (post_name__in, no_found_rows, wp_reset_postdata()), then the MAIN Loop for the news, three per page thanks to the callback, with h3 headings via the part's `$args`. Check / and /page/2/ with the lesson's commands.
- **Done in the room** = page 1 with Visit us and three news items, page 2 with Welcome. **Goes home:** Step 2 (the corrected related box: the reset) and Step 3 (the check), and the pagesco-plan.txt line.

WHICH QUERY? (10M)

Read each request; learners answer P (pre_get_posts), Q (new WP_Query + reset) or G (get_posts()), then one says why.

1. "News in pages of three on the blog home." **P**: the page's own list; pagination agrees.
2. "Three links to our pages above the news." **Q**: an extra list with template tags (the project).
3. "An array of page URLs for a sitemap I build in PHP." **G**: data, no template tags.
4. "Search results: posts only." **P**: the lesson's Exercise 1.
5. "Other posts from this category under each post." **Q**: the related box, with its reset.
6. "Count the News posts for a dashboard number." **Q** or **G** with fields ids; found_posts, or count() of the array.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- What do post_count, found_posts and max_num_pages mean? **Answer:** on this page; in total; pages of this size.
- Why the guard `$query->is_main_query()`? **Answer:** the callback runs for every query on the site.
- /page/3/ answers 404 before your template runs. Who decided? **Answer:** WordPress, from the main query's result.
- After get_posts(), do you reset? **Answer:** no, as long as you pass each post along.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** `2 0`: two pages loaded; found_posts is 0 because no_found_rows skips the counting query; **Q8** B) Every other WP_Query that runs on the blog home (a box, a widget, a block) is also cut to three posts; **Q9** A) 1, 7, 3; **Q10** D) After `get_posts()`, when each WP_Post is passed to the functions you call (`get_the_title( $post )`); **Q11** C) If it changes the page's main list: pre_get_posts on the main query (category_name, posts_per_page); if it adds a list: new WP_Query plus wp_reset_postdata(); **Q12** (a) a secondary WP_Query (or get_posts()) with posts_per_page 3, plus wp_reset_postdata() after a Loop; (b) pre_get_posts on the main query (is_post_type_archive('book'), posts_per_page 12). Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners stuck on argument syntax do TODOs 1, 2 and 4 of the lab only, with the lesson's argument table side by side; TODOs 3 and 5 go home.

Stretch: the archive-rules folder lab and the debug handout; the lesson's Exercise 4 (sticky posts and pagination) and the Bonus Challenge (two News posts per page the right way, then decide whether the shop wants it).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Question 3's pile tells you whether Session 7's warm-up needs the order-taker picture again.

1. In one sentence: what does the Loop do, and what does the_post() do inside it? **Answer:** it walks the posts the main query already found; the_post() moves to the next one and sets the global $post the template tags read.
2. Name a template tag pair and say when you use each half. **Answer:** for example the_title() / get_the_title(): the_ prints HTML-ready text; get_ returns a value you escape yourself, for attributes, strings or further processing.
3. The blog home should show three posts per page. Where does that change go, and which two guards must it have? **Answer:** a pre_get_posts callback (functions.php or a plugin) with is_admin() / $query->is_main_query(), then $query->is_home().

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish Lesson 4.1:** Common Mistakes 2 to 4, the project Steps 2 to 4 (404.php, theme.json, the per-URL check).
- **Finish Lesson 4.2:** query-args.php, Common Mistakes 2 and 4, the project Steps 2 and 3 (the related box with wp_reset_postdata(), the check).
- **Lesson exercises.** 4.1: Exercises 1 to 3 (Exercise 4 is a stretch). 4.2: Exercises 1 to 3 (Exercise 4 is a stretch). Each removes its files; sticky_posts back to [].
- **Both Quick Quizzes** on the lesson pages (answers in the Answer Key).
- **Journal:** both entries. The 4.2 prompt (three book lists for Module 5) is the plan you will use in Lesson 5.2.
- **Labs not reached:** `labs/module-06/l4_1-loop-template-tags/template_tags_starter.php`, `labs/module-06/l4_2-custom-queries/archive_rules_starter/`, the two debug handouts.

BRING LIST FOR SESSION 7

- The course site at the end of Lesson 4.2: pagesco-classic active with functions.php, front-page.php and template-parts/related-posts.php; mu-plugins/ holds only mailpit-smtp.php and pagesco-site.php; sticky_posts is [].
- The project folder with `labs/`; pagesco-plan.txt. Session 7 finishes pagesco-classic (Lesson 4.3) and starts the Part 1 project: think of one question you would ask a new client before building their theme.
- The journal and the workbook.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** index.php drawing every list; printed vs returned; pre_get_posts with guards; the paged custom query's 404.
- **Cut first (saves 5m):** the parts step of the 4.1 demo; show the finished index.php and move on.
- **Cut second (saves 10m):** the "Which query" sort; self-check Q12 and the 4.2 journal cover it.
- **Cut third (saves 15m):** the 4.2 project start; the guided lab already practiced the arguments.

RUNNING SHORT: WHAT TO ADD

- Run `debug_related_box.php` for everyone and fix it together; the reset bug only appears after the loud one is fixed.
- The 4.2 Common Mistake 2 live (the related box without the reset and the wrong navigation), then the fix.
- The 4.1 Exercise 3 live: category.php with found_posts and _n(), for News and Uncategorized.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 7: functions.php: Supports, Assets, Menus & Widgets

How to finish Module 4 and open the Part 1 project. Finish pagesco-classic with functions.php: theme supports, a versioned stylesheet and a deferred script, a menu location and a widget area filled from WP-CLI, accessibility and translation, then hand the site back to pagesco. After the break, brief the Part 1 project, a block theme for the Fernhill Community Garden, and get every learner through Milestone 1 and into Milestone 2.

`1 lesson + project kickoff` `3 hrs session` `Lesson 4.3` `Session 7 of 18 · Part 1`

HOW TO USE THIS GUIDE

- Session 7 has two halves. Before the break, Lesson 4.3 in the usual rhythm: frame, live demo, guided lab, start the project (its Step 3 switches the site back to pagesco). After the break, 15 more minutes of 4.3 (accessibility, translation, Common Mistake 1), then the **Part 1 project** kickoff and the first studio time.
- Type, do not paste. Every command below was run to produce the output printed under it, on WordPress 7.1.3 and PHP 8.5: for 4.3 a course site exactly as Lesson 4.2 leaves it (pagesco-classic active), for the project the site at the end of Lesson 4.3 (pagesco active), the state Session 8 and Part 2 assume.
- The Part 1 project is the Session 8 materials': the brief (learners' copy `labs/module-08/p1-project/BRIEF.md`), the starter `fernhill_starter/`, the self-check `test_fernhill.php`, the model solution and the rubric (the Part 1 section of `final-assessment.html`). This guide runs its kickoff; Session 8's guide runs the studio, the timed assessment and the review.
- Labs for 4.3 live in `labs/module-07/`; the project's in `labs/module-08/p1-project/`. `_starter` files and folders are handouts, `_solution` ones (and `fernhill_solution/`) are yours.

> **THE #1 RISK OF SESSION 7: LEAVING ON THE WRONG THEME**
>
> Today learners switch themes four times: pagesco-classic all through 4.3, the Reading Corner lab theme, back to pagesco at the end of the lesson, then fernhill for the project. A learner who goes home on fernhill or pagesco-classic starts Session 8 (and later Lesson 5.1) with the wrong site, and every output differs. **In the room:** Step 3 of the 4.3 project is not optional and happens before the break; the check-in at 2:40 ends with `wp theme activate pagesco` on every screen. **At home:** the brief's rules repeat it.

BEFORE THE SESSION

- ☐ **Learner sites:** at the end of Lesson 4.2: pagesco-classic active with functions.php, front-page.php and related-posts.php; sticky_posts empty (the warm-up check).
- ☐ **Run the kit checks once:** `python3 labs/check_labs.py module-07` (9 results in about a minute) and `python3 labs/check_labs.py module-08` for the project's starter and model solution (Session 8's labs). On your own project: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-07`.
- ☐ **Read the Part 1 project yourself:** `labs/module-08/p1-project/BRIEF.md`, the starter's TODOs, and the rubric in `final-assessment.html`. Decide the hand-in address and the due date (one week after Session 8) before class.
- ☐ **Projector machine:** a course site at the end of Lesson 4.2 for the demo (it ends at the end of 4.3, ready for the kickoff), VS Code on the themes folder, two terminals, the browser on http://localhost:8080/ with View Source, `lesson_13.html` in a tab.
- ☐ **Files to hand out:** `labs/module-07/` without the `*_solution*` files and folders; from `labs/module-08/p1-project/`: BRIEF.md, fernhill_starter/, test_fernhill.php and fernhill_front.png (NOT fernhill_solution/); the workbook.
- ☐ **Board, before learners arrive:** the theme hook timeline (functions.php included → after_setup_theme: supports, menus → init / widgets_init → query, template → wp_head: wp_enqueue_scripts at 1, styles at 8 → wp_footer: scripts at 20); for after the break, R1 to R8 with an empty "Module" column.

KNOWN TRAPS TODAY

SYMPTOM	CAUSE	FIX
Two `<title>` elements	title-tag support plus the hand-written <title> of Lesson 4.1	Delete the line in header.php (the lesson's sed).
`Notice: Function add_theme_support( 'html5' ) was called incorrectly. You need to pass an array of types.`	add_theme_support('html5') without the array	add_theme_support('html5', array('search-form', ...)).
CSS changes do not show for visitors	No version, or a typed one: ?ver=7.1.3 or a stale number	The theme's Version, raised on every release.
The header lists every page, Sample Page included	A typo in theme_location, or no menu assigned, without has_nav_menu()	Copy the location id; guard with has_nav_menu().
`register_sidebar was called incorrectly ... Defaulting to "sidebar-1"`	register_sidebar() without an id	A fixed id such as footer, never changed.

SYMPTOM	CAUSE	FIX
Three "called incorrectly" notices at the top of every page	wp_enqueue_style() at the top level of a file	A callback on wp_enqueue_scripts; read the first notice.
Fernhill: `Checks passed: 0 of 12` or a decode notice	A trailing comma in theme.json	Lesson 3.2 Mistake 1: remove it; the self-check's line 2.
Learners still on fernhill next week	No switch back at the end of the day	`wp theme activate pagesco`.

SESSION 7 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 6 (the Loop, template tags, queries, pre_get_posts), the readiness check, homework questions.
0:10	10	**4.3** Frame and mental model	The fit-out instructions; the theme hook timeline.
0:20	25	**4.3** Live demo	functions.php in WP-CLI, title-tag and html5, two titles, what block themes get for free, a versioned stylesheet and a deferred script, a menu location filled from WP-CLI, a widget area.
0:45	20	**4.3** Guided lab	`labs/module-07/l4_3-functions-php/corner_theme_starter/`: a classic theme whose functions.php is empty.
1:05	15	**4.3** Start the lesson project	Step 1 (style.css 1.1.0) and Step 3 (pagesco's functions.php, back to pagesco); Step 2's check goes home.
1:20	10	Break	Everyone on pagesco now: the project starts from the end of Lesson 4.3.
1:30	15	**4.3** Accessibility, translation and Mistake 1	The skip link and screen-reader text, the Spanish translation file, a top-level enqueue and its three notices.
1:45	10	Part 1 project kickoff: the brief	Odette Marsh's email, R1 to R8 and the module each comes from, the rules, the rubric's Met line.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:55	10	Part 1 project studio: Milestone 1	Everyone copies the starter, activates fernhill, runs the self-check: `Checks passed: 1 of 12`; a tour of the starter.
2:05	20	Part 1 project studio: Milestone 2 started	The style.css header, the palette and layout in theme.json; the self-check after each save.
2:25	15	Part 1 project studio: plan Milestone 3	On paper: the template map, the two parts and their landmarks, the 404 page; what goes home.
2:40	10	Studio check-in	Self-check lines on the board, due dates (M2 and M3 before Session 8), and wp theme activate pagesco.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 8.
Total	180		Ends at 3:00.

In the room. Lesson 4.3 is estimated at 140 minutes alone; the room gives it 85 (70 before the break, 15 after). Kept: functions.php's role and timing, theme supports, enqueueing with versions and strategies, menus and widget areas with WP-CLI, the accessibility checklist and the translation demo, Common Mistake 1, the project's Steps 1 and 3. Sent home: Common Mistakes 2 to 4, the project's Step 2 (the per-URL check), the exercises. The Part 1 project gets 65 minutes: the brief, Milestone 1 for everyone, Milestone 2 started, Milestone 3 planned on paper, a check-in.

Goes home (about 2 to 3 hours). 4.3: Common Mistakes 2 to 4, the project's Step 2, Exercises 1 to 3, the Quick Quiz, the journal. **The Part 1 project:** Milestone 2 finished and Milestone 3 (templates and parts, the 404 page) before Session 8, about 1 hour 45 minutes; bring the theme and the last self-check output.

Protect, in this order: the switch back to pagesco (Step 3) before the break; Milestone 1 on every machine (`Checks passed: 1 of 12`); the enqueue with its version; the menu location and has_nav_menu(). **Cut first:** the block-theme defaults step and the widget step of the demo (the lab covers widget areas), then the translation demo after the break (keep the skip link), then the M3 planning segment (it becomes homework).

OPENING SEGMENT

0:00 Warm-up and Readiness Check — 10 min

TIMING

6m	Five retrieval questions on Session 6 (W1 to W5, on the slide)
4m	Homework questions; who is not on pagesco-classic (W5)?

THE READINESS CHECK (W5): THE TWO COMMANDS

On the projector's site:

```
wp theme list --status=active --field=name
curl -s http://localhost:8080/ | grep -c 'class="entry-title"'
```

```
pagesco-classic
3
```

Another theme active: `wp theme activate pagesco-classic`. A number other than 3: the 4.2 project's functions.php is missing; copy it from `lesson_12.html` now.

WARM-UP ANSWERS

1. **W1** What does the Loop do, and what does the_post() do inside it? **Answer:** It walks the posts the main query already found; the_post() moves to the next one and sets the global $post (Lesson 4.1).
2. **W2** the_title() or get_the_title() inside echo '<h2>' . ... . '</h2>'? **Answer:** get_the_title(), escaped: the_title() prints on the spot and leaves an empty heading.
3. **W3** posts_per_page 2, three matches: post_count, found_posts, max_num_pages on page 1? **Answer:** 2, 3, 2 (Lesson 4.2).
4. **W4** Name the three guards of a pre_get_posts callback that changes the blog home. **Answer:** is_admin(), $query->is_main_query(), $query->is_home().
5. **W5** Run the two commands on the slide. What should a site ready for today print? **Answer:** pagesco-classic (active since Lesson 4.1), then 3: three news items on the home page (the 4.2 callback).

LESSON PLAN

4.3 functions.php: Supports, Assets, Menus & Widgets

85 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain when functions.php runs and choose the right hook for theme setup, widgets and assets
- Declare theme supports such as title-tag and html5 with add_theme_support(), and say which ones block themes get automatically
- Enqueue a stylesheet and a deferred script with wp_enqueue_style() and wp_enqueue_script(), with dependencies and a version that busts browser caches
- Register a menu location and a widget area, print them with wp_nav_menu() and dynamic_sidebar(), and fill them from WP-CLI
- Add the accessibility and translation basics every theme needs: a skip link, screen-reader text, labeled navigation and strings WordPress can translate

TIMING IN THE ROOM

10m	Frame: the fit-out instructions; the hook timeline
25m	Live demo: supports, two titles, block-theme defaults, the stylesheet and script, the menu, the widget area
20m	Guided lab: `corner_theme_starter/`
15m	Start the lesson project: Step 1 and Step 3
15m	After the break: accessibility, translation, Common Mistake 1

MENTAL MODEL TO SAY

"If the templates are the floor plans of the shop, functions.php is the sheet of instructions the theme hands the building manager when it moves in: we need the shop sign handled by you (theme supports), please deliver these crates of paint and these tools (stylesheets and scripts), there will be a directory board here and a notice board there (a menu location and a widget area). The manager, WordPress, does the actual work at the right moment of each day."

Then the timeline on the board, and the convention: **one small, prefixed function per job, each hooked to its moment.** "That is the real reason a classic template must call wp_head() and wp_footer(): without them, nobody can add a single stylesheet or script."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Starts from the end of Lesson 4.2 (pagesco-classic active). Run commands in the WP-CLI container shell; save files in VS Code from the lesson page.

1. **functions.php runs everywhere (2m).** "It is not a template: WordPress includes it on every request while the theme is active, WP-CLI included."

```
wp eval 'var_dump( function_exists( "pagesco_classic_adjust_main_query" ), did
```

```
bool(true)
int(1)
```

Your 4.2 function exists in a WP-CLI command, and after_setup_theme has fired once. So: almost everything in functions.php belongs in a callback on the right hook.

2. **Theme supports (5m).** Look at the search form a classic theme gets by default:

```
wp eval 'echo get_search_form( array( "echo" => false ) );'
```

```
<form role="search" method="get" id="searchform" class="searchform" action="ht
                <div>
                    <label class="screen-reader-text" for="s">Search for:</lab
                    <input type="text" value="" name="s" id="s" />
                    <input type="submit" id="searchsubmit" value="Search" />
                </div>
            </form>
```

The xhtml form of the early 2000s. Replace functions.php with the lesson's version (pagesco_classic_setup() on after_setup_theme: title-tag, automatic-feed-links, post-thumbnails, responsive-embeds, html5), then count the titles:

```php
<?php
/**
 * Pages & Co. Classic functions and definitions.
 *
 * @package pagesco-classic
 */

/**
 * Sets up translations and the theme features WordPress should provide.
 */
function pagesco_classic_setup() {
    load_theme_textdomain( 'pagesco-classic', get_template_directory() . '/lan

    add_theme_support( 'title-tag' );
    add_theme_support( 'automatic-feed-links' );
    add_theme_support( 'post-thumbnails' );
    add_theme_support( 'responsive-embeds' );
    add_theme_support(
        'html5',
        array( 'search-form', 'comment-form', 'comment-list', 'gallery', 'capt
    );
}
add_action( 'after_setup_theme', 'pagesco_classic_setup' );

/**
 * Shows three posts per page on the blog home.
 *
 * @param WP_Query $query The query WordPress is about to run.
 */
function pagesco_classic_adjust_main_query( $query ) {
    if ( is_admin() || ! $query->is_main_query() ) {
        return;
    }

    if ( $query->is_home() ) {
        $query->set( 'posts_per_page', 3 );
    }
}
add_action( 'pre_get_posts', 'pagesco_classic_adjust_main_query' );
```

```
curl -s http://localhost:8080/about/ | grep '<title>'
```

```
    <title>About Us &#8211; Pages &amp; Co.</title>
    <title>About Us &#8211; Pages &amp; Co.</title>
```

Two: WordPress's, and the one header.php still writes. Delete the hand-written line:

```
sed -i '/<title>/d' wp-content/themes/pagesco-classic/header.php
curl -s http://localhost:8080/about/ | grep '<title>'
```

```
    <title>About Us &#8211; Pages &amp; Co.</title>
```

And the search form is HTML5 now (role, a label, `type="search"`, a screen-reader-text span):

```
<form role="search" method="get" class="search-form" action="http://localhost
                <label>
                    <span class="screen-reader-text">Search for:</span>
                    <input type="search" class="search-field" placeholder="Sea
                </label>
                <input type="submit" class="search-submit" value="Search" />
            </form>
```

3 **What block themes get for free (2m).** "pagesco never declared a support, yet it had a title and an HTML5 form."

```
grep -n -A 14 "^function _add_default_theme_supports" wp-includes/theme.php |
```

```
4400-   add_theme_support( 'post-thumbnails' );
4401-   add_theme_support( 'responsive-embeds' );
4402-   add_theme_support( 'editor-styles' );
4408-   add_theme_support( 'html5', array( 'comment-form', 'comment-list', 'se
4409-   add_theme_support( 'automatic-feed-links' );
```

And the title is not even on that list: block-template.php prints it for block themes unconditionally.

4 **A versioned stylesheet and a deferred script (5m).** Save the lesson's `assets/js/navigation.js` (it adds the class that hides the menu only once it runs), then append the enqueue callback to functions.php:

```php
/**
 * Loads the theme's stylesheet and its menu script.
 */
function pagesco_classic_enqueue_assets() {
    $version = wp_get_theme( get_template() )->get( 'Version' );

    wp_enqueue_style(
        'pagesco-classic-style',
        get_parent_theme_file_uri( 'style.css' ),
        array(),
        $version
    );

    wp_enqueue_script(
        'pagesco-classic-navigation',
        get_parent_theme_file_uri( 'assets/js/navigation.js' ),
        array(),
        $version,
        array(
            'strategy'  => 'defer',
            'in_footer' => true,
        )
    );
}
add_action( 'wp_enqueue_scripts', 'pagesco_classic_enqueue_assets' );
```

```
curl -s http://localhost:8080/ | grep 'pagesco-classic-'
```

```
<link rel='stylesheet' id='pagesco-classic-style-css' href='http://localhost:8
<script data-wp-strategy="defer" defer id="pagesco-classic-navigation-js" src=
```

Point at three choices: `get_parent_theme_file_uri()` (a child theme would otherwise swap the file), the version from `wp_get_theme( get_template() )`, and `defer` + `in_footer`. `?ver=1.0.0` changes when Step 1 raises the Version.

5 **A menu location, filled from WP-CLI (6m).** Append the lesson's register_nav_menus() callback, save the new header.php (the labeled nav, the toggle button, the skip link), then build the menu as the lesson does:

```
wp menu location list
wp menu create "Main menu"
wp menu item add-post main-menu 12
wp menu item add-post main-menu 14
wp menu item add-post main-menu 13
wp menu location assign main-menu primary
wp menu item list main-menu --fields=db_id,title,link
```

```
+----------+--------------+
| location | description  |
+----------+--------------+
| primary  | Primary menu |
+----------+--------------+
Success: Created menu 4.
Success: Menu item added.
Success: Menu item added.
Success: Menu item added.
Success: Assigned location primary to menu main-menu.
+-------+----------+--------------------------------+
| db_id | title    | link                           |
+-------+----------+--------------------------------+
| 27    | About Us | http://localhost:8080/about/   |
| 28    | Events   | http://localhost:8080/events/  |
| 29    | Contact  | http://localhost:8080/contact/ |
+-------+----------+--------------------------------+
```

The menu is a nav_menu term (4 here), the items nav_menu_item posts, the assignment a theme mod. IDs on learners' sites may differ. On the About page:

```
<nav class="site-nav" aria-label="Primary">
            <button type="button" class="menu-toggle" aria-controls="primary-m
            <ul id="primary-menu" class="menu"><li id="menu-item-27" class="me
<li id="menu-item-28" class="menu-item menu-item-type-post_type menu-item-obje
<li id="menu-item-29" class="menu-item menu-item-type-post_type menu-item-obje
</ul>       </nav>
```

current-menu-item and aria-current for free; Appearance > Menus opens again with this theme.

6 **A widget area (5m).** Append the lesson's register_sidebar() callback (a fixed id: footer), save the new footer.php, add a block widget:

```
wp sidebar list --fields=name,id
wp widget add block footer --content='<!-- wp:paragraph --><p>Open Monday to S
wp widget list footer --fields=name,id,position
```

```
+------------------+---------------------+
| name             | id                  |
+------------------+---------------------+
| Footer           | footer              |
| Inactive Widgets | wp_inactive_widgets |
+------------------+---------------------+
Success: Added widget to sidebar.
+-------+---------+----------+
| name  | id      | position |
+-------+---------+----------+
| block | block-7 | 1        |
+-------+---------+----------+
```

block-7: a fresh install parks five default block widgets (2 to 6) in wp_inactive_widgets. The footer now prints:

```
<div class="footer-widgets">
            <section id="block-7" class="widget widget_block widget_text"><p
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Code at the top level of functions.php.** add_theme_support() or wp_enqueue_style() outside a callback. Coach: "Which moment is this for? Put it in a function on that hook." (Self-check Q1 and Q6.)
- **Typed versions and the wrong URL function.** `'1.0'` copied from a tutorial, get_stylesheet_uri() in a parent. Coach: "What happens on your next release? What happens with a child theme?" (Self-check Q3; the lab's TODO 2.)
- **Locations and areas without fixed ids.** A location typo, register_sidebar() without id. Coach: "Which string does the template use? Copy it." (Self-check Q4; `debug_theme_setup.php`.)

GUIDED LAB

Folder: `labs/module-07/l4_3-functions-php/corner_theme_starter/` with `labs/module-07/l4_3-functions-php/test_corner_theme.sh`; your reference is `labs/module-07/l4_3-functions-php/corner_theme_solution/`. A NEW small classic theme (the shop's reading-corner pages) whose templates are finished and already call wp_nav_menu(), get_search_form() and dynamic_sidebar(); its functions.php is empty. Learners write the four jobs of Lesson 4.3 for a theme that is not pagesco-classic.

- **Install and run as shipped (3m).** `cp -r labs/module-07/l4_3-functions-php/corner_theme_starter wordpress/wp-content/themes/pagesco-lab-corner`, `wp theme activate pagesco-lab-corner`, copy the test into `wordpress/`, run it:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Reading Corner' theme.
<title>:      0
search form:  xhtml (id="searchform")
style:        (not enqueued)
script:       (not enqueued)
menu places:  (none)
widget areas: (none)
```

- **TODOs 1 to 4 in pairs (14m), the test after each.** Circulate for: `add_theme_support( 'html5' )` without the array; the version typed as '2.0.0' (works today, wrong tomorrow); `'in_footer' => true` passed as the fifth argument's boolean instead of inside the array; register_sidebar() on after_setup_theme (it works because widgets_init comes later, but the lesson's hook is widgets_init).
- **Done when** the test prints (the solution):

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Reading Corner' theme.
<title>:      1
search form:  html5 (type="search")
style:        style.css?ver=2.0.0
script:       corner.js?ver=2.0.0, defer
menu places:  corner (Reading corner menu)
widget areas: corner-footer
```

- **Clean up (1m):** `wp theme activate pagesco-classic`, remove the folder and the test.
- **Fast finishers:** `labs/module-07/l4_3-functions-php/debug_theme_setup.php` (needs the end of 4.3: after Step 3), and at home `labs/module-07/l4_3-functions-php/menus_widgets_starter.sh`.

START THE LESSON PROJECT

The lesson project is **Finish the Classic Theme, Then Back to pagesco** (about 35 minutes alone). In the room: Step 1 and Step 3 (15 minutes); the Step 2 check goes home.

- Step 1: replace style.css with the lesson's version (Version 1.1.0 and the rules for screen-reader text, the skip link, the notice band, header, menu, content and footer). Check:

```
wp theme list --name=pagesco-classic --fields=name,status,version
curl -s http://localhost:8080/ | grep -o "pagesco-classic-style-css' href='[^']*''
```

```
+-----------------+--------+---------+
| name            | status | version |
+-----------------+--------+---------+
| pagesco-classic | active | 1.1.0   |
+-----------------+--------+---------+
pagesco-classic-style-css' href='http://localhost:8080/wp-content/themes/pagesco-
```

- Step 3: pagesco's first functions.php (the "Pages & Co." pattern category on init), the sed that adds `pagesco` to the three pattern headers, and `wp theme activate pagesco`:

```
 * Categories: pagesco, call-to-action
 * Categories: pagesco, featured
 * Categories: pagesco, banner, featured
Success: Switched to 'Pages &amp;amp; Co.' theme.
pagesco/hero: pagesco, banner, featured
pagesco/featured-books: pagesco, featured
pagesco/call-to-action: pagesco, call-to-action
Pages & Co.
/             titles: 1  notice: 1
/about/       titles: 1  notice: 1
/hello-world/ titles: 1  notice: 1
```

- **Done in the room** = pagesco active, its three patterns in the pagesco category, one title and one notice per page. This is the state Session 8 and Part 2 start from; do not let anyone skip it.

AFTER THE BREAK: ACCESSIBILITY, TRANSLATION AND MISTAKE 1 (15M)

- **Screen-reader text and the skip link (5m).** Show the pagination's visible "Posts pagination" heading (the lesson's section), then the stylesheet's `.screen-reader-text` rule from Step 1 and the skip link in header.php. The lesson's checklist on the slide: skip link, named landmarks, real buttons, visible focus, one h1.
- **Translation (4m).** The literal text domain audit (the lesson's grep: 0) and, if time allows, the es_ES.l10n.php demo on pagesco-classic (switch to it for two minutes and back). The version note: WordPress 6.8+ loads a theme's languages/ itself; the call stays for 6.6 and 6.7 (self-check Q5).
- **Common Mistake 1 (6m).** The lesson's must-use plugin with a top-level wp_enqueue_style(), and the first lines of the home page:

```php
<?php
wp_enqueue_style( 'pagesco-hours', content_url( 'mu-plugins/hours.css' ) );
```

```
curl -s http://localhost:8080/ | head -n 6
rm wp-content/mu-plugins/mistake-enqueue.php
```

```
Notice: Function wp_enqueue_style was called <strong>incorrectly</strong>. Script

Notice: Function wp_is_block_theme was called <strong>incorrectly</strong>. This

Notice: Function is_feed was called <strong>incorrectly</strong>. Conditional que
```

Three notices from one misplaced line; read the first one, which names the handle. The style still loads, which is the real damage: on every request, admin and login included. (The command deletes the file again.)

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Where does add_theme_support('title-tag') go? **Answer:** a functions.php callback on after_setup_theme (the lesson quiz Q1).
- Why the theme's Version as the version argument? **Answer:** it changes exactly when you release; browsers then fetch the new file.
- Who builds the menu, and who decides where it may appear? **Answer:** the owner builds it; the theme registers locations.
- Which hook for register_sidebar()? **Answer:** widgets_init (the lesson quiz Q4).

Workbook self-check, items Q1 to Q6.

Answers: **Q1** C) `after_setup_theme`; **Q2** `pagesco-classic | inactive | 1.1.0`: pagesco is active again; 1.1.0 is the Version the stylesheet is enqueued with (style.css?ver=1.1.0), so raising it on a release makes every browser fetch the new CSS; **Q3** A) The page loads the CHILD's style.css under the parent's handle, and the parent's CSS is gone; **Q4** B) So a location without an assigned menu prints nothing instead of wp_page_menu()'s list of every page; **Q5** D) Not on WordPress 6.8 and later, which load the active theme's languages/ folder itself; it stays for 6.6 and 6.7; **Q6** A "called incorrectly" notice naming the handle (scripts and styles should not be enqueued until wp_enqueue_scripts...), plus two follow-on notices; the real damage: top-level code runs on every request, so the CSS also loads on admin screens and the login page. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners new to hooks do TODOs 1 and 2 of the lab with the lesson's functions.php open side by side; 3 and 4 go home.

Stretch: `debug_theme_setup.php`, then the lesson's Exercise 1 (the login logo) and the Bonus Challenge (add_editor_style).

1:45 Part 1 Project: The Fernhill Community Garden Theme

65 min

TIMING

10m	The brief: the email, R1 to R8, the rules, the Met line
10m	Milestone 1 for everyone, and a two-minute tour of the starter
20m	Milestone 2 started: the style.css header and theme.json
15m	Plan Milestone 3 on paper
10m	Check-in, due dates, back to pagesco

READ THE BRIEF TOGETHER (10M)

Hand out `labs/module-08/p1-project/BRIEF.md`. Read Odette Marsh's email aloud:

“Our website runs on a free theme from 2019 that nobody dares to touch. We would like our own theme, made for us: green and calm, our news on the front page three at a time, a big welcome at the top, a call for Saturday volunteers, and an autumn look we can switch on in October. Our volunteers write news and edit pages in the editor; none of us writes code. And please: nothing that breaks when WordPress updates itself.”

Then the requirement list on the slide. Ask, row by row: "Which module does this come from?" (R1 to R4 Module 3, R5 and R6 Module 4, R8 every module.)

REQ	WHAT	FROM
R1	Design system	Lesson 3.2
R2	Templates and parts	Lessons 3.1, 3.2
R3	Patterns and the front page	Lessons 3.3, 1.3
R4	Style variation	Lesson 3.3
R5	functions.php	Lessons 4.3, 2.3
R6	The main query	Lesson 4.2
R7	404 page	Lesson 3.1
R8	Quality	Lessons 1.3, 3.2

- The rule that matters most: **the theme is a new folder**; nothing in pagesco, the course's other themes or core changes. Open book, AI may explain an error but does not write the theme, and every learner can explain every line (the Session 8 review asks).
- The rubric's Met line, once: **Met** = at least 14 of 20 points with no row at 1. Below that, the learner revises once, within a week of the feedback. The full rubric is in BRIEF.md and in final-assessment.html.
- The garden has no content yet: learners build on their course site's posts and pages. The theme must not care whose content it shows.

MILESTONE 1 FOR EVERYONE (10M)

From the project folder, on every machine:

```
cp -r labs/module-08/p1-project/fernhill_starter wordpress/wp-content/themes/fernh
wp theme activate fernhill
cp labs/module-08/p1-project/test_fernhill.php wordpress/
wp eval-file --use-include test_fernhill.php
```

```
Success: Switched to 'Fernhill' theme.
Fernhill self-check (Part 1 project, BRIEF.md)
TODO  1  Theme header: Fernhill 1.0.0, text domain fernhill, requires WordPress 7
OK    2  A block theme with a valid theme.json, version 3
TODO  3  Palette: base, contrast, primary, secondary, accent (in that order); the
TODO  4  Layout: content 680px, wide 1100px
TODO  5  Templates: index, front-page, single, page, 404 (missing: front-page, si
TODO  6  Template parts: header (area header) and footer (area footer)
TODO  7  Patterns: fernhill/hero and fernhill/volunteer-call in the fernhill cate
TODO  8  Style variation: Autumn
TODO  9  Front page: header, main, the hero heading, the volunteer heading, foote
TODO 10  Stylesheet: fernhill-style, assets/css/fernhill.css?ver=1.0.0
TODO 11  Main query: 3 posts on the front page, 1 on /page/2/ (now: 4 and 0)
TODO 12  404: status 404 and the heading "This path is overgrown"
Checks passed: 1 of 12
```

- Done = `Checks passed: 1 of 12` on every screen (line 2 is OK because the starter's theme.json is valid).
- The tour (2m): ready-made `assets/css/fernhill.css`, `assets/images/leaf.svg`, `parts/header.html`; half-made `patterns/hero.php` (the model pattern for R3: three strings to translate, its category missing); TODOs in style.css and functions.php; a bare theme.json and templates/index.html. "Read every TODO before you type."

MILESTONE 2 STARTED (20M)

- The style.css header (R1: Version 1.0.0, Requires at least 7.1, Requires PHP, Tested up to), then theme.json: the five role colors from the brief, in order, defaultPalette off, content 680px and wide 1100px.

Run the self-check after each save.

- Circulate for JSON commas (a trailing comma = a decode notice and no styles, Lesson 3.2 Mistake 1), hex values typed in the wrong order, and "Requires at least: 7.1" written as "WordPress 7.1". The model for theme.json is pagesco's own (Lesson 3.2).
- **Where most learners are at the end of the 20 minutes:** header and theme.json settings done. The self-check then reads (the starter with the model solution's style.css and theme.json, run for real):

```
Fernhill self-check (Part 1 project, BRIEF.md)
OK    1  Theme header: Fernhill 1.0.0, text domain fernhill, requires WordPress 7
OK    2  A block theme with a valid theme.json, version 3
OK    3  Palette: base, contrast, primary, secondary, accent (in that order); the
OK    4  Layout: content 680px, wide 1100px
TODO  5  Templates: index, front-page, single, page, 404 (missing: front-page, si
TODO  6  Template parts: header (area header) and footer (area footer)
TODO  7  Patterns: fernhill/hero and fernhill/volunteer-call in the fernhill cate
TODO  8  Style variation: Autumn
TODO  9  Front page: header, main, the hero heading, the volunteer heading, foote
TODO 10  Stylesheet: fernhill-style, assets/css/fernhill.css?ver=1.0.0
TODO 11  Main query: 3 posts on the front page, 1 on /page/2/ (now: 4 and 0)
TODO 12  404: status 404 and the heading "This path is overgrown"
Checks passed: 4 of 12
```

Fonts and the element styles (links, buttons, headings) are the rest of M2 and go home; the self-check does not test them, the rubric's design row does.

PLAN MILESTONE 3 ON PAPER (15M)

- In the workbook's planning box: the template map (index, front-page, single, page, 404: which URL uses which, Lesson 3.1), the two parts with their areas, and the three landmarks every template needs (header part with tagName header, a main group, footer part with tagName footer).
- Pairs compare maps for 3 minutes. Ask: "Which template draws /category/news/ in your theme?" (index.html: no archive template is required.)
- Point at the model they have: pagesco's templates and parts (Lesson 3.2). M3 is about an hour at home.

CHECK-IN (10M)

- Each pair calls out their current "Checks passed" number; write the range on the board.
- Due before Session 8: **Milestones 2 and 3** (about 1 hour 45 minutes at home). Bring the theme and the last self-check output. Session 8: studio for Milestones 4 to 6, the timed Part 1 assessment, the review; hand-in one week after Session 8.
- Last command, on every screen: `wp theme activate pagesco` (the self-check switches nothing).

MILESTONES (FROM THE BRIEF)

	MINUTES	WHAT	REQUIREMENTS	WHERE
M1	15	Set up: copy the starter to `wordpress/wp-content/themes/fernhill/`, activate it, run the self-check (`Checks passed: 1 of 12`), read every starter file and its TODOs.	–	Session 7 studio
M2	45	The design system: the style.css header and theme.json (palette, layout, fonts, element styles).	R1	Session 7 studio (start), home
M3	60	Templates and parts: header and footer parts, index, single, page, 404; the per-URL check.	R2, R7	home
M4	45	Patterns and the front page: the two PHP patterns, front-page.html.	R3	home, Session 8 studio
M5	30	functions.php: the pattern category, the stylesheet, the main query.	R5, R6	Session 8 studio
M6	30	The Autumn variation, NOTES.md, the per-URL check and the self-check (`Checks passed: 12 of 12`), the hand-in.	R4, R8	Session 8 studio, home

Answers for items Q7 to Q12 of the workbook self-check (project skills):

Answers: **Q7** A) It names the theme: the header fields WordPress reads (Theme Name, Text Domain, Version, Requires at least ...); **Q8** C) `theme.json`: `styles.elements.link` and `styles.elements.button` with `var:preset|color|primary`; **Q9** B) A `pre_get_posts` callback in functions.php with the three guards: not the admin, the main query only, the blog home only; **Q10** D) A wp_template_part copy in the database wins over the file: move the change into header.html, then delete the copy; **Q11** `styles/autumn.json`, titled Autumn, without blockTypes: the same five color slugs with autumn values, nothing else; **Q12** `wp theme activate pagesco`: the labs, Session 8's assessment and Part 2 (Lesson 5.1) expect pagesco active, the state at the end of Lesson 4.3. Full set with explanations in the Answer Key.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. Name the hook for each: a theme support, a stylesheet, a widget area. **Answer:** after_setup_theme; wp_enqueue_scripts; widgets_init.
2. Why does the enqueue use the theme's Version? **Answer:** the URL changes with each release, so browsers fetch

the new file.

3. Which Fernhill requirement will take you longest, and which lesson will you reread for it? **Answer:** (personal; most say R2/R3: Lessons 3.1 to 3.3).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish Lesson 4.3:** Common Mistakes 2 to 4, the project's Step 2 (the per-URL check), Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz, the journal.
- **Part 1 project, Milestones 2 and 3** (about 1 hour 45 minutes): the design system (fonts and element styles too), then the templates, the two parts and the 404 page, with the per-URL check. Run the self-check often.
- **Switch back** after each work session: `wp theme activate pagesco`. Edit FILES; if you try something in the Site Editor, move it into the file and delete the database copy.
- **Labs not reached:** `labs/module-07/l4_3-functions-php/menus_widgets_starter.sh`, `labs/module-07/l4_3-functions-php/debug_theme_setup.php`.

BRING LIST FOR SESSION 8

- The course site at the end of Lesson 4.3: pagesco active, pagesco-classic finished and inactive; fernhill in wp-content/themes/ with Milestones 2 and 3 done.
- The last self-check output (the "Checks passed" line), your template map, questions for the studio.
- The project folder with `labs/`, the journal, the workbook. Session 8 also has the timed Part 1 assessment (classic themes, open book): reread Lessons 4.1 to 4.3.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** Step 3 (back to pagesco); Milestone 1 on every machine; the enqueue with its version.
- **Cut first (saves 7m):** the block-theme defaults and the widget steps of the demo.
- **Cut second (saves 15m):** the M3 planning segment: the template map goes home with the workbook box.
- **Cut third (saves 10m):** the translation demo after the break; keep the skip link and Mistake 1.

RUNNING SHORT: WHAT TO ADD

- Run `menus_widgets_starter.sh` for everyone (it switches to pagesco-classic and back by itself).
- Start Milestone 3 in the room: the header part's tagName and the main group in index.html.
- The lesson's Exercise 4 live: WP_Classic_To_Block_Menu_Converter on the Main menu.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 8: Part 1 Project and Assessment

How to run the last session of Part 1: a studio for the Part 1 project (the Fernhill Community Garden block theme), the timed Part 1 assessment (finishing the Brightwater Swim School classic theme, 60 minutes, alone), and a review that turns both into a map of Modules 1 to 4. Rubrics, scoring sheets and answer notes are in final-assessment.html. When Part 1 runs on its own, this is the course close.

`0 new lessons` `3 hrs session` `Modules 1–4` `Session 8 of 18 · Part 1`

HOW TO USE THIS GUIDE

- Session 8 teaches no new lesson. It closes Part 1 with three things: a **studio** for the Part 1 project (the Fernhill Community Garden block theme, briefed in Session 7, due one week from today), a **timed assessment** (finishing the Brightwater Swim School classic theme, 60 minutes, alone), and a **review** that turns both into a map of Part 1.
- The project and the assessment are deliberately different: the project is the block-theme side (Module 3, with functions.php and the main query from Module 4) over a week, with help; the assessment is the classic-theme side (Module 4, with escaping and hooks from Modules 1 and 2) in one hour, without help. Together they cover Modules 1 to 4.
- Rubrics, scoring guidance, scoring sheets and the answer notes (the model solutions' real output, and what each common mistake prints) are in `final-assessment.html` at the kit root (sections "Part 1 Project" and "Part 1 Assessment"). Print the scoring sheets before the session.
- Every output in this guide was produced on WordPress 7.1.3, PHP 8.5, at the state Lesson 4.3 leaves (pagesco active): what every learner's site is at the start of today. Labs live in `labs/module-08/`: `p1-project/` (Session 7 handed it out), `p1-assessment/` (hold it back until 1:25 if you can) and `p1-review/`.
- **When Part 1 runs on its own,** this is the last session: the wrap-up becomes the course close (certificates of completion, what next) and the project hand-in is the last contact. The guide marks where.

> **THE #1 RISK OF SESSION 8: THE STUDIO EATS THE ASSESSMENT**
>
> Learners who are behind on the project will ask to keep building through the break. Do not let them: the assessment starts at 1:25 for everyone, and the project has a week left. Say at 0:10: "The studio ends at the break, whatever happens. Your project is due in a week; the assessment is today." Second risk: a learner whose site is broken (the assessment needs a working course site at after:lesson_13). Check every machine during the status round, and keep one spare laptop with a working site, or let the learner pair-run the self-check on yours and hand in code only.

BEFORE THE SESSION

- ☐ **Print** the scoring sheets for the project and the assessment (`final-assessment.html`), one per learner, and the assessment's `BRIEF.md` (`labs/module-08/p1-assessment/BRIEF.md`) on paper if you prefer paper.
- ☐ **Assessment files.** If you handed out the whole `labs/` folder in Session 1, learners already have `p1-assessment/` (without the solution); that is acceptable (open book, and the score comes from the code and the explanation), but the cleaner way is to hold the folder back: put `labs/module-08/p1-assessment/brightwater_starter/`, `test_brightwater.php` and `BRIEF.md` on a USB stick or a shared folder and release them at 1:25. Never hand out `brightwater_solution/` or `fernhill_solution/`.
- ☐ **Run the kit check:** `python3 labs/check_labs.py module-08` (every line PASS: the review pair and handout, and both self-checks once with the starter and once with the solution, on fresh after:lesson_13 sites). With your own project: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-08` (labs/README.md); your site must be at the end of Lesson 4.3.
- ☐ **Projector machine:** the course project running at the end of Lesson 4.3 (pagesco active), two terminals in the project folder, VS Code, browser tabs: http://localhost:8080, the Site Editor, `final-assessment.html` (for you, not the projector).
- ☐ **Board, before learners arrive:** the Part 1 map (Module → skills → where in the project and the assessment; the table below), today's clock (studio until 1:15, assessment 1:35 to 2:35), and the self-check commands for both themes.
- ☐ **A timer** visible to the room for the assessment, and sticky notes for the status round.

THE PART 1 MAP (FOR THE BOARD)

MODULE	WHAT LEARNERS CAN DO	PROJECT (R) · ASSESSMENT (T)	LESSONS
1	WordPress as a PHP application; the local site; PHP the WordPress way (escaping, i18n, prefixes, standards)	R8, every task; T4	1.1 to 1.3

MODULE	WHAT LEARNERS CAN DO	PROJECT (R) · ASSESSMENT (T)	LESSONS
2	The content model, the database, hooks (actions, filters, priority)	R5, R6; T1 to T3	2.1 to 2.3
3	Block themes: anatomy and the hierarchy, theme.json, templates and parts, patterns, variations, child themes	R1 to R4, R7	3.1 to 3.4
4	Classic themes: the Loop and template tags, custom queries and pre_get_posts, functions.php	R5, R6; T1 to T5	4.1 to 4.3

SESSION 8 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Part 1; a status round: every learner reads out the last line of their project self-check.
0:10	15	Part 1 review	The Part 1 map on one board; `request_map_solution.sh` on the projector; two predict-then-run items; questions from the room.
0:25	50	Part 1 project studio	Milestones 4 to 6 of the Fernhill theme: triage by self-check number, three-minute conferences, the per-URL check.
1:15	10	Break	Set out the assessment files (or a USB stick) during the break. Projects closed.
1:25	10	Part 1 assessment: setup	Rules, the bug reports, the hand-in; every learner copies `brightwater_starter`, activates it, runs the self-check (`Checks passed: 2 of 10`).
1:35	60	Part 1 assessment	Brightwater, alone, open book, timed: tasks T1 to T5, the stretch if time. Hand-in at 2:35.
2:35	15	Review and showcase	After the hand-in: the three bug reports explained, two or three Fernhill front pages shown, a Part 1 retrospective.
2:50	10	Wrap-up	Exit ticket, the project due date, Part 2 preparation (or the course close when Part 1 runs alone).
Total	180		Ends at 3:00.

In the room. No lesson: 50 minutes of project studio, 70 minutes of assessment (setup and work), 25 minutes of review before and after, the warm-up, the break and the wrap-up. The project takes about 3 hours 45 minutes in all:

about 35 minutes in Session 7, 50 today, the rest at home (Milestones 2 to 4 between the sessions, the finish in the week after today). The assessment is sized for 50 minutes of tasks plus 10 for setup checks and the hand-in.

Goes home (about 2 to 3 hours). Finish the Fernhill theme (Milestones 5 and 6 and anything left; due one week from today, with NOTES.md); the Part 1 retrospective journal; the self-check items not done in the room; Part 2 preparation (below) or, when Part 1 runs alone, nothing else.

Protect, in this order: the full 60 minutes of the assessment (start it on time, at 1:35 at the latest); the studio's triage (every learner leaves knowing their next milestone); the review of the three bug reports (it is where the assessment teaches). **Cut first:** the predict items in the review, then the showcase (keep the bug walkthrough), then the warm-up questions (keep the status round).

OPENING SEGMENT

0:00 Warm-up and Status Round — 10 min

TIMING

5m	Five retrieval questions (workbook W1 to W5), answers out loud
5m	Status round: every learner reads out the last line of `test_fernhill.php`; you write the numbers on the board

WARM-UP QUESTIONS AND ANSWERS

1. **Name the five extension points of WordPress and the module that covers each.** Hooks (Lesson 2.3), themes (Modules 3 and 4), plugins (Module 5), blocks (Module 6), APIs (Module 7). Part 1 covered the first two.
2. **Block theme or classic theme: where does each keep its templates, and in which language?** Block: templates/*.html and parts/*.html, block markup. Classic: *.php in the theme root (index.php, singular.php...), PHP with template tags.
3. **Which three guards does a pre_get_posts callback for the blog home need?** Not is_admin(); $query->is_main_query(); $query->is_home() (the query's method, not the global function).
4. **Name four escaping functions and where each goes.** esc_html() text, esc_attr() attributes, esc_url() href/src, wp_kses_post() HTML with safe tags (and the _e / __ variants for translated text).
5. **Your project's last self-check line?** Each learner's own number: the status round (see the studio triage).

READING THE STATUS ROUND

- **10 to 12 of 12:** on track. They work on R8 (the per-URL check, NOTES.md) and a stretch item in the studio, and help a neighbor.
- **5 to 9:** the usual. Templates and theme.json done, patterns or functions.php missing: Milestones 4 and 5 in the studio.
- **1 to 4:** behind (often theme.json or the parts). First conference in the studio goes to them; the goal today is Milestone 4 started.
- **No number** (the theme is not on the machine, or the site is broken): fix the site NOW, before the review: the assessment needs it. `wp theme activate pagesco` and the self-check of Lesson 4.3's end (`wp theme list`) first.

SESSION PLAN

0:10 Part 1 Review — 15 min

◆ LIVE DEMO (YOU TYPE, THEY PREDICT)

1. **The map (4m).** Point at the board's Part 1 map. "Every requirement of your project and every task of today's assessment is one row of this table. Nothing is new; the client is." Ask for one example per row from the room.

2. **What WordPress thinks each address is (5m).** "The hierarchy starts with a question WordPress asks about the request; body_class() prints the answer." Run the review lab's solution on the projector (pagesco active), learners predict each line first.

```
cp labs/module-08/p1-review/request_map_solution.sh wordpress/
docker compose run --rm wpcli sh request_map_solution.sh
```

```
/  200  home blog
/page/2/  404  error404
/book-club-dune/  200  single single-post
/about/  200  page-template-default page
/category/news/  200  archive category category-news
/?s=dune  200  search search-results
/no-such-path/  404  error404
```

Two lines to stop on: `/page/2/` is a 404 on pagesco (ten posts per page, four posts: there is no page 2; on Fernhill, three per page, there is), and `/about/` is `page`, which a block theme answers with page.html and a classic theme with singular.php (Lessons 3.1 and 4.1).

3. **Predict, then run (4m).** Two of the workbook's four predict items (P2 and P3), then the answers:

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/page/2/
wp eval '$q = new WP_Query( array( "post_type" => "page", "orderby" => "title"
echo implode( ", ", wp_list_pluck( $q->posts, "post_title" ) ), "\n";'
```

```
404
About Us, Contact, Events, Sample Page
```

P3: a WP_Query of pages, ordered by title, finds only PUBLISHED pages (the Privacy Policy is a draft): the default post_status for an anonymous query is publish.

4. **Questions from the room (2m).** Take two questions about Part 1. Park the rest on the board for the

review at 2:35.

FAST FINISHERS AND ANYONE EARLY

`labs/module-08/p1-review/request_map_starter.sh` (the same report, two TODOs) and `labs/module-08/p1-review/debug_query_review.php` (a latest-news list with one loud bug and three silent ones; a good warm-up for Task 5 of the assessment, so offer it before 1:25, not during). The answer key has both.

SELF-CHECK (WORKBOOK Q1 TO Q12, AT HOME OR IN SPARE MINUTES)

Answers: **Q1** C) `front-page.html`; **Q2** `ABx`; **Q3** B) `templates/index.html`; **Q4** `twentytwentyfive pagesco-tt5-child`; **Q5** A) The database copy of the part (a wp_template_part post) wins over the file: copy what you want into the file, then reset or delete the copy; **Q6** D) 3; **Q7** B) 2 3; **Q8** C) `esc_url()`; **Q9** `Book club: Dune<h2></h2>`; **Q10** D) `wp_enqueue_scripts`; **Q11** A) A style variation the site owner can switch on for the whole site; **Q12** `--wp--preset--color--oxblood: #7a2e2e`. Full set with explanations in the Answer Key.

0:25 Part 1 Project Studio

50 min

TIMING

5m	Goals for the studio, by status group (board)
40m	Work, with three-minute conferences (low numbers first); the self-check after every milestone
5m	Everyone: the per-URL check and one line in NOTES.md about what is left

STUDIO GOALS BY STATUS GROUP

- **1 to 4 of 12:** Milestone 3 finished (parts and templates) and Milestone 4 started (the hero pattern). Show them `hero.php` as the model: one string at a time.
- **5 to 9:** Milestones 4 and 5: the second pattern, front-page.html, then functions.php (the category, the stylesheet, the main query).
- **10 to 12:** Milestone 6 (Autumn, NOTES.md, the per-URL check), then a stretch item. Ask them to explain their pre_get_posts guards to a neighbor.

THE CONFERENCE (THREE MINUTES, AT THE LEARNER'S SCREEN)

1. "Run the self-check." Read the first TODO line together.
2. "Which requirement is that, and which lesson?" (The BRIEF lists both.)
3. "Show me the file." Point at one thing; do not type for them.
4. Write the next step on a sticky note on their lid. Next learner.

THE BLOCKERS YOU WILL SEE (EACH IS THE MODEL SOLUTION WITH ONE CHANGE, SELF-CHECKED)

SYMPTOM	CAUSE	COACH WITH
Checks 2, 3, 4 and 6 TODO at once, a `wp_json_file_decode()` notice	A trailing comma in theme.json	Lesson 3.2, Mistake 1: the notice names the file; JSON allows no comma before `}`
Check 9 TODO, the header missing on the front page only	A part slug typo in one template (`heder`)	Lesson 3.2, Mistake 2: visitors see nothing, a logged-in admin sees "Template part has been deleted or is unavailable"
Check 10 TODO, the stylesheet loads with `?`	No version argument	Lesson 4.3, Mistake 2: the theme's Version, read with wp_get_theme()

SYMPTOM	CAUSE	COACH WITH
`ver=7.1.3`		
All 12 OK, but `wp eval` shows 3 posts for a query that asked for 10	pre_get_posts without the main-query guard	Lesson 4.2, Mistake 3: the self-check cannot see it; the rubric row can
Edits to a template have no effect	A Site Editor copy in the database	Lesson 3.2, Mistake 3; rule R8
Patterns missing from the inserter	The category not registered, or Categories header without `fernhill`	R3/R5; `wp eval` with WP_Block_Patterns_Registry (the answer notes)

WHAT "DONE" LOOKS LIKE

The self-check of the model solution:

```
Fernhill self-check (Part 1 project, BRIEF.md)
OK    1  Theme header: Fernhill 1.0.0, text domain fernhill, requires WordPress 7
OK    2  A block theme with a valid theme.json, version 3
OK    3  Palette: base, contrast, primary, secondary, accent (in that order); the
OK    4  Layout: content 680px, wide 1100px
OK    5  Templates: index, front-page, single, page, 404
OK    6  Template parts: header (area header) and footer (area footer)
OK    7  Patterns: fernhill/hero and fernhill/volunteer-call in the fernhill cate
OK    8  Style variation: Autumn
OK    9  Front page: header, main, the hero heading, the volunteer heading, foote
OK   10  Stylesheet: fernhill-style, assets/css/fernhill.css?ver=1.0.0
OK   11  Main query: 3 posts on the front page, 1 on /page/2/ (now: 3 and 1)
OK   12  404: status 404 and the heading "This path is overgrown"
Checks passed: 12 of 12
```

The per-URL check of Lesson 3.2, adapted to Fernhill's landmarks (it is in the workbook):

```
for url in / /page/2/ /hello-world/ /about/ /category/news/ "/?s=garden" /no-such-
    page=$(curl -s -w '\n%{http_code}' "http://localhost:8080$url")
    echo "$url  $(echo "$page" | tail -n 1)  header:$(echo "$page" | grep -c '<hea
done
```

```
/  200  header:1  main:1  footer:1  h1: Grow food, grow friendships
/page/2/  200  header:1  main:1  footer:1  h1: Grow food, grow friendships
/hello-world/  200  header:1  main:1  footer:1  h1: Hello world!
/about/  200  header:1  main:1  footer:1  h1: About Us
/category/news/  200  header:1  main:1  footer:1  h1: News
/?s=garden  200  header:1  main:1  footer:1  h1: Search results for: &#8220;garde
/no-such-path/  404  header:1  main:1  footer:1  h1: This path is overgrown
```

`/?s=garden` has an h1 here because the model solution adds the stretch `search.html`; without it the search page falls back to index.html, whose archive title prints nothing on a search (fine for Met, a good conversation for Exemplary). The front page, as the model solution draws it on the course site (the thin line at the top is the course's store notice from its must-use plugin):

Free shipping on orders over $35. Open until 9 p.m. on Thursdays. See events

Pages & Co.

About Us Contact Events Sample Page

Grow food, grow friendships

Forty raised beds, a tool shed and a compost corner, looked after by neighbors of every age. No experience needed.

About the garden



News from the garden

Hello world!

October 25, 2026

Welcome to WordPress. This is your first post. Edit or delete it, then start writing!

Book club: Dune

October 1, 2026

Our Thursday book club starts with Dune. Join us at 7 p.m. in the reading corner; the first coffee is on us.

Staff picks for the season

September 15, 2026

Three books we keep pressing into customers’ hands this season: It was a pleasure to burn. Ray Bradbury, Fahrenheit 451

1 2 Next Page

Volunteer on Saturday mornings

Bring gloves and a friend. We start at 9 a.m. at the tool shed, and there is always tea.

Sign up for a workday

Fernhill Community Garden · 48 Orchard Lane · 555-0187

Workdays every Saturday, 9 a.m. to noon

1:25 Part 1 Assessment

70 min

TIMING

10m	Setup at 1:25: rules, the bug reports, the hand-in; copy, activate, self-check (2 of 10)
60m	Work, 1:35 to 2:35; time calls at 2:05, 2:25 and 2:30

SETUP SCRIPT (SAY IT, THEN SHOW IT)

1. "For the next hour you finish a classic theme for a new client, the Brightwater Swim School. Alone, open book: lesson pages, your notes, your own course code. No people, no AI assistants, no code from the web."
2. Read Ken's email aloud (BRIEF.md): "A developer started a classic theme for us and left halfway. The site works, sort of: our menu never shows, the news page lists bare titles, there is no stylesheet, the search page did something strange when my nephew typed HTML into it, and the previous and next links under a post point to the wrong posts. Can you finish it this afternoon?"
3. "Five tasks, T1 to T5, in BRIEF.md; the minutes next to each are a guide. Three of the tasks start from his bug reports: find the bug, do not just make the symptom go away."
4. On the projector, everyone types along: `cp -r labs/module-08/p1-assessment/brightwater_starter wordpress/wp-content/themes/brightwater`, `wp theme activate brightwater`, `cp labs/module-08/p1-assessment/test_brightwater.php wordpress/`, `wp eval-file --use-include test_brightwater.php`. Done = `Checks passed: 2 of 10` on every screen.
5. "The self-check is a help, not your score. I read your code with the rubric. If you run out of time, write one line at the top of functions.php saying what is missing."
6. "Hand-in at 2:35: zip the brightwater folder as brightwater-yourname.zip with selfcheck.txt inside, and put it in the shared folder." Start the timer at 1:35.

THE STARTER, AS LEARNERS SEE IT AT 1:35

```
Success: Switched to 'Brightwater' theme.
Brightwater self-check (Part 1 assessment, BRIEF.md)
OK    1  A classic theme: Brightwater, text domain brightwater
TODO  2  T1 Title: title-tag support, exactly one <title> (now: 1)
TODO  3  T1 wp_body_open(): the store notice band from mu-plugins/pagesco-site.ph
TODO  4  T2 Stylesheet: brightwater-style, style.css?ver=1.0.0
TODO  5  T1 Menu: the primary location shows About Us, Events, Contact in the hea
TODO  6  T3 Blog home: 2 posts with linked h2 titles, a link to page 2 (now: 0 po
TODO  7  T3 /page/2/: status 200 and the other 2 posts
TODO  8  T4 Search: the term is printed as text (&lt;b&gt;), never as HTML
OK    9  T5 Related news on /book-club-dune/: Staff picks, then Welcome
TODO 10  T5 Post navigation on /book-club-dune/: previous Staff picks, next Hello
Stretch: footer widget area not registered
Checks passed: 2 of 10
```

DURING THE HOUR

- **You may answer:** questions about the BRIEF's wording, the setup (a broken site, Docker), and "is this file in the theme?". **You may not:** say where a bug is, or whether code is right. A stuck learner gets: "Which task? What does the self-check say? Which lesson is it from?"
- **Walk the room once every 15 minutes,** silently. Note who is stuck on setup (a site problem is yours to fix, and their clock stops while you do).
- **Time calls** at 2:05 ("30 minutes: T3 should be under way"), 2:25 and 2:30 ("stop coding at 2:33, zip, hand in").
- **Accommodations:** learners with agreed extra time keep working up to 30 more minutes in the review slot (or finish at home within 48 hours, the same folder); the rest of the room does the showcase without them.

WHAT THE MODEL SOLUTION PRINTS (YOUR REFERENCE, NEVER HANDED OUT)

```
Success: Switched to 'Brightwater' theme.
Brightwater self-check (Part 1 assessment, BRIEF.md)
OK    1  A classic theme: Brightwater, text domain brightwater
OK    2  T1 Title: title-tag support, exactly one <title> (now: 1)
OK    3  T1 wp_body_open(): the store notice band from mu-plugins/pagesco-site.ph
OK    4  T2 Stylesheet: brightwater-style, style.css?ver=1.0.0
OK    5  T1 Menu: the primary location shows About Us, Events, Contact in the hea
OK    6  T3 Blog home: 2 posts with linked h2 titles, a link to page 2 (now: 2 po
OK    7  T3 /page/2/: status 200 and the other 2 posts
OK    8  T4 Search: the term is printed as text (&lt;b&gt;), never as HTML
OK    9  T5 Related news on /book-club-dune/: Staff picks, then Welcome
OK   10  T5 Post navigation on /book-club-dune/: previous Staff picks, next Hello
Stretch: footer widget area registered
Checks passed: 10 of 10
```

The bugs behind the three reports: the header asks for a menu location named `main` while the theme registers `primary` (T1); `search.php` prints `get_search_query( false )` raw (T4); `related-news.php` never calls `wp_reset_postdata()`, so `the_post_navigation()` works from the last related post instead of the post on the screen (T5). The full answer notes, the traps and the rubric are in final-assessment.html.

2:35 Review and Showcase

15 min

TIMING

6m	The three bug reports, explained (after every zip is in)
6m	Two or three Fernhill front pages on the projector, one minute each: one decision each learner made
3m	Part 1 retrospective: one thing learned, one to review

THE BUG WALKTHROUGH

- **"Our menu never shows" (T1).** `register_nav_menus()` names the location, `wp_nav_menu( theme_location )` and `has_nav_menu()` ask for it: the names must match, and a typo is silent (Lesson 4.3, Mistake 3).
- **"Something strange when my nephew typed HTML" (T4).** The heading printed the search words as HTML. Show the fixed output: `<h1 class="search-title">Results for: <b>water</b></h1>`. `esc_html()` at the echo, or `get_search_query()` with its default escaping. Every value from a visitor is untrusted (Lessons 1.3 and 5.3).
- **"Previous and next point to the wrong posts" (T5).** The related loop moved the global post. After `wp_reset_postdata()` Dune's navigation is its own: Staff picks for the season Hello world! (Lesson 4.2, Mistake 2).

RETROSPECTIVE PROMPT

"Part 1 took you from a local site to three kinds of theme. Which skill would you now trust yourself with on a client's site, and which would you look up first?" Two or three answers out loud; the rest go in the journal.

CLOSING

EXIT TICKET (3 QUESTIONS, 4 MINUTES)

1. A client wants "news three at a time" on a block theme's front page. Where does the code go, and why not a second query in the template? **Answer:** a pre_get_posts callback in functions.php (guards: not admin, main query, is_home) sets posts_per_page; the Query Loop inherits the main query, so pagination and /page/2/ keep working. A second query would not page, and the main query would still run.
2. Why does every string in a pattern go through `esc_html_e()` even though you wrote it? **Answer:** it makes the text translatable (the text domain) and escapes it at output; a translation is data you did not write.
3. One skill from Part 1 you will use first, and one you want to review. **Answer:** the learner's own; read the review answers before Session 9 and pick the refresher topics from them.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish the Fernhill theme** (Milestones 5 and 6, anything left, `NOTES.md`). Due **one week from today**: the zip `fernhill-<yourname>.zip` with NOTES.md and the last self-check output (`labs/module-08/p1-project/BRIEF.md`, Hand-in).
- **Journal: the Part 1 retrospective** (workbook prompt).
- **Self-check Q1 to Q12** in the workbook, if not done in the room; the answer key is yours to share after the project deadline.
- **Part 2 preparation** (when Part 2 follows): `wp theme activate pagesco` (Lesson 5.1 starts from it), Node 24 on the computer or `docker pull node:24`, `docker pull composer:2`; Session 9 opens with `check_environment.sh --part2`. A site that is not at the end of Lesson 4.3 (unfinished, or broken by experiments) can start Part 2 from the course download `pagesco-part2-start.zip` (its README.txt: about a minute; it saves the old database first).
- **Optional:** `labs/module-08/p1-review/debug_query_review.php` and the request-map lab if you did not reach them.

BRING LIST FOR SESSION 9

- A laptop with the course site at the end of Lesson 4.3, pagesco active (your Fernhill theme can stay installed, inactive).
- Node 24 or the node:24 image, the composer:2 image, a good connection for the first `npm install` of Module 6 later.
- The journal and the workbook; pagesco-plan.txt (Lesson 5.1 reads it again).

WHEN PART 1 RUNS ON ITS OWN: THE COURSE CLOSE

- Replace the Part 2 items of the wrap-up with: what the learners can now do (the Part 1 map), where to go next (Part 2's plugin, blocks and REST modules; the WordPress Developer Resources; the Theme Handbook), and how the project feedback will reach them.
- **Completion:** Part 1 is complete when the project and the assessment are both Met (at least 14 of 20, no row at 1; one revision each within a week). Issue the certificate of completion after the project feedback, not today.

- Ask for course feedback in the last five minutes (paper or a form), and thank the room.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the assessment's 60 minutes, starting by 1:35; the studio triage; the bug walkthrough.
- **Cut first (saves 4m):** the predict items in the review.
- **Cut second (saves 6m):** the showcase; keep the walkthrough and one retrospective answer.
- **Cut third (saves 5m):** the warm-up questions; keep the status round (you need the numbers for the studio).

RUNNING SHORT: WHAT TO ADD

- Run `labs/module-08/p1-review/debug_query_review.php` together before the break: one loud bug, three silent ones; the silent ones are what Task 5 is about.
- In the review: switch Fernhill's style variation to Autumn in the Site Editor on the projector, then show the database copy it made (`wp post list --post_type=wp_global_styles`) and reset it: rule R8 in one minute.
- After the hand-in: ask two learners to show their T3 Loop and compare `the_permalink()` with `esc_url( get_permalink() )`.

Part 2: Plugins to Launch

Sessions 9 to 18. Modules 5 to 9: the pagesco-books plugin (custom post types, security, settings, custom tables), custom blocks with the Interactivity API, the REST API and custom endpoints, external APIs and cron, debugging, testing, performance, internationalization and deployment, ending with the final project: the Pages & Co. website, launched.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 9: Your First Plugin and Custom Post Types

How to teach the first session of Part 2. Open Part 2 for two groups at once (learners coming from Part 1 and learners joining here) with a refresher, a starting note and the Part 2 environment check, then build the pagesco-books plugin the way it will grow for ten sessions (a header, a namespace, Composer's autoloader, one class per feature, a life cycle) and teach WordPress about books: a post type, a genre taxonomy, typed meta and the seeded catalog.

`2 lessons` `3 hrs session` `Lessons 5.1-5.2` `Session 9 of 18 · Part 2`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's ideas) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Every command below was run to produce the output printed under it, on WordPress 7.1.3 / PHP 8.5: the 5.1 demo on the Part 2 starting site (the state Lesson 4.3 leaves, `after:lesson_13`), the 5.2 demo on the site as Lesson 5.1 leaves it. Ask for a prediction before every run and read every message from the left.
- **Type the short files, copy the long ones.** Part 2's classes are longer than anything in Part 1: learners type the main file, composer.json, the `sed` lines and the commands with you, and copy Plugin.php, Feature.php, Activator.php, BookPostType.php, GenreTaxonomy.php and BookMeta.php from the lesson pages, then read each one aloud with you. Typing a 60-line class in the room teaches typing.
- The lessons are the reference: keep `lesson_14.html` and `lesson_15.html` open in browser tabs. Labs live in `labs/module-09/` (learners keep the folder in their project folder): `_starter` files and folders are handouts, `_solution` ones are yours.
- Session 9 opens Part 2, "Plugins to Launch" (Sessions 9 to 18). Some learners come straight from Part 1; some join here. The 30-minute opening serves both: a Part 1 refresher, the "starting with Part 2" note and the Part 2 environment check. Teal blocks are hands-on, amber blocks are watch-outs.

> **THE #1 RISK OF SESSION 9: TWO ROOMS IN ONE**
>
> Learners who finished Part 1 a week ago have the Lesson 4.3 site and the habits (debug.log, WP-CLI, hooks). Learners who join at Part 2 may have neither: their machine may not run the course project, their site may be a fresh install, and "the store notice from Lesson 2.3" means nothing to them. **Before:** send the setup guide (Docker, the project, the Part 2 starting site `pagesco-part2-start.zip` from the course's downloads) and the starting note below to every new learner a week ahead, and ask everyone to reply with the last line of `check_environment.sh --part2`. **In the room:** the check gets 12 minutes; a learner whose site is not where Part 2 starts sits with a ready partner for the demos and installs the Part 2 starting site at the break, not during Lesson 5.1. **Both groups** get the same refresher: R1 and R2 are exactly what Lesson 5.1 builds on.

BEFORE THE SESSION

- ☐ **A week before:** to everyone, "run `bash labs/module-01/l1_0-orientation/check_environment.sh --part2` and reply with its last line"; to learners new in Part 2, also the setup guide (the course project of Lesson 1.2 and the Part 2 starting site, `downloads/pagesco-part2-start.zip`) and the workbook's "Starting with Part 2" box. `docker pull composer:2` and `docker pull node:24` at home, on a good connection (the check pulls them if missing: about half a gigabyte to download, about 2 GB on disk).
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-09` from this kit's folder (16 results in about a minute and a half; every line PASS). With your own course project: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-09`.
- ☐ **Projector machine at the Part 2 starting site** (the end of Lesson 4.3): `pagesco` active, `wp plugin list` shows no pagesco-books yet. If yours is further along, snapshot it first (`wp db export before-s9.sql` and a copy of `wordpress/wp-content/`) and reset to the starting site; after the session you can restore. VS Code on `pagesco-dev` at a large font, two terminals (one for commands, one for `tail -f wordpress/wp-content/debug.log`), `lesson_14.html` and `lesson_15.html` in browser tabs, and `pagesco-seed.zip` downloaded from the course site.
- ☐ **Board, before learners arrive:** the plugin's layout (pagesco-books.php, composer.json, vendor/, uninstall.php, src/Plugin.php, src/Feature.php, src/Activator.php, src/StoreNotice.php); the life-cycle line **activate = set up once · deactivate = pause, keep data · uninstall = remove the plugin's data**; and **new feature = one class in src/ + one line in feature_classes()**.
- ☐ **Print or share:** the workbook (its Start Here page has the Part 2 check, the starting note and the refresher), sticky notes for help flags and the exit ticket.

KNOWN PART 2 TRAPS

SYMPTOM	CAUSE	FIX
`Failed opening required '.../vendor/autoload.php'` on activation	Composer never ran in this copy of the plugin	`composer dump-autoload` in the composer:2 container, in the plugin folder (Lesson 5.1, Common Mistakes 2).
`Class "PagesCo\Books\StoreNotice" not found`, the whole site down	A file name that does not match the class name letter for letter (Storenotice.php)	Rename the file; on macOS and Windows the bug hides until a Linux server (project Step 3).
`docker: invalid reference format` or `$(id -u)` errors in PowerShell	The Linux form of the composer command on Windows PowerShell	Leave out `--user`, write `${PWD}` (Lesson 1.3); better: run it in the Ubuntu terminal where the project lives.
The plugin activates but the activation callback never ran	`register_activation_hook()` inside a callback (plugins_loaded, init)	Top level of the main file, with `__FILE__` (Common Mistakes 3).
/books/ or /books/dune/ is a 404 although the code is right	Rewrite rules saved before the type existed	`wp rewrite flush` now; the activator flushes from then on (5.2, Common Mistakes 1).
Seeded books have no genres	`tax_input` in a script that runs as no user	`wp_set_object_terms()`, as the seeding script does (5.2, Common Mistakes 2).
`Activate the pagesco-books plugin first` from the seeding script	The plugin is inactive, or BookPostType/GenreTaxonomy are not in feature_classes()	`wp plugin list`, then `grep -n "::class," .../src/Plugin.php`.
The store notice shows the Thursday hours twice after the session	StoreNotice listed, but Step 2 (trimming pagesco-site.php) not done yet	Expected half-way through the project: Step 2 is the fix.
`Error: This does not seem to be a WordPress installation` from the seeding command	`pagesco-seed/` unzipped somewhere other than the WordPress root	It belongs in `wordpress/pagesco-seed/`, next to wp-load.php; run from the project folder with the `wp` shortcut.

SESSION 9 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	30	Opening Part 2: refresher, starting note and environment check	Five Part 1 questions, the "starting with Part 2" note for anyone new, `check_environment.sh --part2` on every machine.
0:30	10	**5.1** Frame and mental model	The shop's departments; plugin, must-use plugin, functions.php; the layout of pagesco-books.
0:40	25	**5.1** Live demo	Hello Dolly's header, the main file, Composer's autoloader, Plugin and Feature, the life cycle of a role, a late activation hook.
1:05	15	**5.1** Guided lab	`labs/module-09/l5_1-first-plugin/gift_wrap_starter/` in pairs: a one-file plugin with the whole life cycle.
1:20	15	**5.1** Start the lesson project	The first feature moves in: StoreNotice, Steps 1 and 2 (everything twice, then once); Steps 3 and 4 go home.
1:35	10	Break	Anyone without a working skeleton copies the lesson's files now (the lesson page has every file).
1:45	10	**5.2** Frame and mental model	A new drawer in the same filing cabinet; type, taxonomy, meta; why flushing matters.
1:55	25	**5.2** Live demo	The book type, the rules and a flush, the genre taxonomy, the activator that registers first, six meta keys, stale rules.
2:20	15	**5.2** Guided lab	`labs/module-09/l5_2-custom-post-types/book_quotes_starter/`: a second post type, REST and an activator.
2:35	15	**5.2** Start the lesson project	Fill the shelves: Steps 1 and 2 (the seeding script) and the start of Step 3.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 10.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 135 + 130 = 265 minutes for a learner working alone; after the 30-minute opening, the break and the wrap-up the room has 130. Both lessons keep their shape (model, demo, lab, project start), with two cuts: the long classes are copied, not typed, and the demos skip what learners can read alone.

Goes home (about 2 to 3 hours). 5.1: the project Steps 3 and 4 (the case of the missing class, the final check), Common Mistakes 1 and 2 (direct access, the missing autoloader) if the demo skipped them, the version note on load_plugin_textdomain(). 5.2: "Seeing Books in the Admin" (the meta box and the columns: Session 10 hardens that meta box, so it MUST be done), Common Mistakes 2 and 3, the project Steps 3 and 4. Each lesson's Hands-on Exercises, both Quick Quizzes and both journal entries.

Protect, in this order: the Part 2 check result for every learner; the 5.1 skeleton working on every machine (activated, `int(10)`), because everything in Part 2 is a feature of it; the 5.1 life cycle (activate, deactivate, uninstall); the 5.2 flush (it explains the most common bug of the module). **Cut first:** the late-hook mistake (it is in the lesson), then the 5.2 genre step (show the class, run its block at home), then the 5.2 project start (learners seed at home; it takes eight minutes).

OPENING SEGMENT

0:00 Opening Part 2: Refresher, Starting Note and Environment Check

30 min

TIMING

5m	Welcome to Part 2: what it builds, how the sessions run, who is new
8m	Part 1 refresher: R1 to R5 (think, pair, answer); the starting site on the projector
5m	"Starting with Part 2": what the lessons assume, for learners joining here
12m	Part 2 environment check: `check_environment.sh --part2` on every machine; sort the results

SEQUENCE AND TALKING POINTS

- **Welcome to Part 2 (5m).** Say: "Part 1 made Pages & Co. look right. Part 2 makes it DO things: a plugin, pagesco-books, that grows into a bookstore over ten sessions: books and genres, settings, an events calendar, blocks, REST endpoints, tests, a deployment, and the final project, Reserve & Pick Up." Same deal as Part 1: model, demo, lab and project start in the room, the rest at home. Ask who is new to the course and pair each new learner with a Part 1 learner for today.
- **Part 1 refresher (8m).** The workbook's R1 to R5, one minute alone, then pairs, then answers. Show the starting site on the projector while you go: the notice bar and its hooks, and the themes. R1 and R2 are the two that matter for today: Lesson 5.1's first feature hooks into the notice's filter and end action, and the plugin starts on plugins_loaded.
- **"Starting with Part 2" (5m).** Read the box in the workbook aloud for the new learners (Part 1 learners can check their site against it): what the site must contain, what the lessons assume you can do, and what to skim this week. New learners whose site is not the starting site follow the demos on a partner's screen today and install the Part 2 starting site at the break.
- **Part 2 environment check (12m).** On the projector, then theirs, from the project folder: `bash labs/module-01/l1_0-orientation/check_environment.sh --part2`. Same script as Session 1, plus three lines at the end. Done = the last line says `READY` or `READY WITH NOTES` (no Node 24 on the laptop is a NOTE: the node:24 image works until Module 6). Sort the FIX lines into three piles: Docker or the project (new learners: setup guide, pair today), composer:2 (pull it during the break), and the starting-site line (install the Part 2 starting site at the break; until then, a partner's screen).

REFRESHER ANSWERS (R1 TO R5)

1. **R1** `wp-content/mu-plugins/pagesco-site.php`; `pagesco_store_notice` is the filter (the text), `pagesco_before_store_notice` and `pagesco_store_notice_end` are actions. Lesson 5.1 hooks a plugin feature into the filter and the end action.

2. **R2** Load (wp-settings.php): must-use plugins, plugins, `plugins_loaded`, the theme, then `init`. pagesco-books starts its features on `plugins_loaded`; post types register on `init`.
3. **R3** A `templates/index.html` (Lesson 3.1); its `theme.json` holds the design. Part 2 never edits the theme's looks: books get their template in Module 6.
4. **R4** Because the action fires for every query, in the admin too: without the guards it would change the Books screen, widgets and REST lists (Lesson 4.2). The book archive at /books/ is a main query again.
5. **R5** A plugin (Lessons 3.4 and 4.3: the theme is the looks, the plugin is what the shop does). That rule is why Part 2 builds pagesco-books.

THE STARTING SITE ON THE PROJECTOR (PART 2 BEGINS HERE)

Three commands from the WP-CLI container shell, all on the site as Lesson 4.3 left it:

```
curl -s http://localhost:8080/ | grep -o '<div class="pagesco-notice">.*</div>'
```

```
<div class="pagesco-notice">Free shipping on orders over $35. Open until 9 p.m. o
```

```
wp eval 'foreach ( array( "pagesco_store_notice", "pagesco_store_notice_end" ) as
```

```
pagesco_store_notice: hooked
pagesco_store_notice_end: hooked
```

```
wp theme list --fields=name,status,version
```

```
+-------------------+----------+---------+
| name              | status   | version |
+-------------------+----------+---------+
| pagesco-classic   | inactive | 1.1.0   |
| pagesco-tt5-child | inactive | 1.0.0   |
| pagesco           | active   | 1.0.0   |
| twentytwentyfive  | inactive | 1.5     |
| twentytwentyfour  | inactive | 1.6     |
| twentytwentythree | inactive | 1.7     |
+-------------------+----------+---------+
```

The notice and its two hooked callbacks (the Thursday hours and the events link, which Lesson 5.1 moves into the plugin), the pagesco theme active, the child theme and the classic theme installed. No plugin is active yet: `wp plugin list` shows only Akismet and Hello Dolly, inactive, and the two must-use plugins.

WHAT THE PART 2 CHECK ADDS (READ FROM THE SCRIPT)

`--part2` runs every Session 1 check (Docker, the project, WordPress 7.1, PHP 8.5, WP-CLI, debugging, Mailpit, the ports) and then three more. Their possible lines, as the script prints them:

- `OK Node $nv and npm $(npm --version 2>/dev/null)`
- `CHECK Node $nv: the course uses Node 24 LTS; newer usually works`
- `CHECK no Node 24 on this computer ($nv): use the node:24 image as Lesson 6.1 shows (docker run ... node:24 npm ...)`
- `FIX no Node 24 and no node:24 image: install Node 24 LTS (nvm install 24) before Module 6`
- `OK $(echo`
- `FIX the composer:2 image does not run: docker pull composer:2 (Lesson 1.3)`
- `OK the site as Lesson 4.3 left it: pagesco active, pagesco-classic installed, mu-plugins/pagesco-site.php`
- `FIX the site is not where Part 2 starts (active theme '$theme'): finish Lesson 4.3, or import the Part 2 starting site (the course download pagesco-part2-start.zip: its README.txt)`

Most laptops: `OK` for Composer and the site, and `CHECK no Node 24 on this computer (v20.20.2): use the node:24 image as Lesson 6.1 shows (docker run ... node:24 npm ...)` → `READY WITH NOTES`. That is fine for Sessions 9 and 10; Node is needed from Session 11 (Lesson 6.1). The site line says `FIX the site is not where Part 2 starts (active theme 'twentytwentyfive'): finish Lesson 4.3, or import the Part 2 starting site (the course download pagesco-part2-start.zip: its README.txt)` on a site that is not at the end of Lesson 4.3.

WATCH-OUTS

- **Do not rebuild Part 1 in the room.** A new learner without the starting site cannot catch up by typing Lessons 2.1 to 4.3 today. Pair them, and install the Part 2 starting site at the break (`pagesco-part2-start.zip`: copy the folder into `wordpress/`, then `docker compose run --rm wpcli sh pagesco-part2-start/install.sh`, about a minute).
- **"My site is further along"** (a Part 1 learner who already started Lesson 5.1 at home): fine. They follow the 5.1 demo as a reading and help a new learner; the 5.2 demo is new for everyone.
- **Old habits.** Learners who wrote plugins before often put everything in one file with global functions. Today's structure (a namespace, Composer, one class per feature) is the course's convention from here to the final project; say so once, kindly.

LESSON PLANS

5.1 Your First Plugin

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Write a plugin header WordPress understands, including the version requirements it checks on activation
- Organize a plugin as a short main file plus namespaced classes in src/, loaded by Composer's PSR-4 autoloader
- Start features from one main object on plugins_loaded, and add a new feature by writing one class and listing it
- Use activation and deactivation hooks and uninstall.php for the right jobs, and say what each must never do
- Recognize the typical first-plugin failures: direct file access, a missing autoloader, a class the autoloader cannot find, a hook registered too late

TIMING IN THE ROOM

10m	Frame: the shop's departments; plugin, must-use plugin, functions.php; the layout
25m	Live demo: header and guard, main file, Composer, Plugin and Feature, the role's life cycle, a late hook
15m	Guided lab: `gift_wrap_starter/` in pairs
15m	Start the lesson project: StoreNotice, Steps 1 and 2

MENTAL MODEL TO SAY

Draw the shop on the board: "pagesco-books is the shop's back office. Each feature, the notice additions, the catalog, the events calendar, the settings page, is a department with its own room. The main file is the front door with the shop's name on it: it unlocks, switches on the lights and hands a list of departments to the manager, the Plugin object, who opens each one in the morning, on plugins_loaded. A new department means a new room and one new line on the manager's list, never knocking through the walls of the existing ones."

Then the three relatives in one line each: **must-use plugin** = always on, no switch (Lesson 2.3); **functions.php** = only while that theme is active (Lesson 4.3); **plugin** = the owner switches it on and off, and it can clean up after itself. "Everything the shop DOES goes in a plugin, so it survives the next redesign."

Then the two tools for a plugin that will have fifteen features: **a namespace** (`PagesCo\Books` , the prefix for classes) and **autoloading** (Composer's PSR-4 rule: `PagesCo\Books\Admin\SettingsPage` lives in `src/Admin/SettingsPage.php` , letter for letter).

◆ LIVE DEMO (YOU TYPE, THEY FOLLOW)

On the Part 2 starting site. Learners type along on theirs; the long classes they copy from the lesson page as you reach them. Ask for a prediction before every run.

1. **A plugin is a header and a guard (3m).** "You have met two relatives already: the must-use plugin of Lesson 2.3 and functions.php of Lesson 4.3. A normal plugin sits in between: the owner switches it on and off." In the WP-CLI container shell, show the top of Hello Dolly. Ask first: which line makes it a plugin?

```
sed -n '1,20p' wp-content/plugins/hello.php
```

```
<?php
/**
 * @package Hello_Dolly
 * @version 1.7.2
 */
/*
Plugin Name: Hello Dolly
Plugin URI: http://wordpress.org/plugins/hello-dolly/
Description: This is not just a plugin, it symbolizes the hope and enthusiasm
Author: Matt Mullenweg
Version: 1.7.2
Author URI: http://ma.tt/
Text Domain: hello-dolly
*/

// Don't load directly.
if ( ! defined( 'ABSPATH' ) ) {
    exit;
}
```

The comment block is the header (only `Plugin Name:` is required; WordPress reads it WITHOUT running the file), and the first code is the ABSPATH guard. Everything after that is two functions and two hooks: Lesson 2.3's toolkit.

2. **The main file (4m).** "pagesco-books starts as a folder and a main file of the same name." Learners type it with you, from the lesson page (it is short). Then ask WordPress what it sees:

```php
<?php
/**
 * Plugin Name:       Pages & Co. Books
 * Plugin URI:        https://example.com/pagesco-books/
 * Description:       The bookstore features of the Pages & Co. website: books
 * Version:           1.0.0
 * Requires at least: 6.8
 * Requires PHP:      8.2
 * Author:            Pages & Co.
 * Author URI:        https://example.com/
 * License:           GPL-2.0-or-later
 * License URI:       https://www.gnu.org/licenses/gpl-2.0.html
 * Text Domain:       pagesco-books
 *
 * @package PagesCo\Books
 */

defined( 'ABSPATH' ) || exit;

define( 'PAGESCO_BOOKS_VERSION', '1.0.0' );
define( 'PAGESCO_BOOKS_FILE', __FILE__ );
define( 'PAGESCO_BOOKS_DIR', plugin_dir_path( __FILE__ ) );
```

```
wp plugin list --fields=name,status,version
wp plugin get pagesco-books --fields=name,title,version,status
```

```
+---------------+----------+---------+
| name          | status   | version |
+---------------+----------+---------+
| akismet       | inactive | 5.7.2   |
| hello         | inactive | 1.7.2   |
| pagesco-books | inactive | 1.0.0   |
| mailpit-smtp  | must-use |         |
| pagesco-site  | must-use | 1.0.0   |
+---------------+----------+---------+
+---------+-------------------+
| Field   | Value             |
+---------+-------------------+
| name    | pagesco-books     |
| title   | Pages & Co. Books |
| version | 1.0.0             |
| status  | inactive          |
+---------+-------------------+
```

Point at `Pages & Co. Books` with a plain &: WordPress read the header without running a line. Installed, inactive, and that is right: there is nothing to run yet. Stop on the two requirement lines: they are the reason a PHP 8.1 server refuses the plugin politely (self-check Q1).

3 **Composer writes the autoloader (4m).** "A namespace is the prefix for classes; PSR-4 is the rule that turns a class name into a file name." Write composer.json (learners copy it from the lesson), then run Composer in its container from the project folder:

```
{
    "name": "pagesco/pagesco-books",
    "description": "The bookstore features of the Pages & Co. website.",
    "type": "wordpress-plugin",
    "license": "GPL-2.0-or-later",
    "require": {
        "php": ">=8.2"
    },
    "autoload": {
        "psr-4": {
            "PagesCo\\Books\\": "src/"
        }
    }
}
```

```
docker run --rm -it --user "$(id -u):$(id -g)" -v "$PWD/wordpress:/var/www/htm
```

```
Composer could not detect the root package (pagesco/pagesco-books) version, de
Generating autoload files
Generated autoload files
```

The first line is a harmless note (no version in composer.json). Windows PowerShell: no `--user`, `${PWD}` (Lesson 1.3). Then Feature.php and Plugin.php: learners COPY these two from the lesson page; you read Plugin.php aloud with them: `instance()`, `boot()` with its own filter `pagesco_books_features`, `feature()`, and the empty list in `feature_classes()`.

4 **Boot on plugins_loaded (3m).** "Two lines finish the main file: load the autoloader, hand boot() to plugins_loaded." Append them, activate, and ask PHP two questions:

```
require PAGESCO_BOOKS_DIR . 'vendor/autoload.php';

add_action( 'plugins_loaded', array( PagesCo\Books\Plugin::instance(), 'boot'

wp plugin activate pagesco-books
wp eval 'var_dump( has_action( "plugins_loaded", array( PagesCo\Books\Plugin::
```

```
Plugin 'pagesco-books' activated.
Success: Activated 1 of 1 plugins.
int(10)
bool(true)
```

Hooked at priority 10, and the class is loaded (`false` = "do not autoload just for this question"). The plugin runs, with no features yet.

5 **A role through the life cycle (6m).** "Three moments are different: activation, deactivation, uninstall." Copy Activator.php from the lesson, append the two `register_*_hook()` lines at the TOP LEVEL of the main file, write uninstall.php, then walk the role through all three:

```
register_activation_hook( __FILE__, array( PagesCo\Books\Activator::class, 'ac
register_deactivation_hook( __FILE__, array( PagesCo\Books\Activator::class, '

wp plugin deactivate pagesco-books
wp plugin activate pagesco-books
wp role list --fields=name,role
wp plugin deactivate pagesco-books
wp role list --field=role | grep -c pagesco_bookseller
wp plugin uninstall pagesco-books --skip-delete
wp role list --field=role | grep -c pagesco_bookseller
wp plugin activate pagesco-books
```

```
Plugin 'pagesco-books' deactivated.
Success: Deactivated 1 of 1 plugins.
Plugin 'pagesco-books' activated.
Success: Activated 1 of 1 plugins.
+---------------+--------------------+
| name          | role               |
+---------------+--------------------+
| Administrator | administrator      |
| Editor        | editor             |
| Author        | author             |
| Contributor   | contributor        |
| Subscriber    | subscriber         |
| Bookseller    | pagesco_bookseller |
+---------------+--------------------+
Plugin 'pagesco-books' deactivated.
Success: Deactivated 1 of 1 plugins.
1
Ran uninstall procedure for 'pagesco-books' plugin without deleting.
Success: Uninstalled 1 of 1 plugins.
0
Plugin 'pagesco-books' activated.
Success: Activated 1 of 1 plugins.
```

Read the two numbers: deactivation kept the role (1), uninstall removed it (0), the last activation made it again. Write the life-cycle table on the board: **activate = set up once · deactivate = pause, keep data · uninstall = remove the plugin's data**.

6 **Mistake: a late activation hook (4m).** "A tidy-minded developer moves everything into plugins_loaded, the activation hook too." Save the lesson's one-file demo plugin as wordpress/wp-content/plugins/pagesco-late-hook.php:

```php
<?php
/**
 * Plugin Name: Late Hook Demo
 */

add_action(
    'plugins_loaded',
    function () {
        register_activation_hook( __FILE__, 'pagesco_late_hook_activate' );
    }
);

/**
 * Should run once, on activation.
 */
function pagesco_late_hook_activate() {
    update_option( 'pagesco_late_hook', 'activated' );
}
```

```
wp plugin activate pagesco-late-hook
wp option get pagesco_late_hook
```

```
Plugin 'pagesco-late-hook' activated.
Success: Activated 1 of 1 plugins.
Error: Could not get 'pagesco_late_hook' option. Does it exist?
```

Activated, and the callback never ran: no option, no warning. On the activation request plugins_loaded fired BEFORE WordPress included the new file. Delete the demo:

```
wp plugin deactivate pagesco-late-hook
wp plugin delete pagesco-late-hook
```

```
Plugin 'pagesco-late-hook' deactivated.
Success: Deactivated 1 of 1 plugins.
Deleted 'pagesco-late-hook' plugin.
Success: Deleted 1 of 1 plugins.
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A file name that does not match the class.** `storeNotice.php`, `Store_Notice.php`, `class-store-notice.php` (Lesson 1.3's style). Coach: "Read the class name, swap PagesCo\Books\ for src/, add .php: is that the file?" (Self-check Q3; the psr4_paths lab.)

- **Hooks registered in the wrong place.** The activation hook inside a callback; `add_action( 'init', ... )` inside a class constructor that nothing creates. Coach: "Who calls this line, and when?" For features: only register() adds hooks, and boot() calls it. (Self-check Q6.)
- **Deleting data on deactivation.** "Clean up" in deactivate(). Coach with the life-cycle line on the board: deactivate is a pause. (Self-check Q4; the gift_wrap lab's uninstall.php.)

GUIDED LAB

Folder: `labs/module-09/l5_1-first-plugin/gift_wrap_starter/` (a one-file plugin, `pagesco-gift-wrap`, plus its uninstall.php) with `labs/module-09/l5_1-first-plugin/test_gift_wrap.sh`; your reference is `labs/module-09/l5_1-first-plugin/gift_wrap_solution/`. It practices the whole life cycle on a plugin small enough to hold in your head: the requirement headers, the guard, an activation hook that stores an option once, a filter on the store notice of Lesson 2.3, and uninstall.php. It is not the lesson's project (that is StoreNotice in pagesco-books).

- **Install and run it as shipped (3m).** `cp -r labs/module-09/l5_1-first-plugin/gift_wrap_starter wordpress/wp-content/plugins/pagesco-gift-wrap`, `wp plugin activate pagesco-gift-wrap`, copy the test into wordpress/ and `docker compose run --rm wpcli sh test_gift_wrap.sh`. Everything says TODO, the direct request is a **500** (no guard: the file runs without WordPress and dies on add_filter()), no price is stored.
- **TODOs 1 to 5 in pairs (10m).** After editing, the activation hook must run again: `wp plugin deactivate pagesco-gift-wrap && wp plugin activate pagesco-gift-wrap`, then the test. The test ends with `wp plugin uninstall --deactivate --skip-delete`, so activate it again before the next run. Circulate for the activation hook written inside `pagesco_gift_wrap_notice()` and for `get_option()` without a check for false.
- **Expected when done** (the solution's test):

```
Plugin 'pagesco-gift-wrap' activated.
Success: Activated 1 of 1 plugins.
header: requires WordPress 6.8, PHP 8.2
direct request: 200
stored price: 2.50
notice:
<div class="pagesco-notice">Free shipping on orders over $35. Open until 9 p.m. o
Deactivating 'pagesco-gift-wrap'...
Plugin 'pagesco-gift-wrap' deactivated.
Ran uninstall procedure for 'pagesco-gift-wrap' plugin without deleting.
Success: Uninstalled 1 of 1 plugins.
after uninstall: (not stored)
```

- **Debrief (2m):** "Why 200 now, and not a page?" (the guard ends the request quietly). "Where did the option go after the last command?" (uninstall.php; deactivation alone would have kept it).

- **Fast finishers:** `labs/module-09/l5_1-first-plugin/psr4_paths_starter.php` (the PSR-4 rule written by hand), then `labs/module-09/l5_1-first-plugin/debug_plugin_audit.php` (two Warnings and a silent wrong answer about Requires PHP).
- **Remove it afterwards:** `rm -r wordpress/wp-content/plugins/pagesco-gift-wrap wordpress/test_gift_wrap.sh`.

START THE LESSON PROJECT

The lesson project is **The First Feature Moves In**: the two notice callbacks that Lesson 2.3 promised to move become the plugin's first feature. In the room: Steps 1 and 2 (15 minutes), on top of the skeleton from the demo.

- Step 1: learners copy src/StoreNotice.php, put `StoreNotice::class,` in `feature_classes()` (editor or the lesson's `sed`), and look at the notice: everything twice. Ask why that is good news (the feature works, and Composer did not have to run again for the new class).
- Step 2: replace mu-plugins/pagesco-site.php with the lesson's 1.1.0 version, then check the notice with the plugin on and off:

```
curl -s http://localhost:8080/ | grep -o '<div class="pagesco-notice">.*</div>'
wp plugin deactivate pagesco-books
curl -s http://localhost:8080/ | grep -o '<div class="pagesco-notice">.*</div>'
wp plugin activate pagesco-books
```

```
<div class="pagesco-notice">Free shipping on orders over $35. Open until 9 p.m. o
Plugin 'pagesco-books' deactivated.
Success: Deactivated 1 of 1 plugins.
<div class="pagesco-notice">Free shipping on orders over $35.</div>
Plugin 'pagesco-books' activated.
Success: Activated 1 of 1 plugins.
```

- **Done in the room** = the notice once with the plugin active, and only the free-shipping text with it off: the plugin/must-use split in one picture.
- **Goes home:** Step 3 (rename StoreNotice.php to Storenotice.php on purpose, read the fatal, rename it back) and Step 4 (the final check and the line in pagesco-plan.txt).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Which line makes a PHP file a plugin? **Answer:** a header comment with `Plugin Name:`; WordPress reads it without running the file.
- What does `defined( 'ABSPATH' ) || exit;` stop? **Answer:** a direct request to the file from a

browser, which would run it without WordPress.

- Where does `PagesCo\Books\Admin\SettingsPage` live? **Answer:** src/Admin/SettingsPage.php, exactly that case.
- What may deactivation never do? **Answer:** delete the owner's data; only uninstall.php does that.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** B) WordPress refuses to activate it and says which PHP version the plugin requires; **Q2** `/var/www/html/wp-content/plugins/pagesco-books/`; **Q3** C) A fatal error: Class `PagesCo\Books\Admin\BookColumns` not found; **Q4** A) Delete the Bookseller role and the plugin's options; **Q5** `NULL`: uninstall.php removed the role, so get_role() finds nothing; **Q6** D) Because on the activation request plugins_loaded has already fired before WordPress includes the new plugin's file. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: a learner whose skeleton will not activate reads the error from the left with you: the file and line almost always point at a typo in the main file or a missing vendor/. If the demo ran away from them, copy all seven files from the lesson page at the break and run its checkpoint block.

Stretch: `psr4_paths_starter.php`, `debug_plugin_audit.php`; the lesson's Exercise 3 (switch a feature off from outside with `pagesco_books_features`) and the Bonus Challenge (a Features link on the Plugins screen).

5.2 Custom Post Types, Taxonomies & Meta

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Register a custom post type with register_post_type() on init, with labels, an archive and REST support
- Register a custom taxonomy with register_taxonomy() and attach it to the post type
- Register typed post meta with register_post_meta(), and explain why the keys are public and show_in_rest
- Add a meta box and extra admin columns so editors can see and change a book's details
- Flush rewrite rules at the right moment, and seed content safely with a WP-CLI script

TIMING IN THE ROOM

10m	Frame: a new drawer in the same filing cabinet; type, taxonomy, meta; why rules need a flush
25m	Live demo: the book type, the rules, the genre taxonomy, the activator, six meta keys, stale rules
15m	Guided lab: `book_quotes_starter/`
15m	Start the lesson project: Fill the Shelves, Steps 1 and 2 and the start of Step 3

MENTAL MODEL TO SAY

Back to Lesson 2.1's picture: "wp_posts is one big filing cabinet. A custom post type is a new labeled drawer in it, not a new cabinet: books are stored in the same table as posts and pages, with the same columns, so the Loop, WP_Query, revisions and the block editor work on them for free. A taxonomy is the colored tabs you clip onto the files to group them, a book's genres. Meta is the index card in each file with its own facts: this book's price, its author, its page count."

Then the three functions on the board, all called on init: **register_post_type()** (a kind of content: book), **register_taxonomy()** (a way to group it: genre), **register_post_meta()** (a named, typed field: pagesco_price). Each is one feature class in pagesco-books, the rule from Lesson 5.1.

Then the one thing that surprises everyone: "Registering a type adds URL rules, and WordPress keeps the rules in the database. Until something flushes them, /books/ is a 404." The activation hook gets its second job.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

On the site as Lesson 5.1 leaves it. Learners copy the three classes from the lesson page and type the `sed` and WP-CLI lines with you.

1. **The book post type (5m).** "A custom post type is a new value in wp_posts.post_type: a new drawer in the same filing cabinet." Learners copy src/BookPostType.php from the lesson (it is mostly labels); you

walk the arguments table: public, has_archive + rewrite, show_in_rest + rest_base, supports (custom-fields is required for meta in REST). List it:

```
sed -i '/StoreNotice::class,/a\        BookPostType::class,' wp-content/plugi
grep -n -A 3 "return array(" wp-content/plugins/pagesco-books/src/Plugin.php
```

```
75:     return array(
76-         StoreNotice::class,
77-         BookPostType::class,
78-     );
```

Note the NAME constant: no other file spells 'book'.

2 **Rules, and a flush (4m).** "The type registers on every request now. Does /books/ work?" Ask for predictions (most say yes). It would 404: the rules in the database know nothing about books until something flushes them.

```
wp rewrite flush
wp rewrite list --match=/books/ --fields=match,query,source
```

```
Success: Rewrite rules flushed.
+--------------------------+-----------------------------------------------------
| match                    | query
+--------------------------+-----------------------------------------------------
| books/?$                 | index.php?post_type=book
| (.?.+?)(?:/([0-9]+))?/?$ | index.php?pagename=$matches[1]&page=$matches[2]
| ([^/]+)(?:/([0-9]+))?/?$ | index.php?name=$matches[1]&page=$matches[2]
+--------------------------+-----------------------------------------------------
```

The first rule, `source other`, is the archive. A flush rebuilds the whole rules option: expensive, so WordPress does it only when asked. Then ask WordPress what it knows (scroll past core's types to the last row):

```
wp post-type list --fields=name,label,public,has_archive
wp eval 'echo get_post_type_object( "book" )->labels->add_new_item, "\n";'
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/books/
```

```
+---------------------+-----------------------+--------+-------------+
| name                | label                 | public | has_archive |
+---------------------+-----------------------+--------+-------------+
| post                | Posts                 | 1      |             |
| page                | Pages                 | 1      |             |
| attachment          | Media                 | 1      |             |
| revision            | Revisions             |        |             |
| nav_menu_item       | Navigation Menu Items |        |             |
| custom_css          | Custom CSS            |        |             |
| customize_changeset | Changesets            |        |             |
| oembed_cache        | oEmbed Responses      |        |             |
| user_request        | User Requests         |        |             |
| wp_block            | Patterns              |        |             |
| wp_template         | Templates             |        |             |
| wp_template_part    | Template Parts        |        |             |
| wp_global_styles    | Global Styles         |        |             |
| wp_navigation       | Navigation Menus      |        |             |
| wp_font_family      | Font Families         |        |             |
| wp_font_face        | Font Faces            |        |             |
| book                | Books                 | 1      | books       |
+---------------------+-----------------------+--------+-------------+
Add Book
200
```

3 **The genre taxonomy (3m).** "Genres are the colored tabs on the files." Copy src/GenreTaxonomy.php: the second argument of register_taxonomy() is the type it attaches to, `BookPostType::NAME`. List it and flush once more:

```
sed -i '/BookPostType::class,/a\            GenreTaxonomy::class,' wp-content/
wp rewrite flush
wp taxonomy list --fields=name,label,object_type,hierarchical
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/genre/fiction/
```

```
Success: Rewrite rules flushed.
+----------------------+--------------------+------------------------------
| name                 | label              | object_type
+----------------------+--------------------+------------------------------
| category             | Categories         | post
| post_tag             | Tags               | post
| nav_menu             | Navigation Menus   | nav_menu_item
| post_format          | Formats            | post
| wp_theme             | Themes             | wp_template, wp_template_part,
| wp_template_part_area | Template Part Areas | wp_template_part
| wp_pattern_category  | Pattern Categories | wp_block
| genre                | Genres             | book
+----------------------+--------------------+------------------------------
404
```

/genre/fiction/ is a 404 only because no genre terms exist yet: the seeding script creates them.

4 **Flushing where it belongs (5m).** "A site owner will not run wp rewrite flush. The activation hook will, but on that one request init has NOT run for our plugin." Replace Activator.php with the lesson's version (it registers the type and taxonomy itself, then flushes; deactivate() unregisters and flushes). Run the cycle:

```
wp plugin deactivate pagesco-books
wp plugin activate pagesco-books
curl -s -o /dev/null -w 'books active: %{http_code}\n' http://localhost:8080/b
wp plugin deactivate pagesco-books
curl -s -o /dev/null -w 'books off:    %{http_code}\n' http://localhost:8080/b
wp plugin activate pagesco-books
```

```
Plugin 'pagesco-books' deactivated.
Success: Deactivated 1 of 1 plugins.
Plugin 'pagesco-books' activated.
Success: Activated 1 of 1 plugins.
books active: 200
Plugin 'pagesco-books' deactivated.
Success: Deactivated 1 of 1 plugins.
books off:    404
Plugin 'pagesco-books' activated.
Success: Activated 1 of 1 plugins.
```

With the plugin off, /books/ is a 404: no broken links left behind, and the books stay in the database.

5 **Six meta keys (4m).** "The index card in each file: author, price, pages." Copy src/BookMeta.php: the keys are DATA in `fields()`, one loop registers them. Stop on the box "Why the keys do not start with an underscore" (public keys for REST and block bindings).

```
sed -i '/GenreTaxonomy::class,/a\          BookMeta::class,' wp-content/plugi
wp eval 'foreach ( array_keys( get_registered_meta_keys( "post", "book" ) ) as
```

```
pagesco_author
pagesco_price
pagesco_pages
pagesco_published
pagesco_in_stock
pagesco_isbn
```

6 **Mistake: stale rules (3m).** "The commonest custom-post-type bug: the code is right and single books 404." Save the rules once without the type, then flush:

```
wp eval 'unregister_post_type( "book" ); flush_rewrite_rules();'
curl -s -o /dev/null -w 'before flush: %{http_code}\n' http://localhost:8080/b
wp rewrite flush
curl -s -o /dev/null -w 'after flush:  %{http_code}\n' http://localhost:8080/b
```

```
before flush: 404
Success: Rewrite rules flushed.
after flush:  200
```

An EMPTY rules option would not show the bug (WordPress rebuilds it); stale rules do. Fix: flush on activation, never on init (self-check Q10).

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **"It's a 404, my code is wrong."** Coach: "When did the rules last change? `wp rewrite list --match=/books/`." Then the fix in the right place: the activator. (Self-check Q10.)
- **The activator that only flushes.** It trusts init to have registered the type; on the activation request it has not. Coach with the warning box "Registering is not enough on activation". (The book_quotes lab's TODO 4.)
- **Taxonomy or meta?** A "genre" meta key, a "price" taxonomy. Coach with the lesson's test: "Would you want a page that lists everything with this value?" (Self-check Q9.)

GUIDED LAB

Folder: `labs/module-09/l5_2-custom-post-types/book_quotes_starter/` (a plugin, `pagesco-quotes`) with `labs/module-09/l5_2-custom-post-types/test_book_quotes.sh`; your reference is `labs/module-09/l5_2-custom-post-types/book_quotes_solution/`. A second post type, `pagesco_quote`, for favorite lines from the shop's books: learners make it public with an archive at

/quotes/, put it in REST, register one public meta key, and write the activator that registers BEFORE it flushes. Same three functions as the lesson, on new data, in procedural style.

- **Install and run it as shipped (3m).** `cp -r labs/module-09/l5_2-custom-post-types/book_quotes_starter wordpress/wp-content/plugins/pagesco-quotes`, `wp plugin activate pagesco-quotes`, copy the test, run it: the archive is a 404 and REST says `rest_no_route`.
- **TODOs 1 to 4 in pairs (9m).** After each change: deactivate and activate the plugin (the activator must run), then the test. Circulate for `has_archive => true` (the archive is then /pagesco_quote/), for a meta key without `show_in_rest`, and for an activator that calls flush_rewrite_rules() alone.
- **Expected when done** (the solution's test):

```
Plugin 'pagesco-quotes' activated.
Success: Activated 1 of 1 plugins.
archive /quotes/: 200
REST /wp/v2/quotes:
[{"title":{"rendered":"A truth universally acknowledged"},"meta":{"pagesco_quote_
quotes left: 0
```

- **Debrief (3m):** "What did the activator have to do that init does on every other request?" (register first). "Why is the meta key in REST without a leading underscore?" (a protected key cannot be written through REST without an auth callback, and block bindings refuse it).
- **Fast finishers:** `labs/module-09/l5_2-custom-post-types/debug_meeting_type.php` (three registration bugs). `shelf_report_starter.php` needs the seeded catalog: it is homework after the project.
- **Remove it afterwards:** `wp plugin deactivate pagesco-quotes`, then `rm -r wordpress/wp-content/plugins/pagesco-quotes wordpress/test_book_quotes.sh` (deactivation unregisters the type and flushes the rules again).

START THE LESSON PROJECT

The lesson project is **Fill the Shelves**: the eleven classics of the Pages & Co. catalog, seeded with one WP-CLI script so the whole class has identical data (book IDs 30 to 40). In the room: Steps 1 and 2 and the start of Step 3 (15 minutes).

- Step 1: download pagesco-seed.zip from the lesson page and unzip it into `wordpress/pagesco-seed/` (next to wp-load.php). Read seed-books.php together for two minutes: the guard for the plugin, fixed term order, `import_id` for fixed IDs, `wp_set_object_terms()` instead of tax_input.
- Step 2: run it from the project folder:

```
wp eval-file pagesco-seed/seed-books.php
```

```
Genre 5: Fiction
Genre 6: Science Fiction
Genre 7: Classic
Genre 8: Fantasy
Genre 9: Dystopian
Book 30: Dune
Book 31: 1984
Book 32: To Kill a Mockingbird
Book 33: The Great Gatsby
Book 34: Pride and Prejudice
Book 35: The Hobbit
Book 36: Brave New World
Book 37: The Catcher in the Rye
Book 38: Lord of the Rings
Book 39: Fahrenheit 451
Book 40: Animal Farm
Success: 11 books in the catalog.
```

- Step 3, the start: the term list (five genres, eleven Fiction) and the REST view of Dune:

```
wp term list genre --fields=term_id,name,slug,count --orderby=term_id
```

```
+---------+-----------------+-----------------+-------+
| term_id | name            | slug            | count |
+---------+-----------------+-----------------+-------+
| 5       | Fiction         | fiction         | 11    |
| 6       | Science Fiction | science-fiction | 3     |
| 7       | Classic         | classic         | 6     |
| 8       | Fantasy         | fantasy         | 2     |
| 9       | Dystopian       | dystopian       | 3     |
+---------+-----------------+-----------------+-------+
```

```
{
  "id": 30,
  "slug": "dune",
  "title": {
    "rendered": "Dune"
  },
  "genres": [
    5,
    6
  ],
  "meta": {
    "pagesco_author": "Frank Herbert",
    "pagesco_price": 14.99,
    "pagesco_pages": 412,
    "pagesco_published": "1965-08-01",
    "pagesco_in_stock": true,
    "pagesco_isbn": ""
  }
}
```


- **Done in the room** = `Success: 11 books in the catalog.` and Dune's meta in REST with real types (a number, a boolean).
- **Goes home:** the lesson's "Seeing Books in the Admin" section (BookMetaBox and BookColumns: copy, list, look at the Books screen), the rest of Step 3 (the column check) and Step 4 (`rm -r pagesco-seed`). Session 10 hardens that meta box: it must exist by then.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Where are books stored? **Answer:** in wp_posts, with post_type book; their meta in wp_postmeta; genres in the term tables.
- Why does the activator register the type itself? **Answer:** on the activation request the plugin's own init callback has not run, so the rules would be flushed without the type.
- Why `pagesco_price` and not `_pagesco_price` ? **Answer:** an underscore key is protected: hidden from the Custom Fields box, not writable through REST without an auth callback, refused by block bindings.
- What happens to the books when the plugin is deactivated? **Answer:** they stay in the database; the type is unregistered, so they vanish from the admin and their URLs 404 until it is active again.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** B) `/books/` ; **Q8** `''` (an empty string); **Q9** A) As a taxonomy on the book type; **Q10** D) It rewrites the whole rules option on every page load: a needless, slow database write; **Q11** `[{"title":`

`{"rendered":"The Hobbit"}},{"title":{"rendered":"Lord of the Rings"}}]`; **Q12** C) `wp_set_object_terms( $id, $genres, 'genre' )` after the insert. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners who fell behind in the demo copy the three classes and the activator from the lesson page and run its `sed` blocks in order; the project's seeding script tells them if the type or taxonomy is missing. Pair them with someone whose `wp post-type list` shows book.

Stretch: `debug_meeting_type.php`, then (after seeding) `shelf_report_starter.php`; the lesson's Exercise 4 (one more meta key from a throwaway must-use plugin) and the Bonus Challenge (a sortable Price column).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note. Sort the notes before Session 10: question 2's "not yet" pile tells you how much of the warm-up goes to the life cycle.

1. Name the three files every new pagesco-books feature touches, and what each gets. **Answer:** a new class in src/ that implements Feature (its hooks in register()), one line in Plugin::feature_classes(), and (if it stores data) a line in uninstall.php.
2. What does each moment do to the Bookseller role: activation, deactivation, uninstall? **Answer:** creates it, keeps it, removes it.
3. /books/ is a 404 right after you registered the book type. Why, and where does the permanent fix go? **Answer:** the rewrite rules in the database were saved before the type existed; flush them, and flush in the activator (after registering the type) from then on.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 5.1: Steps 3 and 4 (the missing class on purpose, the final check, the line in pagesco-plan.txt). 5.2: "Seeing Books in the Admin" (BookMetaBox and BookColumns) and Steps 3 and 4 (the column check, then `rm -r pagesco-seed`). Session 10 starts from exactly this site.
- **Lesson exercises.** 5.1: Exercises 1 to 3 (an old server, a feature the plugin's way, switching a feature off); Exercise 4 is a stretch. 5.2: Exercises 1 to 3 (by author, by genre, the safe re-run); Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages (answers in the Answer Key).
- **Journal:** both entries. The 5.1 prompt asks for the next three features as class names: keep the list, Sessions 10 and 11 build three of them.

- **Labs not reached:** `labs/module-09/l5_1-first-plugin/psr4_paths_starter.php`, `labs/module-09/l5_1-first-plugin/debug_plugin_audit.php`, `labs/module-09/l5_2-custom-post-types/debug_meeting_type.php`, and after the project `labs/module-09/l5_2-custom-post-types/shelf_report_starter.php`.
- **Before the Next Lesson (from 5.2):** pagesco-books lists six features, eleven books are published, /books/ and /genre/fiction/ load, and the Book details box shows on a book.

BRING LIST FOR SESSION 10

- A laptop whose Part 2 check says READY (or READY WITH NOTES), with the site at the end of Lesson 5.2: `wp post list --post_type=book --format=count` prints 11 and the Book details box is on the book screen.
- Two browsers or a private window: Session 10 logs in as admin, maria (editor) and devon (author) side by side.
- The project folder with `labs/`, `pagesco-plan.txt`, the journal and the workbook.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the Part 2 check result for every learner; a working 5.1 skeleton on every machine; the 5.1 life cycle; the 5.2 flush.
- **Cut first (saves 4m):** the 5.1 late-hook mistake: point at Common Mistakes 3 on the page.
- **Cut second (saves 3m):** the 5.2 genre step: learners copy GenreTaxonomy.php and run its block at the start of the project.
- **Cut third (saves 15m):** the 5.2 project start: the seeding script is eight minutes at home and the lesson prints every output.

RUNNING SHORT: WHAT TO ADD

- Lesson 5.1's Common Mistakes 2 live (rename vendor/, activate, read the stack trace: `plugin_sandbox_scrape()` is the useful frame), then put it back.
- Lesson 5.1's Exercise 3 together: switch StoreNotice off from a must-use plugin with the plugin's own filter.
- Lesson 5.2's Common Mistakes 3: the REST API refusing "lots" as a page count, and the draft it leaves behind.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 10: Security and the Settings API

How to teach Session 10. Make the pagesco-books plugin safe with the four habits of WordPress security (sanitize in, escape out, a nonce for genuine requests, a capability for allowed users) and $wpdb->prepare(), showing each attack turn into a dead end on the course site, then give the shop a settings page with the Settings API, trace a save through options.php, and let the store notice read the settings.

2 lessons | 3 hrs session | Lessons 5.3–5.4 | Session 10 of 18 · Part 2

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's ideas) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Every command below was run to produce the output printed under it, on WordPress 7.1.3 / PHP 8.5: the 5.3 demo on the site as Lesson 5.2 leaves it (`after:lesson_15`), the 5.4 demo on the site as Lesson 5.3 leaves it. Ask for a prediction before every run and read every message from the left.
- **Type the short things, copy the long ones** (Session 9's rule): learners type the WP-CLI lines and the `sed` lines; they copy BookMetaBox.php, StockAction.php, Settings.php, SettingsPage.php and pagesco-save.sh from the lesson pages, then read them aloud with you.
- Keep `lesson_16.html` and `lesson_17.html` open in browser tabs. Labs live in `labs/module-10/`: `_starter` files are handouts, `_solution` files are yours.
- **Two browsers.** Today logs in as admin, maria (editor, `maria-pass`) and devon (author, `devon-pass`). A normal and a private window on the projector make the capability demos visible.

> **THE #1 RISK OF SESSION 10: SECURITY AS A LIST OF RULES**
>
> Five functions, four habits and six vulnerabilities fit on one slide and leave nobody's head the next day. The cure is the lesson's own method: every defense is shown turning a working attack into a dead end, on this very site. Do the attacks for real (the stored script tag, the forged request, the wrong user), keep the data flow drawing on the board all session (**genuine? allowed? sanitize → store → escape**), and send every learner question back to it: "which of the five boxes is this?" Lesson 5.4 then shows the same five boxes, built into options.php.

BEFORE THE SESSION

- ☐ **Homework check by email or at the door:** `wp post list --post_type=book --format=count` prints 11, and the Book details box shows on a book (Lesson 5.2's "Seeing Books in the Admin"). Lesson 5.3 replaces that meta box's save: without it, Step 1 of the project has nothing to harden.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-10` (12 results in about a minute; every line PASS). With your own course project: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-10`.
- ☐ **Projector machine at the end of Lesson 5.2** (`after:lesson_15`): eleven books, six features. Two browser windows: admin in one, maria in a private window; `lesson_16.html` and `lesson_17.html` in tabs; a terminal for `tail -f wordpress/wp-content/debug.log`.
- ☐ **Board, before learners arrive:** the data flow **request → genuine? (nonce) → allowed? (capability) → sanitize → store → escape → page**; the escaping table (text esc_html · attribute esc_attr · link esc_url · allowed HTML wp_kses_post); the two names that look alike, **group** `pagesco_books` and **page slug** `pagesco-books`.
- ☐ **Print or share:** the workbook, sticky notes for the exit ticket.

KNOWN TRAPS OF SESSION 10

SYMPTOM	CAUSE	FIX
"The link you followed has expired." on a real click	The link was built without wp_nonce_url(), or with a different action string than check_admin_referer()	The same action string on both sides: `self::ACTION . '_' . $post->ID`.
The meta box no longer saves anything	The save returns at the nonce check: the field name differs between render() and save()	`pagesco_books_details_nonce` in both; `wp_nonce_field( action, name )`.
`Error: The pagesco-books options page is not in the allowed options list.`	`settings_fields()` got the page slug instead of the group	`settings_fields( Settings::GROUP )`; `do_settings_sections( self::SLUG )` (5.4, Common Mistakes 1).
`Warning: Undefined array key "show_events_link"` in debug.log on save	The sanitize callback reads an unticked checkbox	`! empty( $input['show_events_link'] )` (Common Mistakes 2).
The notice sentence vanished after a WP-CLI change	`update_option()` with half the array	Save the form again (or `wp option patch update ...`).

SYMPTOM	CAUSE	FIX
pagesco-save.sh prints `options.php answered: 200` and an error page	Not logged in (wrong password) or the plugin is inactive	admin / password; `wp plugin list`.
Maria sees the settings page	A capability other than manage_options (edit_posts, read)	The capability is for what the page can change: manage_options.

SESSION 10 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	15	Warm-up	Five retrieval questions on Session 9; homework check on the projector (eleven books, six features, the Book details box).
0:15	10	**5.3** Frame and mental model	The shop's back door; one data flow: genuine? allowed? sanitize, store, escape.
0:25	30	**5.3** Live demo	Escaping a stored script tag, sanitizers, the nonce round trip, a prepared query, three mistakes.
0:55	15	**5.3** Guided lab	`labs/module-10/l5_3-security/staff_note_starter.php`: four requests against one save handler.
1:10	15	**5.3** Start the lesson project	Harden the plugin: Steps 1 and 2 (the hardened save, the stock action); Step 3 goes home.
1:25	10	Break	Learners whose meta box is missing copy BookMetaBox.php from Lesson 5.2 now.
1:35	10	**5.4** Frame and mental model	One labeled box on the shared shelf; menus and capabilities; the Settings API as data.
1:45	30	**5.4** Live demo	Settings with defaults, the page and its 403, the form, the save round trip, half an array.
2:15	15	**5.4** Guided lab	`labs/module-10/l5_4-settings-api/newsletter_settings_starter/`: a newsletter screen, saved like a browser saves it.
2:30	20	**5.4** Start the lesson project	Settings the shop can change: Steps 1 and 2 (the notice reads the settings, the admin changes it).
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 11.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 130 + 125 = 255 minutes for a learner working alone; after the warm-up, the break and the wrap-up the room has 145. Both lessons keep their shape; the demos run the attacks and skip the reading (the vulnerability table, the FAQ).

Goes home (about 2 to 3 hours). 5.3: the project Step 3 (the forged request and the checkpoint), the vulnerability table and the FAQ. 5.4: the project Step 3 (uninstall.php deletes the option; the uninstall test) and the lesson's Common Mistakes 1 (the group and the slug) if the demo skipped it. Each lesson's Hands-on Exercises, both Quick Quizzes, both journal entries.

Protect, in this order: the 5.3 data flow with two attacks shown for real (the stored script tag and the forged request); the hardened save (Step 1: it is the code every later handler copies); the 5.4 round trip through options.php (the save script) with one value refused. **Cut first:** the 5.3 sanitizer step (it is in the workbook's worked example), then 5.4's half-array mistake (one sentence: "update_option replaces the whole value"), then the 5.3 project Step 2 (the stock action goes home; the forged request needs it, so Step 3 goes too).

OPENING SEGMENT

0:00 Warm-up and Homework Check — 15 min

TIMING

8m	Five retrieval questions on Session 9 (W1 to W5), think-pair-share
7m	Homework check: the site Session 9 handed on, on the projector and on every laptop

SEQUENCE AND TALKING POINTS

- **Retrieval (8m).** The workbook's W1 to W5: two minutes alone, then pairs, then answers. W1 and W4 matter most today: every handler of this session is one feature class, and W4 is the save you harden in 25 minutes.
- **Homework check (7m).** On the projector, then on theirs, two commands from the WP-CLI container shell:

```
wp post list --post_type=book --format=count; echo
```

```
11
```

```
grep -c '::class,' wp-content/plugins/pagesco-books/src/Plugin.php
```

```
6
```

- Eleven books and six feature lines. Then open a book in wp-admin: the Book details box must be there. Anyone without it copies BookMetaBox.php and BookColumns.php from Lesson 5.2 and runs its `sed` line at the break.

WARM-UP ANSWERS (W1 TO W5)

1. **W1** A class in `src/` implementing `Feature` (hooks in `register()`), one line in `Plugin::feature_classes()`, and uninstall.php if it stores data.
2. **W2** On the activation request the plugin's own init callback has not run, so the rules would be rebuilt without the type.
3. **W3** `''`: meta is stored as strings, false as an empty string. REST casts it to false.
4. **W4** `wp_nonce_field( 'pagesco_books_save_details', ... )` in `render()` and `wp_verify_nonce()` in `save()`: today's lesson hardens that save.

5 **W5** Nothing: they stay in wp_posts; the type is unregistered, so they leave the admin and /books/ is a 404 until it is active again.

5.3 Security in WordPress

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Sanitize input on the way in and escape output on the way out, choosing the right function for each context
- Protect every state change with a nonce and a current_user_can() capability check
- Write safe database queries with $wpdb->prepare() and explain how SQL injection works
- Name the common WordPress vulnerabilities (XSS, CSRF, SQL injection, broken access control) and the defense against each
- Harden a real save handler and an admin action in the pagesco-books plugin

TIMING IN THE ROOM

10m	Frame: the shop's back door; the data flow; the four habits plus prepare()
30m	Live demo: escaping, sanitizers, the nonce, a prepared query, three mistakes
15m	Guided lab: `staff_note_starter.php` in pairs
15m	Start the lesson project: Steps 1 and 2

MENTAL MODEL TO SAY

"Your plugin's write handlers are the shop's back door, where deliveries come in. A **nonce** is checking the delivery has the token you handed the courier this morning, so it is not a stranger with a forged clipboard. The **capability** check is making sure the person signing for it works here. **Sanitizing** is unwrapping and inspecting the goods before they go on the shelf. **Escaping** is how you display a customer's name in the window: you print it as text, never as a sign someone else wrote for you. Skip any one step and the shop is exposed in a different way."

Then the flow on the board, one box per habit, left to right: **request → genuine? (nonce) → allowed? (capability) → sanitize → store → escape → page**, and under the store box: **SQL by hand →** `$wpdb->prepare()`. Every demo step below lands in one box: say which before you run it.

And one sentence on where WordPress helps: "The block editor and the REST API check nonces and capabilities for you. The moment you write your own form handler, admin action or AJAX endpoint, those checks are your job."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

On the site as Lesson 5.2 leaves it. Learners type along; ask for a prediction before every run.

1. **A stored script tag, escaped (4m).** "Every value from outside is untrusted. The author of a book comes from a form." Set Dune's author to a string full of HTML and print it the way the Books column does:

```
wp post meta update 30 pagesco_author 'Herbert & Sons <script>alert(1)</script
wp eval 'ob_start(); do_action( "manage_book_posts_custom_column", "pagesco_au
wp post meta update 30 pagesco_author 'Frank Herbert'
```

```
Success: Updated custom field 'pagesco_author'.
Herbert &amp;amp; Sons &amp;lt;script&amp;gt;alert(1)&amp;lt;/script&amp;gt;
Success: Updated custom field 'pagesco_author'.
```

Entities, not tags: `esc_html()` in BookColumns::print_column() did its job, and the last command put the real author back. Show the anti-example from the lesson (the same column without escaping) and say: "Stored XSS runs for every editor who opens the Books list."

2. **Sanitizers at work (3m).** "Escaping protects output; sanitizing protects what you store." Three deliberately bad inputs:

```
wp eval '$in = array( "price" => "-5.50", "pages" => "412 pages", "author" =>
echo "price:  ", max( 0, (float) $in["price"] ), "\n";
echo "pages:  ", absint( $in["pages"] ), "\n";
echo "author: ", sanitize_text_field( $in["author"] ), "\n";'
```

```
price:  0
pages:  412
author: O'Brien Press
```

Then the two gaps in Lesson 5.2's save, on the board: **autosaves** (no form fields, would wipe the values) and **validating** a date, not just cleaning it.

3. **The nonce round trip (3m).** "Is this request genuine?" The two functions behind every form token:

```
wp eval 'wp_set_current_user( 2 );
$token = wp_create_nonce( "pagesco_books_save_details" );
echo "verify good token: ", var_export( (bool) wp_verify_nonce( $token, "pages
echo "verify bad token:  ", var_export( (bool) wp_verify_nonce( "deadbeef", "p
```

```
verify good token: true
verify bad token:  false
```

In a form you never call them by hand: `wp_nonce_field()` prints the field, `check_admin_referer()` verifies it and stops with a 403. Then the second question, separately:

`current_user_can()`.

4 **A prepared query (4m).** "When you write SQL by hand, every value goes through a placeholder." Show the anti-example (concatenation) first, then run the safe report and read the SQL prepare() built:

```
wp eval 'global $wpdb;
$author    = "Frank Herbert";
$max_price = 20;
$sql = $wpdb->prepare(
    "SELECT p.post_title, pm_price.meta_value AS price
     FROM {$wpdb->posts} p
     INNER JOIN {$wpdb->postmeta} pm_author ON pm_author.post_id = p.ID AND pm
     INNER JOIN {$wpdb->postmeta} pm_price  ON pm_price.post_id  = p.ID AND pm
     WHERE p.post_type = %s AND pm_author.meta_value = %s AND pm_price.meta_va
    "book",
    $author,
    $max_price
);
echo $sql, "\n---\n";
foreach ( $wpdb->get_results( $sql ) as $row ) { echo $row->post_title, " $",
```

```
SELECT p.post_title, pm_price.meta_value AS price
     FROM wp_posts p
     INNER JOIN wp_postmeta pm_author ON pm_author.post_id = p.ID AND pm_autho
     INNER JOIN wp_postmeta pm_price  ON pm_price.post_id  = p.ID AND pm_price
     WHERE p.post_type = 'book' AND pm_author.meta_value = 'Frank Herbert' ANI
---
Dune $14.99
```

`'Frank Herbert'` was quoted, `20` went in as a number. One rule: no variable ever goes directly into a query string.

5 **Mistake: the wrong context (2m).** "Escaping is not one function."

```
wp eval '$url = home_url( "/books/?genre=sci-fi&sort=price" );
echo "esc_url:  ", esc_url( $url ), "\n";
echo "esc_html: ", esc_html( $url ), "\n";'
```

```
esc_url:  http://localhost:8080/books/?genre=sci-fi&#038;sort=price
esc_html: http://localhost:8080/books/?genre=sci-fi&amp;sort=price
```

esc_url() keeps a usable link (the & becomes & for an attribute); esc_html() turns it into display text.

6 **Mistake: a nonce but no capability (2m).** "A nonce proves the request is genuine, not that the user is allowed."

```
wp eval 'wp_set_current_user( 2 );
echo "editor edit Dune (30):   ", var_export( current_user_can( "edit_post", 3
wp_set_current_user( 3 );
echo "author edit Dune (30):   ", var_export( current_user_can( "edit_post", 3
```

```
editor edit Dune (30):   true
author edit Dune (30):   false
```

7 **Mistake: trusting an insert (2m).** "Database functions assume you already checked."

```
wp eval 'wp_set_current_user( 3 );
$id = wp_insert_post( array( "post_type" => "book", "post_title" => "Snuck in"
echo "created book as author Devon: ", ( $id ? "ID $id" : "failed" ), "\n";
wp_delete_post( $id, true );'
```

```
created book as author Devon: ID 41
```

An author created a published book (ID 41); the last line deleted it. Broken access control without any error message.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Escaping at input instead of output.** `esc_html()` in the save, raw `echo` in the column. Coach: "Where does the value land? Escape there, at the last moment, even if you sanitized it." (Self-check Q1, Q2.)
- **One check standing in for another.** A nonce instead of a capability, a capability instead of a nonce, `is_admin()` as a permission check (it only says the request is for an admin screen). Coach with the flow: "Which box does this function fill?" (Self-check Q3, Q5.)
- **Sanitizing a value that needs validating.** A date run through sanitize_text_field(). Coach: "Clean it, or judge it?" (Self-check Q6.)

GUIDED LAB

File: `labs/module-10/l5_3-security/staff_note_starter.php`; your reference is `labs/module-10/l5_3-security/staff_note_solution.php`. A save handler for a private staff note on a book, attacked by four requests the script plays as different users: no token, a forged token, the wrong user (Devon may not edit Dune) and a script tag from the right user. As shipped it saves everything. Learners add the nonce check, the capability check and the sanitizing in the order the lesson's hardened save uses.

- **Run it as shipped (3m).** `cp labs/module-10/l5_3-security/staff_note_starter.php wordpress/` then `wp eval-file --use-include staff_note_starter.php`: four times `saved`, the last one with `<script>` stored.
- **TODOs 1 to 4 in pairs (9m).** Circulate for the nonce action string (it must be the one the form was made with: `'pagesco_lab_note_' . $post_id`), for `current_user_can( 'edit_post' )` without the post ID, and for escaping instead of sanitizing.
- **Expected when done:**

```
no token, as Maria              refused: no token    stored: (nothing)
forged token, as Maria          refused: bad token   stored: (nothing)
real token, as Devon            refused: not allowed stored: (nothing)
real token, as Maria, with tag  saved                stored: Signed copy, ask at
```

- **Debrief (3m):** "Which request would a CSRF attack send?" (the forged token). "Why does Devon have a REAL token and still fail?" (he may open the form; he may not edit this book). "And the script tag?" (sanitize_text_field() stripped it; the column would still escape on output).
- **Fast finishers:** `labs/module-10/l5_3-security/prepared_search_starter.php` (placeholders, LIKE and a hostile value), then `labs/module-10/l5_3-security/debug_book_search.php` (two misused prepare() calls and a silent bug).

START THE LESSON PROJECT

The lesson project is **Harden the Plugin**: the real meta box save, and a "Mark sold out" row action built the safe way. In the room: Steps 1 and 2 (15 minutes).

- Step 1: learners replace BookMetaBox.php with the lesson's hardened version (copy it) and read the four numbered checks in save() aloud: real submission, not an autosave; our nonce; may edit this book; sanitize and validate. Check it:

```
php -l wp-content/plugins/pagesco-books/src/Admin/BookMetaBox.php
wp eval '$GLOBALS["current_screen"] = WP_Screen::get( "book" );
do_action( "add_meta_boxes_book", get_post( 30 ) );
echo isset( $GLOBALS["wp_meta_boxes"]["book"]["normal"]["high"]["pagesco-book-deta
```

```
No syntax errors detected in wp-content/plugins/pagesco-books/src/Admin/BookMetaB
Book details box registered
```

- Step 2: StockAction.php (copy it): the link built with wp_nonce_url() and only for users who may edit the book; the handler with check_admin_referer(), the capability, wp_safe_redirect(). List it and check who sees the link:

```
sed -i '/Admin\\BookColumns::class,/a\         Admin\\StockAction::class,' wp-cor
wp eval 'wp_set_current_user( 2 );
$a = apply_filters( "post_row_actions", array(), get_post( 30 ) );
echo "editor sees stock action: ", isset( $a["pagesco_stock"] ) ? "yes\n" : "no\n"
wp_set_current_user( 3 );
$a = apply_filters( "post_row_actions", array(), get_post( 30 ) );
echo "author sees stock action: ", isset( $a["pagesco_stock"] ) ? "yes\n" : "no\n"
```

```
editor sees stock action: yes
author sees stock action: no
```

- **Done in the room** = the box registered and the link for the editor only. **Goes home:** Step 3 (the forged request: a 403 "The link you followed has expired.", and the checkpoint), then a real click on "Mark sold out" in the browser.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- When do you escape? **Answer:** at the point of output, for the context it lands in.
- What does a nonce prove, and what does it not? **Answer:** that the request came from a page this site served to this user; not that the user is allowed.
- Does wp_insert_post() check capabilities? **Answer:** no; check current_user_can() yourself first.
- Why does the save skip autosaves? **Answer:** an autosave carries no form fields and would wipe the stored values.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** C) `esc_attr()` ; **Q2** `Pride & Prejudice <2nd ed.>` ; **Q3** B) The request came from a page this site served to this user, for this action, recently; **Q4** `SELECT ID FROM wp_posts WHERE post_title = 'Ender\'s Game'` ; **Q5** A) `current_user_can( 'edit_post', $book_id )` before the update; **Q6** D) Validating it: DateTime::createFromFormat('Y-m-d', $v) must give back exactly $v, else store ''. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a learner lost in the functions gets one card: **out = esc_* · in = sanitize_* · genuine = nonce · allowed = current_user_can · SQL = prepare**. Pair them on the lab with the flow drawing in front of them.

Stretch: `prepared_search_starter.php` , `debug_book_search.php` ; the lesson's Exercise 2 (esc_js) and Exercise 4 (a capability-gated bulk change), and the Bonus Challenge (a Duplicate row action).

5.4 Admin Pages, Settings API & Options

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Store a plugin's settings as one array option with defaults, and read it safely with get_option() and wp_parse_args()
- Add an admin page with add_options_page() (or a top-level menu) protected by the right capability
- Build a settings form with the Settings API: register_setting(), sections, fields, settings_fields() and do_settings_sections()
- Validate submitted values in a sanitize callback and report problems with add_settings_error()
- Trace what happens between Save Changes and the database: options.php, the nonce, the allowed options list
- Decide what uninstall.php removes, and remove the plugin's own settings

TIMING IN THE ROOM

10m	Frame: one labeled box on the shared shelf; menus and capabilities; the Settings API as data
30m	Live demo: Settings, the page and its 403, the form, the save round trip, half an array
15m	Guided lab: `newsletter_settings_starter/`
20m	Start the lesson project: Steps 1 and 2

MENTAL MODEL TO SAY

"wp_options is a shelf in the back office that every plugin shares. You could put each of your settings on it loose, four little notes with your name on them, or one labeled box, pagesco_books_settings, with all your notes inside. One box is easier to find, to load in one go, and to throw away when you leave."

Then the Settings API in four pieces on the board, next to Lesson 5.3's flow: **register_setting()** (the option, its group, its sanitize callback, its default) · **add_settings_section() / add_settings_field()** (the form as data, on admin_init) · **settings_fields() + do_settings_sections() + submit_button()** (the form on your page) · **options.php** (core's handler: nonce, capability, allowed list, update_option(), your callback). "Lesson 5.3's five boxes, built in."

And the two names that look alike, circled: **group** `pagesco_books` (settings_fields) and **page slug** `pagesco-books` (do_settings_sections, the URL).

LIVE DEMO (YOU TYPE, THEY FOLLOW)

On the site as Lesson 5.3 leaves it, with admin in one browser and maria in a private window.

1. **No such option (2m).** "Settings live in wp_options. Ask for ours before it exists."

```
wp option get pagesco_books_settings
```

```
Error: Could not get 'pagesco_books_settings' option. Does it exist?
```

2 **One class knows the option (6m).** "One labeled box on the shared shelf." Learners copy src/Settings.php from the lesson; walk its three parts: `defaults()`, `get()` with `wp_parse_args()`, and `register_setting()` on init (group, type object, sanitize callback, default). List it and ask again:

```
sed -i '/Admin\\StockAction::class,/a\          Settings::class,' wp-content/p
wp option get pagesco_books_settings
wp db query "SELECT COUNT(*) FROM wp_options WHERE option_name = 'pagesco_book
wp eval 'var_dump( PagesCo\Books\Settings::get( "events_limit" ) );'
```

```
array (
  'notice_extra' => 'Open until 9 p.m. on Thursdays.',
  'show_events_link' => true,
  'books_per_page' => 6,
  'events_limit' => 5,
)
COUNT(*)
0

int(5)
```

The defaults answer, the table has NO row (0), and `Settings::get()` returns a real int. Nothing is written until someone saves (self-check Q8).

3 **A page, and who may open it (5m).** "An admin page is a callback on admin_menu, guarded by a capability." Copy the lesson's first SettingsPage.php (placeholder body), list it, and look at the Settings menu as admin:

```
sed -i '/Settings::class,/a\            Admin\\SettingsPage::class,' wp-conten
grep -n "::class," wp-content/plugins/pagesco-books/src/Plugin.php
```

```
76:         StoreNotice::class,
77:         BookPostType::class,
78:         GenreTaxonomy::class,
79:         BookMeta::class,
80:         Admin\BookMetaBox::class,
81:         Admin\BookColumns::class,
82:         Admin\StockAction::class,
83:         Settings::class,
84:         Admin\SettingsPage::class,
```

```
http://localhost:8080/wp-admin/options-general.php   (View Source: the Setting
```

```
<a href='options-general.php' class="wp-first-item current" aria-current="page
<a href='options-connectors.php'>Connectors</a>
<a href='options-writing.php'>Writing</a>
<a href='options-reading.php'>Reading</a>
<a href='options-discussion.php'>Discussion</a>
<a href='options-media.php'>Media</a>
<a href='options-permalink.php'>Permalinks</a>
<a href='options-privacy.php'>Privacy</a>
<a href='options-general.php?page=pagesco-books'>Pages &#038; Co. Books</a>
```

Then the same page as Maria, the editor:

```
http://localhost:8080/wp-admin/options-general.php?page=pagesco-books   (as ma
```

```
HTTP/1.1 403 Forbidden

<div class="wp-die-message">Sorry, you are not allowed to access this page.</
```

A 403 before your callback runs: manage_options guards the menu item and the URL (self-check Q11).

4 **The form as data (6m).** "You could print a form by hand. For options, WordPress has done the work." Replace SettingsPage.php with the lesson's full version (copy it), read add_fields() aloud: two sections, four fields, `label_for`, and render() with `settings_fields()`, `do_settings_sections()`, `submit_button()`. The rows it builds:

```
http://localhost:8080/wp-admin/options-general.php?page=pagesco-books   (View
```

```
<tr><th scope="row"><label for="pagesco-notice-extra">Notice addition</label><
<tr><th scope="row"><label for="pagesco-show-events-link">Events link</label><
<tr><th scope="row"><label for="pagesco-books-per-page">Books per list</label>
<tr><th scope="row"><label for="pagesco-events-limit">Events to show</label></,
```

Inputs named `pagesco_books_settings[...]`: that array syntax makes PHP deliver the four values as one array, stored as one option.

5 **What Save Changes does (6m).** "Press Save Changes, or let a script do exactly what the browser does." Save the lesson's pagesco-save.sh in the WordPress root (copy it), then send the four fields with an Events value the callback must refuse:

```
sh pagesco-save.sh 'pagesco_books_settings[notice_extra]=Open until 9 p.m. on
wp option get pagesco_books_settings
```

```
options.php answered: 302 http://localhost:8080/wp-admin/options-general.php?p
Events to show must be between 1 and 20.
array (
  'notice_extra' => 'Open until 9 p.m. on Thursdays.',
  'show_events_link' => true,
  'books_per_page' => 8,
  'events_limit' => 5,
)
```

Read from the top: 302 = nonce, capability and allowed options list passed; your error instead of "Settings saved."; Books per list stored as 8, Events to show kept at 5. Draw the options.php flow from the lesson's diagram on the board.

6 **Mistake: half an array from code (3m).** "The form always sends every field. Code does not have to."

```
wp eval 'update_option( "pagesco_books_settings", array( "events_limit" => 8 )
wp option get pagesco_books_settings
```

```
array (
  'notice_extra' => '',
  'show_events_link' => false,
  'books_per_page' => 8,
  'events_limit' => 8,
)
```

The sentence is now empty and the link is off, with no error anywhere: update_option() replaced the whole value and the callback treated every missing key as empty. Put the form's values back:

```
sh pagesco-save.sh 'pagesco_books_settings[notice_extra]=Open until 9 p.m. on
```

```
options.php answered: 302 http://localhost:8080/wp-admin/options-general.php?p
Settings saved.
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **The group and the slug swapped.** "not in the allowed options list". Coach: "settings_fields() takes the FIRST argument of register_setting()." (Lesson quiz Q1.)
- **A sanitize callback that returns only what changed**, or `$input` unchanged. Coach: "It returns the complete value to store; what happens to a key you leave out?" (Self-check Q10; the newsletter lab's TODO 2.)
- **The wrong capability.** `edit_posts` so editors can "help". Coach: "What can this page change? Site-wide behavior: manage_options." (Self-check Q11.)

GUIDED LAB

Folder: `labs/module-10/l5_4-settings-api/newsletter_settings_starter/` (one must-use plugin file) with `labs/module-10/l5_4-settings-api/test_newsletter_settings.sh`; your reference is `labs/module-10/l5_4-settings-api/newsletter_settings_solution/`. A Settings > Newsletter screen in procedural style: one array option, a sanitize callback with an allowed list, two missing fields. The test logs in and saves the form exactly as the browser would (like the lesson's pagesco-save.sh): a sign-up text with tags, a frequency the screen does not offer, the checkbox unticked; then it opens the page as Maria.

- **Install and run it as shipped (3m).** `cp labs/module-10/l5_4-settings-api/newsletter_settings_starter/pagesco-lab-newsletter.php wordpress/wp-content/mu-plugins/`, copy the test into wordpress/, `docker compose run --rm wpcli sh test_newsletter_settings.sh`: one row, "Settings saved." and the hostile values stored as sent.
- **TODOs 1 to 3 in pairs (9m).** Open the screen in the browser too (Settings > Newsletter). Circulate for a callback that returns `$clean` with only two keys, for `in_array()` without `true`, and for a field callback named in add_settings_field() but spelled differently below.
- **Expected when done** (the solution's test):

```
rows: 3
<label for="pagesco-lab-signup-text">Sign-up text</label>
<label for="pagesco-lab-frequency">Frequency</label>
<label for="pagesco-lab-show-form">Sign-up form</label>
options.php answered: 302
Frequency must be weekly or monthly.
stored: {"signup_text":"Join our list","frequency":"monthly","show_form":false}
as maria: 403
```

- **Debrief (3m):** "Why `monthly` and not empty?" (the callback refused daily and kept the saved value). "Why `show_form: false`?" (the box was not sent). "Who checked the nonce?" (options.php).
- **Fast finishers:** `labs/module-10/l5_4-settings-api/debug_newsletter_sanitize.php`; after the project, `labs/module-10/l5_4-settings-api/settings_probe_starter.php`.
- **Remove it afterwards:** `rm wordpress/test_newsletter_settings.sh wordpress/wp-content/mu-plugins/pagesco-lab-newsletter.php` (the test deleted its option).

START THE LESSON PROJECT

The lesson project is **Settings the Shop Can Change**: the store notice reads the settings, the staff change it from the admin, and uninstalling cleans up. In the room: Steps 1 and 2 (20 minutes).

- Step 1: replace StoreNotice.php with the lesson's version (the callbacks keep their names; only their bodies change). With the saved values equal to the defaults, the notice must look as it has since Lesson 2.3:

```
http://localhost:8080/   (View Source: the notice)
```

```
<div class="pagesco-notice">Free shipping on orders over $35. Open until 9 p.m. o
```

- Step 2: marketing's request. In the browser: Notice addition "Signed copies of Dune this week.", Events link unticked, Save Changes. Or with the script, `<b>` tags included on purpose:

```
sh pagesco-save.sh 'pagesco_books_settings[notice_extra]=Signed copies of <b>Dune<
```

```
options.php answered: 302 http://localhost:8080/wp-admin/options-general.php?page
Settings saved.
```

```
<div class="pagesco-notice">Free shipping on orders over $35. Signed copies of Du
```

- The new sentence without the tags (sanitize_text_field() removed them on the way in) and no link.

Nobody touched PHP. Then put the usual notice back the same way.

- **Done in the room** = the notice changed from the admin and changed back. **Goes home:** Step 3 (uninstall.php deletes the option, the uninstall test, then `rm pagesco-save.sh`).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Why does `wp option get pagesco_books_settings` answer although the table has no row? **Answer:** register_setting()'s default is handed out while the row is missing.
- Where does the form post, and what does that file check? **Answer:** wp-admin/options.php: the nonce, manage_options, the allowed options list; then it saves through your sanitize callback.
- What arrives for an unticked checkbox? **Answer:** nothing; missing means off.
- What should uninstall.php do with the settings option, and with the books? **Answer:** delete the option (the plugin's own); keep the books (the shop's content).

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** A) Core's own handler, wp-admin/options.php, before it calls your sanitize callback; **Q8** `{"notice_extra":"Open until 9 p.m. on Thursdays.","show_events_link":true,"books_per_page":6,"events_limit":5}`; **Q9** C) Nothing: an unticked checkbox is not sent at all; **Q10** `{"notice_extra":"","show_events_link":false,"books_per_page":6,"events_limit":7}`: 0 is out of range, so books_per_page keeps the saved value (6, the default); notice_extra and show_events_link were not in the array, so they come back empty and false; **Q11** D) The manage_options capability passed to add_options_page(); **Q12** B) A counter of page views, updated on every request. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners who fall behind copy Settings.php and the full SettingsPage.php, run the two `sed` lines, and open the page in the browser: the form is the feedback. The newsletter lab then repeats every piece on a smaller scale.

Stretch: `debug_newsletter_sanitize.php` and `settings_probe_starter.php`; the lesson's Exercise 2 (a stylesheet only on your page), Exercise 4 (the settings in REST) and the Bonus Challenge (Reset to defaults).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note. Before Session 11, sort question 1's notes: anyone who cannot name the four checks gets the support card in the warm-up.

1. Name the four checks a handler that changes data makes, in order. **Answer:** genuine (nonce: check_admin_referer() / wp_verify_nonce()), allowed (current_user_can() for this object), sanitize and validate every value, and escape whatever it prints; SQL by hand through $wpdb->prepare().
2. What does options.php do between Save Changes and the database? **Answer:** checks the nonce of the group, manage_options and the allowed options list, then update_option() runs your sanitize callback, then it redirects back with settings-updated=true.
3. Your sanitize callback refuses a value. What should it store, and how does the user find out? **Answer:** the old (saved) value, and add_settings_error() queues a message the page prints above the form.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 5.3: Step 3 (the forged request, the checkpoint) and a real "Mark sold out" click in the browser. 5.4: Step 3 (uninstall.php deletes the option; the uninstall test; `rm pagesco-save.sh`). Session 11 starts from exactly this site: nine features.
- **Lesson exercises.** 5.3: Exercises 1 to 3 (the date validator, esc_js, a prepared count); Exercise 4 is a stretch. 5.4: Exercises 1 to 3 (wp option patch, a stylesheet on one page, a top-level menu); Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages (answers in the Answer Key).
- **Journal:** both entries. The 5.3 prompt traces one untrusted value through a handler you wrote; the 5.4 prompt lists everything the site stores and who owns it (Session 11 adds a table to that list).
- **Labs not reached:** `labs/module-10/l5_3-security/prepared_search_starter.php`, `labs/module-10/l5_3-security/debug_book_search.php`, `labs/module-10/l5_4-settings-api/debug_newsletter_sanitize.php`, and after the project `labs/module-10/l5_4-settings-api/settings_probe_starter.php`.
- **For Session 11:** install Node.js 24 LTS if `check_environment.sh --part2` said CHECK for Node (or `docker pull node:24`): Lesson 6.1 starts with it.

BRING LIST FOR SESSION 11

- The site at the end of Lesson 5.4: nine features, the store notice reading the settings, `wp option get pagesco_books_settings` printing the defaults.
- Node 24 on the laptop (`node --version` prints v24), or the node:24 image pulled. Session 11 runs `npm install` (about 540 MB of build tools): do it at home on a good connection if you can, after the session's first half shows you how.
- The project folder with `labs/`, the journal and the workbook.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** two attacks shown for real; the hardened save; the 5.4 round trip with one value refused.
- **Cut first (saves 3m):** the 5.3 sanitizer step (the workbook's worked example covers the same functions).
- **Cut second (saves 3m):** 5.4's half-array mistake: say the one sentence, point at Common Mistakes 3.
- **Cut third (saves 7m):** the 5.3 project Step 2: the stock action and its forged request go home together.

RUNNING SHORT: WHAT TO ADD

- Lesson 5.3's Exercise 4 together: a capability-gated bulk change, run as Maria, then as Devon.
- Lesson 5.4's Common Mistakes 1 live: swap the group for the slug, save, read "not in the allowed options list", swap back.
- Lesson 5.4's Exercise 4: the settings in /wp/v2/settings, and the schema that makes REST refuse a string for an integer.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 11: Custom Tables, Shortcodes and Block Setup

How to teach Session 11. Finish Module 5 with shortcodes that return escaped HTML and a custom events table built with dbDelta(), read and written with $wpdb and migrated by a schema version, then open Module 6: Node 24, create-block in the existing plugin, the @wordpress/scripts build and its manifest, and the Blocks feature that registers the Book Card.

2 lessons | 3 hrs session | Lessons 5.5-6.1 | Session 11 of 18 · Part 2

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame → live demo (you type, they type along) → guided lab (a kit lab, new practice) → start the lesson's own project. Exercises, quiz and journal are homework.
- Every command below was run to produce the output printed under it, on WordPress 7.1.3 / PHP 8.5: the 5.5 demo on the site as Lesson 5.4 leaves it (`after:lesson_17`), the 6.1 demo on the site as Lesson 5.5 leaves it, with Node 24 in the node:24 container (create-block, npm install and the build really ran).
- **Type the short things, copy the long ones**: the WP-CLI, sed and npm lines are typed; BookShortcode.php, EventsTable.php, the Activator, the test scripts, package.json and Blocks.php are copied from the lesson pages and read aloud.
- Keep `lesson_18.html` and `lesson_19.html` open. Labs live in `labs/module-11/`: `_starter` files and folders are handouts, `_solution` ones are yours.
- Session 11 is the hinge between PHP and JavaScript: the first half finishes Module 5 (data), the second starts Module 6 (blocks). Lesson 6.1 needs Node 24 and a 540 MB npm install: the warm-up starts that download.

> **THE #1 RISK OF SESSION 11: THE NPM INSTALL EATS LESSON 6.1**
>
> Twenty laptops downloading 540 MB of build tools at once over venue Wi-Fi can take longer than the whole lesson. **Before:** Session 10's bring list asked for Node 24; email again two days ahead. **In the warm-up:** every learner creates pagesco-books/package.json from Lesson 6.1 (it does not depend on Lesson 5.5) and starts `npm install` in a second terminal, so the download runs through Lesson 5.5. **In the demo:** if a laptop is still installing, the learner follows on the projector and builds at the project start. The shelf_label lab needs no Node at all (its build/ is prebuilt).

BEFORE THE SESSION

- ☐ **Homework check by email:** the site at the end of Lesson 5.4 (nine features, `wp option get pagesco_books_settings` prints the defaults), and `node --version` printing v24 (or `docker pull node:24` done).
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-11` (19 results in about two minutes; every line PASS).
- ☐ **Projector machine at the end of Lesson 5.4** (`after:lesson_17`), Node 24 installed, `npm install` for pagesco-books already done once at home (so the npm cache is warm), VS Code, two terminals, the post editor in a browser tab.
- ☐ **Board:** the storage table (content = post type, groups = taxonomy, site-wide values = option, many uniform rows = custom table); the kit-workshop-shelf drawing: **src/blocks → npm run build → build/blocks + manifest → Blocks feature on init → the editor and the front end**.
- ☐ **Print or share:** the workbook, sticky notes.

KNOWN TRAPS OF SESSION 11

SYMPTOM	CAUSE	FIX
The shortcode's output appears above the page	The callback echoes	Build a string and return it (ob_start() / ob_get_clean() for template-style HTML).
A new column never appears	schema() changed, DB_VERSION not raised	Raise DB_VERSION; the next request migrates.
`Unknown column` after renaming a column in schema()	dbDelta() only adds; the old column stays and the new one is empty	An explicit ALTER TABLE step for that version.
`npx` asks "Ok to proceed?" and seems to hang	The first create-block download	Answer y.
A second plugin `book-card` appears in the Plugins list	create-block without --no-plugin, in wp-content/plugins	`rm -r wp-content/plugins/book-card` ; scaffold with --no-plugin --target-dir.
"The specified manifest file does not exist." above every page	No build/ (fresh checkout, npm never ran)	`npm install && npm run build` in the plugin folder.
The editor shows the old block after a change	build/ is stale	`npm run build` , or keep `npm start` running.

SYMPTOM	CAUSE	FIX
`npm` or `node` not found in Ubuntu on Windows	Node installed in Windows, not in WSL	Install Node inside WSL (nvm), or use the node:24 image.
EACCES errors from npm in the docker run line	The container ran as root, or HOME is not writable	The lesson's `--user "$(id -u):$(id -g)" -e HOME=/tmp`.

SESSION 11 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	15	Warm-up	Five retrieval questions on Session 10; homework check; `node --version` on every laptop and the npm install started early.
0:15	10	**5.5** Frame and mental model	A placeholder card in the shop window; when a custom table is right.
0:25	25	**5.5** Live demo	The book card shortcode and its attributes, the events table with dbDelta(), a row's life with $wpdb, a schema change without a version.
0:50	15	**5.5** Guided lab	`labs/module-11/l5_5-custom-tables/wishlist_table_starter.php`: a throwaway table from CREATE to DROP.
1:05	15	**5.5** Start the lesson project	The events calendar: Steps 1 and 2 (the migration to version 2, the table dropped on uninstall).
1:20	10	Break	Laptops whose npm install has not finished keep it running; nobody starts it now on venue Wi-Fi without asking.
1:30	10	**6.1** Frame and mental model	A kit, a workshop and a shelf; what a block is for a developer.
1:40	35	**6.1** Live demo	Node 24, create-block --no-plugin, package.json and the build, the manifest, the Blocks feature, a missing build, a stale build.
2:15	15	**6.1** Guided lab	`labs/module-11/l6_1-block-setup/shelf_label_starter/`: register a prebuilt block from its manifest.
2:30	20	**6.1** Start the lesson project	The Book Card's first draft: Step 1 (make it the shop's, rebuild) and a look in the editor (Step 2).
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 12.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 140 + 135 = 275 minutes for a learner working alone; the room has 145. Both lessons keep their shape; Lesson 6.1's demo gets 35 minutes because the toolchain is new for everyone.

Goes home (about 2 to 3 hours). 5.5: project Steps 3 and 4 (the [pagesco_events] shortcode, the seeded events, the Events page) and Common Mistakes 1 and 3 if skipped. 6.1: the project Steps 3 and 4 (the block on the front end, the final check) and the FAQ. Each lesson's Hands-on Exercises, both Quick Quizzes and both journal entries.

Protect, in this order: a working build and a registered Book Card on every laptop (Sessions 12 and 13 build on it); the 5.5 table with a versioned migration; one shortcode that returns. **Cut first:** the 5.5 attribute cases (the self-check covers them), then the 6.1 stale-build mistake (say the sentence), then the 5.5 project Step 2 (uninstall.php goes home).

OPENING SEGMENT

0:00 Warm-up, Homework Check and Node — 15 min

TIMING

7m	Five retrieval questions on Session 10 (W1 to W5)
8m	Homework check, then `node --version` and the early `npm install`

SEQUENCE AND TALKING POINTS

- **Retrieval (7m).** W1 to W5 in the workbook, then answers. W4 (%i) and W5 (uninstall) lead straight into Lesson 5.5.
- **Homework check (3m).** From the WP-CLI container shell:

```
grep -c '::class,' wp-content/plugins/pagesco-books/src/Plugin.php
```

```
9
```

```
wp option get pagesco_books_settings --format=json
```

```
{"notice_extra":"Open until 9 p.m. on Thursdays.","show_events_link":true,"books_
```

- **Node and the early install (5m).** Everyone: `node --version` (the course runs v24.21.0; any v24 is right). Then create wordpress/wp-content/plugins/pagesco-books/package.json from Lesson 6.1 (copy it) and start `npm install` in that folder in a second terminal (or the lesson's docker run line with `npm install`). It runs while we do Lesson 5.5.

WARM-UP ANSWERS (W1 TO W5)

1. **W1** Genuine (nonce), allowed (`current_user_can()` for this object), sanitize and validate, escape on output; SQL by hand through `$wpdb->prepare()`.
2. **W2** The group (`pagesco_books`), the first argument of `register_setting()`; `do_settings_sections()` takes the slug.
3. **W3** Nothing: missing means off (`! empty( ... )`).
4. **W4** `%i` (WordPress 6.2+), today's lesson builds a whole table with it.

5 **W5** The plugin's own data (the settings option, the role); the shop's books stay. Today the events table joins the "remove" list.

5.5 Shortcodes, $wpdb & Custom Tables

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Register shortcodes with add_shortcode() whose callbacks return escaped HTML, with attributes handled by shortcode_atts()
- Decide when a custom table is the right storage, and when post types, meta, taxonomies or options are
- Create and upgrade a table with dbDelta() and a schema version stored in an option
- Read and write rows with $wpdb: insert(), update(), delete(), get_row(), get_var(), get_results(), always through prepare() or format arrays
- Remove the table and its version option in uninstall.php

TIMING IN THE ROOM

10m	Frame: the placeholder card; when a custom table is right
25m	Live demo: BookShortcode, attributes, the table, a row's life, a schema change without a version
15m	Guided lab: `wishlist_table_starter.php`
15m	Start the lesson project: Steps 1 and 2

MENTAL MODEL TO SAY

"Picture the shop window with a small card that reads 'Book of the Month goes here'. Whoever dresses the window does not need to know the price or the stock; every morning someone swaps the card for the real book. A shortcode is that card: the editor writes `[pagesco_book id="30"]`, and on every page view your callback swaps it for the book's current title, price and stock."

Then the second picture: "Post types, meta, taxonomies and options give you admin screens, REST, revisions and export for free. A table of your own gives all that up, so it is the last choice, for many rows with the same few fields, queried by those fields: events, later bookings and holds." And the one rule for both: **values reach SQL only through placeholders**.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

On the site as Lesson 5.4 leaves it.

1. **A shortcode returns, never echoes (4m).** "A shortcode is a placeholder card in the shop window." Learners copy src/Shortcodes/BookShortcode.php; read render() aloud: shortcode_atts(), absint(), a check that it is a published book, every value escaped, a returned string. List it and run it by hand:

```
sed -i '/Admin\\SettingsPage::class,/a\        Shortcodes\\BookShortcode::cla
wp eval 'echo do_shortcode( "[pagesco_book id=\"30\"]" ), "\n";'
```

```
<span class="pagesco-book"><a href="http://localhost:8080/books/dune/">Dune</a
```

2 **Four ways to write the tag (3m).** "Attributes arrive as strings, or as an empty string." Save the lesson's test file in the WordPress root and run it:

```
<?php
$cases = array(
    '[pagesco_book id="33"]',              // another book
    '[pagesco_book id="12"]',              // ID 12 is the About page, not a
    '[pagesco_book]',                      // no attributes at all
    '[pagesco_book ID="35" color="red"]',  // upper-case name, unknown attrib
);
foreach ( $cases as $case ) {
    echo $case, "\n  => '", do_shortcode( $case ), "'\n";
}
```

```
[pagesco_book id="33"]
  => '<span class="pagesco-book"><a href="http://localhost:8080/books/the-grea
[pagesco_book id="12"]
  => ''
[pagesco_book]
  => ''
[pagesco_book ID="35" color="red"]
  => '<span class="pagesco-book"><a href="http://localhost:8080/books/the-hobb
```

A page that is not a book, no attribute at all: an empty string, never an error message in a public page. Upper-case names are lowercased; undeclared attributes dropped.

3 **When a table is right (2m).** Put the lesson's storage table on the board (post type, taxonomy, option, custom table) and ask where events go before showing it. Many uniform rows, listed by date, never written in the editor: a table, with everything a post type gives for free now written by hand.

4 **The table, with dbDelta() (6m).** Learners copy src/EventsTable.php (version 1) and the new Activator.php; point at the four formatting rules of dbDelta() and at name() built from $wpdb->prefix. List the feature and reactivate:

```
sed -i '/Admin\\SettingsPage::class,/a\        EventsTable::class,' wp-conten
wp plugin deactivate pagesco-books
wp plugin activate pagesco-books
wp option get pagesco_books_db_version
```

```
Plugin 'pagesco-books' deactivated.
Success: Deactivated 1 of 1 plugins.
Plugin 'pagesco-books' activated.
Success: Activated 1 of 1 plugins.
1
```

```
DESCRIBE wp_pagesco_events;
```

```
+------------+-----------------+------+-----+---------+----------------+
| Field      | Type            | Null | Key | Default | Extra          |
+------------+-----------------+------+-----+---------+----------------+
| id         | bigint unsigned | NO   | PRI | NULL    | auto_increment |
| title      | varchar(200)    | NO   |     | NULL    |                |
| starts_at  | datetime        | NO   | MUL | NULL    |                |
| location   | varchar(200)    | NO   |     |         |                |
| created_at | datetime        | NO   |     | NULL    |                |
+------------+-----------------+------+-----+---------+----------------+
```

Version 1 recorded, starts_at indexed (MUL).

5 **A row's whole life (4m).** "Six methods, every value through a placeholder." The lesson's script in the WordPress root:

```php
<?php
use PagesCo\Books\EventsTable;

global $wpdb;
$table = EventsTable::name();

$id = EventsTable::add( 'Test event', '2027-10-31 12:00:00', 'Back office' );
echo "insert: new ID $id\n";

$row = $wpdb->get_row( $wpdb->prepare( 'SELECT title, location FROM %i WHERE i
echo "get_row: {$row->title} in the {$row->location}\n";

$changed = $wpdb->update( $table, array( 'location' => 'Café' ), array( 'id' =
echo "update: $changed row(s) changed\n";

echo 'prepare: ', $wpdb->prepare( 'SELECT COUNT(*) FROM %i WHERE location = %s

$deleted = $wpdb->delete( $table, array( 'id' => $id ), array( '%d' ) );
echo "delete: $deleted row(s) deleted\n";
echo 'rows left: ', $wpdb->get_var( $wpdb->prepare( 'SELECT COUNT(*) FROM %i',
```

```
insert: new ID 1
get_row: Test event in the Back office
update: 1 row(s) changed
prepare: SELECT COUNT(*) FROM `wp_pagesco_events` WHERE location = 'Children\
delete: 1 row(s) deleted
rows left: 0
```

Point at the prepare: line: the table in backticks from %i, the apostrophe escaped. Then `rm events-crud.php`.

6 **Mistake: a schema change without a version (4m).** "The shop wants a host per event. Add the column, reload, and..."

```
sed -i 's/^\t\t\tcreated_at datetime NOT NULL,$/&\n\t\t\thost varchar(100) NOT
grep -n "host varchar" wp-content/plugins/pagesco-books/src/EventsTable.php
curl -s -o /dev/null http://localhost:8080/
wp option get pagesco_books_db_version
wp db query "SELECT COUNT(*) AS host_columns FROM information_schema.COLUMNS W
```

```
75:         host varchar(100) NOT NULL,
1
host_columns
0
```

0 host columns: maybe_upgrade() only runs when DB_VERSION is higher. Take the line out again (`sed -i '/host varchar(100)/d' ...`). The project does it right.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A shortcode that echoes.** Coach: "Where does echo print, and when does the callback run?" (the open_hours lab shows it).
- **Hard-coding wp_ or interpolating values into SQL.** Coach: "Who controls this string? $wpdb->prefix is WordPress's; anything from a visitor goes through a placeholder." (Self-check Q3.)
- **A schema change without DB_VERSION**, or a rename expected from dbDelta(). Coach with the two mistakes of the lesson: dbDelta only ADDS, and only when asked. (Self-check Q5.)

GUIDED LAB

File: `labs/module-11/l5_5-custom-tables/wishlist_table_starter.php`; your reference is `labs/module-11/l5_5-custom-tables/wishlist_table_solution.php`. A throwaway wishlist table (which reader wants which book) from CREATE to DROP in one script: dbDelta() in the right shape, inserts with format arrays, a JOIN with wp_posts through prepare() and %i, an update with a where format, then DROP TABLE and a proof it is gone. Not the events table (the project's).

- **Run it as shipped (2m):** five TODO lines.
- **TODOs 1 to 5 in pairs (10m),** running after each. Circulate for `PRIMARY KEY (id)` with one space, a comma after the last column, and values written into the SQL instead of the format array. If a run stops half-way: `wp db query "DROP TABLE IF EXISTS wp_pagesco_lab_wishlist"`.
- **Expected when done:**

```
Created table wp_pagesco_lab_wishlist
rows: 4, last ID 4
Dune: 2 reader(s)
Fahrenheit 451: 1 reader(s)
The Hobbit: 1 reader(s)
updated: 1 row(s)
tables left: 0
```

- **Debrief (3m):** "What did dbDelta() return, and what would it return on a second run?" (an empty array: nothing to change). "Which line would break if a reader were named O'Brien?" (none: placeholders).
- **Fast finishers:** the must-use plugin `labs/module-11/l5_5-custom-tables/open_hours_starter/` with `labs/module-11/l5_5-custom-tables/test_open_hours.sh`, then `labs/module-11/l5_5-custom-tables/debug_events_report.php` (after the project's seeding).

START THE LESSON PROJECT

The lesson project is **The Events Calendar**. In the room: Steps 1 and 2 (15 minutes).

- Step 1: replace EventsTable.php with version 2 (copy it: DB_VERSION 2, a capacity column, add() and find() learn it). Do NOT reactivate; load a page and check:

```
curl -s -o /dev/null http://localhost:8080/
wp option get pagesco_books_db_version
```

```
2
```

```
SELECT COLUMN_NAME, COLUMN_TYPE, COLUMN_DEFAULT
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'wp_pagesco_events'
ORDER BY ORDINAL_POSITION;
```

```
+-------------+-------------------+----------------+
| COLUMN_NAME | COLUMN_TYPE       | COLUMN_DEFAULT |
+-------------+-------------------+----------------+
| id          | bigint unsigned   | NULL           |
| title       | varchar(200)      | NULL           |
| starts_at   | datetime          | NULL           |
| location    | varchar(200)      |                |
| created_at  | datetime          | NULL           |
| capacity    | smallint unsigned | 0              |
+-------------+-------------------+----------------+
```

- capacity was appended at the end: a fresh install from version 2 has the statement's order instead, which is why nobody SELECTs *.
- Step 2: replace uninstall.php (the table and both options go, the books stay) and test it while the table is still empty:

```
wp plugin uninstall pagesco-books --deactivate --skip-delete
wp db query "SELECT COUNT(*) AS events_tables FROM information_schema.TABLES WHERE
wp option list --search='pagesco_books*'
wp plugin activate pagesco-books
wp option get pagesco_books_db_version
```

```
Deactivating 'pagesco-books'...
Plugin 'pagesco-books' deactivated.
Ran uninstall procedure for 'pagesco-books' plugin without deleting.
Success: Uninstalled 1 of 1 plugins.
events_tables
0

+-------------+--------------+
| option_name | option_value |
+-------------+--------------+
+-------------+--------------+
Plugin 'pagesco-books' activated.
Success: Activated 1 of 1 plugins.
2
```

- **Goes home:** Step 3 (EventsShortcode, the seeding script, six events) and Step 4 (the Events page shows five, sized by the setting).

CHECKS FOR UNDERSTANDING

- What must a shortcode callback do with its HTML? **Answer:** return it, escaped; never echo.
- When is a custom table the right storage? **Answer:** many rows with the same few fields, queried by those fields, not edited as content.
- How does an update add a column on a live site? **Answer:** a higher DB_VERSION; maybe_upgrade() on init runs install() and dbDelta() adds it.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** `{"limit":"3","location":""}`; **Q2** A) A REST route, admin screens, revisions and export; **Q3** `SELECT title FROM `wp_pagesco_events` WHERE location = 'Café'`; **Q4** B) `$wpdb->insert_id`; **Q5** C) Because a plugin update replaces the files but does not run the activation hook again; **Q6** `<ul class="pagesco-events"><li class="pagesco-event"><strong>Book club: Dune</strong> <time datetime="2027-11-04T19:00">Thursday, November 4, 7:00 pm</time> <span class="pagesco-event__where">Reading corner, 20 seats</span></li></ul>`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners new to SQL write the CREATE TABLE on paper first, column by column, against DESCRIBE's output. **Stretch:** the open_hours lab and debug_events_report.php; Exercise 3 (an enclosing shortcode) and Exercise 4 (lookup by slug through the shortcode_atts filter).

6.1 Block Development Setup

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Describe a block type as WordPress sees it: block.json metadata, an editor script with edit and save, styles, and (later) PHP rendering
- Scaffold a block into an existing plugin with npx @wordpress/create-block and --no-plugin, and read every generated file
- Build blocks with @wordpress/scripts, and say what each file in build/ is for, including index.asset.php and the blocks manifest
- Register all built blocks from a plugin feature with wp_register_block_types_from_metadata_collection()
- Recognize the setup mistakes that cost beginners an afternoon: a second plugin by accident, a missing build, and stale build files

TIMING IN THE ROOM

10m	Frame: a kit, a workshop and a shelf; the four parts of a block type
35m	Live demo: Node, create-block, package.json and the build, the manifest, the Blocks feature, two mistakes
15m	Guided lab: `shelf_label_starter/`
20m	Start the lesson project: Step 1 and a look in the editor

MENTAL MODEL TO SAY

"The source files in src/blocks/book-card/ are a flat-pack kit: modern JavaScript with JSX and SCSS that no browser reads as is. The build is the workshop: @wordpress/scripts assembles the kit into plain JavaScript and CSS in build/blocks/book-card/. WordPress only ever looks at the shelf, the build/ folder, through the registration call in your plugin. Change the kit and nothing happens on the shelf until the workshop runs again."

Then the four parts on the board: **block.json** (metadata, read by PHP and the editor) · **edit** (the editing view, a React component) · **save** (the HTML stored in post_content) · **styles** (and later a PHP render). "A static block's front end needs no JavaScript: visitors get the stored HTML."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

On the site as Lesson 5.5 leaves it, with Node 24.

1. **Core's own blocks (2m).** "A block type is block.json, an editor script with edit and save, styles." How many does core register?

```
wp eval 'echo count( WP_Block_Type_Registry::get_instance()->get_all_registere
```

```
116
```

Every one a block.json in wp-includes/blocks plus prebuilt JavaScript. A static block's front end needs no JavaScript at all.

2. **Node 24 (2m).** In the plugin's folder (or with the node:24 image, the lesson's docker run line):

```
node --version
npm --version
```

```
v24.21.0
11.19.0
```

3. **create-block --no-plugin (5m).** "Every tutorial creates a new plugin. We add a block to the one we have." Run the lesson's command in the plugin folder, then look at what it wrote:

```
npx @wordpress/create-block@latest book-card --no-plugin --target-dir=src/bloc
```

```
Creating a new WordPress block in the /var/www/html/wp-content/plugins/pagesc

Creating a "block.json" file.

Done: WordPress block Book Card bootstrapped in the /var/www/html/wp-content/

Code is Poetry
```

```
ls wp-content/plugins/pagesco-books/src/blocks/book-card
cat wp-content/plugins/pagesco-books/src/blocks/book-card/block.json
```

```
block.json   edit.js      editor.scss  index.js     save.js      style.scss
{
    "$schema": "https://schemas.wp.org/trunk/block.json",
    "apiVersion": 3,
    "name": "pagesco/book-card",
    "version": "0.1.0",
    "title": "Book Card",
    "category": "widgets",
    "icon": "smiley",
    "description": "A book recommended by the Pages & Co. staff.",
    "example": {},
    "supports": {
        "html": false
    },
    "textdomain": "pagesco-books",
    "editorScript": "file:./index.js",
    "editorStyle": "file:./index.css",
    "style": "file:./style-index.css",
    "viewScript": "file:./view.js"
}
```

Read block.json with the lesson's field table; then index.js, edit.js (useBlockProps()), save.js (a pure function of the attributes).

4 **package.json, install, build (10m).** Learners whose `npm install` ran during the warm-up already have node_modules/. Show package.json (the three flags on the build script), then build:

```
{
    "name": "pagesco-books",
    "version": "1.0.0",
    "private": true,
    "description": "The blocks of the Pages & Co. Books plugin.",
    "license": "GPL-2.0-or-later",
    "scripts": {
        "build": "wp-scripts build --webpack-src-dir=src/blocks --output-path=
        "start": "wp-scripts start --webpack-src-dir=src/blocks --output-path=
    },
    "devDependencies": {
        "@wordpress/scripts": "^37.0.0"
    }
}
```

```
npm run build
```

```
> pagesco-books@1.0.0 build
> wp-scripts build --webpack-src-dir=src/blocks --output-path=build/blocks --h

assets by chunk 1.48 KiB (name: book-card/index)
  assets by path book-card/*.css 102 bytes
    asset book-card/index-rtl.css 51 bytes [emitted] (name: book-card/index)
    asset book-card/index.css 51 bytes [emitted] (name: book-card/index)
  asset book-card/index.js 1.23 KiB [emitted] [minimized] (name: book-card/inc
...
webpack 5.112.2 compiled successfully in 990 ms
```

(The install added 1261 packages here; read its warnings once: deprecated packages deep in the toolchain, the audit count of build tools, npm 11's install-scripts rule.) What the build wrote:

```
cd wp-content/plugins/pagesco-books
find build -type f | sort
cat build/blocks/blocks-manifest.php
```

```
build/blocks/blocks-manifest.php
build/blocks/book-card/block.json
build/blocks/book-card/index-rtl.css
build/blocks/book-card/index.asset.php
build/blocks/book-card/index.css
build/blocks/book-card/index.js
build/blocks/book-card/style-index-rtl.css
build/blocks/book-card/style-index.css
build/blocks/book-card/view.asset.php
build/blocks/book-card/view.js
<?php
// This file is generated. Do not modify it manually.
return array(
    'book-card' => array(
        '$schema' => 'https://schemas.wp.org/trunk/block.json',
        'apiVersion' => 3,
...
```

```
cat wp-content/plugins/pagesco-books/build/blocks/book-card/index.asset.php
```

```
<?php return array(
    'dependencies' => array(
        'react-jsx-runtime',
        'wp-block-editor',
        'wp-blocks',
        'wp-i18n'
    ),
    'version' => 'e4dd14a0fd30d395bc2c'
);
```

5 **The Blocks feature (5m).** "Registering the built blocks is one more feature class." Copy src/Blocks.php: one call, wp_register_block_types_from_metadata_collection(), on init. List it and ask the registry:

```
sed -i '/Shortcodes\\EventsShortcode::class,/a\        Blocks::class,' wp-con
wp eval '$type = WP_Block_Type_Registry::get_instance()->get_registered( "page
echo $type->title, " (", $type->category, ")\n";
echo "editor script: ", implode( ", ", $type->editor_script_handles ), "\n";
echo "styles: ", implode( ", ", $type->style_handles ), " / editor: ", implode
echo "view script: ", implode( ", ", $type->view_script_handles ), "\n";
echo wp_scripts()->registered["pagesco-book-card-editor-script"]->src, "\n";'
```

```
Book Card (widgets)
editor script: pagesco-book-card-editor-script
styles: pagesco-book-card-style / editor: pagesco-book-card-editor-style
view script: pagesco-book-card-view-script
http://localhost:8080/wp-content/plugins/pagesco-books/build/blocks/book-card/
```

Each file of block.json became a registered script or style with a handle from the block name. Open the post editor and insert the Book Card.

6 **Mistake: a missing build (3m).** "You check the plugin out from Git, where build/ is not committed, and forget to build."

```
mv wp-content/plugins/pagesco-books/build wp-content/plugins/pagesco-books/bui
curl -s http://localhost:8080/ | head -n 2
```

```
Notice: Function WP_Block_Metadata_Registry::register_collection was called <s
```

A "called incorrectly" notice above the page (and above every admin screen with display on). Put the folder back.

7 **Mistake: editing src/ and expecting WordPress to notice (3m).** "The smiley has to go." Change the icon in src/ and ask WordPress:

```
sed -i 's/"icon": "smiley"/"icon": "book"/' wp-content/plugins/pagesco-books/s
grep '"icon"' wp-content/plugins/pagesco-books/src/blocks/book-card/block.json
wp eval 'echo WP_Block_Type_Registry::get_instance()->get_registered( "pagesco
```

```
    "icon": "book",
smiley
```

The kit changed, the shelf did not. `npm run build > /dev/null`, ask again:

```
wp eval 'echo WP_Block_Type_Registry::get_instance()->get_registered( "pagesco
```

```
book
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A second plugin by accident.** create-block without --no-plugin. Coach: "`wp plugin list`: how many plugins do you have now?" (Self-check Q12.)
- **Editing src/ and reloading.** Coach with the shelf: "Did the workshop run?" (The core_blocks_tour lab and Mistake 3.)
- **npm in the wrong folder.** `npm run build` in wordpress/ or the project folder: "Missing script: build". Coach: "Where is package.json?"

GUIDED LAB

Folder: `labs/module-11/l6_1-block-setup/shelf_label_starter/` (a small plugin, `pagesco-shelf-label`, with one static block and its build/ prebuilt) with `labs/module-11/l6_1-block-setup/test_shelf_label.sh`; your reference is `labs/module-11/l6_1-block-setup/shelf_label_solution/`. TODO 1 registers the build from its manifest, exactly the Blocks feature's call; it needs no Node, so learners still installing can do it. TODO 2 (keywords, then a rebuild) needs Node.

- **Install and run (3m):** `cp -r labs/module-11/l6_1-block-setup/shelf_label_starter wordpress/wp-content/plugins/pagesco-shelf-label`, `wp plugin activate pagesco-shelf-label`, copy the test, run it: not registered, no stylesheet, yet the saved HTML is on the page.
- **TODO 1 (5m),** then the test; **TODO 2 (5m)** for laptops with Node.
- **Expected when done** (the solution's test):

```
Plugin 'pagesco-shelf-label' activated.
Success: Activated 1 of 1 plugins.
Shelf Label (text, icon tag)
keywords: shelf, sign
handles: pagesco-shelf-label-editor-script, pagesco-shelf-label-style
front end: 1 stylesheet(s)
<p class="wp-block-pagesco-shelf-label">Staff picks</p>
```

- **Debrief (2m):** "Why did the page show the label even unregistered?" (the HTML is in the post). "And why no stylesheet?" (no registered style handle).
- **Fast finishers:** `labs/module-11/l6_1-block-setup/core_blocks_tour_starter.sh`, then `labs/module-11/l6_1-block-setup/debug_block_registration.php`. Remove the lab plugin afterwards (`wp plugin deactivate pagesco-shelf-label`, `rm -r ...`).

START THE LESSON PROJECT

The lesson project is **The Book Card's First Draft**. In the room: Step 1 and Step 2 (20 minutes).

- Step 1: replace block.json, edit.js, save.js, style.scss and editor.scss from the lesson (copy them), delete view.js, build and list:

```
rm src/blocks/book-card/view.js
npm run build > /dev/null
ls src/blocks/book-card build/blocks/book-card
```

```
build/blocks/book-card:
block.json
index-rtl.css
index.asset.php
index.css
index.js
style-index-rtl.css
style-index.css

src/blocks/book-card:
block.json
edit.js
editor.scss
index.js
save.js
style.scss
```

- Step 2: reload the post editor, insert the Book Card, open the Code editor view.

- **Goes home:** Step 3 (the block on the front end: its CSS inlined, and only where the block is used) and Step 4 (the final check and the line in pagesco-plan.txt).

CHECKS FOR UNDERSTANDING

- Which folder does WordPress read? **Answer:** build/blocks and its blocks-manifest.php, never src/.
- What is index.asset.php for? **Answer:** the script handles index.js depends on, and a version hash.
- Does a visitor's browser run save()? **Answer:** no; the stored HTML is in the post.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** D) `index.asset.php`, the dependency list and version hash; **Q8** `pagesco-book-card-style`; **Q9** A) `npm run build`: the minified production files, without .map files; **Q10** C) Their stored comments still say wp:pagesco/book-card, a block that no longer exists: the editor cannot handle them; **Q11** `3`: wp-block-editor, wp-blocks and wp-i18n; the fourth dependency, react-jsx-runtime, does not start with wp-; **Q12** B) A plugin PHP file, a package.json and node_modules/. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a learner without Node follows the demo on the projector and does the shelf_label lab's TODO 1; they build at home with the node:24 image. **Stretch:** core_blocks_tour and the debug handout; Exercise 4 (a second block, no new PHP) and the Bonus Challenge (lint and format scripts).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. Why must a shortcode callback return its HTML instead of echoing it? **Answer:** it runs while the content is filtered, before the page is printed (in a block theme before the <head>); echoed output lands outside the content.
2. You add a column to schema(). What else must change, and why? **Answer:** DB_VERSION must go up, so maybe_upgrade() runs install() on the next request; updates do not rerun activation.
3. Name the path of the Book Card from your files to the editor. **Answer:** src/blocks/book-card → npm run build → build/blocks + blocks-manifest.php → the Blocks feature registers them on init → the editor (and the front end gets the saved HTML and style-index.css).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 5.5: Steps 3 and 4 (EventsShortcode, the seeding script, the Events page with five events). 6.1: Steps 3 and 4 (the front end, the final check). Session 12 starts from exactly this site: thirteen features and a built Book Card.
- **Lesson exercises.** 5.5: Exercises 1 to 3; Exercise 4 is a stretch. 6.1: Exercises 1 to 3; Exercise 4 is a stretch.

- **Both Quick Quizzes** and **both journal entries.**
- **Labs not reached:** `labs/module-11/l5_5-custom-tables/open_hours_starter/`, `labs/module-11/l5_5-custom-tables/debug_events_report.php`, `labs/module-11/l6_1-block-setup/core_blocks_tour_starter.sh`, `labs/module-11/l6_1-block-setup/debug_block_registration.php`.

BRING LIST FOR SESSION 12

- The site at the end of Lesson 6.1: `npm run build` works in pagesco-books, the Book Card (book icon) is in the inserter, /events/ lists five events.
- Node 24 (or the node:24 image); the project folder with `labs/`; the journal and the workbook.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** a working build and a registered Book Card; the versioned migration; a shortcode that returns.
- **Cut first (saves 3m):** the attribute cases of 5.5 (self-check Q1 covers them).
- **Cut second (saves 3m):** 6.1's stale-build mistake: one sentence.
- **Cut third (saves 7m):** 5.5 project Step 2: uninstall.php goes home.

RUNNING SHORT: WHAT TO ADD

- Lesson 5.5's Common Mistakes 3 live: dbDelta() "renaming" a column adds a second one.
- Lesson 6.1's Exercise 2 (your block among core's widgets) and Exercise 3 (the text domain survives the build).
- `npm start` on the projector: change the editor text, reload the editor, watch it change.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 12: Static and Dynamic Blocks

How to teach the session where the Book Card becomes a real block and the shop gets its first dynamic one: where a static block keeps each attribute and why its stored HTML is a contract, then a render.php that queries the catalog on every view and core blocks bound to the books' own data.

2 lessons | 3 hrs session | Lessons 6.2-6.3 | Session 12 of 18 · Part 2

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's ideas) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Every command below was run on WordPress 7.1.3 / PHP 8.5 to produce the output printed under it: Lesson 6.2's demo on the site exactly as Lesson 6.1 left it (`after:lesson_19`), Lesson 6.3's on the site after Lesson 6.2 (`after:lesson_20`). Browser-only results (the editor's validation messages) are quoted from the lesson page, which recorded them in a real 7.1 editor. Ask for a prediction before every run.
- This session is half JavaScript files the editor runs and half PHP the server runs. Learners copy the .js, .json and .scss files from the lesson pages (they are long, and the lessons explain them line by line); the room's time goes to the PHP side, the stored markup and the mental models, which are what people get wrong alone.
- Labs live in `labs/module-12/` (learners keep the folder in their project folder, next to compose.yaml): `_starter` files and folders are handouts, `_solution` files and folders are yours. The block labs need no Node: the Next Events block registers a block.json and a render.php from PHP and is tested with do_blocks().
- Part 2, Session 4 of 10. Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (6.2, 6.3) match the course site.

> THE #1 RISK OF SESSION 12: THE BUILD EATS THE SESSION
>
> Both lessons change files that only matter after `npm run build`, and every laptop builds differently: native Node 24, the node:24 container, a stale `npm start` in another terminal. Ten learners with a missing build means ten editors saying "This block has encountered an error". **Before:** the warm-up's W3 to W5 are a build check; anyone whose `ls wp-content/plugins/pagesco-books/build/blocks` does not show the manifest gets a partner. **In the room:** you type every file, and the room may watch instead of typing (the files are on the lesson pages); what they must type is the PHP (render.php, BookBindings.php) and the commands. **After:** the two lesson projects need the built blocks; send the Before the Next Lesson checks home in writing.

BEFORE THE SESSION

- ☐ **Projector site at** `after:lesson_19`: the end of Lesson 6.1 (the 6.1 Book Card, `npm run build` working). Know your build command: native `npm run build` in wordpress/wp-content/plugins/pagesco-books, or Lesson 6.1's `docker run --rm ... node:24 npm run build`. Keep a second terminal for the WP-CLI container shell (`docker compose run --rm wpcli sh`).
- ☐ **Rehearse once:** Lesson 6.2's sections on a scratch copy of the site (the deprecation step in particular: reload Card test, read the Console, add deprecated.js, reload again), then Lesson 6.3's render.php and BookBindings. The whole session's demo takes three builds.
- ☐ **Files to hand out:** `labs/module-12/` WITHOUT the `*_solution.*` files and the `card_price_solution/` and `next_events_solution/` folders. **Run the kit check once:** `python3 labs/check_labs.py module-12` (15 results, about two minutes).
- ☐ **Projector:** VS Code on pagesco-dev at a large font, the browser with the editor (admin / password), View Source, and the developer tools' Console; `lesson_20.html` and `lesson_21.html` in tabs (their screenshots show the Settings and Styles panels if your editor misbehaves).
- ☐ **Board, before learners arrive:** a two-column table **HTML (source + selector) | comment (no source, only if not default)**, and the line **static = printed card · dynamic = departure board · binding = shelf label**.

KNOWN TRAPS OF THIS SESSION

SYMPTOM	CAUSE	FIX
"This block has encountered an error" or the old card after a change	No build since the change, or the build failed	`npm run build` in the plugin folder; read its last line. `npm start` keeps building while you work.
Notices about `register_collection` / `blocks-manifest.php` above wp-admin, the editor broken	build/ is missing (Lesson 6.1's trap)	Build once; never commit or copy the plugin without build/.
"Block contains unexpected or invalid content"	save() changed for content that exists (or a selector catches a nested element, Mistake 2)	A deprecated entry with the old attributes, supports and save; or delete test posts while still developing.
`Warning: Undefined array key "title"` above the doctype	PHP read an HTML-sourced attribute from `$block['attrs']`	Read the HTML with the HTML API, or make the value a comment attribute, or make the block dynamic.
`Warning: Undefined array key "count"` inside the page	render.php read an attribute that has no default	`isset()` or `??`; and fix it in src/, not in build/ (the next build wipes build/).

SYMPTOM	CAUSE	FIX
A bound paragraph shows "Author" or "Price"	Wrong meta key, a key without show_in_rest, or a source without `uses_context`	`get_registered_meta_keys( 'post', 'book' )`; check `uses_context`; choose fallbacks that look wrong.
List items print "Stock" instead of the value	WordPress older than 7.1	Bind paragraphs instead (the lesson's version note).
create-block hangs or fails in the room	It downloads its template (npm) on venue Wi-Fi	Create the folder and the five files by hand, as the demo does; scaffold at home.

SESSION 12 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 5.5 and 6.1, homework check, the build working on every laptop.
0:10	10	**6.2** Frame and mental model	The printed card with a sticky note: attributes in the HTML or in the comment; save as a contract.
0:20	25	**6.2** Live demo	Three sourced attributes, what the post stores and what PHP sees; supports; a validation error and its deprecation; Mistake 1.
0:45	15	**6.2** Guided lab	`labs/module-12/l6_2-static-blocks/attribute_sources_starter.php`: where core's own blocks keep each attribute.
1:00	15	**6.2** Start the lesson project	Step 1: the Hobbit staff pick on the Staff picks post, for every laptop whose card builds.
1:15	10	Break	Laptops still building stay open; the facilitator helps with Node and the build.
1:25	10	**6.3** Frame and mental model	The printed card and the departure board; what a dynamic block stores.
1:35	20	**6.3** Live demo: render.php	Stored prices go stale; the Book List's files, build and is_dynamic(); a page of comments; the Books per list setting.
1:55	20	**6.3** Guided lab	`labs/module-12/l6_3-dynamic-blocks/next_events_starter/`: a dynamic block of your own, no JavaScript, tested with do_blocks().

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:15	15	**6.3** Live demo: block bindings	What can be bound; core/post-meta's raw values; the shop's pagesco/book source.
2:30	20	**6.3** Start the lesson project	Steps 1 and 2: single-book.html with live details, checked on Dune and Gatsby.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 13 (Checkpoint 2).
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 140 + 145 = 285 minutes for a learner working alone; after the warm-up, the break and the wrap-up the room has 150. Lesson 6.2 gets 65 minutes: the storage model, a demo through validation, a lab on core's own attributes, and Step 1 of the staff pick. Lesson 6.3 gets 85: render.php and bindings each get their own demo, a lab builds a dynamic block of the learner's own, and Steps 1 and 2 of the book pages.

Goes home (about 2 to 3 hours). Lesson 6.2's project Steps 2 and 3 (the front end, the comparison with Lesson 1.3's helper, deleting Card test) and Common Mistakes 2 and 3 to read; Lesson 6.3's "The Book List in the Editor" section (the controls and core-data) and "Variations" (variations.php and a build), the project's Step 3, Common Mistakes 2 and 3; both lessons' Hands-on Exercises, Quick Quizzes and journal entries; Before the Next Lesson for both.

Protect, in this order: the HTML-or-comment model with the empty `array ()` from PHP; the validation error and the deprecation that fixes it; render.php with the setting changing a page nobody touched; core/post-meta's raw values next to pagesco/book. **Cut first:** the supports step of the 6.2 demo (the workbook's worked example covers it), then the 6.2 project start (Step 1 goes home), then the stale-price step of the 6.3 demo (tell it).

OPENING SEGMENT

0:00 Warm-up: Lessons 5.5 and 6.1

10 min

TIMING

6m	Five retrieval questions (workbook Warm-up box), answers out loud
4m	Homework check: Session 11's projects; on every laptop `ls wp-content/plugins/pagesco-books/build/blocks` in the WP-CLI container

QUESTIONS AND ANSWERS

1. **W1 (Lesson 5.5).** A shortcode callback ends with `echo $html;` instead of `return $html;`. What goes wrong? **Answer:** The HTML appears at the top of the content (wherever the filter runs), not where the shortcode is: shortcodes RETURN their output. The [pagesco_events] callback returns it.
2. **W2 (Lesson 5.5).** Why does EventsTable::find() build its SQL with `$wpdb->prepare()` even though only the plugin calls it? **Answer:** Because the location comes from shortcode attributes, REST arguments (Lesson 7.2) and anyone who edits a post: every value goes through placeholders, every time.
3. **W3 (Lesson 6.1).** Where does `npm run build` write, and which file does WordPress read to register the plugin's blocks? **Answer:** Into build/blocks/ (`blocks-manifest.php` and `book-card` on the projector site), and build/blocks/blocks-manifest.php, read by wp_register_block_types_from_metadata_collection() in the Blocks feature.
4. **W4 (Lesson 6.1).** You add a second block folder in src/blocks/ and build. What PHP do you add to register it? **Answer:** None: the manifest lists every built block and the Blocks feature registers them all. Today's Book List proves it.
5. **W5 (Lesson 6.1).** You edited src/blocks/book-card/edit.js, reloaded the editor, and nothing changed. First question? **Answer:** Did you build? The editor loads build/blocks/book-card/index.js. `npm start` rebuilds on every save while you work.

On the projector site the build folder holds:

```
ls wp-content/plugins/pagesco-books/build/blocks
```

```
blocks-manifest.php  book-card
```

Anyone who sees no `book-card` folder builds now (or pairs). Laptops that cannot build at all do today's PHP labs and type the PHP parts of the demos; the JavaScript halves wait for the build at home.

6.2 Static Blocks: Attributes, Controls & Supports

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Declare attributes in block.json and choose where each one is stored: in the saved HTML (source and selector) or in the block's comment
- Make text editable on the canvas with RichText and save it with RichText.Content
- Add settings to the block sidebar with InspectorControls and components from @wordpress/components
- Opt into color, spacing and typography block supports and read the classes and styles they save
- Explain block validation, and change save safely with a deprecated entry and migrate

TIMING IN THE ROOM

10m	Frame: the printed card with a sticky note; HTML or comment; the stored HTML is a contract
25m	Live demo: three attributes, the stored post and PHP's view, supports, validation and a deprecation, Mistake 1
15m	Guided lab: `attribute_sources_starter.php`
15m	Start the lesson project: Step 1, the Hobbit staff pick

MENTAL MODEL TO SAY

Hold up a printed shop card: "A static block in a post is this card with a sticky note on the back. The printed side is the HTML: every visitor sees it, and the bookseller's words are printed right on it. The sticky note is the comment JSON: the few settings that are not words, such as heading level 2. The editor reads both to rebuild the card for editing. PHP can read the sticky note easily; to find the words it would have to read the print."

Then the board table: **source + selector = in the HTML** (title, author, the link's href); **no source = in the comment, and only when it differs from the default** (headingLevel, showBadge, colors). "Lesson 1.3's pagesco_book_card() took the same data as arguments. The difference is WHERE a block keeps them."

Then the contract: "Every time the editor opens a post it runs your CURRENT save() with the attributes it read and compares the result with the stored HTML. Different means invalid. Once content exists, save() may not change without describing the old version: a deprecation."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Lesson 6.2's own files and commands, typed live. The block files are long: show them on the lesson page, paste them into VS Code, and spend the time on what each part STORES. Outputs come from the site exactly as Lesson 6.1 left it.

1. **Before: the 6.1 card (2m).** "Lesson 6.1's card has no data of its own. Ask WordPress which attributes it has." Type the registry one-liner from the lesson in the WP-CLI container (`docker compose run --rm wpcli sh`):

```
wp eval 'echo implode( ", ", array_keys( WP_Block_Type_Registry::get_instance(
```

```
lock, metadata, className, style
```

Four attributes WordPress adds to every block, none of the card's own. Write **HTML or comment?** on the board.

2. **Three sourced attributes (6m).** "A bookseller types the title, the author and why she loves the book, right on the card." Replace block.json with the lesson's first version (show the `attributes` part on the projector), then edit.js (RichText for each field, `allowedFormats`), save.js (RichText.Content) and style.scss, all from the lesson page, and build in the plugin folder:

```
{
    "$schema": "https://schemas.wp.org/trunk/block.json",
    "apiVersion": 3,
    "name": "pagesco/book-card",
    "version": "0.2.0",
    "title": "Book Card",
    "category": "widgets",
    "icon": "book",
    "description": "A book recommended by the Pages & Co. staff.",
    "keywords": [ "staff pick", "recommendation" ],
    "textdomain": "pagesco-books",
    "attributes": {
        "title": {
            "type": "string",
            "source": "html",
            "selector": ".wp-block-pagesco-book-card__title"
        },
        ...
```

Then `npm run build > /dev/null` in wordpress/wp-content/plugins/pagesco-books (natively, or with Lesson 6.1's node:24 container) and the registry again:

```
wp eval 'echo implode( ", ", array_keys( WP_Block_Type_Registry::get_instance(
```

```
title, author, pick, lock, metadata, className, style
```

3 **What the post stores, and what PHP sees (5m).** In the editor: a post "Card test" with a card for Fahrenheit 451 (the lesson's text, the last words in bold). Open the Code editor (Ctrl+Shift+Alt+M) and read it out loud:

```
<!-- wp:pagesco/book-card -->
<div class="wp-block-pagesco-book-card"><h3 class="wp-block-pagesco-book-card_
<!-- /wp:pagesco/book-card -->
```

The front end prints exactly that HTML (View Source of /card-test/):

```
<div class="wp-block-pagesco-book-card"><h3 class="wp-block-pagesco-book-card_
```

Now ask PHP with the lesson's `read-card.php` (`wp eval-file read-card.php`):

```
<?php
$post  = get_page_by_path( 'card-test', OBJECT, 'post' );
$block = parse_blocks( $post->post_content )[0];

echo $block['blockName'], "\n";
var_export( $block['attrs'] );
echo "\n";
```

```
pagesco/book-card
array (
)
```

"An empty array: everything is in the HTML." That is the sticky note, and it is empty.

4 **Supports: features for free (5m).** "The shop wants a sage card for summer." Jump to the lesson's supports version of block.json (it already has the link and the heading level of the controls section; build the controls section's edit.js and save.js with it). Build, then the registry once more:

```
wp eval 'echo implode( ", ", array_keys( WP_Block_Type_Registry::get_instance(
```

```
title, author, pick, url, headingLevel, lock, metadata, className, style, back
```

Three new attributes came from supports: `backgroundColor`, `textColor`, `fontSize`. In the editor's Styles tab pick Sage, font size S and padding Medium; the stored card:

```
<div class="wp-block-pagesco-book-card has-sage-background-color has-backgrou
```

And where the class gets its color, the theme's global styles:

```
curl -s http://localhost:8080/card-test/ | grep -o '\.has-sage-background-colo
```

```
.has-sage-background-color{background-color: var(--wp--preset--color--sage) !
```

5 **The contract: validation and a deprecation (5m).** "Now the shop wants a Staff pick badge on every card." Add the lesson's showBadge version (block.json, edit.js, save.js, style.scss), build, and reload Card test in the editor with the Console open (F12). The card shows **Block contains unexpected or invalid content**; the Console says why:

```
Block validation: Expected tag name `p`, instead saw `a`.
Block validation: Block validation failed for `pagesco/book-card` (Object).

Content generated by `save` function:

<div class="wp-block-pagesco-book-card has-sage-background-color has-backgroun

Content retrieved from post body:

<div class="wp-block-pagesco-book-card has-sage-background-color has-backgroun
```

Read the first line: the new save() starts with the badge `<p>`, the stored card with the link `<a>`. Then create the lesson's `deprecated.js` (the old attributes, supports and save, plus `migrate()` that sets `showBadge: false`), the new index.js, build, reload: the card opens normally and the Console logs "Block successfully updated". Nothing is saved until someone updates the post.

6 **Mistake 1: a sourced attribute in PHP (2m).** "A PHP developer wants to log every card's title." Save the lesson's throwaway must-use plugin `mistake-card-title.php`, then in the container shell:

```
curl -s http://localhost:8080/card-test/ | head -n 2
curl -s http://localhost:8080/card-test/ | grep -o '<!-- Book Card:[^>]*>'
```

```
Warning: Undefined array key "title" in /var/www/html/wp-content/mu-plugins/m
<!-- Book Card:  -->
```

The title is not in the comment, so it is not in `$block['attrs']`: a Warning above the doctype and an empty comment. In PHP a static block is finished HTML; the HTML API reads it (Exercise 1, and the fast finishers' `card_price` lab). `rm wp-content/mu-plugins/mistake-card-title.php`.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Reading the card's text in PHP from** `$block['attrs']` (the demo's Mistake 1). Coach with the board: "Is the title on the sticky note or on the print?" The fixes: the HTML API, a comment attribute, or a

dynamic block. (Self-check Q2, the debug handout.)

- **Changing save() and blaming the editor.** "The block broke after my change." Coach: "What does the Console's first validation line say, and does content already exist?" Test posts can be deleted while developing; client content needs a deprecation. (Self-check Q3.)
- **Translating text in save() or using a too-wide selector** (Mistakes 3 and 2). Coach: "Will save() return the same HTML in Spanish? Does the selected element contain only its own value?" Insert, save, RELOAD before trusting a change. (Self-check Q6.)

GUIDED LAB

File: learners copy `labs/module-12/l6_2-static-blocks/attribute_sources_starter.php` into wordpress/ and run it with `wp eval-file --use-include attribute_sources_starter.php`; your reference is `labs/module-12/l6_2-static-blocks/attribute_sources_solution.php`. It practices the lesson's central question on blocks the learner did not write: for core/heading, core/quote and core/button, which attribute lives in the HTML and which in the comment, read from the registry; then what parse_blocks() returns for a stored heading and image.

- **Run it as shipped (2m).** Every line says TODO; the skip list (the attributes WordPress adds to many blocks) is given.
- **TODOs 1 to 4 in pairs (10m).** TODO 1 is the registry call from the demo. TODOs 2 and 3 are one if/else on `isset( $attr['source'] )`; circulate for `array_key_exists()` (core/image's isDecorative defaults to false, and `isset()` would miss a false default). Before TODO 4, ask for a prediction: will the image's src be in the attrs?
- **Expected when done:**

```
core/heading content: in the HTML (rich-text, h1,h2,h3,h4,h5,h6)
core/heading level: in the comment (default 2)
core/quote value: in the HTML (html, blockquote)
core/quote citation: in the HTML (rich-text, cite)
core/button tagName: in the comment (default "a")
core/button type: in the comment (default "button")
core/button url: in the HTML (attribute, a @href)
core/button title: in the HTML (attribute, a,button @title)
core/button text: in the HTML (rich-text, a,button)
core/button linkTarget: in the HTML (attribute, a @target)
core/button rel: in the HTML (attribute, a @rel)
parse_blocks() sees:
core/heading {"level":3}
core/image {"id":42,"sizeSlug":"large"}
```

- **Debrief (3m):** "Core uses a fourth source the lesson did not list: `rich-text`. Where does it store?" (In the HTML, like html.) "Why is the button's `tagName` in the comment?" (It is a setting, not text.) "The image's src is not in the attrs: who reads it back?" (The editor, through the selector.)

- **Fast finishers:** the folder lab `labs/module-12/l6_2-static-blocks/card_price_starter/` (a must-use plugin that reads a static card's link with the HTML API and adds the live price after it; test with `labs/module-12/l6_2-static-blocks/test_card_price.sh`), then `labs/module-12/l6_2-static-blocks/debug_card_report.php`.

START THE LESSON PROJECT

The lesson project is **A Staff Pick on the Staff Picks Post**: the finished card for The Hobbit under Lesson 2.1's list. It needs the block as the lesson's sections leave it (badge, deprecation and all): laptops whose card is at that point start it; the others finish the sections at home first.

- Step 1 (12 minutes in the lesson): insert a Book Card after the list, type the title, author and recommendation ("everyone" in italics), set the link, H3 and the badge in Card settings, Sage in Styles. Update the post.
- **Done in the room** = the Code editor shows the card's comment with only `{"backgroundColor":"sage"}` (the lesson prints the whole post). Ask why headingLevel and showBadge are not there. (Both are at their defaults.)
- **Goes home:** Step 2 (the front end), Step 3 (the comparison table with Lesson 1.3's helper, deleting Card test, the parse_blocks check) and the pagesco-plan.txt line.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Where is a card's title stored, and where is its heading level? **Answer:** the title in the HTML (inside the selector's element); the level in the comment, and only when it is not 3.
- Which hook gives a block's wrapper the supports' classes and styles? **Answer:** `useBlockProps()` in edit and `useBlockProps.save()` in save (Quiz Q3 of the lesson).
- What happens to the front end when a card turns invalid in the editor? **Answer:** nothing: the front end prints the stored HTML.
- What three things does a deprecated entry repeat? **Answer:** the old attributes, the old supports and the old save (plus an optional migrate).

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** C) In the saved HTML: the `href` of the first `<a>` inside the block; **Q2** `{"showBadge":false}`; **Q3** C) Wrapping the author in a `<cite>` in save(); **Q4** A) Inside `<BlockControls>` in edit; **Q5** B) The class `has-sage-background-color` in the HTML and `"backgroundColor":"sage"` in the comment; **Q6** save() must return the same HTML for the same attributes in every language: a translated string makes cards invalid as soon as someone edits in another locale. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners new to React read edit.js and save.js side by side with the lesson's explanation and type only the attribute names; pair them with someone who has written JSX. For the lab, write the `isset( $attr['source'] )` branch on the board.

Stretch: the `card_price` folder lab and `debug_card_report.php`; then the lesson's Exercise 3 (a Plain style variation) and the Bonus Challenge (the heading level in the toolbar: does it need a deprecation?).

6.3 Dynamic Blocks & Block Bindings

85 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain when a block should be dynamic, and what a dynamic block stores in post_content
- Render a block on the server with render.php: attributes, a WP_Query through get_posts(), get_block_wrapper_attributes(), and escaping
- Preview a dynamic block in the editor with ServerSideRender, and fill its controls from the REST API through @wordpress/core-data
- Offer ready-made versions of a block as variations, generated in PHP
- Bind core blocks to post meta with core/post-meta, and write a custom binding source with register_block_bindings_source()

TIMING IN THE ROOM

10m	Frame: the printed card and the departure board; what a dynamic block stores
20m	Live demo A: stale prices, the Book List's files and build, a page of comments, the setting
20m	Guided lab: the `next_events_starter/` plugin
15m	Live demo B: what can be bound, core/post-meta's raw values, pagesco/book
20m	Start the lesson project: single-book.html, Steps 1 and 2

MENTAL MODEL TO SAY

"A static block is a printed card: the shop prints it once, and it says the same thing until someone reprints it. A dynamic block is a station's departure board: what is stored is only 'show the next trains from platform 2', and the board looks up the timetable whenever someone glances at it. The post keeps the instruction, the comment; your render.php is the board."

Then the comparison on the board, from the lesson's table: stored = **comment + HTML** vs **comment only**; HTML made by **save(), once** vs **render.php, every view**; changing markup = **a deprecation** vs **edit render.php**; plugin off = **HTML stays** vs **nothing shows**.

Before demo B, the third picture: "A bound block is a shelf label with an empty slot. The design is fixed in the template; the slot says 'price of the book on this shelf', and a clerk, the source, fills it every time someone looks."

LIVE DEMO A: RENDER.PHP (YOU TYPE, THEY FOLLOW)

From the site as Lesson 6.2 left it. Learners type render.php themselves (it is the PHP of today); the other four files they paste.

1. **Stored HTML goes stale (4m).** "Put Dune on sale, as a bookseller would, and compare the front page with the book club post."

```
wp post meta update 30 pagesco_price 12.99
curl -s http://localhost:8080/ | grep -A 10 '>Dune<' | grep -o '\$[0-9.]*'
curl -s http://localhost:8080/book-club-dune/ | grep -o 'pagesco-book__price">
```

```
Success: Updated custom field 'pagesco_price'.
$14.99
pagesco-book__price">$12.99
```

Lesson 3.3's pattern still says $14.99: it was typed into a file. The shortcode reads the meta on every view. A static block is like the pattern; today's Book List is like the shortcode. Put the price back (`wp post meta update 30 pagesco_price 14.99`).

2. **The Book List's files and the build (6m).** In the room, skip create-block (it downloads its template on venue Wi-Fi): create src/blocks/book-list/ in VS Code and save the lesson's five files: block.json (three settings, no source), render.php (read it top to bottom, it is Module 4 and 5), style.scss, the first edit.js (ServerSideRender only) and index.js. Build and list the build folder:

```
npm run build > /dev/null
ls build/blocks/book-list
```

```
block.json
index.asset.php
index.js
render.php
style-index-rtl.css
style-index.css
```

render.php was COPIED into build/: the copy WordPress runs (Mistake 2's trap). No PHP change, the manifest registers it:

```
wp eval '$type = WP_Block_Type_Registry::get_instance()->get_registered( "page
echo $type->title, ": ", $type->is_dynamic() ? "dynamic" : "static", "\n";
echo implode( ", ", array_keys( $type->get_attributes() ) ), "\n";'
```

```
Book List: dynamic
count, genre, orderBy, lock, metadata, className, style, backgroundColor, text
```

3 **A page of comments (5m).** A page "List test" with two Book Lists (one as inserted, one Fantasy by price). The Code editor shows only this:

```
<!-- wp:pagesco/book-list /-->

<!-- wp:pagesco/book-list {"genre":"fantasy","orderBy":"price"} /-->
```

And View Source shows two finished lists (8 items: six books, the Books per list default, then the two fantasy books cheapest first):

```
<ul class="wp-block-pagesco-book-list">
            <li class="wp-block-pagesco-book-list__item"><span class="pagesco
            <li class="wp-block-pagesco-book-list__item"><span class="pagesco
            <li class="wp-block-pagesco-book-list__item"><span class="pagesco
            <li class="wp-block-pagesco-book-list__item"><span class="pagesco
            <li class="wp-block-pagesco-book-list__item"><span class="pagesco
            <li class="wp-block-pagesco-book-list__item"><span class="pagesco
    </ul>
...
```

Shown here up to the end of the first list. Every item is Lesson 5.5's BookShortcode::render(), escaped where printed: the 5.5 promise kept.

4 **The shop changes its mind (3m).** "Books per list: 4." From the container (the whole option as JSON, Lesson 5.4's Common Mistake 3), count the items, delete the option again:

```
wp option update pagesco_books_settings '{"notice_extra":"Open until 9 p.m. on
curl -s http://localhost:8080/list-test/ | grep -c '<li class="wp-block-pagesc
wp option delete pagesco_books_settings
```

```
Success: Updated 'pagesco_books_settings' option.
6
Success: Deleted 'pagesco_books_settings' option.
```

Four plus two: the first list followed the setting, nobody touched the page. That is why count has no default in block.json.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Reading an attribute that has no default** (`$attributes['count']`): a Warning INSIDE the page content, with curly quotes, naming build/.../render.php. Coach: "Which attributes get a default from block.json? Only the ones that declare one." (The lab's TODO 1, the debug handout.)
- **Fixing the file the message names**, build/blocks/book-list/render.php. Coach: "What does the next build

do to build/?" It empties it and copies src/ again. Edit src/, build (or keep `npm start` running).

- **A binding that shows its fallback.** Coach with three questions: is the key exactly one of BookMeta::fields()? Is it registered with show_in_rest and without an underscore? Does a custom source declare `uses_context`? (Self-check Q10, Q11; the `core_sources` lab.)

GUIDED LAB

Folder: `labs/module-12/l6_3-dynamic-blocks/next_events_starter/`, a small plugin whose block `pagesco/next-events` lists the next events from Lesson 5.5's table; your reference is `labs/module-12/l6_3-dynamic-blocks/next_events_solution/`. It practices render.php on data the lesson did not use: an attribute without a default that falls back to a setting (Events to show), a clamp, a query through the plugin's own class, `get_block_wrapper_attributes()` and escaping. It registers block.json from PHP and has no editor script, so it needs no build: the test renders it with do_blocks(), exactly as a page view would.

- **Install and run as shipped (4m).** `cp -r labs/module-12/l6_3-dynamic-blocks/next_events_starter wordpress/wp-content/plugins/pagesco-next-events`, `wp plugin activate pagesco-next-events`, `cp labs/module-12/l6_3-dynamic-blocks/test_next_events.php wordpress/`, `wp eval-file --use-include test_next_events.php`. Every block prints its TODO paragraph.
- **TODOs 1 to 4 (12m),** in the copy in wordpress/wp-content/plugins/pagesco-next-events/block/render.php. Circulate for `$attributes['count']` without `isset()` (the Warning lands in the output: it is the lesson's Mistake 1 made for real) and for `echo $event->title` without `esc_html()`.
- **Expected when done** (the solution):

```
Plugin 'pagesco-next-events' activated.
Success: Activated 1 of 1 plugins.
== no attributes
<ul class="wp-block-pagesco-next-events">
            <li><time datetime="2027-11-04T19:00">Nov 4</time> Book club: Dune</l
            <li><time datetime="2027-11-06T10:30">Nov 6</time> Story time: The Ho
            <li><time datetime="2027-11-11T18:30">Nov 11</time> An evening with A
            <li><time datetime="2027-11-18T19:30">Nov 18</time> Poetry open mic</
            <li><time datetime="2027-12-02T19:00">Dec 2</time> Book club: Fahrenh
    </ul>
== 2 at the Café
<ul class="wp-block-pagesco-next-events">
            <li><time datetime="2027-11-11T18:30">Nov 11</time> An evening with A
            <li><time datetime="2027-11-18T19:30">Nov 18</time> Poetry open mic</
    </ul>
== rooftop
<p class="wp-block-pagesco-next-events">No events at this location yet.</p>
count 99: 6 events
```

- **Debrief (4m):** "Where did `class="wp-block-pagesco-next-events"` come from?" (get_block_wrapper_attributes().) "Why does count 99 show six?" (The clamp allows ten; the table has six.) "What would you need to insert this block in the editor?" (An editor script, built as in Lesson 6.1.) Then `wp plugin deactivate pagesco-next-events` and `rm -r` the folder.
- **Fast finishers:** `labs/module-12/l6_3-dynamic-blocks/core_sources_starter.php` (what core/post-meta and core/post-data return for bound paragraphs, rendered with a made-up context), then `labs/module-12/l6_3-dynamic-blocks/debug_genre_counts.php`.

LIVE DEMO B: BLOCK BINDINGS (YOU TYPE, THEY FOLLOW)

1. **What can be bound (3m).** "Core blocks can show data they do not store."

```
wp eval 'foreach ( array( "core/paragraph", "core/heading", "core/list-item",
```

```
core/paragraph: content
core/heading: content
core/list-item: content
core/image: id, url, title, alt, caption
core/button: url, text, linkTarget, rel
```

List items arrived in 7.1 (the lesson's version note: on 6.8 to 7.0 they print their fallback). Core's four sources:

```
wp eval 'foreach ( get_all_registered_block_bindings_sources() as $name => $so
```

```
core/pattern-overrides (Pattern Overrides)
core/post-data (Post Data)
core/post-meta (Post Meta)
core/term-data (Term Data)
```

2 **core/post-meta on book pages (5m).** Create the lesson's first single-book.html (an author paragraph and three list items bound to `core/post-meta`) and view Gatsby's page:

```
<main id="wp--skip-link--target" class="wp-block-group has-global-padding is-
    <h1 class="wp-block-post-title">The Great Gatsby</h1>

    <p class="pagesco-book-author wp-block-paragraph">F. Scott Fitzgerald</p>

    <ul class="wp-block-list pagesco-book-facts">
    <li>8.99</li>

    <li></li>

    <li>180</li>
    </ul>

</main>
```

Binding works and is not formatting: `8.99` with no dollar sign, an EMPTY item for the false stock, raw pages. core/post-meta reads only keys registered with show_in_rest and without a leading underscore.

3 **The shop's own source (7m).** Save the lesson's `src/BookBindings.php`. Read `uses_context`, the visibility check and the five fields aloud. Then register it as the fourteenth feature and list the sources:

```
sed -i '/\t\t\tBlocks::class,/a\            BookBindings::class,' wp-content/p
grep -c "::class," wp-content/plugins/pagesco-books/src/Plugin.php
wp eval 'foreach ( get_all_registered_block_bindings_sources() as $name => $so
```

```
14
core/pattern-overrides (Pattern Overrides)
core/post-data (Post Data)
core/post-meta (Post Meta)
core/term-data (Term Data)
pagesco/book (Book details)
```

No build: PHP only. The project rewrites single-book.html with `pagesco/book` for the price and the facts.

START THE LESSON PROJECT

The lesson project is **Book Pages with Live Details**: single-book.html with the author on core/post-meta, the price and four facts on pagesco/book, and the New Arrivals variation of the Book List underneath. In the room: Steps 1 and 2 (20 minutes).

- Step 1: replace single-book.html with the lesson's project version (paste it; read the `metadata.bindings` comments aloud with the room).
- Step 2: View Source of /books/dune/ and of Gatsby's facts. What the projector shows for Gatsby:

```
<ul class="wp-block-list pagesco-book-facts">
    <li>Sold out</li>

    <li><a href="http://localhost:8080/genre/classic/" rel="tag">Classic</a>, <a

    <li>180 pages</li>

    <li>First published in 1925</li>
    </ul>
```

- **Done in the room** = Dune shows $14.99, In stock, two genre links, 412 pages and 1965, then three new arrivals. (The New Arrivals comment needs the Book List block, which the demo built.)
- **Goes home:** Step 3 (put Dune on sale and watch both pages follow, delete List test, the checks), the variations section and build, the editor section with the List settings panel.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demos and the lab:

- What does a page with a Book List store? **Answer:** only the comment with the attributes; the HTML is made on every view.
- Why does render.php use `get_posts()` and not a Loop with `the_post()`? **Answer:** it leaves the global $post alone, so a block in the middle of someone else's Loop breaks nothing; and it loads the meta in one query.
- core/post-meta or pagesco/book for the price? **Answer:** pagesco/book: core/post-meta prints the raw 8.99; the shop's source formats it.
- When does variations.php run? **Answer:** only when something asks for the variations (the editor, through REST), not on page views (lesson quiz Q5).

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** C) Run `npm run build` (render.php is copied into build/), then nothing: every page follows on its next view; **Q8** `4`; **Q9** D) get_block_wrapper_attributes() returns attributes that are already escaped (class

and style from the supports); **Q10** D) Nothing: an empty list item; **Q11** A) Its fallback text, with no error; **Q12** `First published in 1945`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners who did not reach the Book List type only render.php and copy the rest; in the lab, give them the solution's line for TODO 1 and let them do TODOs 2 to 4. For the bindings demo, the board table: **source → args → value or null**.

Stretch: `core_sources_starter.php` and `debug_genre_counts.php`; the lesson's Exercise 2 (an ISBN field that stays quiet) and Exercise 3 (only books in stock); the Bonus Challenge (registerBlockBindingsSource() in the editor).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note. Sort them before Session 13: question 2's pile tells you whether the checkpoint's Part C (blocks) needs a five-minute review in the warm-up.

1. A static block's title and heading level: where is each stored? **Answer:** the title in the saved HTML (found by its selector), the heading level in the block comment (only when it differs from its default).
2. Name two things a dynamic block's render.php must do with its attributes. **Answer:** treat them as input (isset or a default for optional ones, cast, clamp, sanitize) and escape everything it prints; plus starting the wrapper with get_block_wrapper_attributes().
3. A bound paragraph shows its fallback text on a book page. Name two causes. **Answer:** the meta key is wrong, not registered with show_in_rest or protected (core/post-meta), or a custom source lacks uses_context or returns null.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 6.2: Steps 2 and 3 (and Step 1 if the block was not ready). 6.3: Step 3, plus the editor section (List settings) and the variations section with its build, so New Arrivals is in the inserter.
- **Lesson exercises.** 6.2: Exercises 1 and 2 (the HTML API, register_block_type_args); Exercise 3 is a stretch. 6.3: Exercises 1 and 2 (a binding without a browser, the ISBN field); Exercise 3 is a stretch.
- **Both Quick Quizzes** on the lesson pages (five questions each; answers are in the Answer Key).
- **Journal:** both entries (where every piece of the card's data lives; sorting the site's book data into typed, rendered and bound).
- **Before the Next Lesson (from both lessons):** the Staff picks card valid in the editor, Card test and List test gone, `npm run build` run on the final files; refresh JavaScript getters and `event.preventDefault()` for

Lesson 6.4.

- **Checkpoint 2 is in Session 13** (Modules 5 and 6, 30 minutes, open notes): reread Lessons 5.2 to 5.5 and today's two lessons; bring a site at the end of Lesson 6.3.
- **Optional:** `labs/module-12/l6_2-static-blocks/card_price_starter/`, `labs/module-12/l6_3-dynamic-blocks/core_sources_starter.php` and both debug handouts.

BRING LIST FOR SESSION 13

- A site at the end of Lesson 6.3 (book pages with live details, the Book List with its variations) and a working `npm run build`.
- The project folder with `labs/` (Session 13 adds `labs/module-13/`, including the checkpoint, handed out in the room).
- pagesco-plan.txt, the journal and the workbook; Maria's and Devon's passwords are the course's (maria-pass, devon-pass), Lesson 7.1 creates application passwords from them.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the empty `array ()` from PHP; the validation error and its deprecation; render.php with the setting; core/post-meta next to pagesco/book.
- **Cut first (saves 5m):** the supports step of the 6.2 demo; the workbook's worked example shows the supports and the style engine.
- **Cut second (saves 15m):** the 6.2 project start: Step 1 goes home with Steps 2 and 3.
- **Cut third (saves 4m):** the stale-price step of the 6.3 demo: tell the story, show the lesson's output.

RUNNING SHORT: WHAT TO ADD

- Lesson 6.2's Exercise 1 live: the HTML API reads the Hobbit card's title and link (`WP_HTML_Processor::create_fragment()`, `next_token()`).
- Lesson 6.3's ServerSideRender request with curl as admin: the block-renderer route returns render.php's HTML as JSON; then an orderBy outside the enum gets a 400.
- Run `debug_genre_counts.php` for everyone and fix it together; the silent bug (a reversed clamp) is the best discussion.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 13: The Interactivity API and Using the REST API

How to teach the session where the shop's pages react without reloading and its data answers over HTTP: the Interactivity API's server-first model (directives the server already evaluates, a small store that takes over), then the REST API's routes, headers, authentication and errors, and Checkpoint 2 on Modules 5 and 6.

`2 lessons` `3 hrs session` `Lessons 6.4-7.1` `Checkpoint 2` `Session 13 of 18 · Part 2`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's ideas) → start the lesson's own project. Today ends with **Checkpoint 2 at 2:20**: 30 minutes, scored, open notes, on Modules 5 and 6. The lesson exercises, quiz and journal are homework.
- Every command below was run on WordPress 7.1.3 / PHP 8.5 to produce the output printed under it: Lesson 6.4's demo on the site after Lesson 6.3 (`after:lesson_21`), Lesson 7.1's on the site after Lesson 6.4 (`after:lesson_22`). Browser recordings (the filter's clicks, fetch() without a nonce) are the lesson's own, made with Playwright on a 7.1 site. REST requests shown as `curl --user "maria:$PASS"` were sent with a real application password; yours prints its own password once.
- Labs live in `labs/module-13/`: `_starter` files are handouts, `_solution` files are yours, and the whole `checkpoint-2/` folder stays with you until 2:20. The 6.4 labs need no browser and no build: `wp_interactivity_process_directives()` runs the server half of the Interactivity API on any HTML.
- Part 2, Session 5 of 10. Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (6.4, 7.1) match the course site.

> **THE #1 RISK OF SESSION 13: TWO BIG LESSONS SQUEEZE THE CHECKPOINT**
>
> Lessons 6.4 and 7.1 are written for 140 and 140 minutes; the room has 115 for both, and Checkpoint 2 needs its full 30. A demo that runs ten minutes over eats the checkpoint, and a checkpoint started at 2:30 is unfair to the slow, careful learners it should reward. **Start the checkpoint at 2:20 whatever happens.** The cuts are listed under If You Run Long; the first is the 6.4 project start (5 minutes), then the browser step of the 6.4 demo, then the apiFetch step of the 7.1 demo. Both lessons' projects mostly go home anyway: say so at 0:00.

BEFORE THE SESSION

- ☐ **Projector site at** `after:lesson_21` (the end of Lesson 6.3, a working build). Rehearse 6.4's files and build once on a scratch copy; then put the projector site at `after:lesson_22` for 7.1, or keep going on the same site (the 6.4 demo creates the Book Filter, which 7.1 does not need).
- ☐ **Two terminals:** the project folder (`wp` shortcut, the build) and the WP-CLI container shell (`docker compose run --rm wpcli sh`) for curl. Keep `PASS` and `DEVON_PASS` in the shell once created; never in a file under wordpress/.
- ☐ **Files to hand out:** `labs/module-13/` WITHOUT the `*_solution.*` files and WITHOUT the `checkpoint-2/` folder. At 2:20 hand out `checkpoint-2/checkpoint_starter.php` only (shared folder or USB stick), and the printed Part D page.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-13` (14 results, about two minutes), and run `checkpoint_solution.php` on your site so you know the four lines it prints.
- ☐ **Print:** the workbook's Checkpoint 2 page for every learner, and one rubric per learner (answer key, "Checkpoint 2: Rubric and Model Answer").
- ☐ **Board, before learners arrive: state · derived state · context**; the status table **400 the request · 401 who are you? · 403 you may not · 404 no such route or item**.

KNOWN TRAPS OF THIS SESSION

SYMPTOM	CAUSE	FIX
The filter reloads the page on every click; no view.js in build/	package.json without `--experimental-modules`	Add it to both scripts, rebuild, check `ls build/blocks/book-filter | grep view` (Mistake 1).
The filter reloads the page, Console empty, view.js built	The store's namespace does not match data-wp-interactive	One spelling in render.php and view.js (Mistake 3); watch for page loads, not just the result.
Console warning about synchronous `event.preventDefault()`	The action is not wrapped in withSyncEvent()	`selectGenre: withSyncEvent( ( event ) => { ... } )` (Mistake 2).
`rest_not_logged_in` with `--user`	The account password instead of an application password, a password with a missing character, or a revoked one	Create one with `wp user application-password create ... --porcelain`; Users > Profile shows Last Used (Mistake 3).

SYMPTOM	CAUSE	FIX
`rest_not_logged_in` in the browser although logged in	fetch() without X-WP-Nonce	wp.apiFetch() (dependency wp-api-fetch), or send the nonce yourself.
`$PASS` empty in the next command	Each `docker compose run --rm wpcli sh` is a new shell	Set PASS again in the shell you are using; keep the password in a password manager.
The widget's list never appears	The script is not enqueued on index.php, or a typo in the widget ID	View Source: `#pagesco-stock-widget-js`; the Console shows the error.

SESSION 13 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 6.2 and 6.3, homework check, the checkpoint's rules in one sentence.
0:10	10	**6.4** Frame and mental model	The stage and the lighting desk: server-rendered HTML with directives, a store that takes over.
0:20	25	**6.4** Live demo	The Book Filter's files and the module build; what the server already did for ?genre=fantasy; the state JSON; the browser recording.
0:45	15	**6.4** Guided lab	`labs/module-13/l6_4-interactivity-api/open_today_starter.php`: directives and derived state, processed by the server.
1:00	5	**6.4** Start the lesson project	Step 1: the Find a Book page; Steps 2 and 3 go home.
1:05	10	Break	Checkpoint files stay closed until 2:20.
1:15	10	**7.1** Frame and mental model	The service counter: routes, endpoints, JSON, status codes, who is asking.
1:25	30	**7.1** Live demo	The index and the routes, the headers, _fields and pages; 401, cookie and nonce, an application password; 400, 403, 404.
1:55	15	**7.1** Guided lab	`labs/module-13/l7_1-rest-api/catalog_queries_starter.sh`: six questions for the catalog, six requests.
2:10	10	**7.1** Start the lesson project	Step 1: the inventory script sells the last Fahrenheit 451 and restocks it.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:20	30	Checkpoint 2 (Modules 5 and 6)	`labs/module-13/checkpoint-2/checkpoint_starter.php` plus the written Part D: scored, open notes, on their own.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 14.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 140 + 140 = 280 minutes for a learner working alone; after the warm-up, the break, the checkpoint and the wrap-up the room has 115. Lesson 6.4 gets 55 minutes: the server-first model, a demo through what curl already sees, a lab on server-processed directives and the Find a Book page. Lesson 7.1 gets 65: the vocabulary, a long demo of the counter's answers and refusals, a lab of six requests and the inventory script's first sale. Then the checkpoint, on its own, scored.

Goes home (about 3 hours: this is a heavy week). Lesson 6.4's project Steps 2 and 3 (test it like a customer, with and without JavaScript; clean up) and Common Mistakes 1 to 3 to reproduce; Lesson 7.1's "apiFetch" section in practice and the project's Steps 2 to 4 (the Stock at a glance widget); both lessons' Hands-on Exercises, Quick Quizzes and journal entries. Say at 0:00 that the 7.1 widget is the main homework.

Protect, in this order: the checkpoint's 30 minutes (start at 2:20); what the server already did for ?genre=fantasy (the server-first idea); 401 versus 403 and the application password; the guided labs. **Cut first:** the 6.4 project start (the page is one paste at home), then the browser step of the 6.4 demo (the workbook has the recording), then the apiFetch step of the 7.1 demo.

OPENING SEGMENT

0:00 Warm-up: Lessons 6.2 and 6.3 — 10 min

TIMING

6m	Five retrieval questions (workbook Warm-up box), answers out loud
4m	Homework check: the staff pick and the book pages; the checkpoint's rules: 2:20, 30 minutes, open notes, on your own

QUESTIONS AND ANSWERS

1. **W1 (Lesson 6.2).** The Hobbit card on the Staff picks post: which of its attributes does PHP's parse_blocks() see? **Answer:** Only the comment's: `{"backgroundColor":"sage"}`. Title, author, recommendation and link are in the HTML; headingLevel and showBadge are at their defaults.
2. **W2 (Lesson 6.2).** You changed the card's save(). When do you need a deprecated entry, and when not? **Answer:** When content with the old HTML exists that must stay editable (a client's posts): always. While the block exists only on a test site: delete the test posts instead.
3. **W3 (Lesson 6.3).** The Book List stores only `<!-- wp:pagesco/book-list /-->`. Who decides how many books it shows? **Answer:** render.php, on every view: no count means the Books per list setting (6), clamped to 1 to 24.
4. **W4 (Lesson 6.3).** Which line makes WordPress pass the book's ID to a paragraph bound to pagesco/book? **Answer:** `'uses_context' => array( 'postId', 'postType' )` in register_block_bindings_source(); without it the fallback stays, silently.
5. **W5 (Lesson 6.3).** What does the Hobbit's page show as its price, and where does the value come from? **Answer:** `$10.99`: the large paragraph is bound to pagesco/book's price field, which formats pagesco_price on every view.

The projector site's answers to W1 and W5:

```
wp eval '$b = wp_list_filter( parse_blocks( get_page_by_path( "staff-picks", OBJEC
```

```
{"backgroundColor":"sage"}
```

```
curl -s http://localhost:8080/books/the-hobbit/ | grep -o 'has-large-font-size wp-
```

```
has-large-font-size wp-block-paragraph">$10.99
```

6.4 The Interactivity API

55 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain how the Interactivity API splits the work: server-rendered HTML with directives, then hydration by a store in a script module
- Scaffold an interactive block into the plugin, and build its viewScriptModule with --experimental-modules
- Set state and per-element context in PHP with wp_interactivity_state() and wp_interactivity_data_wp_context(), including derived state the server can compute
- Write a store with store(), state getters, getContext() and actions, and connect it with data-wp-on, bind, class and text
- Build interactive features that stay accessible and keep working without JavaScript

TIMING IN THE ROOM

10m	Frame: the stage and the lighting desk; server first, then hydration; state, derived state, context
25m	Live demo: the module build, render.php, the store, what the server already did, the browser
15m	Guided lab: `open_today_starter.php`
5m	Start the lesson project: the Find a Book page

MENTAL MODEL TO SAY

"The server builds the stage: every book is on it, every genre link, finished and visible. Directives are cables from each light to a fader on the lighting desk: 'this book is lit while isVisible is true'. The store is the desk: move the genre fader and every light wired to it changes at once. You only ever touch the faders; the cables do the rest."

Then the two consequences on the board: **the server evaluates directives too** (the page is right before any JavaScript) and **one shared runtime**, loaded as a module only where an interactive block is. Then the design question before any code: "What is the ONE thing that changes when a customer clicks?" (the genre). "Everything else is derived. What does each element need to know about itself?" (its slug, its genres: context).

LIVE DEMO (YOU TYPE, THEY FOLLOW)

From the site as Lesson 6.3 left it. The JavaScript and JSON files are pasted from the lesson; render.php is read aloud; the outputs are what curl, the build and the page really return.

1. **The module build (4m).** "Interactive blocks are built a second way." The scaffold (`--template @wordpress/create-block-interactive-template`) downloads its template, so in the room create src/blocks/book-filter/ by hand. First the plugin's package.json: both scripts gain `--experimental-modules`:

```
"build": "wp-scripts build --webpack-src-dir=src/blocks --output-path=build/bl
        "start": "wp-scripts start --webpack-src-dir=src/blocks --output-path=
```

   "Without it, view.js is silently not built" (Common Mistake 1). Then the lesson's block.json: `"interactivity": true` and `"viewScriptModule": "file:./view.js"`.

2. **render.php: state, derived state, context, directives (8m).** Paste the lesson's render.php and read it in four parts with the room: the data (books, genres, each book's genre slugs); `?genre=` validated against the real slugs; `wp_interactivity_state()` with two values and three CLOSURES; the HTML with `data-wp-interactive`, `wp_interactivity_data_wp_context()` and the directives. Write on the board: **state = genre · derived = isVisible, isSelected, visibleCount · context = slug, genres**.

3. **The store (4m).** Paste view.js: the same three derived values as getters, and one action, `selectGenre`, wrapped in `withSyncEvent()` because it calls `preventDefault()`. Then style.scss, index.js, edit.js (ServerSideRender) and build:

```
npm run build > /dev/null
ls build/blocks/book-filter
```

```
block.json
index.asset.php
index.js
render.php
style-index-rtl.css
style-index.css
view.asset.php
view.js
```

   view.js and view.asset.php: the module was built. Create a page "Filter test" with only `<!-- wp:pagesco/book-filter /-->`.

4. **What the server already did (5m).** "curl runs no JavaScript. Ask for the fantasy books."

```
curl -s 'http://localhost:8080/filter-test/?genre=fantasy' > filter.html
grep -o 'Showing <span data-wp-text="state.visibleCount">[0-9]*</span> of [0-9
grep -o '<a aria-current="true" class="is-selected"' filter.html
grep -c '<li hidden' filter.html
```

```
Showing <span data-wp-text="state.visibleCount">2</span> of 11 books
<a aria-current="true" class="is-selected"
9
```

The server ran the directives: the count says 2, the Fantasy link has aria-current and the class, nine books are hidden. Then the state that travels with the page:

```
grep -A 1 'id="wp-script-module-data-@wordpress/interactivity"' filter.html
rm filter.html
```

```
<script id="wp-script-module-data-@wordpress/interactivity" type="application/
{"state":{"pagesco/book-filter":{"genre":"fantasy","bookGenres":[["classic","c
```

`genre` and `bookGenres` as values; the closures arrive as `{}` and are listed in `derivedStateClosures`: the store must bring its own getters. And the module only where the block is:

```
curl -s http://localhost:8080/filter-test/ | grep -o 'id="pagesco-book-filter-
curl -s http://localhost:8080/filter-test/ | grep -o '"@wordpress/interactivit
curl -s http://localhost:8080/about/ | grep -c 'pagesco-book-filter-view-scrip
```

```
id="pagesco-book-filter-view-script-module-js-module"
"@wordpress/interactivity":"http://localhost:8080/wp-includes/js/dist/script-m
0
```

5 **In the browser (4m).** Open /filter-test/ with the Console, click Fantasy, Science Fiction, All books. Then the lesson's Playwright recording, for the room to compare with what they saw:

```
Loaded                 Showing 11 of 11 books current: All books       addr
                       visible: all 11
Clicked Fantasy        Showing 2 of 11 books  current: Fantasy         addr
                       visible: Lord of the Rings, The Hobbit
Clicked Science Fiction  Showing 3 of 11 books  current: Science Fiction  addr
                       visible: Brave New World, Dune, Fahrenheit 451
Clicked All books      Showing 11 of 11 books current: All books       addr
                       visible: all 11

Page loads: 1
Console: (empty)
```

"Three filters, one page load, an empty Console." With JavaScript switched off the same click reloads the page and the server filters it: progressive enhancement, almost for free.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **No `--experimental-modules`:** "compiled successfully", no view.js, and a filter that reloads the page. Coach: "Is view.js in build/? Does the page load on every click?" (Lesson quiz Q1.)
- **A JavaScript getter that disagrees with its PHP closure:** the page changes on hydration (a flash). Coach: "Write the closure and the getter side by side; same inputs, same answer." (Self-check Q2, the guided lab.)
- **Per-element data in the state, or a typo in the namespace.** Coach: "Is this the same for every element (state) or one element's own (context)? Is the namespace spelled the same in render.php and view.js?" (Self-check Q4, the debug handout.)

GUIDED LAB

File: `labs/module-13/l6_4-interactivity-api/open_today_starter.php`; your reference is `labs/module-13/l6_4-interactivity-api/open_today_solution.php`. An opening-hours box with a folded week: the learner writes two derived-state closures and the directives, and `wp_interactivity_process_directives()` does on the server exactly what WordPress does for a block with `interactivity: true`. It practices the server half of the lesson, which is the half PHP developers own, with no build and no browser.

- **Run it as shipped (2m):** the paragraph says TODO, and the three days carry only their context.
- **TODOs 1 to 4 in pairs (9m).** Before each run, predict which `<li>` gets `hidden`, `aria-current` and the class. Circulate for `wp_interactivity_state( 'pagesco/hours' )` inside the closures (it works, but the namespace is already known: no argument needed) and for a forgotten `!`.
- **Expected when done:**

```
<div data-wp-interactive="pagesco/hours"><p data-wp-text="state.todayText">Today
<button aria-expanded="false" data-wp-on--click="actions.toggleWeek" data-wp-bind
<li aria-current="false" hidden data-wp-context='{"day":"Friday"}' data-wp-bind--
<li aria-current="true" class="is-today" data-wp-context='{"day":"Saturday"}' dat
<li aria-current="false" hidden data-wp-context='{"day":"Sunday"}' data-wp-bind--
</ul></div>
```

- **Debrief (4m):** "aria-current says false on Friday: why not simply absent?" (aria- attributes get the words true and false.) "Which directive did the server NOT touch?" (data-wp-on--click.) "What would the browser's store need?" (A getter for each closure and the toggleWeek action: the lesson's view.js pattern; optional at home.)
- **Fast finishers:** `labs/module-13/l6_4-interactivity-api/event_list_starter.php` (data-wp-each repeats a template from a list in the state), then `labs/module-13/l6_4-interactivity-api/debug_stock_badges.php`.

START THE LESSON PROJECT

The lesson project is **A Find a Book Page**: the Book Filter on its own page, which the header menu picks up by itself. In the room only Step 1 (5 minutes), on laptops whose build has view.js.

- Step 1: a page "Find a Book" with the lesson's paragraph and the Book Filter. Then ask the server for the dystopian books, as text:

```
http://localhost:8080/find-a-book/?genre=dystopian   (the count, as text)
```

```
Showing 3 of 11 books
```

- **Goes home:** Step 2 (test it like a customer: every genre, a reload on ?genre=classic, the keyboard, JavaScript off) and Step 3 (delete Filter test, the checks, the plan line).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- What is the state of the Book Filter, and what is derived? **Answer:** the state is the genre (and the books' genres); isVisible, isSelected and visibleCount are derived.
- Why are the genres links and not buttons? **Answer:** without JavaScript the link is followed and the server renders the filtered page; with it, the action prevents the navigation.
- Which directives does the server process? **Answer:** interactive, context, bind, class, style, text, each; not on, watch, init, run.
- What does the page load only where the block is? **Answer:** the view.js script module (and the import map entry for @wordpress/interactivity).

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** B) From the state JSON that render.php's wp_interactivity_state() put into the page; **Q2** D) `data-wp-on--click="actions.selectGenre"`; **Q3** `8`; **Q4** A) In the element's context, `wp_interactivity_data_wp_context( array( 'slug' => $slug ) )`; **Q5** C) Wrapping the action in withSyncEvent(); **Q6** The link is followed (a page load) and the server renders the filtered page: the links are real ?genre= links and render.php reads the address. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners new to JavaScript stores read view.js next to the render.php closures (the board's three columns) and type only render.php. In the lab, give TODO 1's closure on the board; TODO 2 is the same shape.

Stretch: `event_list_starter.php` and `debug_stock_badges.php`; the lesson's Exercise 3 (a heading that follows the filter) and the Bonus Challenge (hide sold-out books, with a no-JavaScript fallback).

7.1 Using the REST API

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain routes, endpoints, methods, JSON responses and status codes, and list the routes your site offers
- Read the catalog from core endpoints and shape the answer with _fields, _embed, context, filters and the pagination headers
- Authenticate the right way: cookie plus nonce for logged-in JavaScript, application passwords for other programs
- Call the REST API from JavaScript with apiFetch, and handle its errors
- Read REST errors (400, 401, 403, 404) and tell "who are you?" from "you may not"

TIMING IN THE ROOM

10m	Frame: the service counter; base, namespace, route, endpoint, response; the request's trip
30m	Live demo: the index and routes, headers and pages, 401, cookie and nonce, an application password, 400, 403, 404, apiFetch
15m	Guided lab: `catalog_queries_starter.sh`
10m	Start the lesson project: the inventory script sells the last copy

MENTAL MODEL TO SAY

"The shop floor is the website: customers browse the shelves, which is HTML. At the back is a service counter with a list of what it handles: 'ask for a book by its number', 'change a price'. Anyone may ask the counter for public information. For anything that changes the shop, the clerk first checks who you are, authentication, and then whether your role allows it, permission. What you get back is a neat package, JSON, with a slip stapled to it: the status code and the headers."

Then the five words on the board with the site's examples: **base** /wp-json/ · **namespace** wp/v2 · **route** /wp/v2/books/(?P<id>[\d]+) · **endpoint** GET or POST on a route · **response** JSON + status + headers. And the trip: authentication → route and method → arguments validated → permission → callback → JSON. "Lesson 7.2 builds counters of the shop's own."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

From the site as Lesson 6.4 left it, in the WP-CLI container shell (curl is there, http://localhost:8080 is the site). Learners type every curl: that is the skill.

1. **Ask the API about itself (4m).** "Every site has a service counter at /wp-json/." Three fields of the index:

```
curl -s http://localhost:8080/wp-json/   (the fields name, url and namespaces)
```

```
{
  "name": "Pages &amp;amp; Co.",
  "url": "http://localhost:8080",
  "namespaces": [
    "oembed/1.0",
    "wp/v2",
    "wp-site-health/v1",
    "wp-block-editor/v1",
    "wp-abilities/v1"
  ]
}
```

Then the routes Lesson 5.2's show_in_rest built for books and genres:

```
wp eval 'foreach ( array_keys( rest_get_server()->get_routes() ) as $route ) {
```

```
/wp/v2/books
/wp/v2/books/(?P<id>[\d]+)
/wp/v2/books/(?P<id>[\d]+)/autosaves
/wp/v2/books/(?P<parent>[\d]+)/autosaves/(?P<id>[\d]+)
/wp/v2/genres
/wp/v2/genres/(?P<id>[\d]+)
```

2 **The catalog, and its headers (5m).** "Ten books, and the totals travel in headers."

```
curl -s -D - -o books.json http://localhost:8080/wp-json/wp/v2/books | grep -E
wc -c < books.json
rm books.json
```

```
HTTP/1.1 200 OK
Link: <http://localhost:8080/wp-json/wp/v2/books?page=2>; rel="next"
X-WP-Total: 11
X-WP-TotalPages: 2
13202
```

13202 bytes for ten books. Ask for less with `_fields`, and for a page with `per_page` and `page`:

```
curl -s 'http://localhost:8080/wp-json/wp/v2/books?_fields=id,title.rendered,m
```

```
[{"id":40,"title":{"rendered":"Animal Farm"},"meta":{"pagesco_price":8.49}},{"
```

```
curl -s -i 'http://localhost:8080/wp-json/wp/v2/books?per_page=4&page=2&_field
```

```
HTTP/1.1 200 OK
Link: <http://localhost:8080/wp-json/wp/v2/books?per_page=4&page=1&_fields=id>
X-WP-Total: 11
X-WP-TotalPages: 3

[{"id":36},{"id":35},{"id":34},{"id":33}]
```

3 **Who is asking? 401 (3m).** "Now change something, anonymously."

```
curl -s -X POST -H 'Content-Type: application/json' -d '{"meta":{"pagesco_pric
```

```
{"code":"rest_cannot_edit","message":"Sorry, you are not allowed to edit this
```

401: "who are you?" Then the cookie and the nonce, from the container (the lesson's block: a cookie file for Maria, then users/me without, with and with a made-up nonce):

```
curl -s -o /dev/null -c jar.txt http://localhost:8080/wp-login.php
curl -s -o /dev/null -b jar.txt -c jar.txt --data-urlencode log=maria --data-u
curl -s -b jar.txt 'http://localhost:8080/wp-json/wp/v2/users/me?_fields=id,na
NONCE=$(curl -s -b jar.txt 'http://localhost:8080/wp-admin/admin-ajax.php?acti
curl -s -b jar.txt -H "X-WP-Nonce: $NONCE" 'http://localhost:8080/wp-json/wp/v
curl -s -b jar.txt -H 'X-WP-Nonce: 0123456789' 'http://localhost:8080/wp-json/
rm jar.txt
```

```
{"code":"rest_not_logged_in","message":"You are not currently logged in.","dat
{"id":2,"name":"Maria Lopez"}
{"code":"rest_cookie_invalid_nonce","message":"Cookie check failed","data":{"s
```

4 **An application password (6m).** "A program has no cookie and must never know Maria's real password." Create one for the inventory script (your 24 characters will differ; put them in `PASS` in the container shell):

```
wp user application-password create maria "Inventory script" --porcelain
```

```
PeVINOjKoM7cd6O2FzYsnzqd
```

Who am I, the sale that was refused a minute ago, and the book page that follows at once through Lesson 6.3's binding:

```
curl -s --user "maria:$PASS" 'http://localhost:8080/wp-json/wp/v2/users/me?_fi
```

```
{"id":2,"name":"Maria Lopez"}
```

```
curl -s --user "maria:$PASS" -X POST -H 'Content-Type: application/json' -d '{
```

```
{"id":30,"meta":{"pagesco_price":12.99}}
```

```
curl -s http://localhost:8080/books/dune/ | grep -o 'has-large-font-size wp-bl
```

```
has-large-font-size wp-block-paragraph">$12.99
```

Put the price back (the lesson's next command, `14.99`). "Basic authentication only encodes: HTTPS, or the local environment, as here."

5 **When the answer is no (7m).** "Every error has the same shape: a code, a message, a status." Too many books:

```
curl -s -i 'http://localhost:8080/wp-json/wp/v2/books?per_page=500'
```

```
HTTP/1.1 400 Bad Request

{"code":"rest_invalid_param","message":"Invalid parameter(s): per_page","data'
```

Devon, an author, with his own application password, on Maria's book:

```
curl -s --user "devon:$DEVON_PASS" -X POST -H 'Content-Type: application/json'
```

```
{"code":"rest_cannot_edit","message":"Sorry, you are not allowed to edit this
```

The same code as the anonymous POST, but **403**: "I know you, and no." Two 404s and a context only editors get:

```
curl -s http://localhost:8080/wp-json/wp/v2/bookz; echo
curl -s http://localhost:8080/wp-json/wp/v2/books/99; echo
```

```
{"code":"rest_no_route","message":"No route was found matching the URL and re
{"code":"rest_post_invalid_id","message":"Invalid post ID.","data":{"status":
```

```
curl -s 'http://localhost:8080/wp-json/wp/v2/books/30?context=edit'; echo
```

```
{"code":"rest_forbidden_context","message":"Sorry, you are not allowed to edi
```

6 **apiFetch, on the board (5m).** Show the lesson's apiFetch fragment (read, write, catch) and the recording of Common Mistake 2: a plain `fetch()` on Maria's Dashboard is anonymous, `wp.apiFetch()` sends the nonce.

```
> await ( await fetch( '/wp-json/wp/v2/users/me?_fields=id,name' ) ).json()
{"code":"rest_not_logged_in","message":"You are not currently logged in.","dat
> await wp.apiFetch( { path: '/wp/v2/users/me?_fields=id,name' } )
{"id":2,"name":"Maria Lopez"}

Network:  GET /wp-json/wp/v2/users/me?_fields=id,name                       n
          GET /wp-json/wp/v2/users/me?_fields=id%2Cname&_locale=user
```

"When an API says 'not logged in' while you are, check the nonce first."

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- `_fields` **without** `_links`**, so** `_embed` **embeds nothing.** No error, no genres. Coach: "What does _embed follow? Is it still in the answer?" (Self-check Q10.)
- **fetch() without the nonce:** "not logged in" on a screen where Maria plainly is. Coach: "Which header proves the request comes from your own page?" (Self-check Q12; lesson quiz Q2.)
- **The account password with** `--user`**:** 401 rest_not_logged_in, indistinguishable from a typo in an application password. Coach: "Which password did you create for this program? Users > Profile: was it ever used?"

GUIDED LAB

File: `labs/module-13/l7_1-rest-api/catalog_queries_starter.sh`; your reference is `labs/module-13/l7_1-rest-api/catalog_queries_solution.sh`. Six questions for the catalog, each one curl request to a core route shaped with the lesson's parameters or read from a header: a genre's ID by its slug, its books by title, a search, the pages, a status code, the biggest genres. It practices reading the API from the outside, as any client does.

- **Run it as shipped (2m):** `cp labs/module-13/l7_1-rest-api/catalog_queries_starter.sh wordpress/`, `docker compose run --rm wpcli sh catalog_queries_starter.sh`: only the

total works.

- **TODOs 1 to 6 in pairs (10m),** each request tried alone in a container shell first. Circulate for `&` in an unquoted URL (the shell starts a background job: quote every URL) and for `genres=dystopian` (the filter takes the ID, which is why TODO 1 exists).
- **Expected when done:**

```
Books in the catalog: 11
The Dystopian genre:
[{"id":9,"name":"Dystopian"}]
Its books, by title:
[{"slug":"1984"},{"slug":"brave-new-world"},{"slug":"fahrenheit-451"}]
Search for farm:
[{"id":40,"title":{"rendered":"Animal Farm"}}]
The site's pages, oldest first:
[{"slug":"sample-page"},{"slug":"about"},{"slug":"contact"},{"slug":"events"},{"s
Book 99: 404
The two biggest genres:
[{"count":11,"name":"Fiction"},{"count":6,"name":"Classic"}]
```

- **Debrief (3m):** "Why does the genre filter take an ID when the site's addresses use slugs?" (Core's routes link resources by ID; Lesson 7.2's own route takes the slug.) "How would you get all eleven titles if per_page stopped at 4?" (Loop up to X-WP-TotalPages: the lesson's Exercise 1.)
- **Fast finishers:** `labs/module-13/l7_1-rest-api/who_may_starter.sh` (five requests as anonymous, Devon, Maria and admin, with throwaway application passwords it revokes), then `labs/module-13/l7_1-rest-api/debug_stock_list.php`.

START THE LESSON PROJECT

The lesson project is **Stock at a Glance**: the inventory script's credentials in use, then a Dashboard widget that reads and updates books with apiFetch. In the room: Step 1 (10 minutes), the two requests a stock system sends.

- Step 1: the last Fahrenheit 451 is sold (POST with Maria's "Inventory script" password), the book's page says so, and the delivery puts it back:

```
curl -s --user "maria:$PASS" -X POST -H 'Content-Type: application/json' -d '{"met
```

```
{"id":39,"title":{"rendered":"Fahrenheit 451"},"meta":{"pagesco_in_stock":false}}
```

```
<li>Sold out</li>
```

```
curl -s --user "maria:$PASS" -X POST -H 'Content-Type: application/json' -d '{"met
```

```
{"id":39,"title":{"rendered":"Fahrenheit 451"},"meta":{"pagesco_in_stock":true}}
```

- **Done in the room** = both answers and the page's `<li>Sold out</li>` in between. Ask: "Which pages followed without anyone touching them?" (The book page, the Book List, Find a Book: they all read the meta.)
- **Goes home:** Steps 2 to 4: `Admin\StockWidget` as the fifteenth feature, `assets/js/stock-widget.js` (plain JavaScript, no build), trying it as Maria; then the checks. The widget's frame, as Maria's Dashboard source shows it on the projector site:

```
<div class="inside">
        <p class="pagesco-stock-widget__status" aria-live="polite">Loading the ca
        <ul class="pagesco-stock-widget__books"></ul>
        </div>
```

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Where do the totals of a collection travel? **Answer:** in the headers X-WP-Total and X-WP-TotalPages (and Link for the neighbors), so the body stays a plain list.
- Cookie without a nonce, cookie with a wrong nonce: what does each get? **Answer:** treated as anonymous (401 for anything protected); 403 rest_cookie_invalid_nonce.
- Devon gets rest_cannot_edit. 401 or 403, and what does it mean? **Answer:** 403: WordPress knows him and his role says no.
- Why one application password per program? **Answer:** revoking one (a lost laptop, a retired script) cuts off exactly that program.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** C) 401 rest_forbidden_context; **Q8** `X-WP-TotalPages: 2`; **Q9** A) 403 rest_cookie_invalid_nonce: the page must fetch a fresh nonce; **Q10** D) _fields removed _links, so there was nothing to follow, and _embedded is cut off too; **Q11** B) WP_ENVIRONMENT_TYPE is local; elsewhere WordPress offers them only on HTTPS; **Q12** The REST nonce (X-WP-Nonce) for the logged-in user, and the API's root URL (also _locale=user, a JSON body and the server's error object on failure). Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a card with the three request shapes: `curl -s URL` (read), `curl -s -I URL` (headers only), `curl -s --user "maria:$PASS" -X POST -H 'Content-Type: application/json' -d '{...}' URL` (write). For the lab, TODO 1 and 2 together on the projector.

Stretch: `who_may_starter.sh` and `debug_stock_list.php`; the lesson's Exercise 3 (batch/v1) and Exercise 4 (register_rest_field); the Bonus Challenge (a search box with a debounce and AbortController).

CHECKPOINT 2

2:20 Checkpoint 2: Modules 5 and 6

30 min

TIMING

2m	Hand out `checkpoint-2/checkpoint_starter.php` and the printed Part D page; read the rules aloud
25m	Parts A to C in the file, Part D on paper; on their own, open notes (lesson pages, their own plugin)
3m	Collect: the file (renamed `checkpoint_NAME.php`, left in wordpress/ until you have run it) and the Part D page

THE TASK (THE WORKBOOK'S CHECKPOINT PAGE)

The Weekly Picks report. The shop's newsletter wants four lines built from the plugin's data. Learners write the bodies of four functions; the main section is given and must not change; the script changes nothing on the site. Run with `wp eval-file --use-include checkpoint_NAME.php` on a site at the end of Lesson 6.3 (the Book List block must exist).

- **Part A (Lessons 5.2, 5.3):** `pagesco_cp_cheap_books( float $max )`: the books in stock that cost at most $max, cheapest first, as "Title ($8.49)": one get_posts() with a meta_query (price as a number, stock '1') and orderby meta_value_num.
- **Part B (Lesson 5.5):** `pagesco_cp_events_at( string $location )`: how many events take place at a location: one COUNT query on the events table through `$wpdb->prepare()` (the main section tries a hostile location).
- **Part C (Lessons 6.2, 6.3):** `pagesco_cp_list_titles( string $genre )`: render a Book List block for the genre, cheapest first, with do_blocks() and return its titles; `pagesco_cp_cards_in( string $slug )`: how many Book Cards a post stores and the first one's comment attributes.
- **Part D (written):** three questions on the workbook page, one on Module 5, two on Module 6.

As shipped, the starter runs without a message and prints:

```
A  Under $10, in stock: TODO
B  Café: -1 events · Rooftop: -1 events · Reading corner'; DROP TABLE x; --: -1 e
C1 Fantasy, cheapest first: TODO
C2 Staff picks post: 0 Book Card, comment attributes {"TODO":true}
```

The model answer prints:

```
A  Under $10, in stock: Pride and Prejudice ($7.99), Animal Farm ($8.49), Brave N
B  Café: 2 events · Rooftop: 0 events · Reading corner'; DROP TABLE x; --: 0 even
C1 Fantasy, cheapest first: The Hobbit, Lord of the Rings
C2 Staff picks post: 1 Book Card, comment attributes {"backgroundColor":"sage"}
```

RULES AND SCORING

- Open notes: the lesson pages, their own plugin and pagesco-plan.txt; no neighbor, no AI assistant. Questions about the wording only.
- Score with the rubric in the Answer Key: five rows, 0 to 2 points each. Run each learner's file once on your projector site (it changes nothing); compare with the model output line by line.
- **8 to 10:** on track for Session 14. **5 to 7:** name the weakest row in your feedback and point to its lesson. **Under 5:** a one-to-one before Session 15; Module 7 builds directly on 5.3 and 5.5.
- A learner whose site is not at the end of Lesson 6.3 (no Book List): score Part C from the code, and say so on the sheet.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Right after the checkpoint, so keep it short: three questions on today's lessons, on a sticky note.

1. In the Book Filter, name the state and one piece of derived state. **Answer:** the state is the selected genre (and the books' genres); derived: isVisible, isSelected or visibleCount.
2. 401 or 403: which means "who are you?" **Answer:** 401 (not authenticated); 403 is "I know you, and no".
3. Which credential does a partner's nightly price script use? **Answer:** an application password of a user whose role can edit books, over HTTPS.

HOMEWORK (ABOUT 3 HOURS)

- **Finish both lesson projects.** 6.4: Steps 2 and 3 (test it like a customer, with JavaScript off; clean up). 7.1: Steps 2 to 4, the Stock at a glance widget, and the checks.
- **Lesson exercises.** 6.4: Exercises 1 and 2 (every filter from the server, the state registry); Exercise 3 is a stretch. 7.1: Exercises 1 and 2 (the catalog page by page, a draft round trip with a throwaway password); 3 and 4 are stretches.
- **Both Quick Quizzes** on the lesson pages (answers are in the Answer Key).
- **Journal:** both entries (state, derived state and context for the filter; every program that might talk to the site).
- **Before the Next Lesson (7.1):** the widget works for Maria, Fahrenheit 451 is in stock; keep the "Inventory script" password (in `$PASS`, never in wordpress/) and create Devon's "Test" password for Lesson 7.2; reread Lesson

5.3's sanitize and validate and Lesson 5.5's EventsTable::find() and add().

- **Optional:** `labs/module-13/l6_4-interactivity-api/event_list_starter.php`, `labs/module-13/l7_1-rest-api/who_may_starter.sh` and both debug handouts.

BRING LIST FOR SESSION 14

- A site at the end of Lesson 7.1 (the widget, fifteen features) and Maria's "Inventory script" and Devon's "Test" application passwords in your password manager.
- The project folder with `labs/`; Session 14's labs call the site's own REST API over HTTP, no internet needed.
- pagesco-plan.txt, the journal and the workbook.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the checkpoint's 30 minutes (start at 2:20 even if a lab is unfinished); what the server already did for ?genre=fantasy; 401 versus 403 and the application password.
- **Cut first (saves 5m):** the 6.4 project start: the Find a Book page is one paste at home.
- **Cut second (saves 4m):** the browser step of the 6.4 demo: the workbook and the lesson have the recording.
- **Cut third (saves 5m):** the apiFetch step of the 7.1 demo: the project's widget at home teaches it.

RUNNING SHORT: WHAT TO ADD

- Lesson 6.4's Exercise 1 live: the count for every genre, `romance` and an encoded `<script>`, from curl.
- Lesson 7.1's Exercise 3: restock two books in one batch request, read the answer's per-request status, undo it.
- Run `who_may_starter.sh` for everyone and fill in the table together before running the solution.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 14: Custom Endpoints, External APIs and Cron

How to teach the session where the shop opens API counters of its own and starts calling another company's: routes with arguments, schemas, errors and permission checks, then a careful HTTP client, transients that survive a broken supplier, WP-Cron jobs scheduled once, and an ability for assistants.

2 lessons | 3 hrs session | Lessons 7.2–7.3 | Session 14 of 18 · Part 2

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's ideas) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Every command below was run on WordPress 7.1.3 / PHP 8.5 to produce the output printed under it: Lesson 7.2's demo on the site after Lesson 7.1 (`after:lesson_23`), Lesson 7.3's on the site after Lesson 7.2 (`after:lesson_24`). REST requests shown as `curl --user "maria:$PASS"` were sent with a real application password. Timing outputs (the timeout demo) differ by a few milliseconds on every run.
- No internet is needed: the supplier is the course's sandbox inside the site, and the kit's 7.3 labs call the site's own pagesco/v1 API over HTTP, as if it were another company's. Only the lesson's last exercise (Open Library) needs a connection.
- Labs live in `labs/module-14/`: `_starter` files and folders are handouts, `_solution` files and folders are yours.
- Part 2, Session 6 of 10. Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (7.2, 7.3) match the course site.

> **THE #1 RISK OF SESSION 14: TWO LONG LESSONS OF PHP CLASSES**
>
> Lessons 7.2 and 7.3 are written for 145 and 150 minutes and are the plugin's longest files so far (BooksEndpoint, EventsEndpoint, Client, Availability, DailySync, Abilities). Typing them in the room would take the whole session. **Paste the classes, read them aloud, and type only the commands and the curl requests**: the skill is reading answers and errors, not typing arrays. Both projects' later steps go home (7.2 Steps 3 and 4, 7.3 Steps 3 and 4); say so at 0:00.

BEFORE THE SESSION

- ☐ **Projector site at** `after:lesson_23` (the end of Lesson 7.1, the Stock at a glance widget). Keep Maria's "Inventory script" password in `PASS` and Devon's "Test" password in `DEVON_PASS` in the WP-CLI container shell (`docker compose run --rm wpcli sh`), never in a file under wordpress/.
- ☐ **Rehearse** 7.2's two versions of BooksEndpoint and the EventsEndpoint, and 7.3's sandbox, Client and Availability, on a scratch copy; have the lesson pages open to paste from.
- ☐ **Files to hand out:** `labs/module-14/` WITHOUT the `*_solution.*` files and the `opening_hours_solution/` folder. **Run the kit check once:** `python3 labs/check_labs.py module-14` (15 results, about two minutes).
- ☐ **Checkpoint 2:** bring the scored rubrics and give each learner a minute in the warm-up with their weakest row.
- ☐ **Board, before learners arrive:** the four desks **validate → sanitize → permission → callback**; **phone = wp_remote_get · sticky note = transient · closing checklist = WP-Cron**.

KNOWN TRAPS OF THIS SESSION

SYMPTOM	CAUSE	FIX
A route answers, but `X-WP-DoingItWrong` in the headers	No permission_callback	Always write the key; `__return_true` only for public reads (Mistake 1).
per_page=500 accepted	A custom sanitize_callback replaced the schema's checks	Keep `'validate_callback' => 'rest_validate_request_arg'` next to it (Mistake 2).
500 Internal Server Error with `"data":null`	A WP_Error without a status	`array( 'status' => 404 )` on every WP_Error a callback returns (Mistake 3).
"REST API routes must be registered on the rest_api_init action"	register_rest_route() on init	Hook rest_api_init.
`Trying to access array offset on null` after a remote call	Only is_wp_error() was checked; the body was HTML or an error	Check the status and the decoded shape too (Lesson 7.3, Mistake 1).
Pages hang for seconds	A remote call on page views, with the default 5-second timeout	Read a transient; call from a cron job; set a short timeout.
The job ran four times	wp_schedule_event() on every request	Schedule on activation, guarded with wp_next_scheduled() (Mistake 2).

SYMPTOM	CAUSE	FIX
The job "ran" but did nothing	The callback was added only inside is_admin()	Add cron callbacks unconditionally (Mistake 3).

SESSION 14 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 6.4 and 7.1, homework check (the widget), Checkpoint 2 returned or announced.
0:10	10	**7.2** Frame and mental model	A window with a printed menu: route, arguments, schema, and the staff-only sign.
0:20	25	**7.2** Live demo	A first route; arguments, schema and errors the schema writes; pages and 404s; a route without permission_callback.
0:45	20	**7.2** Guided lab	`labs/module-14/l7_2-custom-endpoints/opening_hours_starter/`: a route of your own with an enum argument.
1:05	15	**7.2** Start the lesson project	Steps 1 and 2: the events route, read by anyone.
1:20	10	Break	Leave the projector on the Events API's 401 and 403.
1:30	10	**7.3** Frame and mental model	The phone, the sticky note and the closing checklist.
1:40	30	**7.3** Live demo	The sandbox and its key; three ways to fail and a timeout; transients and the read-through; WP-Cron's list.
2:10	20	**7.3** Guided lab	`labs/module-14/l7_3-external-apis-cron/remote_answers_starter.php`: one careful client, five kinds of answer.
2:30	20	**7.3** Start the lesson project	Steps 1 and 2: the ISBNs, the client and the cached Supplier column.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 15.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 145 + 150 = 295 minutes for a learner working alone; after the warm-up, the break and the wrap-up the room has 150. Lesson 7.2 gets 70 minutes: a route, its arguments and errors, a lab with a

route of the learner's own, and the events route's read side. Lesson 7.3 gets 80: the sandbox and the three failures, transients and cron, a lab of five answers, and the client with its cached column.

Goes home (about 2 to 3 hours). Lesson 7.2's project Steps 3 and 4 (the refused POSTs, then event 7 through the API) and the section "Calling It from the Front End" to read; Lesson 7.3's project Steps 3 and 4 (the daily sync with activation, the check-stock ability) and the Abilities section; both lessons' Hands-on Exercises, Quick Quizzes and journal entries; Before the Next Lesson for both.

Protect, in this order: what the schema turns away and the order of the four desks; Mistake 1 (a route without permission_callback); the three ways to fail; the read-through and its brief failure cache. **Cut first:** the Abilities step of the 7.3 demo, then the transients CLI step (the lab does it), then the 7.2 project start (Steps 1 and 2 go home).

OPENING SEGMENT

0:00 Warm-up: Lessons 6.4 and 7.1 — 10 min

TIMING

6m	Five retrieval questions (workbook Warm-up box), answers out loud
4m	Homework check: the Stock at a glance widget on every laptop; the scored Checkpoint 2 rubrics handed back

QUESTIONS AND ANSWERS

1. **W1 (Lesson 6.4).** curl asks /find-a-book/?genre=fantasy. How many books does the server send with the hidden attribute? **Answer:** `9` of eleven: the server evaluated data-wp-bind--hidden for each book (two fantasy books stay visible).
2. **W2 (Lesson 6.4).** Why does selectGenre need `withSyncEvent()`? **Answer:** It calls event.preventDefault(); since 6.8 actions may run after the event, too late to stop the link.
3. **W3 (Lesson 7.1).** A client gets 403 rest_cannot_edit. Fix the request, the credentials or the role? **Answer:** The role: 403 means "I know you, and no" (Devon on Maria's book). 401 would be the credentials.
4. **W4 (Lesson 7.1).** How does a client learn how many pages the catalog has at per_page=5, without reading the books? **Answer:** A HEAD request (`curl -I`) and the header: `X-WP-TotalPages: 3` on the projector site.
5. **W5 (Lesson 7.1).** Why can the Stock at a glance widget POST to /wp/v2/books with plain apiFetch, and an outside script cannot? **Answer:** apiFetch sends Maria's cookie AND the page's wp_rest nonce; a script has neither, so it needs an application password.

The projector site's answers to W1 and W4:

```
curl -s 'http://localhost:8080/find-a-book/?genre=fantasy' | grep -c '<li hidden'
```

```
9
```

```
curl -s -I 'http://localhost:8080/wp-json/wp/v2/books?per_page=5' | grep -i '^X-WP
```

```
X-WP-TotalPages: 3
```

LESSON PLANS

7.2 Custom REST Endpoints

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Register routes with register_rest_route() on rest_api_init, as a feature class of the plugin
- Declare arguments with types, defaults, limits and enums, and add your own validate_callback and sanitize_callback without losing the built-in checks
- Describe responses with a JSON Schema, and return WP_REST_Response objects with headers and status codes
- Return errors as WP_Error with the right status, and know what WordPress does with one that has none
- Write a real permission_callback for every route, and explain why __return_true is only for public reads

TIMING IN THE ROOM

10m	Frame: a window with a printed menu; route, arguments, schema, permission; the four desks
25m	Live demo: a first route, OPTIONS, fantasy by price, four 400s, 400 and 404, Mistake 1
20m	Guided lab: the `opening_hours_starter/` plugin
15m	Start the lesson project: the events route, Steps 1 and 2

MENTAL MODEL TO SAY

"Core's counter handles everything for everyone, so its menu is long and its packages are big. The shop opens a window of its own with a short printed menu: 'books, by genre or stock, sorted three ways, ten details each' and 'events: read them, or add one if you work here'. The menu is the route and its arguments; the card describing each package is the schema; the sign 'staff only' on the second item is the permission callback. A window without such a sign serves anyone who walks up."

Then the four desks on the board, in order: **validate** (types, ranges, enums, your validate_callback) → **sanitize** (strings become integers, defaults fill in) → **permission** (may this user?) → **callback** (gets clean, typed values). "A request that fails at a desk never reaches the next."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

From the site as Lesson 7.1 left it. Paste the classes from the lesson; type the commands. The container shell is where curl runs.

1. **A first route (5m).** "A counter of the shop's own." Save the lesson's first `src/Rest/BooksEndpoint.php` (one route, `__return_true`, `prepare_book()` with casts), register it as the sixteenth feature and ask the index:

```
sed -i '/\t\t\tAdmin\\StockWidget::class,/a\           Rest\\BooksEndpoint::c
grep -c "::class," wp-content/plugins/pagesco-books/src/Plugin.php
curl -s http://localhost:8080/wp-json/ | grep -o '"namespaces":\[[^]]*\]'
```

```
16
"namespaces":["oembed\/1.0","pagesco\/v1","wp\/v2","wp-site-health\/v1","wp-b
```

`pagesco/v1` is in the list. `GET /wp-json/pagesco/v1/books` answers 6 books (the Books per list setting), ten typed fields each, in about the size core needed for one book. Then the full version of the class from the lesson: two routes, `collection_args()`, `validate_genre()`, the schema.

2 **The menu: OPTIONS (3m).** "Clients read the menu from the route itself."

```
curl -s -X OPTIONS http://localhost:8080/wp-json/pagesco/v1/books | php -r '$r
```

```
page      integer  The page of results.
per_page  integer  Books per page. Default: the Books per list setting.
genre     string   Only books in this genre (a genre slug).
in_stock  boolean  true: only books in stock; false: only sold-out books.
orderby   string   The order of the books.
schema: id, title, slug, link, author, price, in_stock, pages, published, genr
```

Every argument with its type and description, and the schema's fields: documentation for free.

3 **Questions core could not answer (3m).** The fantasy books, cheapest first (title and price of each):

```
curl -s 'http://localhost:8080/wp-json/pagesco/v1/books?genre=fantasy&orderby=
```

```
[
  {
    "title": "The Hobbit",
    "price": 10.99
  },
  {
    "title": "Lord of the Rings",
    "price": 15.99
  }
]
```

4 **What the schema turns away (5m).** "Four requests, each wrong in a different way. Predict which needed code of yours."

```
curl -s 'http://localhost:8080/wp-json/pagesco/v1/books?genre=romance'; echo
curl -s 'http://localhost:8080/wp-json/pagesco/v1/books?per_page=500'; echo
curl -s 'http://localhost:8080/wp-json/pagesco/v1/books?orderby=pages'; echo
curl -s 'http://localhost:8080/wp-json/pagesco/v1/books?in_stock=maybe'; echo
```

```
{"code":"rest_invalid_param","message":"Invalid parameter(s): genre","data":{"
{"code":"rest_invalid_param","message":"Invalid parameter(s): per_page","data"
{"code":"rest_invalid_param","message":"Invalid parameter(s): orderby","data"
{"code":"rest_invalid_param","message":"Invalid parameter(s): in_stock","data"
```

Only the genre: validate_genre(). The other three WordPress wrote from the schema, before the callback ran. Then pages and the two kinds of not-found:

```
curl -s 'http://localhost:8080/wp-json/pagesco/v1/books?page=9'; echo
curl -s -i http://localhost:8080/wp-json/pagesco/v1/books/14 | sed -n '1p;$p';
```

```
{"code":"pagesco_invalid_page","message":"The page number is larger than the r
HTTP/1.1 404 Not Found
{"code":"pagesco_book_not_found","message":"No book was found with that ID.","
```

"Book 14 is the Events page. The 404 does not say so: never tell visitors what exists behind an ID they may not see."

5 **Mistake 1: a route without permission_callback (5m).** Save the lesson's scratch must-use plugin (a /hours route with no permission_callback) and look at the headers:

```
curl -s -i http://localhost:8080/wp-json/pagesco/v1/hours | grep -E '^(HTTP|X-
curl -s http://localhost:8080/wp-json/pagesco/v1/hours; echo
```

```
HTTP/1.1 200 OK
X-WP-DoingItWrong: register_rest_route (since 5.5.0; The REST API route defin
{"thursday":"9 a.m. to 9 p.m."}
```

It works, and the only sign is a header nobody reads. Outside a REST request the notice prints as usual:

```
wp eval 'rest_get_server();'
```

```
Notice: Function register_rest_route was called <strong>incorrectly</strong>.
```

Remove the file. Then the rule on the board: **__return_true only for public reads; every write checks a capability.** Show the lesson's `can_manage_events()` fragment.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **No permission_callback, or __return_true on a write.** Coach: "Who may call this, and what stops everyone else?" Look at the response headers of your own routes. (Self-check Q5.)
- **A sanitize_callback that switches the schema off** (absint on per_page). Coach: "Which function checks the maximum now?" Keep `rest_validate_request_arg` as the validate_callback. (The debug handout.)
- **A WP_Error without a status, or an error that leaks existence.** Coach: "What status does a client see? What does the message tell a stranger?" (Self-check Q6.)

GUIDED LAB

Folder: `labs/module-14/l7_2-custom-endpoints/opening_hours_starter/`, a plugin with `GET /pagesco/v1/opening-hours`: the route is registered with its permission (a public read); the learner adds the argument (a type and an enum, NO default: "not sent" means the whole week) and the callback's two answers. Your reference is `labs/module-14/l7_2-custom-endpoints/opening_hours_solution/`; the test is `labs/module-14/l7_2-custom-endpoints/test_opening_hours.sh`.

- **Install and run as shipped (4m):** `cp -r labs/module-14/l7_2-custom-endpoints/opening_hours_starter wordpress/wp-content/plugins/pagesco-opening-hours`, `wp plugin activate pagesco-opening-hours`, copy the test into wordpress/ and `docker compose run --rm wpcli sh test_opening_hours.sh`: every request answers the TODO object.
- **TODOs 1 to 3 (12m).** Circulate for a default of '' in the enum (the error then reads "is not one of , monday") and for `$request->get_param()` versus `$request['day']` (both fine).
- **Expected when done:**

```
Plugin 'pagesco-opening-hours' activated.
Success: Activated 1 of 1 plugins.
== Saturday
{"day":"saturday","closed":false,"opens":"10:00","closes":"16:00"}
== Sunday
{"day":"sunday","closed":true}
== A day that does not exist
{"code":"rest_invalid_param","message":"Invalid parameter(s): day","data":{"statu
== The whole week: how many days, and Thursday
7
{"day":"thursday","closed":false,"opens":"9:00","closes":"21:00"}
== What OPTIONS says about the argument
"day":{"description":"One day of the week, in lowercase English. Without it: the
```

- **Debrief (4m):** "Which line of yours produced the 400?" (None: the enum.) "What does an argument without a default hold when the client does not send it?" (null.) Then deactivate and remove the plugin.

- **Fast finishers:** `labs/module-14/l7_2-custom-endpoints/schema_rules_starter.php` (what the schema accepts and turns query strings into), then `labs/module-14/l7_2-custom-endpoints/debug_staff_picks_route.php`.

START THE LESSON PROJECT

The lesson project is **The Events API**: GET and POST /pagesco/v1/events on Lesson 5.5's table, through EventsTable. In the room: Steps 1 and 2 (15 minutes).

- Step 1: paste `src/Rest/EventsEndpoint.php`, read `can_manage_events()`, `validate_starts_at()` and `prepare_event()` aloud, register it as the seventeenth feature.
- Step 2: read the events (the lesson's request, id, title and starts_at of each):

```
[
  {
    "id": 1,
    "title": "Book club: Dune",
    "starts_at": "2027-11-04T19:00:00"
  },
  {
    "id": 2,
    "title": "Story time: The Hobbit",
    "starts_at": "2027-11-06T10:30:00"
  },
  {
    "id": 3,
    "title": "An evening with Ana Ruiz",
    "starts_at": "2027-11-11T18:30:00"
  },
  {
    "id": 4,
    "title": "Poetry open mic",
    "starts_at": "2027-11-18T19:30:00"
  },
  {
    "id": 5,
    "title": "Book club: Fahrenheit 451",
    "starts_at": "2027-12-02T19:00:00"
  }
]
```

- **Goes home:** Step 3 (anonymous 401, Devon 403, Maria's bad date: one error at a time) and Step 4 (event 7 through the API, 201, and the checks).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- In which order do the desks run? **Answer:** validate, sanitize, permission, callback.
- Why WP_Query and not get_posts() in get_books()? **Answer:** found_posts and max_num_pages for the X-WP-Total headers.
- What does rest_authorization_required_code() return? **Answer:** 401 when nobody is logged in, 403 when someone is.
- What does a WP_Error without a status become? **Answer:** a 500 with "data":null (lesson quiz Q5).

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** B) POST; **Q2** `200 6 2`; **Q3** A) 400 rest_missing_callback_param: arguments are validated before the permission check; **Q4** D) validate_callbacks run first; the schema checks of the other arguments run in the sanitizing step, which a failed validation never reaches; **Q5** C) A check of `current_user_can( 'edit_others_posts' )` that returns a WP_Error with rest_authorization_required_code(); **Q6** An endpoint must not tell visitors what exists behind an ID they may not see: drafts, private books and other post types all get the same 404. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: the board's four desks and one route skeleton (methods, callback, permission_callback, args) to copy; in the lab, TODO 1's argument array on the board.

Stretch: `schema_rules_starter.php` and `debug_staff_picks_route.php`; the lesson's Exercise 1 (rest_do_request) and Exercise 3 (DELETE an event); the Bonus Challenge (PATCH a book's price).

7.3 External APIs, Transients & WP-Cron

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Call an external API with wp_remote_get() and wp_remote_post(), and handle each way it can fail: no answer, an error status, or an answer that is not what you expected
- Keep API keys in wp-config.php and choose wp_safe_remote_get() when a URL comes from a user
- Cache answers in transients, explain where they are stored and how a persistent object cache changes that, and cache failures briefly too
- Schedule recurring work with WP-Cron, start and stop it on activation and deactivation, and inspect and run it with WP-CLI
- Register an ability with the Abilities API and run it over the REST API

TIMING IN THE ROOM

10m	Frame: the phone, the sticky note, the closing checklist
30m	Live demo: the sandbox, three ways to fail, transients, WP-Cron, abilities
20m	Guided lab: `remote_answers_starter.php`
20m	Start the lesson project: Steps 1 and 2

MENTAL MODEL TO SAY

"When a bookseller wants to know whether the distributor has more copies of The Great Gatsby, she calls. Sometimes the line is busy, sometimes nobody answers, sometimes the person reads the wrong list. A good bookseller writes the answer on a sticky note with the time, so the next colleague does not call again, and at closing time somebody works through a checklist."

Then the three pieces on the board: **the phone** = wp_remote_get() and friends; **the sticky note** = a transient, which may fall off the wall early, so you always check; **the closing checklist** = WP-Cron, which WordPress works through when the next visitor walks in after closing time. "A page that shows supplier data never waits for the supplier when a fresh note exists, and the nightly job keeps the note fresh."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

From the site as Lesson 7.2 left it. Scratch scripts are saved in wordpress/ and run with `wp eval-file`; delete them at the end of the demo (the lesson's `rm` line).

1. **A supplier to call (5m).** "Riverside Books is fictional, so the course gives you its sandbox: three files in wordpress/supplier-sandbox/, plain PHP." Paste the lesson's .htaccess, availability.php and orders.php; then the key and the address go into wp-config.php, never into code:

```
wp config set PAGESCO_BOOKS_SUPPLIER_URL 'http://localhost:8080/supplier-sandb
wp config set PAGESCO_BOOKS_SUPPLIER_KEY 'sandbox-7f3a9c'
```

```
Success: Added the constant 'PAGESCO_BOOKS_SUPPLIER_URL' to the 'wp-config.php
Success: Added the constant 'PAGESCO_BOOKS_SUPPLIER_KEY' to the 'wp-config.php
```

Try the sandbox with curl first, as with any documentation open:

```
curl -s -i 'http://localhost:8080/supplier-sandbox/v1/availability?isbns=97807
curl -s -H 'X-Api-Key: sandbox-7f3a9c' 'http://localhost:8080/supplier-sandbox
```

```
HTTP/1.1 401 Unauthorized
{"error":"invalid_api_key","message":"Send a valid API key in the X-Api-Key he

{"supplier":"Riverside Books (sandbox)","items":[{"isbn":"9780743273565","ava
```

2 **wp_remote_get and its answer (4m).** The lesson's `supplier-ask.php` (`wp eval-file supplier-ask.php`):

```php
<?php
$response = wp_remote_get(
    'http://localhost:8080/supplier-sandbox/v1/availability?isbns=978074327356
    array(
        'timeout' => 5,
        'headers' => array( 'X-Api-Key' => PAGESCO_BOOKS_SUPPLIER_KEY ),
    )
);

if ( is_wp_error( $response ) ) {
    echo 'No answer: ', $response->get_error_message(), "\n";
    return;
}

echo implode( ', ', array_keys( $response ) ), "\n";
echo wp_remote_retrieve_response_code( $response ), ' ', wp_remote_retrieve_re
echo wp_remote_retrieve_header( $response, 'content-type' ), "\n";

$data = json_decode( wp_remote_retrieve_body( $response ), true );
print_r( $data['items'][0] );
```

```
headers, body, response, cookies, filename, http_response
200 OK
application/json; charset=utf-8
Array
(
    [isbn] => 9780743273565
    [available] => 0
    [next_delivery] => 2027-11-15
)
```

An array, read with the wp_remote_retrieve_*() helpers; the body is a string you decode and check.

3. **Three ways to fail (6m).** "The supplier will have bad days. Provoke all three." The lesson's `supplier-failures.php`:

```
Wrong key:  response 401 {"error":"invalid_api_key","message":"Send a valid AP
Down:       503, Retry-After 120, JSON: NULL
No host:    WP_Error http_request_failed: cURL error 6: Could not resolve host
```

A 401 and a 503 are normal responses, not WP_Errors; the 503 is HTML, so json_decode() gives NULL; only the missing host is a WP_Error. Then the slow cousin (`supplier-slow.php`, a 2-second timeout against a 4-second answer): `cURL error 28: Operation timed out after 2066 milliseconds` (your milliseconds differ) and 1.9 seconds lost for nothing. Pages read caches; jobs call.

4. **Transients (6m).** "A sticky note with an expiry date." With WP-CLI, see where it lives:

```
wp transient set pagesco_books_demo 'Gatsby: back November 15' 600
wp transient get pagesco_books_demo
wp db query "SELECT option_name, autoload FROM wp_options WHERE option_name LI
wp transient delete pagesco_books_demo
wp transient get pagesco_books_demo
```

```
Success: Transient added.
Gatsby: back November 15
option_name autoload
_transient_timeout_pagesco_books_demo   off
_transient_pagesco_books_demo   off

Success: Transient deleted.
Warning: Transient with key "pagesco_books_demo" is not set.
```

Two rows in wp_options, not autoloaded; the last command warns on purpose. No persistent object cache here (`wp_using_ext_object_cache()` prints NULL), so transients live in the database; with

Redis they would live in memory and could vanish early. Then the read-through: the lesson's `cache-demo.php` makes one request for three views:

```
View 1: next delivery 2027-11-15, HTTP requests so far: 1
View 2: next delivery 2027-11-15, HTTP requests so far: 1
View 3: next delivery 2027-11-15, HTTP requests so far: 1
```

"Cache failures too, briefly, and never store false."

5 **WP-Cron (6m).** "The closing checklist, worked through by the next visitor." Core's events, sorted by recurrence (WP-CLI lists them by next run, which changes):

```
wp cron event list --fields=recurrence,hook --format=csv | sed 1d | sort
wp cron schedule list
wp cron test
```

```
"1 day",delete_expired_transients
"1 day",recovery_mode_clean_expired_keys
"1 day",wp_privacy_personal_data_cleanup_requests
"1 day",wp_scheduled_auto_draft_delete
"1 day",wp_scheduled_delete
"1 hour",wp_privacy_delete_old_export_files
"1 week",wp_delete_temp_updater_backups
"1 week",wp_site_health_scheduled_check
"12 hours",wp_update_plugins
"12 hours",wp_update_themes
"12 hours",wp_update_user_counts
"12 hours",wp_version_check
+------------+-------------+----------+
| name       | display     | interval |
+------------+-------------+----------+
| hourly     | Once Hourly | 3600     |
| twicedaily | Twice Daily | 43200    |
| daily      | Once Daily  | 86400    |
| weekly     | Once Weekly | 604800   |
+------------+-------------+----------+
Success: WP-Cron spawning is working as expected.
```

Three rules on the board: **times are earliest times · no visitors, no jobs (system cron in 8.4) · jobs run without a user**. And the warning: `wp cron event run` moves a recurring event; fire the hook with `do_action()` instead.

6 **The Abilities API, in one look (3m).** "A third way to open the shop to programs, for assistants." Core's three:

```
wp eval 'foreach ( wp_get_abilities() as $ability ) { echo $ability->get_name(
```

```
core/get-site-info: Get Site Information
core/get-user-info: Get User Information
core/get-environment-info: Get Environment Info
```

The project adds pagesco/check-stock (Step 4, at home).

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Trusting the answer:** only is_wp_error() checked, then `$data['items']` on a maintenance page: two Warnings and a confident "0 copies". Coach: "Which of the three failures does this line handle?" (The guided lab, the debug handout.)
- **Scheduling on every request:** four daily events after three commands. Coach: "Where does a plugin do things exactly once?" Activation, guarded by wp_next_scheduled(). (The cron lab.)
- **A cache that never hits:** storing false, or checking `if ( $cached )` for a value that can be 0. Coach: "What does get_transient() return when nothing is there?" (Self-check Q8, the debug handout.)

GUIDED LAB

File: `labs/module-14/l7_3-external-apis-cron/remote_answers_starter.php`; your reference is `labs/module-14/l7_3-external-apis-cron/remote_answers_solution.php`. One careful client function, `pagesco_lab_ask()`, against five answers: the shop's own pagesco/v1 (Lesson 7.2) called over HTTP like a stranger's API (data, a 400, a 404), a web page that is not JSON, and a host that does not exist. It is the lesson's Client::decode() written by the learner, on an API they know inside out.

- **Run it as shipped (2m):** five lines of `pagesco_todo`.
- **TODOs 1 to 4 (14m).** Circulate for the status check written as `!== 200` (fine here) and for the message taken from the JSON only (the HTML page has none: fall back to wp_remote_retrieve_response_message()).
- **Expected when done:**

```
genre=fantasy     2 books: The Hobbit, Lord of the Rings
genre=romance     pagesco_remote_http_error (400): Invalid parameter(s): genre
books/99          pagesco_remote_http_error (404): No book was found with that ID
a web page        pagesco_remote_bad_response: The answer is not JSON.
shop.invalid      http_request_failed: cURL error 6: Could not resolve host: shop
```

- **Debrief (4m):** "Which of the five is a WP_Error before your code touches it?" (Only shop.invalid.) "Which would a page view wait longest for?" (A timeout: 3 seconds here.)

- **Fast finishers:** `labs/module-14/l7_3-external-apis-cron/events_cache_starter.php` (a read-through transient that also caches a failure), `labs/module-14/l7_3-external-apis-cron/cron_plan_starter.sh`, then `labs/module-14/l7_3-external-apis-cron/debug_event_counts.php`.

START THE LESSON PROJECT

The lesson project is **The Supplier Sync**: ISBNs, a client, a cached Supplier column, a nightly job and an ability. In the room: Steps 1 and 2 (20 minutes).

- Step 1: `pagesco-isbns.php` gives the eleven books their ISBNs (run once, then delete it).
- Step 2: paste `src/Supplier/Client.php` and `src/Supplier/Availability.php`, register Availability, ask the supplier once (the lesson's `refresh()` command), then the Books screen as Maria. The Supplier cells, in title order:

```
31 copies
22 copies
7 copies
24 copies
15 copies
3 copies
18 copies
5 copies
None; next delivery November 15, 2027
9 copies
12 copies
```

- **Done in the room** = Gatsby says "None; next delivery November 15, 2027" and Catcher "5 copies".
- **Goes home:** Step 3 (DailySync, Activator, deactivate and activate, the past event removed) and Step 4 (the check-stock ability over REST, Maria yes, Devon no).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- A 503 from the supplier: WP_Error or array? **Answer:** an array; only no answer at all is a WP_Error (lesson quiz Q1).
- Where does a transient live on this site, and on one with Redis? **Answer:** two non-autoloaded rows in wp_options; in the object cache, where it may be evicted early.
- A job due at 3:00 on a shop without night visitors: when does it run? **Answer:** with the first request after 3:00.
- Why does the cron callback check no capability? **Answer:** cron runs without a user, and nobody can trigger it with input of their own.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** D) An empty string; **Q8** `bool(false)`; **Q9** A) `wp_safe_remote_get()`: it refuses private networks and odd ports; **Q10** B) `wp eval 'do_action( "pagesco_books_daily_sync" );'`; **Q11** C) In wp-config.php (or an environment variable it reads), on each site; **Q12** `| pagesco_books_daily_sync | 1 day |`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a card with the three checks in order (is_wp_error, the status, the shape) and the three transient functions. In the lab, TODO 2 and 3 on the projector.

Stretch: `events_cache_starter.php`, `cron_plan_starter.sh`, `debug_event_counts.php`; the lesson's Exercise 2 (invalidate the note when an ISBN changes) and Exercise 3 (a six-hour recurrence); the Bonus Challenge (a Reorder row action).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Learners answer in the workbook's Exit Ticket box or on a sticky note.

1. Name the four desks a REST request passes, in order. **Answer:** validate, sanitize, permission, callback.
2. Name the three ways a call to another server can fail. **Answer:** no answer (WP_Error: DNS, refused, timeout); an error status (a normal response); a body that is not the data you expect.
3. Where does a plugin schedule its daily job, and what guards it? **Answer:** in the activation callback, guarded by wp_next_scheduled(); cleared on deactivation.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 7.2: Steps 3 and 4 (event 7 through the API). 7.3: Steps 3 and 4 (the daily sync, the check-stock ability) and the checks.
- **Lesson exercises.** 7.2: Exercises 1 and 2 (rest_do_request, the schema check); Exercise 3 is a stretch. 7.3: Exercises 1 to 3 (an order through the client, invalidation, a six-hour recurrence); Exercise 4 needs the internet.
- **Both Quick Quizzes** on the lesson pages (answers are in the Answer Key).
- **Journal:** both entries (an endpoint the shop does not have yet; a real service for a client's site).
- **Before the Next Lesson (7.3):** the Supplier column and one daily event; no scratch scripts or must-use plugins left (`ls wordpress wordpress/wp-content/mu-plugins`); Node 22.13 or newer for wp-env in Lesson 8.1 (`node --version`).
- **Optional:** `labs/module-14/l7_2-custom-endpoints/schema_rules_starter.php`, `labs/module-14/l7_3-external-apis-cron/events_cache_starter.php`, `labs/module-14/l7_3-external-`

`apis-cron/cron_plan_starter.sh` and both debug handouts.

BRING LIST FOR SESSION 15

- A site at the end of Lesson 7.3 (twenty features, the sandbox, one daily event) and Node 22.13+ for wp-env.
- The project folder with `labs/`; pagesco-plan.txt, the journal and the workbook.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the four 400s and the four desks; Mistake 1; the three ways to fail; the read-through.
- **Cut first (saves 3m):** the Abilities step of the 7.3 demo (Step 4 at home).
- **Cut second (saves 6m):** the transients CLI step: `events_cache` and the read-through show the same.
- **Cut third (saves 15m):** the 7.2 project start: Steps 1 and 2 go home with 3 and 4.

RUNNING SHORT: WHAT TO ADD

- Lesson 7.2's Exercise 1 live: rest_do_request() for a genre and a missing book.
- Lesson 7.3's Common Mistake 2 live: three wp commands, four events, then wp_clear_scheduled_hook().
- Run `debug_staff_picks_route.php` together; the silent bug (absint switching off the maximum) is the best discussion.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 15: Debugging, Testing and Performance

How to teach Lessons 8.1 and 8.2: debugging with evidence instead of guesses (the debug log, a hook that logs, a backtrace, recovery mode, --skip-plugins), checks that machines run (PHPUnit in wp-env, shown as recorded; PHPCS to zero findings), and performance work that starts with a number: the query report, the object cache, N+1, and the Core Web Vitals of the front page.

2 lessons | 3 hrs session | Lessons 8.1–8.2 | Session 15 of 18 · Part 2

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's ideas) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Type, do not paste. Every command below was run to produce the output printed under it, on WordPress 7.1.3, PHP 8.5, MySQL 9.7 and WP-CLI 2.12: Lesson 8.1's demos on the site as Lesson 7.3 leaves it, Lesson 8.2's on the site as Lesson 8.1 leaves it, which is what your projector site has at that point. Four kinds of output are **recordings quoted from the lessons**, and say so: the PHPUnit runs in wp-env, the SQLite Object Cache, the browser measurements (LCP, CLS, INP) and the PHPCS reports of the project.
- The lessons are the reference. Keep `lesson_26.html` and `lesson_27.html` open in browser tabs. Labs live in `labs/module-15/` (learners keep the folder in their project folder, next to compose.yaml): `_starter` files and `next_event_starter/` are handouts, `_solution` files and `next_event_solution/` are yours.
- Each lesson was written for 150 minutes of work alone (Lesson 8.2: 150); the room has 130 minutes for both. The room does the mental models, the demos, two labs and the first three steps of each project; the tests in wp-env (8.1 Step 4), the browser measurements (8.2 Step 4) and the exercises go home. The Agenda says exactly what.
- Part 2 (Sessions 9 to 18) builds on the site each lesson leaves; a learner who is a lesson behind does the guided labs from their own state and catches up on the projects at home. Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (8.1, 8.2) match the course site.

THE #1 RISK OF SESSION 15: THE TOOLING EATS THE SESSION

Lesson 8.1's project installs 34 Composer packages (PHPUnit, PHPCS, the WordPress Coding Standards) from the internet, and its tests need Node.js 22.13 or newer for wp-env, which downloads images and WordPress on its first start. Lesson 8.2 installs a plugin from WordPress.org. On venue Wi-Fi, all of it at once is half an hour of progress bars. **Before:** ask learners to run `composer update` in the plugin folder at home (Step 1 of Lesson 8.1, with the alias), and to start wp-env once (`npx @wordpress/env start` , then `stop`). **In the room:** the 8.1 project starts with `composer show --direct` on every machine; whoever has no packages yet runs `composer update` during the demo, not during the project. wp-env runs at home only. **Instead of installing in the room:** Query Monitor and the SQLite Object Cache are shown from the lesson's recorded cards; nothing later depends on either.

BEFORE THE SESSION

- ☐ **A week before:** remind learners that Lesson 8.1's Step 1 (`composer update`) and a first `npx @wordpress/env start` need the internet and Node.js 24 (Session 9's Part 2 check); ask them to do both at home and to bring the site as Lesson 7.3 leaves it (the 7.3 project finished, `wp cron event list` shows `pagesco_books_daily_sync`).
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-15` from this kit's folder (every line PASS; 14 results in about two minutes). With your own course project: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-15` (labs/README.md); `labs/module-15/l8_1-debugging-testing/test_next_event.sh` needs the site after Lesson 8.1's Step 2.
- ☐ **Projector machine:** the course site as Lesson 7.3 leaves it, a copy of it (or a database export) as Lesson 8.1 leaves it for the second half (`wp db export ~/s15-after-8-1.sql` after your own run-through of 8.1's project, and its plugin folder), the `wp` and `composer` shortcuts, VS Code on the plugin folder, two terminals (one for commands, one for `tail -f wordpress/wp-content/debug.log`), a browser with the Books screen as Maria, http://localhost:8025 (Mailpit) and Chrome's DevTools, plus `lesson_26.html` and `lesson_27.html` .
- ☐ **Rehearse the crash once:** activating `pagesco-crash` takes the whole site down for WP-CLI; know the `--skip-plugins` line by heart before you do it in front of the room, and delete the two recovery-mode options afterwards so the next run sends a fresh email.
- ☐ **Board, before learners arrive:** "symptom → suspects → evidence"; the debug constants table (local | production); the three piles of PHPCS findings (fix · auto-fix · document); the measuring loop "measure, change one thing, measure again"; and the request with its three places where time goes (server, network, browser).
- ☐ **Print or share:** the workbook (Session 15), sticky notes for help flags and the exit ticket.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`composer update` hangs or fails with a network error	No internet, or a proxy	Pair with a neighbor whose plugin has vendor/ for the lab; run Step 1 at home. Nothing in the room's labs needs the packages except `test_next_event.sh`.
`Cannot create cache directory` in Composer's output	The course's composer container cannot write its cache folder	Harmless (Lesson 8.1's cards show it as `...`).
`phpunit` in wp-env: `Call to undefined method PHPUnit\Framework\TestSuite::empty()`	The container's global PHPUnit 10.5	`vendor/bin/phpunit` (Lesson 8.1, Common Mistake 2).
`npx @wordpress/env start` says the Node.js version is too old	wp-env 12 needs Node.js 22.13 or newer	Node.js 24 as in Lesson 6.1 (Session 9's check); at home.
`npx wp-env` installs something odd	The unscoped package is not WordPress's	Always `npx @wordpress/env`.
Every WP-CLI command dies with the crash plugin's fatal error	WP-CLI loads all active plugins	`wp --skip-plugins=pagesco-crash plugin deactivate pagesco-crash`.
No recovery email the second time	Rate-limited: one per day per error (`recovery_mode_email_last_sent`)	`wp option delete recovery_mode_email_last_sent recovery_keys`, as the lesson does.
The query report's counts are one lower than the lesson's	`can_compress_scripts` is created the first time an administrator opens wp-admin in a browser	Expected (the lesson says so); every comparison still holds.
The Events page still makes a query after the events cache	No persistent object cache: the cache lasts one request	Expected; it pays off with Redis or Memcached (Lesson 8.4).
`Function print_emoji_styles is deprecated` after removing the emoji script	Only two of the three remove_action lines	All three lines of Exercise 1.

SYMPTOM	CAUSE	FIX
`Undefined global variable` in a lab script that uses `global $pagesco_x`	`wp eval-file --use-include` runs the file inside a function: its variables are local	Pass values as arguments, or a static variable in a function (`fake_supplier` does); `global $wpdb;` still works.

SESSION 15 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 7.2 and 7.3, homework check (the 7.3 project and its pagesco-plan.txt note).
0:10	10	**8.1** Frame and mental model	Debugging is detective work: evidence, not guesses. The debug constants, local against production. Unit and integration tests; a fake network.
0:20	20	**8.1** Live demo	"Supplier not reachable" solved from the log, a notice traced to its file, a crash and recovery mode, --skip-plugins, the public debug.log; the recorded PHPUnit run.
0:40	15	**8.1** Guided lab	`labs/module-15/l8_1-debugging-testing/fake_supplier_starter.php`: the supplier client against four fake answers.
0:55	20	**8.1** Start the lesson project	Lint and Test, Steps 1 to 3: development packages, the ruleset, the first report, fix, auto-fix or document, until composer lint exits 0. Step 4 (wp-env and the tests) goes home.
1:15	10	Break	Machines still running composer update keep running. Leave the "three piles" table (fix, auto-fix, document) on the board.
1:25	10	**8.2** Frame and mental model	Measure before you optimize: the server, the network, the browser. The object cache, its persistent version, transients.
1:35	20	**8.2** Live demo	First byte, SAVEQUERIES and a query report, the object cache within a request, N+1 counted, the lazy hero image; the persistent cache and the browser numbers as recorded in the lesson.
1:55	15	**8.2** Guided lab	`labs/module-15/l8_2-performance-caching/query_count_starter.php`: predict, then count, what five everyday calls cost.
2:10	25	**8.2** Start the lesson project	A Faster Shop, Steps 1 to 3: the events cache with last-changed, the settings row, update_postmeta_cache(). Step 4 (the hero image, measured in the browser) and Step 5 go home.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:35	15	**8.1** + **8.2** Debrief	Before and after: lint 58 to 0, queries per page, 12 to 2. What each check proves and what it cannot. The Quality and Performance notes in pagesco-plan.txt.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 16.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 150 + 150 = 300 minutes for a learner working alone, and after the warm-up, the break, the debrief and the wrap-up the room has 130. Each lesson keeps its shape (model, demo, lab, project start); the demos lean on the lessons' recorded cards for what needs the internet or a browser, and each project starts with the steps that are typing and reading (8.1 Steps 1 to 3, 8.2 Steps 1 to 3), not the ones that wait for downloads or a browser.

Goes home (about 2 to 3 hours). Lesson 8.1: Step 4, the test suite in wp-env (twelve tests green, one broken on purpose and fixed), the Quality note in pagesco-plan.txt. Lesson 8.2: Step 4 (the hero image's aspect ratio and preload, CLS and LCP measured in DevTools) and Step 5 (the before-and-after table, SAVEQUERIES and the report removed, `composer lint` 0). Both lessons' Hands-on Exercises, both Quick Quizzes, both journal entries, both "Before the Next Lesson" lists, and the labs not reached.

Protect, in this order: the 8.1 "Supplier not reachable" case (the habit of evidence the whole lesson teaches); the 8.1 project to a clean `composer lint` (every later lesson keeps it at 0); the 8.2 query report and the missing option (the measurement that finds what reading cannot); the 8.2 project Step 1 (the last-changed pattern). **Cut first:** the 8.1 public-log step and the 8.2 first-byte step (both are one curl the learners repeat at home), then the recorded PHPUnit cards (show only the green run), then the 8.2 lazy-image mistake (tell it, with the numbers).

OPENING SEGMENT

0:00 Warm-up and Homework Check — 10 min

TIMING

6m	Five retrieval questions on Lessons 7.2 and 7.3 (workbook, Warm-up), pairs compare
4m	Homework check: the 7.3 project's checkpoint (`wp cron event list`, the Supplier column), the APIs note in pagesco-plan.txt

QUESTIONS AND ANSWERS

1. **W1 (Lesson 7.3).** Name the three ways a call to the supplier can go wrong, and what wp_remote_get() returns for each. **Answer:** No answer at all (a WP_Error: DNS, refused, timeout); an answer that is not a success (an array with a 401 or 503 status); a success whose body is not the JSON you expect. Only the first is a WP_Error.
2. **W2 (Lesson 7.3).** The Supplier column reads a transient. What happens when the transient has vanished? **Answer:** Availability::get() reads through: it calls the supplier, stores the answer again (12 hours, 5 minutes after an error) and returns it. A transient may vanish at any time, so code never relies on it being there.
3. **W3 (Lesson 7.3).** Why does the lesson run the daily job with wp eval 'do_action("pagesco_books_daily_sync");' instead of wp cron event run? **Answer:** wp cron event run on a recurring event reschedules it to now plus the interval: the 03:00 job would move to whenever you ran it.
4. **W4 (Lesson 7.2).** Your REST route returns a WP_Error without a status. What does the client get? **Answer:** A 500. The route's callback should give its WP_Error array('status' => 404) (or the right code); permission callbacks use rest_authorization_required_code().
5. **W5 (Lesson 7.3).** Where do the supplier's address and key live on your site, and why there? **Answer:** In wp-config.php (PAGESCO_BOOKS_SUPPLIER_URL and _KEY, set with wp config set): configuration per environment, never in the plugin's code. Lesson 8.4 moves them to the server's environment.

WATCH-OUTS

- **A learner without the 7.3 state** (no supplier sandbox, no daily job): Lesson 8.1's case cannot be reproduced on their site. Pair them for the demo; their guided lab works anyway (`fake_supplier` fakes the supplier), and the 7.3 project is the first homework item.

- **W2 and W3 come back in today's demos:** the transient holds the evidence of the 8.1 case, and the 8.2 project's invalidation proof runs the daily job with `do_action()`.

LESSON PLANS

8.1 Debugging & Testing

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Configure WP_DEBUG, the debug log and error display for development and production, and keep the log private
- Trace a problem with error_log(), hooks such as http_api_debug and doing_it_wrong_run, and wp_debug_backtrace_summary()
- Recover from a fatal error with recovery mode and WP-CLI's --skip-plugins, and use Query Monitor to look inside a request
- Write PHPUnit integration tests with WP_UnitTestCase, factories and pre_http_request, and run them in a wp-env test environment
- Set up PHPCS with the WordPress Coding Standards for a plugin, fix what it finds, and justify the exceptions with a ruleset or phpcs:ignore

TIMING IN THE ROOM

10m	Frame: the detective (symptom, suspects, evidence); the debug constants; unit and integration tests, and why tests never use the network
20m	Live demo: the supplier case from the log, a backtrace, a crash and recovery mode, --skip-plugins, the public log, the recorded PHPUnit run
15m	Guided lab: `fake_supplier_starter.php`
20m	Start the lesson project: Steps 1 to 3 (composer update, phpcs.xml.dist, fix, auto-fix, document, 0 findings)

MENTAL MODEL TO SAY

"Debugging is detective work. A symptom: the Supplier column says 'not reachable'. Suspects: the network, the address, the key, the code. Evidence rules them out, one by one. The worst habit is changing code until the symptom goes away. The best is making the program tell you what it is doing." Write **symptom → suspects → evidence** on the board.

Then the two professional habits in one sentence each: "When something goes wrong, find out why with evidence: the log, a hook that logs, a backtrace, recovery mode. Before anything goes wrong, let machines check the code: tests that run the plugin inside a real WordPress, and PHPCS, which reads every line for what tests rarely catch."

Then the testing picture: **unit tests** run one function without WordPress (fakes for WordPress), **integration tests** run your code in a freshly installed WordPress with a test database: WP_UnitTestCase, a rollback after each test, factories for data. "And one rule: tests never use the network. pre_http_request answers instead, so a test decides exactly what the supplier says, including its bad days."

◆ LIVE DEMO (YOU TYPE, THEY FOLLOW)

In the WP-CLI container shell (`docker compose run --rm wpcli sh`) and the browser, on the site as Lesson 7.3 leaves it. The outputs below come from that site; on yours the time stamps differ and everything else matches.

1. **The debug constants (2m).** "Since Lesson 1.2 your site runs with debugging on. Here is all of it, from the WP-CLI container."

```
wp eval 'foreach ( array( "WP_DEBUG", "WP_DEBUG_LOG", "WP_DEBUG_DISPLAY", "SCR
```

```
WP_DEBUG          true
WP_DEBUG_LOG      true
WP_DEBUG_DISPLAY  true
SCRIPT_DEBUG      true
environment       local
error_log         /var/www/html/wp-content/debug.log
```

   Put the lesson's local-against-production table on the board next to it: display on here, never on a live site; the log here, outside the web root there (Lesson 8.4 sets that up).

2. **A case: "Supplier not reachable" (5m).** "A colleague edited wp-config.php. A few minutes later Maria's Books screen says the supplier is not reachable. No guessing: evidence." Break the key by one character, then look as Maria:

```
wp config set PAGESCO_BOOKS_SUPPLIER_KEY 'sandbox-7f3a9d'
wp transient delete pagesco_books_supplier_availability
```

```
Success: Updated the constant 'PAGESCO_BOOKS_SUPPLIER_KEY' in the 'wp-config.p
Success: Transient deleted.
Supplier not reachable
```

   Evidence 1, what the code already recorded (Availability keeps the error in its transient):

```
wp eval 'echo get_transient( "pagesco_books_supplier_availability" )["error"],
```

```
The supplier answered 401: Send a valid API key in the X-Api-Key header.
```

Evidence 2, make the code talk: save the lesson's temporary logger as `wordpress/wp-content/mu-plugins/pagesco-debug-http.php`, delete the cached failure, reload the Books screen, and read the new line of debug.log:

```
<?php
/**
 * Plugin Name: Pages & Co. Debug: HTTP requests (temporary)
 */

add_action(
    'http_api_debug',
    function ( $response, $context, $transport, $args, $url ) {
        $key = $args['headers']['X-Api-Key'] ?? '';
        error_log(
            sprintf(
                'HTTP %s %s, key %s, answer %s',
                $args['method'],
                strtok( $url, '?' ),
                '' === $key ? 'none' : substr( $key, 0, 8 ) . '…',
                is_wp_error( $response ) ? $response->get_error_message() : wp
            )
        );
    },
    10,
    5
);
```

```
[26-Oct-2026 13:22:07 UTC] HTTP GET http://localhost:8080/supplier-sandbox/v1/
```

"Right address, a key is sent, the answer is 401: the key is wrong, not the network or the code. And notice what the logger does NOT write: the whole key." The fix, the logger removed, the column refreshed:

```
wp config get PAGESCO_BOOKS_SUPPLIER_KEY
wp config set PAGESCO_BOOKS_SUPPLIER_KEY 'sandbox-7f3a9c'
rm wp-content/mu-plugins/pagesco-debug-http.php
wp eval 'echo PagesCo\Books\Plugin::instance()->feature( PagesCo\Books\Supplie
```

```
sandbox-7f3a9d
Success: Updated the constant 'PAGESCO_BOOKS_SUPPLIER_KEY' in the 'wp-config.p
no error
```

3 **Who called that? A backtrace (4m).** "A 'called incorrectly' notice names the function, not the code that misused it. On a site with thirty plugins that is the question." Save the lesson's two must-use plugins (the one that asks too early, and the `0-` logger that loads before it), request /about/, read debug.log:

```
<?php
/**
 * Plugin Name: Too Early (a mistake to find)
 */

if ( is_search() ) {
    remove_action( 'wp_head', 'wp_generator' );
}

<?php
/**
 * Plugin Name: Pages & Co. Debug: who did it wrong? (temporary)
 */

add_action(
    'doing_it_wrong_run',
    function ( $function_name ) {
        error_log( "$function_name called incorrectly. Call stack: " . wp_debu
    }
);
```

The new lines of debug.log:

```
[26-Oct-2026 13:22:08 UTC] is_search called incorrectly. Call stack: require(
[26-Oct-2026 13:22:08 UTC] PHP Notice:  Function is_search was called <strong>
```

Read the first line from the left, outermost call first: WordPress loading, then `include_once('/mu-plugins/pagesco-too-early.php')`. That is the file. Then `rm wp-content/mu-plugins/pagesco-too-early.php wp-content/mu-plugins/0-pagesco-debug-backtrace.php`.

4 **When the site goes down (5m).** "A plugin that crashes on every request, but only on init, so activation succeeds." Save `wordpress/wp-content/plugins/pagesco-crash/pagesco-crash.php` from the lesson, activate it, open the login page (a 'protected' page, where WordPress sends the recovery email):

```
wp plugin activate pagesco-crash
```

```
Plugin 'pagesco-crash' activated.
Success: Activated 1 of 1 plugins.
```

The login page (WP_DEBUG_DISPLAY shows PHP's own message; a live site shows only "There has been a critical error" with status 500):

```
Fatal error: Uncaught Error: Call to undefined function pagesco_crash_missing_
Stack trace:
#0 /var/www/html/wp-includes/class-wp-hook.php(353): pagesco_crash_test('')
#1 /var/www/html/wp-includes/class-wp-hook.php(377): WP_Hook->apply_filters(NU
#2 /var/www/html/wp-includes/plugin.php(523): WP_Hook->do_action(Array)
#3 /var/www/html/wp-settings.php(779): do_action('init')
#4 /var/www/html/wp-config.php(142): require_once('/var/www/html/w...')
#5 /var/www/html/wp-load.php(50): require_once('/var/www/html/w...')
#6 /var/www/html/wp-login.php(12): require('/var/www/html/w...')
#7 {main}
  thrown in /var/www/html/wp-content/plugins/pagesco-crash/pagesco-crash.php o
```

Mailpit, the newest message (the `...` lines are the recovery link, different every time, and the PHP version):

```
From: WordPress <wordpress@example.com>
To: wordpress@example.com
Subject: [Pages & Co.] Your Site is Experiencing a Technical Issue

Howdy!

WordPress has a built-in feature that detects when a plugin or theme causes a

In this case, WordPress caught an error with one of your plugins, Pages & Co.

First, visit your website (http://localhost:8080/) and check for any visible :

Please contact your host for assistance with investigating this issue further.

If your site appears broken and you can't access your dashboard normally, Wor

...

To keep your site safe, this link will expire in 1 day. Don't worry about that

When seeking help with this issue, you may be asked for some of the following
WordPress version 7.1.3
Active theme: Pages &amp;amp; Co. (version 1.0.0)
Current plugin: Pages & Co. Crash Test (version )
...

Error Details
=============
An error of type E_ERROR was caused in line 9 of the file /var/www/html/wp-cor
Stack trace:
#0 /var/www/html/wp-includes/class-wp-hook.php(353): pagesco_crash_test('')
#1 /var/www/html/wp-includes/class-wp-hook.php(377): WP_Hook->apply_filters(NU
#2 /var/www/html/wp-includes/plugin.php(523): WP_Hook->do_action(Array)
#3 /var/www/html/wp-settings.php(779): do_action('init')
#4 /var/www/html/wp-config.php(142): require_once('/var/www/html/w...')
#5 /var/www/html/wp-load.php(50): require_once('/var/www/html/w...')
#6 /var/www/html/wp-login.php(12): require('/var/www/html/w...')
#7 {main}
  thrown
```

"Now fix it from the command line." It does not work, because WP-CLI loads the same plugins (the stack is trimmed here):

```
wp plugin deactivate pagesco-crash
```

```
Fatal error: Uncaught Error: Call to undefined function pagesco_crash_missing_
Stack trace:
#0 /var/www/html/wp-includes/class-wp-hook.php(353): pagesco_crash_test('')
#1 /var/www/html/wp-includes/class-wp-hook.php(377): WP_Hook->apply_filters(NU
#2 /var/www/html/wp-includes/plugin.php(523): WP_Hook->do_action(Array)
...
  thrown in /var/www/html/wp-content/plugins/pagesco-crash/pagesco-crash.php (
Error: There has been a critical error on this website.Learn more about troubl
```

`--skip-plugins` loads WordPress without it:

```
wp --skip-plugins=pagesco-crash plugin deactivate pagesco-crash
wp plugin delete pagesco-crash
wp option delete recovery_mode_email_last_sent recovery_keys
```

```
Plugin 'pagesco-crash' deactivated.
Success: Deactivated 1 of 1 plugins.
Deleted 'pagesco-crash' plugin.
Success: Deleted 1 of 1 plugins.
Success: Deleted 'recovery_mode_email_last_sent' option.
Success: Deleted 'recovery_keys' option.
```

5 **Mistake 1: a debug log anyone can download (1m).** "Ask for the log like a stranger would."

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/wp-content/debu
curl -s http://localhost:8080/wp-content/debug.log | grep -c 'PHP '
```

```
200
27
```

200, and the second number is how many PHP messages a stranger can read (yours differs). Locally it does not matter; on a server, the log goes outside the web root (Lesson 8.4).

6 **The tests, as recorded (3m).** "The lesson runs its twelve tests in a wp-env test environment on port 8889: a WordPress of its own, because the test library reinstalls its database on every run." Show the lesson's recorded run on the projector (wp-env needs Node.js and a minute to start; learners do it at home in Step 4):

```
npx @wordpress/env run cli --env-cwd=wp-content/plugins/pagesco-books vendor/b
```

```
ℹ Starting 'vendor/bin/phpunit' on the cli container.

Installing...
Running as single site... To run multisite, use -c tests/phpunit/multisite.xml
Not running ajax tests. To execute these, use --group ajax.
Not running ms-files tests. To execute these, use --group ms-files.
Not running external-http tests. To execute these, use --group external-http.
PHPUnit 9.6.38 by Sebastian Bergmann and contributors.

............                                                      12 / 12 (100

Time: 00:00.095, Memory: 46.50 MB

OK (12 tests, 30 assertions)
✔ Ran `vendor/bin/phpunit` in 'cli'. (in 2s 20ms)
```

Then the red run after the one-character slip in Client.php (`$code > 599`): the test names the exact code it expected. "A test that only asked 'is it an error?' would have passed."

```
PHPUnit 9.6.38 by Sebastian Bergmann and contributors.

.........F..                                                      12 / 12 (100

Time: 00:00.084, Memory: 46.50 MB

There was 1 failure:

1) PagesCo\Books\Tests\SupplierClientTest::test_turns_an_error_status_into_a_w
Failed asserting that two strings are identical.
--- Expected
+++ Actual
@@ @@
-'pagesco_supplier_http_error'
+'pagesco_supplier_bad_response'

/var/www/html/wp-content/plugins/pagesco-books/tests/SupplierClientTest.php:85

FAILURES!
Tests: 12, Assertions: 29, Failures: 1.
✖ Command failed with exit code 1
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Changing code until the symptom disappears.** A learner edits Client.php to "fix" the 401. Coach: "What is your evidence? What did the transient say, what did the log say?" The answer was in wp-config.php all along. (Self-check Q2, the debug handout.)

- **Logging secrets.** The quick logger writes the whole API key, or the whole request. Coach: "Who reads this log, and for how long does it live?" Eight characters are enough to compare. (The debug handout's silent bug.)
- **Making PHPCS quiet instead of making the code right.** A plugin-wide exclude, or `// phpcs:ignore` without a rule and a reason. Coach with the three piles on the board: fix, auto-fix, document; every exception names its rule and says why. (Self-check Q5, the lesson quiz Q4.)

GUIDED LAB

File: learners copy `labs/module-15/l8_1-debugging-testing/fake_supplier_starter.php` into wordpress/ and run it with `wp eval-file --use-include`; your reference is `labs/module-15/l8_1-debugging-testing/fake_supplier_solution.php`. It practices the lesson's central testing idea on the learner's own site, without wp-env: each check fakes one answer of the supplier with `pre_http_request` and asserts the exact error code the client returns.

- **Run it as shipped (2m).** `cp labs/module-15/l8_1-debugging-testing/fake_supplier_starter.php wordpress/` then `wp eval-file --use-include fake_supplier_starter.php`: check 1 passes, checks 2 to 4 print TODO.
- **Checks 2 to 4 in pairs (10m).** Each is one `pagesco_lab_fake()` call (the answer) and one `pagesco_lab_check()` (the condition). Circulate for `get_error_data()['status']` compared with `'401'` (a string: `===` fails) and for check 4, where the fake returns a WP_Error instead of an array.
- **Expected when done:**

```
PASS a good answer is keyed by ISBN
PASS a 401 becomes pagesco_supplier_http_error with the status in its data
PASS a 200 that is not JSON becomes pagesco_supplier_bad_response
PASS a network failure comes back unchanged
4 of 4 checks passed, 4 requests answered by the fake, none by the network
```

- **Debrief (3m):** "Why does the last line say none by the network?" (The filter answered every request; the sandbox was never called: the test would pass on a train.) "Which check would have caught the `$code > 599` slip of the lesson's red run?" (Check 2.)
- **Fast finishers:** `labs/module-15/l8_1-debugging-testing/debug_supplier_log.php` (two Warnings and a silent bug in what it logs), then the folder lab `labs/module-15/l8_1-debugging-testing/next_event_starter/` with `labs/module-15/l8_1-debugging-testing/test_next_event.sh` once their project has the packages (a must-use plugin to zero PHPCS findings).

START THE LESSON PROJECT

The lesson project is **Lint and Test the Plugin**. In the room: Steps 1 to 3 (20 minutes), the coding standards to zero findings; the test suite (Step 4) needs wp-env and goes home.

- Step 1: the new composer.json, the `composer` alias, `composer update` (or `composer show --direct` for learners who ran it at home), three packages listed.
- Step 2: `phpcs.xml.dist` from the lesson, then `composer lint -- --report=summary`. The recorded report (yours matches it, file for file):

```
Composer could not detect the root package (pagesco/pagesco-books) version, defau

PHP CODE SNIFFER REPORT SUMMARY
----------------------------------------------------------------------
FILE                                                  ERRORS  WARNINGS
----------------------------------------------------------------------
uninstall.php                                         0       3
src/EventsTable.php                                   1       5
src/Plugin.php                                        0       1
src/blocks/book-filter/render.php                     18      3
src/blocks/book-list/render.php                       13      2
src/blocks/book-list/variations.php                   4       0
src/Rest/BooksEndpoint.php                            3       3
src/Rest/EventsEndpoint.php                           2       0
----------------------------------------------------------------------
A TOTAL OF 41 ERRORS AND 17 WARNINGS WERE FOUND IN 8 FILES
----------------------------------------------------------------------
PHPCBF CAN FIX 6 OF THESE SNIFF VIOLATIONS AUTOMATICALLY
----------------------------------------------------------------------

...

Script phpcs handling the lint event returned with error code 2
```

- Step 3: the decision table, read aloud row by row before anyone edits: `composer lint:fix` (6 fixed), the parameter rename, the phpcs:ignore comments (the lesson's sed commands, or by hand), `npm run build`, the full ruleset, then `composer lint; echo "exit status $?"` until it says 0.
- **Done in the room** = `exit status 0` and the Find a Book count line still `Showing 2 of 11 books`. **Goes home:** Step 4 (.wp-env.json, phpunit.xml.dist, the bootstrap, four test classes, the green run, the red run on purpose) and the Quality note.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- The Books screen says "Supplier not reachable". Where do you look first? **Answer:** at what the code already recorded: the transient's `error` (the 401 message), then make the code talk with an

`http_api_debug` logger.

- Which page triggers the recovery email: the home page or wp-login.php? **Answer:** a protected page such as wp-login.php or wp-admin; and at most once a day per error.
- Why does a test fake the supplier instead of calling the sandbox? **Answer:** so the test controls the answer, bad days included, and passes or fails because of your code only (lesson quiz Q3).
- PHPCS flags `echo get_block_wrapper_attributes();`. Wrap it in esc_html()? **Answer:** no: core escapes it; a `phpcs:ignore` for that rule on that line, with the reason (lesson quiz Q4).

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** B) A file outside the web root, such as `/home/pagesco/logs/wordpress.log`; **Q2** `pagesco_supplier_http_error 503`; **Q3** D) `wp --skip-plugins=pagesco-crash plugin deactivate pagesco-crash`; **Q4** A) Must-use plugins load in alphabetical order: it must be there before the plugin that asks too early; **Q5** C) Excluded that rule for those two file patterns in phpcs.xml.dist, with a comment: WordPress includes them inside a function, so the variables are local; **Q6** `npx @wordpress/env run cli --env-cwd=wp-content/plugins/pagesco-books vendor/bin/phpunit`: the plugin's own PHPUnit 9.6; the container's global phpunit is 10.5, which the WordPress 7.1 test library does not support. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners new to testing do the guided lab with the lesson's SupplierClientTest open next to it: each `pagesco_lab_check()` is one `assertSame()`. For PHPCS, hand out the decision table as the only reference: every finding goes into one of three piles.

Stretch: `debug_supplier_log.php` and the PHPCS folder lab; the lesson's Exercise 1 (SettingsTest) and Exercise 3 (what PHPCS sees in insecure code); the Bonus Challenge (PHPCompatibilityWP and a render_block() test).

8.2 Performance & Caching

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Measure what a request costs with SAVEQUERIES and $wpdb->queries, read an EXPLAIN of a meta query, and recognize missing options and N+1 queries
- Explain the object cache, its persistent version and transients, choose between them, and cache your own query with a key that is invalidated when the data changes
- Describe how WordPress 7.1 loads block CSS, script modules and classic scripts, and use wp_preload_resources and fetchpriority where they help
- Explain srcset, sizes, lazy loading and why block templates get no automatic image optimization
- Measure LCP, CLS and interaction timing in a browser, and say honestly what a local lab measurement can and cannot tell you

TIMING IN THE ROOM

10m	Frame: the line at the register; three places where time goes; object cache, persistent cache, transients
20m	Live demo: first byte, SAVEQUERIES and the query report, the object cache, N+1, the lazy hero image, the recorded numbers
15m	Guided lab: `query_count_starter.php`
25m	Start the lesson project: Steps 1 to 3 (the events cache, the settings row, update_postmeta_cache)

MENTAL MODEL TO SAY

"A busy Saturday at Pages & Co., a long line at the register. You could hire a second cashier, buy a faster card reader or move the gift wrapping. A good manager first stands next to the register for ten minutes with a notepad: if the time goes into wrapping, the card reader money is wasted." Then the rule on the board: **measure, change one thing, measure again**.

Then the three places, each with its tool: "The **server** builds the HTML (SAVEQUERIES, the query report). The **network** carries it and every file it names (View Source, the Network panel). The **browser** turns it into pixels (Core Web Vitals: LCP, INP, CLS)."

Then the caches, as a table on the board: the **object cache** remembers within one request; a **persistent object cache** (an object-cache.php drop-in: Redis, Memcached) keeps it between requests; a **transient** outlives the request on any site, in wp_options or in the persistent cache. "And every cache needs an invalidation plan: a key that knows when the data changed."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

From the project folder and the WP-CLI container, on the site as Lesson 8.1 leaves it. Recorded cards (the persistent cache, the browser numbers) are marked as such.

1. **How long until the first byte? (2m).** "Before any idea, a number." Three pages, the server's share of the time (yours differ, and differ between two runs: run it twice):

```
for page in / /books/dune/ /find-a-book/; do
  curl -s -o /dev/null -w "$page  %{http_code}  %{size_download} bytes  first
done
```

```
/  200  84295 bytes  first byte after 0.398305 s
/books/dune/  200  80250 bytes  first byte after 0.057047 s
/find-a-book/  200  83202 bytes  first byte after 0.053838 s
```

"The first one is slower: PHP compiling WordPress after a restart. Never trust one measurement; change one thing at a time."

2. **What a page costs the database (5m).** "SAVEQUERIES in wp-config.php records every query of the request, with its caller." One get_posts() from WP-CLI:

```
wp config set SAVEQUERIES true --raw
wp eval 'global $wpdb; $start = count( $wpdb->queries ); get_posts( array( "po
```

```
Success: Added the constant 'SAVEQUERIES' to the 'wp-config.php' file with the
SELECT wp_posts.ID FROM wp_posts WHERE 1=1 AND wp_posts.po WP_Query->get_posts
SELECT wp_posts.* FROM wp_posts WHERE ID IN (40,39)        _prime_post_caches
SELECT post_id, meta_key, meta_value FROM wp_postmeta WHER update_meta_cache
SELECT DISTINCT t.term_id, tr.object_id FROM wp_terms AS t WP_Term_Query->get_
SELECT t.*, tt.* FROM wp_terms AS t INNER JOIN wp_term_tax _prime_term_caches
```

Five queries for two books: the IDs, the posts, their meta, their genres (two). Then the lesson's query report, `wordpress/wp-content/mu-plugins/pagesco-query-report.php`, and five addresses as an anonymous visitor:

```
for page in / /books/dune/ /find-a-book/ /events/ "/wp-json/pagesco/v1/books?o
  curl -s -o /dev/null "http://localhost:8080$page"
done
```

```
[26-Oct-2026 13:22:22 UTC] /: 24 queries; missing options: WPLANG, can_compres
[26-Oct-2026 13:22:22 UTC] /books/dune/: 27 queries; missing options: WPLANG,
[26-Oct-2026 13:22:22 UTC] /find-a-book/: 27 queries; missing options: WPLANG,
[26-Oct-2026 13:22:22 UTC] /events/: 24 queries; missing options: WPLANG, can_
[26-Oct-2026 13:22:22 UTC] /wp-json/pagesco/v1/books: 14 queries; missing opt:
```

Point at the missing options: `pagesco_books_settings` costs one query on every page and every REST request, to learn that there is nothing. "No code review finds that; a measurement does. The project removes it."

3. **What WordPress remembers, and for how long (4m).** "Why only two dozen? Because WordPress caches." Two meta values, one query; the same WP_Query twice:

```
wp eval 'global $wpdb; $before = $wpdb->num_queries; get_post_meta( 30, "pages
wp eval 'global $wpdb; $args = array( "post_type" => "book", "meta_key" => "pa
```

```
1 query for two meta values
first query: 6 database queries, the same query again: 0
```

"But in PHP's memory, which ends with the request." Two WP-CLI commands are two requests:

```
wp eval 'wp_cache_set( "note", "Gatsby: back November 15", "pagesco_demo" ); v
wp eval 'var_dump( wp_cache_get( "note", "pagesco_demo" ) );'
```

```
string(24) "Gatsby: back November 15"
bool(false)
```

Then the recorded persistent cache (the lesson's SQLite Object Cache needs the internet; show the card): after `wp cache flush`, round 1 fills the cache, round 2 is the steady state of a busy site:

```
[26-Oct-2026 06:01:59 UTC] /: 23 queries; missing options: WPLANG, can_compres
[26-Oct-2026 06:02:00 UTC] /books/dune/: 11 queries; missing options: WPLANG,
[26-Oct-2026 06:02:00 UTC] /find-a-book/: 10 queries; missing options: WPLANG,
[26-Oct-2026 06:02:00 UTC] /events/: 6 queries; missing options: WPLANG, can_c
[26-Oct-2026 06:02:00 UTC] /wp-json/pagesco/v1/books: 5 queries; missing optio
[26-Oct-2026 06:02:00 UTC] /: 0 queries; missing options: WPLANG, can_compress
[26-Oct-2026 06:02:00 UTC] /books/dune/: 0 queries; missing options: WPLANG, c
[26-Oct-2026 06:02:00 UTC] /find-a-book/: 0 queries; missing options: WPLANG,
[26-Oct-2026 06:02:00 UTC] /events/: 1 queries; missing options: WPLANG, can_c
[26-Oct-2026 06:02:00 UTC] /wp-json/pagesco/v1/books: 0 queries; missing optio
```

4. **Mistake 2: N+1, counted (3m).** "The supplier feature collects the catalog's ISBNs. What does that cost?" Save the lesson's `wordpress/pagesco-count-isbns.php` (it calls the private method

through ReflectionMethod) and run it:

```
wp eval-file pagesco-count-isbns.php
```

```
11 ISBNs, 12 queries:
  1 x SELECT wp_posts.ID FROM wp_posts WHERE N=N AND wp_posts.post_type = 'boo
 11 x SELECT post_id, meta_key, meta_value FROM wp_postmeta WHERE post_id IN (
```

"One query for the IDs, then one per book. Eleven books, eleven; five thousand, five thousand. The fix is two lines, in the project."

5 **Mistake 3: lazy-loading the image that matters most (4m).** "The hero image, as the theme's pattern wrote it:"

```
<img src="http://localhost:8080/wp-content/themes/pagesco/assets/images/book-1
```

No width, no height, no fetchpriority: WordPress skips block templates. Then the lesson's "lazy-load everything" must-use plugin, `mistake-lazy.php`:

```php
<?php
/**
 * Plugin Name: Mistake: Lazy-Load Every Image
 */

add_filter(
    'render_block_core/image',
    function ( $html ) {
        $image = new WP_HTML_Tag_Processor( $html );
        if ( $image->next_tag( 'img' ) ) {
            $image->set_attribute( 'loading', 'lazy' );
        }
        return $image->get_updated_html();
    }
);
```

```
<img loading="lazy" src="http://localhost:8080/wp-content/themes/pagesco/asset
```

In the lesson's lab measurement (Chromium, Slow 4G) LCP went from about 1480 ms to about 2012 ms: "no message tells you". Then `rm pagesco-count-isbns.php wp-content/mu-plugins/mistake-lazy.php wp-content/mu-plugins/pagesco-query-report.php; wp config delete SAVEQUERIES`.

6. **Core Web Vitals, honestly (2m).** Show the lesson's console snippet and its recorded numbers: LCP about 1480 ms with the hero image as the LCP element, CLS 0.101 (over 0.1), interactions on the Book Filter 16 ms. Say: "Lab numbers compare a change on one machine. Field data from real visitors, after the launch, decides the rest."

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Optimizing without a number.** "I added a cache, it feels faster." Coach: "What was the count before, and after? Same page, same machine, twice." (The guided lab, self-check Q7.)
- **A cache nobody clears.** A learner caches the shortcode's HTML for a day. Coach: "Add an event. When does the page show it?" Then the last-changed pattern: the key changes when the data does. (Lesson Mistake 1, self-check Q10, the debug handout.)
- **Trusting a warm cache.** N+1 disappears behind a persistent cache and comes back after `wp cache flush`. Coach: "Measure with an empty cache." (Self-check Q9.)

GUIDED LAB

File: `labs/module-15/l8_2-performance-caching/query_count_starter.php`; your reference is `labs/module-15/l8_2-performance-caching/query_count_solution.php`. It practices the lesson's measuring habit on five calls learners already write every day: each row runs a call twice and prints its queries, so the object cache becomes visible. The rule of the lab: **write the prediction in the comment before you run.**

- **Run it as shipped (2m).** `cp labs/module-15/l8_2-performance-caching/query_count_starter.php wordpress/`, `wp eval-file --use-include query_count_starter.php`: row 1 (an autoloaded option: 0 and 0), then TODO.
- **Rows 2 to 6 in pairs (10m).** Predict, then write the `pagesco_lab_row()` call. Circulate for row 5: `pagesco_lab_forget_meta()` first, or row 4's WP_Query has already cached some of the meta and the count is lower.
- **Expected when done:**

```
call                                                                first
get_option( 'blogname' )                                                0
get_option( 'pagesco_lab_missing' )                                     1
get_post_meta( 33, 'pagesco_price' ) and 'pagesco_author'               1
WP_Query: the three cheapest books                                      6
get_post_meta() over 11 books, one by one                              11
get_post_meta() over 11 books, after update_postmeta_cache()            1
```

- **Debrief (3m):** "Which predictions were wrong?" (Usually row 2: a missing option costs one query, then none; and row 4's six.) "Which row is the supplier feature's lookup before the project, and which after?" (Rows 5 and 6.)

- **Fast finishers:** `labs/module-15/l8_2-performance-caching/cheapest_cache_starter.php` (the last-changed pattern on a query of their own), `labs/module-15/l8_2-performance-caching/page_weight_starter.sh` (what the pages download), then `labs/module-15/l8_2-performance-caching/debug_events_cache.php`.

START THE LESSON PROJECT

The lesson project is **A Faster Shop, with Numbers**: four improvements, each measured. In the room: Steps 1 to 3 (25 minutes); the hero image (Step 4) needs a browser with DevTools and time, and goes home with Step 5.

- Before Step 1: the query report and SAVEQUERIES from the demo stay on (the lesson says so). Learners who skipped the demo create both now.
- Step 1: the new EventsTable.php (type it: the key, `is_array()`, `flush_cache()` in install, add and delete_before), the measurement, and the invalidation proof with the past event and `do_action( "pagesco_books_daily_sync" )`. Learners without the SQLite Object Cache (no internet) skip its rounds: the proof works without it.
- Step 2: the new Activator.php, deactivate and activate, the empty autoloaded row (`a:0:{}`, `auto`), the five addresses again: one query fewer each.
- Step 3: Mistake 2's script, the two lines with sed (or by hand), 12 queries to 2, `refresh()` still finds 11 books.
- **Done in the room** = the settings row exists and `pagesco-count-isbns.php` says 2 queries. **Goes home:** Step 4 (pattern edit, `pagesco_preload_hero_image()`, measured with the console snippet and Slow 4G) and Step 5 (the table, the instruments removed, `composer lint` 0).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- The report lists pagesco_books_settings under missing options on every request. What does it cost? **Answer:** one query per request; an autoloaded row removes it (lesson quiz Q1).
- A value stored with wp_cache_set(), no persistent cache: how long does it live? **Answer:** until the end of the request (lesson quiz Q2, self-check Q8).
- Why the group's last-changed value in the key? **Answer:** every write renews it, so every older cached result becomes unreachable without anyone deleting it (lesson quiz Q3).
- A laptop measurement says LCP 1.4 s. What do you know? **Answer:** a baseline to compare a change against on the same machine; field data decides the rest (lesson quiz Q5).

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** `1`; **Q8** B) `bool(false)`; **Q9** C) 501 queries; the fix is update_postmeta_cache($ids) before the loop: 2 queries; **Q10** B) A cached empty result, array(), would count as a miss: a location without events would

query the database on every call; **Q11** D) The hero image had no reserved space; the Image block's aspectRatio attribute fixed it; **Q12** `wp_preload_resources`, with `'fetchpriority' => 'high'` (and `'as' => 'image'`). Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: a learner who is lost in the numbers gets one question per row: "first time, second time, why the difference?" The table in the lesson's Step 5 is the map; every row of it is one step.

Stretch: `cheapest_cache_starter.php` and `debug_events_cache.php`; the lesson's Exercise 3 (the price query at two thousand books) and the Bonus Challenge (caching the catalog route, and the case its key misses).

DEBRIEF

2:35 Before and After: What the Checks Prove — 15 min

◆ SEQUENCE

- **Two numbers per lesson (5m).** Ask pairs for their before and after: PHPCS 58 findings to 0 (or how far they got); `pagesco-count-isbns.php` 12 queries to 2; the missing option gone. Write them on the board next to the morning's "measure, change one thing, measure again".
- **What each check proves, and what it cannot (5m).** composer lint: the code follows the standard and escapes, prepares, checks nonces where it can see it; not that it works. The tests: the behaviors they name, on a fresh WordPress; not the browser, not the theme. The query report: the server's work; not what the visitor feels. Lab Web Vitals: a comparison on one machine; not the field. "Which of these would have caught the 401 of this morning?" (None: the log did. Debugging and testing are two different habits.)
- **The notes (5m).** Learners write the Quality and Performance headings into pagesco-plan.txt now, with today's numbers. Lesson 9.1's final project asks for exactly this kind of proof ("composer lint clean, the PHPUnit suite green in wp-env, the numbers before and after").

CLOSING

◆ EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note and leave it on the way out. Question 2's pile tells you whether Session 16's warm-up repeats the last-changed pattern.

1. A live site shows "There has been a critical error on this website." Name two places to look, and one command that still works when a plugin is the cause. **Answer:** the server's error log (or debug.log on development), the recovery mode email; `wp --skip-plugins=<plugin> plugin deactivate <plugin>`.
2. Why does EventsTable::find() put `wp_cache_get_last_changed( 'pagesco_events' )` into its cache key? **Answer:** every change to the table renews it (flush_cache()), so after a write every old key is unreachable and no stale list is ever read.
3. Name one thing a lab measurement on your laptop can tell you, and one it cannot. **Answer:** whether a change made the page better or worse on the same machine; not what real visitors on their phones experience (field data, after the launch).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 8.1: Step 4 (the test environment, twelve tests green, the red run on purpose, the fix) and the Quality note; if Steps 1 to 3 did not finish, those first. 8.2: Step 4 (the hero image, measured) and Step 5 (the table, the instruments removed, `composer lint` 0), and the Performance note.
- **Lesson exercises.** 8.1: Exercises 1 and 3 (SettingsTest; what PHPCS sees in insecure code); Exercise 2 is a stretch. 8.2: Exercises 1 and 2 (the emoji script; the inline CSS budget); Exercise 3 is a stretch. Remove the exercise test files afterwards if you want your counts to match the later lessons.
- **Both Quick Quizzes** on the lesson pages (five questions each; answers are in the Answer Key).
- **Journal:** both entries. The 8.1 prompt asks about the last bug you chased; the 8.2 prompt asks you to measure a slow site before you guess.
- **Before the Next Lesson (from 8.2):** `wp config get SAVEQUERIES` says it is not defined, `wp plugin list --status=dropin` is empty, and the twelve tests pass once more in wp-env.
- **Optional:** the labs not reached in class: `labs/module-15/l8_1-debugging-testing/debug_supplier_log.php`, the PHPCS folder lab `next_event_starter/`, `labs/module-15/l8_2-performance-caching/cheapest_cache_starter.php`, `labs/module-15/l8_2-performance-caching/page_weight_starter.sh` (run it before and after your Step 4) and `labs/module-15/l8_2-performance-caching/debug_events_cache.php`.

BRING LIST FOR SESSION 16

- The site as Lesson 8.2 leaves it: `composer lint` exits 0, the events cache, the settings row, the hero image with its aspect ratio and preload, and no SAVEQUERIES or query report left.
- An internet connection at home before the session: Lesson 8.3 installs the Spanish core translation (`wp language core install es_ES`) and the Plugin Check plugin, and Lesson 8.4 downloads the rehearsal server `pagesco-live.zip` from the course site. Download the ZIP at home.
- Git on your computer (on Windows inside WSL: `git --version`), Node.js 24 and the node:24 image (Lesson 6.1) for `npm ci` in Lesson 8.4.

- pagesco-plan.txt with the Quality and Performance notes, the journal, and the workbook.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the 8.1 supplier case; the 8.1 project to `exit status 0`; the 8.2 query report with the missing option; the 8.2 project Step 1.
- **Cut first (saves 3m):** the 8.1 public-log step and the 8.2 first-byte step: one curl each, at home.
- **Cut second (saves 3m):** the recorded red PHPUnit run (show it in the debrief instead, or leave it to Step 4 at home).
- **Cut third (saves 4m):** the 8.2 lazy-image mistake: tell it with the lesson's numbers instead of typing it.
- **Cut fourth (saves 10m):** the debrief shrinks to the notes in pagesco-plan.txt.

RUNNING SHORT: WHAT TO ADD

- The 8.1 Exercise 3 live: `scratch-search.php` linted with the project's ruleset, every finding sorted into the three piles by the room.
- The 8.2 EXPLAIN of the price query, read from the innermost line out, and the question "what grows with the catalog?"
- Run `page_weight_solution.sh` for everyone before and after a volunteer's Step 4, on the projector.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 16: Distribution and Deployment

How to teach Lessons 8.3 and 8.4 and hand out the final project: translator-friendly strings, accessibility checks that are honest about their limits and a release ZIP that ships only what runs; then the shop's code in Git, release builds from a tag, a migration previewed and the production configuration, rehearsed on a second WordPress; and the Reserve & Pick Up brief, milestones and rubric.

`2 lessons` `3 hrs session` `Lessons 8.3–8.4` `Session 16 of 18 · Part 2`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame → live demo → guided lab → start the lesson's own project; exercises, quiz and journal are homework. Then a third block: **the final project's brief**, handed out today so that learners arrive at Session 17 with a plan.
- Type, do not paste. Every command below was run to produce the output under it on WordPress 7.1.3, PHP 8.5, MySQL 9.7 and WP-CLI 2.12: Lesson 8.3's demos on the site as Lesson 8.2 leaves it, Lesson 8.4's on the site as Lesson 8.3 leaves it. **Recordings quoted from the lessons** say so: Plugin Check and the Spanish front end (they need the internet), the keyboard walk and axe audits (a browser), and everything on the pagesco-live rehearsal server.
- The lessons are the reference: keep `lesson_28.html`, `lesson_29.html` and `lesson_30.html` open. Labs live in `labs/module-16/`: `_starter` files are handouts, `_solution` files are yours. Each lesson was written for 150 minutes of work alone (Lesson 8.4: 150); the room has 120 for both, plus 25 for the brief.
- Git, the release builds and the rehearsal server run on the learner's own computer (the WP-CLI container has no git); the kit's labs practice the server-side half (WP-CLI, `wp db`, cron) on the development site.

> **THE #1 RISK OF SESSION 16: THE LAUNCH NEEDS THE EARLIER RELEASE**
>
> Lesson 8.4's project commits and tags "Pages & Co. Books 1.1.0": it needs Lesson 8.3's release finished (Steps 3 to 5: the Spanish files, the accessibility fixes, the readme and the ZIP), which does not fit in the room. **So the room starts Lesson 8.4's project at Step 3** (the rehearsal server: download, .env, `docker compose up -d`), which needs nothing from 8.3; Steps 1 and 2 go home after 8.3's Step 5. **Before:** learners download pagesco-live.zip at home (Session 15's bring list) and check `git --version`. **In the room:** nobody types `git commit` before `wp plugin list` shows pagesco-books 1.1.0.

BEFORE THE SESSION

- ☐ **A week before:** remind learners of the bring list (the site as Lesson 8.2 leaves it, pagesco-live.zip downloaded, Git, Node.js 24, internet at home for es_ES and Plugin Check).
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-16` (every line PASS; 14 results in about two minutes); with your project, `WP_LABS_COMPOSE=~/pagesco-dev ...` (labs/README.md). Lesson 8.4's labs need the site after Lesson 8.3.
- ☐ **Projector machine:** the site as Lesson 8.2 leaves it, and a copy as Lesson 8.3 leaves it (a database export and the plugin folder) for the second half; `pagesco-live` unzipped next to `pagesco-dev`, its .env made and its images pulled (`docker compose pull` in its folder) so that `up -d` takes seconds; Git configured; `lesson_30.html` printed or on screen for the brief.
- ☐ **Print:** the workbook (Session 16), and one copy of Lesson 9.1's brief, milestones table, security checklist and rubric per pair (or the final project part of `final-assessment.html`).
- ☐ **Board:** the publisher's three jobs (i18n · l10n · a11y); "code goes up · content comes down · configuration stays"; the release recipe (tag → git archive → composer install --no-dev → npm ci, build, plugin-zip).

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`wp language core install es_ES` or `wp plugin install plugin-check` fails	No internet	Show the lesson's recorded cards; do it at home. The labs need neither.
make-pot warns but "Success"	Warnings do not fail the command	Read them; Mistake 1.
The Spanish JSON strings vanish from the PO	make-json without --no-purge	`wp i18n make-json ... --no-purge`; rebuild from the backup PO.
`Failed opening required ... deep_copy.php` after installing the ZIP	The development autoloader	`composer dump-autoload --no-dev` before plugin-zip, then `composer dump-autoload`.
`git: command not found` in `docker compose run --rm wpcli sh`	Git runs on the computer, not in the WP-CLI container	A terminal on the computer in `wordpress/wp-content` (Windows: inside WSL).
`docker compose up -d` in pagesco-live: port 8081 already allocated	Something else on 8081	Stop it; the lesson's addresses assume 8081.

SYMPTOM	CAUSE	FIX
`docker compose restart wordpress` breaks wpcli and cron in pagesco-live	network_mode: service:wordpress	`docker compose up -d` instead (the compose file says so).
`wp db export` on pagesco-live: error 1044 LOCK TABLES	Least privilege, on purpose (Step 6)	Grant LOCK TABLES as root, as the lesson does; delete the 831-byte failed file.
`wp db import` prints Access denied lines but Success	A dump with LOCK TABLES, an account without it	Read the whole output; grant the right, import again.

SESSION 16 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 8.1 and 8.2, homework check (composer lint 0, the twelve tests, the Performance table).
0:10	10	**8.3** Frame and mental model	A publisher's three jobs: manuscript (i18n), translations (l10n), editions for every reader (a11y). Distribution as the professional baseline.
0:20	20	**8.3** Live demo	make-pot and a POT entry, the JSON hashes, contrast computed, untranslatable strings caught, the uninstall without its taxonomy, the ZIP with the development autoloader; the recorded audits and Plugin Check.
0:40	15	**8.3** Guided lab	`labs/module-16/l8_3-i18n-a11y-distribution/plural_strings_starter.php`: four strings rewritten until the translator's file translates them.
0:55	15	**8.3** Start the lesson project	Release 1.1.0, Steps 1 and 2: the delete-all opt-in, and the uninstall tested both ways from a backup. Steps 3 to 5 go home.
1:10	10	Break	Learners who have not downloaded pagesco-live.zip start the download now.
1:20	10	**8.4** Frame and mental model	Code goes up, content comes down, configuration stays. Development, staging, production.
1:30	20	**8.4** Live demo	The development configuration, development mode and the pattern cache, the secrets, the .gitignore and Git, a dry run, serialized data, a dump and .git in the web root; the recorded rehearsal.
1:50	15	**8.4** Guided lab	`labs/module-16/l8_4-deployment-maintenance/migration_preview_starter.sh`: a move to a staging

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
			address, previewed without changing anything.
2:05	20	**8.4** Start the lesson project	Launch Day, Step 3: the rehearsal server set up and started (download, .env, docker compose up -d). Steps 1 and 2 (after 8.3's release) and 4 to 7 go home.
2:25	25	Final-project brief (Lesson 9.1)	Reserve & Pick Up handed out: the brief, the hand-in, the ten milestones, the security checklist, the rubric; first plan in pairs; what Session 17 starts with.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, the full bring list for Session 17.
Total	180		Ends at 3:00.

In the room. The lessons estimate 150 + 150 = 300 minutes alone; after the warm-up, the break, the brief and the wrap-up the room has 120. Each lesson keeps its shape; Lesson 8.3's project starts with the two steps that need no internet (the opt-in and its test), Lesson 8.4's with the one step that does not depend on 8.3's release (the rehearsal server).

Goes home (about 2 to 3 hours). Lesson 8.3: Steps 3 to 5 (Spanish, the three accessibility fixes, version 1.1.0 with readme, ZIP and Plugin Check) and the Release note. Then Lesson 8.4: Steps 1 and 2 (commit, tag, build release 1, the launch database), Step 4 (launch), 5 (check), 6 (backup and restore) and 7 (release 2), and the Deployment note. Both lessons' exercises, quizzes, journals, and the final project's planning (the brief's homework below).

Protect, in this order: the 8.4 frame and the Git step ("code goes up, content comes down" is the idea the final project deploys by); the final-project brief (Session 17 starts building); the 8.3 uninstall tested both ways; the 8.4 dry run. **Cut first:** the 8.3 Plugin Check recordings, then the 8.4 pattern-cache step (tell it), then the 8.3 contrast step.

OPENING SEGMENT

0:00 Warm-up and Homework Check — 10 min

TIMING

6m	Five retrieval questions on Lessons 8.1 and 8.2 (workbook, Warm-up), pairs compare
4m	Homework check: `composer lint` 0, the twelve tests green, the Performance table in pagesco-plan.txt

QUESTIONS AND ANSWERS

1. **W1 (Lesson 8.1).** On a live site, where do PHP's messages go, and why not to wp-content/debug.log? **Answer:** To a log outside the web root (or the server's log), display always off: wp-content/debug.log is a public download (200 for anyone).
2. **W2 (Lesson 8.1).** Which command runs the plugin's tests, and in which site? **Answer:** npx @wordpress/env run cli --env-cwd=wp-content/plugins/pagesco-books vendor/bin/phpunit, in the plugin's own wp-env test site on port 8889, never in the site you develop on.
3. **W3 (Lesson 8.2).** get_posts() with 'fields' => 'ids', then get_post_meta() for each ID: what is that called, and the fix? **Answer:** N+1 queries; update_postmeta_cache($ids) before the loop loads all the meta in one query.
4. **W4 (Lesson 8.2).** What makes a cached result of EventsTable::find() unreachable after a new event is added? **Answer:** add() calls flush_cache(), which renews wp_cache_get_last_changed('pagesco_events'), and that value is part of every key.
5. **W5 (Lesson 8.2).** The theme's hero image got two fixes. Which, and what did each change? **Answer:** The Image block's aspectRatio (CLS 0.101 to 0.001: the space is reserved) and a preload with fetchpriority high in wp_preload_resources (LCP a little earlier).

WATCH-OUTS

- **No tests run yet** (wp-env did not start at home): fine for today; they must be green before Session 17, because Lesson 9.1's Milestone 7 adds five.
- **A learner still on 8.1's project:** the 8.3 labs run on the 8.2 state; they finish 8.2's Steps 4 and 5 before 8.3's Step 5.

8.3 Internationalization, Accessibility & Distribution

60 min

OBJECTIVES (FROM THE LESSON PAGE)

- Generate a POT file with wp i18n make-pot, write translator-friendly strings, and produce .mo, .l10n.php and JavaScript .json translations with WP-CLI
- Explain how WordPress 7.1 finds a plugin's translations, for PHP (Domain Path) and for scripts (wp_set_script_translations() and block.json)
- Check a theme and its blocks for accessibility with contrast ratios, a keyboard walk and an automated axe audit, and say what automated checks miss
- Prepare a plugin for distribution: a valid readme.txt, version numbers, the directory guidelines, a release ZIP without development files, and a clean Plugin Check report
- Write an uninstall routine that removes content only when an administrator has opted in

TIMING IN THE ROOM

10m	Frame: manuscript, translations, editions for every reader
20m	Live demo: make-pot, the JSON hashes, untranslatable strings, contrast, the uninstall without its taxonomy, the ZIP's autoloader, Plugin Check as recorded
15m	Guided lab: `plural_strings_starter.php`
15m	Start the lesson project: Steps 1 and 2

MENTAL MODEL TO SAY

"A publisher who buys a novel does three things before every reader can have it. The manuscript is prepared for translators: nothing that only works in English. Translators produce the Spanish edition. And there are editions for readers who read differently: large print, an audiobook." On the board: **i18n** = your code, every string through a translation function with the text domain, in a form a translator can work with; **l10n** = translation files, from anyone, later; **a11y** = a quality of every page.

Then distribution in one sentence: "Pages & Co. Books is installed on one site, and it is still distributed: to staging, to the live site, to the next developer, as a ZIP with a version number. Building it to the directory's standard is the professional baseline."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

In the WP-CLI container, on the site as Lesson 8.2 leaves it.

1. **Strings to a POT file (4m).** "A translator never reads your code: they get a POT file." From the WP-CLI container:

```
wp i18n make-pot wp-content/plugins/pagesco-books /tmp/pagesco-books.pot --exc
grep -c '^msgid ' /tmp/pagesco-books.pot
grep -B3 -A4 '^msgid "%d seat"' /tmp/pagesco-books.pot
```

```
Plugin file detected.
Success: POT file successfully generated.
132
#. translators: %d: number of seats.
#: src/Shortcodes/EventsShortcode.php:65
#, php-format
msgid "%d seat"
msgid_plural "%d seats"
msgstr[0] ""
msgstr[1] ""
```

Read the entry from the top: the translators comment you wrote in Lesson 5.5, the file and line, php-format, singular and plural. Then why `--exclude=src/blocks,tests`: JavaScript translations are matched to the script the browser loads, so the POT points at build/. The names those JSON files will need:

```
for script in assets/js/stock-widget.js build/blocks/book-list/index.js; do
  echo "$(printf '%s' "$script" | md5sum | cut -d' ' -f1)  $script"
done
```

```
cfe2bc3ef2a0800531dc26491d4caca6  assets/js/stock-widget.js
1ef6e759fb00b1b9d64a64056a083f27  build/blocks/book-list/index.js
```

2. **Mistake 1: strings a translator cannot translate (4m).** "Four typical mistakes in a scratch file, wordpress/pagesco-strings-test/strings.php. Count what make-pot notices."

```php
<?php
/**
 * Plugin Name: Strings Test
 * Text Domain: pagesco-books
 */

$shown = 3;
$total = 11;
$genre = 'Fantasy';

echo sprintf( __( 'Showing %s of %s books', 'pagesco-books' ), $shown, $total
echo __( 'Books in ', 'pagesco-books' ) . $genre;
echo sprintf( __( '%d books', 'pagesco-books' ), $shown );
echo __( $genre, 'pagesco-books' );
```

```
wp i18n make-pot pagesco-strings-test /tmp/strings.pot
grep '^msgid' /tmp/strings.pot
```

```
Plugin file detected.
Warning: The string "Showing %s of %s books" contains placeholders but has no
Warning: Multiple placeholders should be ordered. (strings.php:11)
Warning: The string "%d books" contains placeholders but has no "translators:'
Success: POT file successfully generated.
msgid ""
msgid "Strings Test"
msgid "Showing %s of %s books"
msgid "Books in "
msgid "%d books"
```

Two warnings for four mistakes, and exit 0. The silent two are worse: `"Books in "` is a fragment, and `__( $genre )` produced no entry at all. Then `rm -r pagesco-strings-test /tmp/strings.pot`.

3 **Contrast, computed (3m).** "Accessibility check one: numbers, not taste." Save the lesson's `wordpress/pagesco-contrast.php`:

```
wp eval-file pagesco-contrast.php
```

```
ink      on paper     13.9:1  AAA   body text
oxblood  on paper      8.8:1  AAA   links
slate    on paper      5.8:1  AA    dates, small text
ink      on sage      12.0:1  AAA   header band
paper    on ink       13.9:1  AAA   notice bar, footer
paper    on oxblood    8.8:1  AAA   buttons
```

4.5:1 for normal text, 3:1 for large text and control outlines. Then the two checks no script does: the keyboard walk and an axe audit, recorded in the lesson (show its cards): the selected genre's focus is invisible, the Navigation nests a list, the notice is outside any landmark.

4 **Mistake 2: an uninstall that cannot find the genres (3m).** "uninstall.php runs with the plugin NOT loaded. Reproduce that with --skip-plugins:"

```
wp --skip-plugins=pagesco-books eval '$genres = get_terms( array( "taxonomy" =
wp --skip-plugins=pagesco-books eval 'echo count( get_posts( array( "post_type
```

```
invalid_taxonomy: Invalid taxonomy.
11 books
```

Books found (WP_Query just compares a column), genres not (an unregistered taxonomy is a WP_Error): an uninstall that loops over that deletes nothing, silently. The project's uninstall.php registers the taxonomy first.

5 **Mistake 3: the development autoloader in a ZIP (3m).** "Copy what a careful ZIP would contain, and load it as WordPress would."

```
mkdir -p /tmp/zip-test/vendor
cp -r wp-content/plugins/pagesco-books/vendor/autoload.php wp-content/plugins/
php -r 'require "/tmp/zip-test/vendor/autoload.php";'
```

```
[26-Oct-2026 14:03:20 UTC] PHP Warning:  require(/tmp/zip-test/vendor/composer
[26-Oct-2026 14:03:20 UTC] PHP Fatal error:  Uncaught Error: Failed opening re
Stack trace:
...
  thrown in /tmp/zip-test/vendor/composer/autoload_real.php on line 41
```

The stack frames with Composer's generated class name are trimmed. `composer dump-autoload --no-dev` before packaging (8.3), or a clean `composer install --no-dev` (8.4). Then `rm -r /tmp/zip-test`.

6 **Plugin Check, as recorded (3m).** "The review team's own tool (it needs the internet to install). On the plugin as Lesson 8.2 left it:"

```
wp plugin check pagesco-books --format=csv
```

```
     60 WARNING,WordPress.NamingConventions.PrefixAllGlobals.NonPrefixedVariab
      7 ERROR,missing_direct_file_access_protection
      4 WARNING,WordPress.DB.DirectDatabaseQuery.DirectQuery
      3 WARNING,WordPress.DB.SlowDBQuery.slow_db_query_tax_query
      3 WARNING,WordPress.DB.SlowDBQuery.slow_db_query_meta_key
      2 WARNING,WordPress.DB.DirectDatabaseQuery.NoCaching
      2 ERROR,wp_function_not_compatible_with_requires_wp
      2 ERROR,application_detected
      1 WARNING,WordPress.DB.SlowDBQuery.slow_db_query_meta_query
      1 WARNING,WordPress.DB.DirectDatabaseQuery.SchemaChange
      1 ERROR,plugin_header_invalid_plugin_uri_domain
      1 ERROR,plugin_header_invalid_author_uri_domain
      1 ERROR,no_plugin_readme
      1 ERROR,hidden_files
```

Read the errors as a release checklist (the ABSPATH guards, Requires at least 6.9, the placeholder URIs, the readme, development files). And on the unpacked release ZIP at the end of the project:

```
   280261                    72 files
Success: Checks complete. No errors found.
     30 WARNING,WordPress.NamingConventions.PrefixAllGlobals.NonPrefixedVariab
      4 WARNING,WordPress.DB.DirectDatabaseQuery.DirectQuery
      2 WARNING,WordPress.DB.SlowDBQuery.slow_db_query_tax_query
      2 WARNING,WordPress.DB.SlowDBQuery.slow_db_query_meta_key
      2 WARNING,WordPress.DB.DirectDatabaseQuery.NoCaching
      1 WARNING,WordPress.DB.SlowDBQuery.slow_db_query_meta_query
      1 WARNING,WordPress.DB.DirectDatabaseQuery.SchemaChange
Deactivating 'plugin-check'...
Plugin 'plugin-check' deactivated.
Uninstalled and deleted 'plugin-check' plugin.
Success: Uninstalled 1 of 1 plugins.
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Concatenating translated fragments.** Coach: "Read it in a language where the genre comes first." One sentence, placeholders, a translators comment. (Self-check Q1, the guided lab.)
- **Trusting a tool that found nothing.** axe passes, so "it is accessible". Coach: "Did the keyboard walk find the focus problem? Did axe?" (Lesson quiz Q3.)
- **Shipping the working copy.** A ZIP of the plugin folder, with tests, rulesets and the development autoloader. Coach with `release_check.sh`. (Self-check Q5.)

GUIDED LAB

File: `labs/module-16/l8_3-i18n-a11y-distribution/plural_strings_starter.php`; your reference is `labs/module-16/l8_3-i18n-a11y-distribution/plural_strings_solution.php`. A translator's Spanish file is given (written to /tmp, loaded for the lab's own text domain): the learner changes the CODE until it can be translated, which is the developer's half of i18n.

- **Run it as shipped (2m):** `cp labs/module-16/l8_3-i18n-a11y-distribution/plural_strings_starter.php wordpress/`, `wp eval-file --use-include plural_strings_starter.php`: the es lines are English.
- **TODO 1 to 4 in pairs (10m):** _n(), numbered placeholders, _x() with two contexts, a whole sentence. Circulate for _n() without the count passed to sprintf() too, and for msgids that differ by a space from the translator's.
- **Expected when done:**

```
en  1 book in stock | 9 books in stock
en  Showing 3 of 11 books
en  Order | Order
en  Books in Fantasy
es  1 libro disponible | 9 libros disponibles
es  De 11 libros, se muestran 3
es  Orden | Pedido
es  Libros de Fantasy
```

- **Debrief (3m):** "Which string did the translator turn around?" (%2$s first.) "Which two of the starter's strings could never be translated, whatever the file?" (The fragment and the concatenated count.)
- **Fast finishers:** `labs/module-16/l8_3-i18n-a11y-distribution/a11y_scan_starter.sh`, then `labs/module-16/l8_3-i18n-a11y-distribution/debug_seats_spanish.php`; `labs/module-16/l8_3-i18n-a11y-distribution/release_check_starter.sh` after the project's Step 5.

START THE LESSON PROJECT

Release 1.1.0. In the room: Steps 1 and 2 (15 minutes): the opt-in and the uninstall tested both ways from a backup.

- Step 1: the new Settings.php, SettingsPage.php and uninstall.php; the three section headings; `delete_data` false.
- Step 2, in ONE container shell (`docker compose run --rm wpcli sh`: the backup lives in its /tmp): the counts, the export, the uninstall with `--skip-delete`, the import. The recorded default run:

```
books   genres  book_meta   events_table
11  5   66  1

Success: Exported to '/tmp/before-uninstall.sql'.
Deactivating 'pagesco-books'...
Plugin 'pagesco-books' deactivated.
Ran uninstall procedure for 'pagesco-books' plugin without deleting.
Success: Uninstalled 1 of 1 plugins.
books   genres  book_meta   events_table
11  5   66  0

Success: Imported from '/tmp/before-uninstall.sql'.
```

And with the opt-in:

```
Success: Updated 'pagesco_books_settings' option.
Deactivating 'pagesco-books'...
Plugin 'pagesco-books' deactivated.
Ran uninstall procedure for 'pagesco-books' plugin without deleting.
Success: Uninstalled 1 of 1 plugins.
books   genres  book_meta   events_table
0   0   0   0

Success: Imported from '/tmp/before-uninstall.sql'.
books   genres  book_meta   events_table
11  5   66  1
```

- **Done in the room** = 11 books after both imports. **Goes home:** Steps 3 (Spanish; internet), 4 (accessibility fixes), 5 (version, readme, ZIP, Plugin Check).

CHECKS FOR UNDERSTANDING

- Which string can a translator translate well? **Answer:** a whole sentence with numbered placeholders and a translators comment (lesson quiz Q1).
- What is the long part of the JSON file's name? **Answer:** the MD5 of the script's path in the plugin (lesson quiz Q2).
- axe reports nothing. What do you know? **Answer:** the machine-checkable rules pass; keyboard and screen reader tests are still needed (lesson quiz Q3).
- Why `composer dump-autoload --no-dev` before plugin-zip? **Answer:** the development autoloader requires files the ZIP does not contain (lesson quiz Q5).

Workbook self-check, items Q1 to Q6.

Answers: **Q1** C) Numbered placeholders, `'Showing %1$s of %2$s books'`, with a translators comment naming both; **Q2** `languages/pagesco-books-es_ES-f9065c2ff2c4d291b5b56d1be2a45f3f.json`; **Q3** A)

Normal-size text in slate on paper meets WCAG AA (4.5:1), but not AAA (7:1); **Q4** D) During uninstall the plugin is not loaded: its classes, post type and taxonomy are not there; **Q5** B) `phpcs.xml.dist`; **Q6** It keeps the JavaScript strings in the PO file; without it make-json moves them into the JSON files and removes them from the PO. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: the string rules table of the lesson is the only reference for the lab: each TODO is one row.

Stretch: the lesson's Exercise 1 (_n() in the Stock widget) and Exercise 3 (the must-use plugin translated); the Bonus Challenge (the skip link first again).

8.4 Deployment & Maintenance

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Choose a hosting type for a WordPress project by what it needs, and explain how code, content and configuration move between development, staging and production
- Keep a WordPress project's own code in Git with a .gitignore, and build release files from a tagged commit with composer install --no-dev, npm ci, npm run build and the plugin ZIP
- Move a database with wp db export, wp db import and wp search-replace, including serialized values and a plugin's own tables
- Configure WordPress for production: debugging and development mode off, the environment type, secrets outside the code, a real cron job, file editing disabled and a least-privilege database account
- Back up a site and prove the backup with a restore, deploy an update, and keep a maintenance checklist

TIMING IN THE ROOM

10m	Frame: code goes up, content comes down, configuration stays
20m	Live demo: the configuration, development mode, secrets, Git, a dry run, serialized data, what the web root hands out, the recorded rehearsal
15m	Guided lab: `migration_preview_starter.sh`
20m	Start the lesson project: Step 3, the rehearsal server

MENTAL MODEL TO SAY

"Pages & Co. moves into a bigger store. The carpenter builds the shelves in the workshop and carries them in finished. The books come along once, on moving day, and from then on the booksellers add and sell them in the store. The alarm code is set for the new building and never painted on a shelf." On the board: **code goes up** (Git, a release build, a deploy) · **content comes down** (the database moves once, then only copies from live) · **configuration stays** (written per environment).

Then the consequence that surprises everyone once: "An edit in the live Site Editor is content. Deploy a new header.html and Maria's header stays." (Self-check Q7.)

LIVE DEMO (YOU TYPE, THEY FOLLOW)

On the site as Lesson 8.3 leaves it; Git in a terminal on the computer, in `wordpress/wp-content` (here the node:24 container ran it, as the lesson's runner does).

1. **The development configuration (3m).** "Your site has the opposite of a production configuration, on purpose."

```
wp eval 'foreach ( array( "WP_DEBUG", "WP_DEBUG_DISPLAY", "SCRIPT_DEBUG", "WP_
```

```
WP_DEBUG             true
WP_DEBUG_DISPLAY     true
SCRIPT_DEBUG         true
WP_ENVIRONMENT_TYPE  'local'
WP_DEVELOPMENT_MODE  'theme'
DISALLOW_FILE_EDIT   (not defined)
DISABLE_WP_CRON      (not defined)
```

Fill in the lesson's production column on the board: debug off, no SCRIPT_DEBUG, production, no development mode, DISALLOW_FILE_EDIT true, DISABLE_WP_CRON true with a real cron job, secrets on the server only.

2. **Development mode and the pattern cache (4m).** "Why the theme's Version matters on a live site." Development mode off, one page view fills the cache:

```
wp config delete WP_DEVELOPMENT_MODE
curl -s -o /dev/null http://localhost:8080/
wp db query "SELECT option_name FROM wp_options WHERE option_name LIKE '_site_
```

```
Success: Deleted the constant 'WP_DEVELOPMENT_MODE' from the 'wp-config.php' f
option_name
_site_transient_wp_theme_files_patterns-ab0ac91d73cd40af5667134910f64f93
```

Now a new pattern file, `wordpress/wp-content/themes/pagesco/patterns/gift-cards.php` (the lesson's), and the question "does WordPress know it?", before and after one changed Version line:

```
wp eval 'var_dump( WP_Block_Patterns_Registry::get_instance()->is_registered(
sed -i 's/^Version: 1.0.0$/Version: 1.1.0/' wp-content/themes/pagesco/style.cs
wp eval 'var_dump( WP_Block_Patterns_Registry::get_instance()->is_registered(
```

```
bool(false)
bool(true)
```

Then `rm wp-content/themes/pagesco/patterns/gift-cards.php; wp config set WP_DEVELOPMENT_MODE theme`. On a live site: raise the theme's Version with every release that changes the theme.

3. **Where the secrets are (2m).** "A secret is anything that lets someone act as the site."

```
grep -n -e "define( 'WP_DEBUG'" -e "WORDPRESS_CONFIG_EXTRA" -e "SUPPLIER" wp-c
```

```
116:define( 'WP_DEBUG', !!getenv_docker('WORDPRESS_DEBUG', '') );
127:if ($configExtra = getenv_docker('WORDPRESS_CONFIG_EXTRA', '')) {
131:define( 'PAGESCO_BOOKS_SUPPLIER_URL', 'http://localhost:8080/supplier-sand
132:define( 'PAGESCO_BOOKS_SUPPLIER_KEY', 'sandbox-7f3a9c' );
```

The sandbox key in wp-config.php is fine here (not in Git, a fake supplier). Live: the server's wp-config.php or environment, different per environment; the supplier-sandbox folder never ships.

4 **The shop's code in Git (4m).** "Git keeps your own source and nothing else." Save the lesson's `wordpress/wp-content/.gitignore` (ignore everything, let the shop's folders back in), then in `wordpress/wp-content` on your computer:

```
git init -q -b main
git config user.name "Your Name"
git config user.email "you@example.com"
git add .
git status --short | wc -l
git ls-files | cut -d/ -f1-2 | sort | uniq -c
```

```
91
      1 .gitignore
      1 mu-plugins/pagesco-site.php
     66 plugins/pagesco-books
     23 themes/pagesco
```

And why a file is missing:

```
git check-ignore -v mu-plugins/mailpit-smtp.php plugins/pagesco-books/vendor p
```

```
.gitignore:7:/mu-plugins/*  mu-plugins/mailpit-smtp.php
.gitignore:19:/plugins/pagesco-books/vendor/    plugins/pagesco-books/vendor
.gitignore:15:/plugins/*    plugins/akismet
.gitignore:3:/* uploads
```

mailpit-smtp.php can never reach the live site: the rule guarantees it.

5 **A migration, dry run (3m).** "WordPress stores full URLs in the database. What would the move to 8081 change?"

```
wp search-replace 'http://localhost:8080' 'http://localhost:8081' --all-tables
```

```
+------------+--------------+--------------+------+
| Table      | Column       | Replacements | Type |
+------------+--------------+--------------+------+
| wp_options | option_value | 2            | PHP  |
| wp_posts   | post_content | 6            | SQL  |
| wp_users   | user_url     | 1            | SQL  |
+------------+--------------+--------------+------+
Success: 9 replacements to be made.
```

Then serialized data: the length is rewritten with the string (s:34 → s:40), which a plain SQL REPLACE() breaks:

```
wp option add pagesco_demo_links '{"shop":"http://localhost:8080/find-a-book/"
wp db query "SELECT option_value FROM wp_options WHERE option_name = 'pagesco_
wp search-replace 'http://localhost:8080' 'https://www.pagesco.example' wp_opt
wp option delete pagesco_demo_links
```

```
Success: Added 'pagesco_demo_links' option.
option_value
a:1:{s:4:"shop";s:34:"http://localhost:8080/find-a-book/";}

pagesco_demo_links', 'a:1:{s:4:\"shop\";s:40:\"https://www.pagesco.example/fi
Success: Deleted 'pagesco_demo_links' option.
```

6 **Mistakes 1 and 3: what the web root hands out (3m).** "Ask like a stranger."

```
wp db export pagesco-dev.sql
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/pagesco-dev.sql
curl -s http://localhost:8080/pagesco-dev.sql | grep -o "'admin','[$]wp[$]2y[$
```

```
Success: Exported to 'pagesco-dev.sql'.
200
'admin','$wp$2y$12
```

The whole database, password hashes included. `rm pagesco-dev.sql`. And the repository in wp-content:

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/wp-content/.git
curl -s http://localhost:8080/wp-content/.git/HEAD
```

```
200
ref: refs/heads/main
```

On your machine leave it; on a server: deploy release files, never the repository, and deny hidden files (the rehearsal's Apache does).

7 **The rehearsal, as recorded (1m).** Show the lesson's recorded Step 5 card from pagesco-live: the production settings checked on the running site.

```
debug false, environment production, development mode "", file editor off, app
Error: The DISABLE_WP_CRON constant is set to true. WP-Cron spawning is disab
Grants for pagesco_app@%
GRANT USAGE ON *.* TO `pagesco_app`@`%`
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER ON `pagesco`

Error: Query failed: --------------
CREATE DATABASE test
--------------

ERROR 1044 (42000) at line 1: Access denied for user 'pagesco_app'@'%' to data
```

Debug off, production, no editor, no application passwords without HTTPS, WP-Cron's spawning off, a database account limited to its database. Learners make this card themselves at home.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Deploying the database again with an update.** Coach: "Where were today's orders made?" A later deploy is code only. (Lesson quiz Q1.)
- **A migration without the plugin's table.** Coach: "Count the CREATE TABLE lines before you import." (Self-check Q8, the debug handout.)
- **A backup nobody restored.** Coach: "How big is the file? When did you last restore one?" (Self-check Q12, `restore_drill`.)

GUIDED LAB

File: `labs/module-16/l8_4-deployment-maintenance/migration_preview_starter.sh`; your reference is `labs/module-16/l8_4-deployment-maintenance/migration_preview_solution.sh`. A move to a staging address, previewed with dry runs and exports to /tmp: the habit of counting before importing, without touching the database.

- **Run it as shipped (2m):** `cp labs/module-16/l8_4-deployment-maintenance/migration_preview_starter.sh wordpress/`, `docker compose run --rm wpcli sh migration_preview_starter.sh`.
- **TODO 1 to 4 in pairs (10m):** never a search-replace without `--dry-run` or `--export`. Circulate for `--format=count` and for `> /dev/null` on the exports.
- **Expected when done:**

```
tables WordPress registers: 12, all with the prefix: 13
the tables only the second list has: wp_pagesco_events
dry run, every table with the prefix, GUIDs left alone: 9 replacements
export without --all-tables-with-prefix: 12 tables
export with --all-tables-with-prefix: 13 tables
new address in the export: 15, old address left (the GUIDs): 34
options with the address inside a serialized value: 0
the database still says: http://localhost:8080
```

- **Debrief (3m):** "What is the 34?" (The GUIDs, left alone on purpose.) "Which line would have caught Common Mistake 2?"
- **Fast finishers:** `labs/module-16/l8_4-deployment-maintenance/restore_drill_starter.sh`, `labs/module-16/l8_4-deployment-maintenance/cron_audit_starter.php`, then `labs/module-16/l8_4-deployment-maintenance/debug_launch_check.sh`.

START THE LESSON PROJECT

Launch Day, Rehearsed. In the room: Step 3 only (20 minutes), because Steps 1 and 2 need Lesson 8.3's release 1.1.0.

- Unzip pagesco-live.zip next to pagesco-dev, `cp .env.example .env` and two long random passwords, `mkdir wordpress deploy backups`, on Linux and WSL the two lines and compose.override.yaml, `docker compose up -d`.
- Read the folder's table together (compose.yaml, .env, create-app-user.sh, apache-8081.conf, php-production.ini): every line of the production configuration is in one of them.
- **Done in the room** = four containers started and http://localhost:8081 answering (WordPress's installer: nothing is installed, on purpose). **Goes home:** Steps 1, 2, 4 to 7, in that order, after 8.3's Step 5. The recorded results learners compare with:

```
+------------+--------------+--------------+------+
| Table      | Column       | Replacements | Type |
+------------+--------------+--------------+------+
| wp_options | option_value | 2            | PHP  |
| wp_posts   | post_content | 6            | PHP  |
| wp_users   | user_url     | 1            | PHP  |
+------------+--------------+--------------+------+
Success: Made 9 replacements and exported to release/pagesco-live.sql.
http://localhost:8080
13
     34 http://localhost:8080
     15 http://localhost:8081
```

```
Server: Apache
403
404
```

```
0
Pages & Co. Books 1.1.1 and theme 1.1.1: HTTPS for the supplier, Update URI
Pages & Co. release 1: theme 1.1.0, Pages & Co. Books 1.1.0, site plugin 1.2.0
release-1       First launch
release-2       Plugin 1.1.1, theme 1.1.1
```

CHECKS FOR UNDERSTANDING

- A bug fix after a month live: what do you deploy? **Answer:** only the plugin ZIP built from the release tag (lesson quiz Q1).
- Export without --all-tables-with-prefix: what is missing? **Answer:** the plugin's events table (lesson quiz Q2).
- A new pattern, the same theme Version: what happens? **Answer:** unknown until the cached list expires (lesson quiz Q3).
- DISABLE_WP_CRON and no cron job: the nightly sync? **Answer:** never runs (lesson quiz Q5).

Workbook self-check, items Q7 to Q12.

Answers: **Q7** B) Maria's header: Site Editor changes are saved in the database, and they win over the theme's files; **Q8** `12 13`: `wp_pagesco_events`, the plugin's own table, which WordPress does not register; **Q9** A) The GUID is an identifier feed readers use to recognize posts, not a link: it must not change; **Q10** D) Git cannot let a file back in when its folder is ignored: the folder is let in, its contents ignored again, then the one subfolder let in; **Q11** `composer install --no-dev --optimize-autoloader`: the development packages (PHPUnit, PHPCS), so vendor/composer has no autoload_files.php that requires them; **Q12** C) It fails with error 1044, and leaves a small header-only file under the backup's name. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: one card with the release recipe in order (tag, git archive, composer install --no-dev -o, npm ci, build, plugin-zip) and the launch in order (code first, then the database).

Stretch: the lesson's three exercises (launch check as code, nightly backup script, smoke test), and the Bonus Challenge (a CI workflow on release-* tags).

2:25 The Final-Project Brief: Reserve & Pick Up

25 min

TIMING

8m	The brief: Maria's requirements, read aloud, and what learners hand in
7m	The ten milestones, the security checklist and the rubric
8m	First plan, in pairs: nouns and verbs, requirements to layers
2m	What Session 17 starts with, and what to plan tonight

OBJECTIVES (FROM THE LESSON PAGE)

- Turn a client's brief into data, rules, routes, blocks and screens, and decide where each rule is enforced
- Build a feature through every layer of the course: a custom table, settings, a service class with a custom hook, REST routes with real permission callbacks, form handlers, a staff screen, a dynamic and an interactive block, and a theme pattern with live data
- Make it secure and private by default: nonces and capabilities on every change, ownership in every query, escaping, and personal data kept no longer than needed
- Prove the feature with coding standards, PHPUnit tests, a smoke test from outside, accessibility checks and a performance measurement
- Release it from a tagged commit and deploy it as code only, then present your work against a rubric

SEQUENCE AND TALKING POINTS

- **The brief (8m).** Hand out the brief (`lesson_30.html`, "The Brief"; or the final project part of `final-assessment.html`). Read Maria's eight requirements aloud and ask after each: "Which layer handles that?" Then the hand-in: the Git repository with tag release-3 (plugin and theme 1.2.0), both ZIPs, the proof (composer lint, PHPUnit, the smoke test on both sites, the performance numbers), RESERVE-AND-PICKUP.md, a five-minute showcase. The lesson estimates 300 minutes; Sessions 17 and 18 give it studio time, the rest is homework.
- **Milestones and assessment (7m).** Put the milestones table on the slide (read from the lesson) and say: "Each ends with something you can check and commit. Three sittings: 1 to 4, 5 to 7, 8 to 10." Then the security checklist (it carries the most points) and the rubric: 80 of 100 is portfolio-ready.
- **First plan (8m).** In pairs, the From Requirements to Code habit: underline the nouns (the data: a hold, a customer, a book, a pickup date), the verbs (reserve, cancel, collect, email), and fill a requirements-to-layers table: rule, door (form, REST, block), screen. Pairs share one decision each: why a custom table, why "expired" is computed.

- **What comes next (2m).** "Session 17 starts at Milestone 1 with your plan in your hands, on the site and server as Lesson 8.4 leaves them. Tonight: finish 8.3 and 8.4 first, then read the brief again and write your plan." Read the bring list.

MILESTONES (FROM THE LESSON PAGE)

#	MILESTONE	USES LESSONS	TIME
1	Plan, and measure before you build	1.1, 8.2	15 min
2	The data: a holds table and two settings	5.1, 5.4, 5.5	25 min
3	The rules, and the email	1.3, 2.3, 5.3, 1.2	30 min
4	REST routes, form handlers and the staff screen	5.3, 5.4, 7.1, 7.2	30 min
5	Two blocks: Reserve for Pickup and My Holds	3.2, 6.1, 6.3, 6.4	45 min
6	Live prices on the front page	3.3, 6.3	15 min
7	Quality: standards, tests, speed, accessibility	8.1, 8.2, 8.3	30 min
8	Release 1.2.0	8.3, 8.4	20 min
9	Deploy to the rehearsal server	8.4, 7.3	25 min
10	Write-up and showcase	all	15 min

250 minutes of building; the brief, the mistakes, exercises, quiz and journal make up the rest of the lesson's 300.

RUBRIC (FROM THE LESSON PAGE)

AREA	POINTS
Features	20
Data and rules	15
Security and privacy	20
Blocks and experience	15
Code quality	10
Testing	10
Release and deploy	5
Write-up and showcase	5

Each area: 0 (missing), half (meets) or all (exceeds). The lesson's table has the meets and exceeds descriptions.

WATCH-OUTS

- **Do not start building today.** A learner who types code for holds tonight on a site still at 8.2 builds on the wrong state; the plan is tonight's work.
- **The reference solution** (`downloads/pagesco-final`, and each milestone's "Reference solution" panel) exists: say so, and say how to use it: try first, then compare.
- **Privacy is a requirement**, not a nice-to-have: holds name customers (requirement 8, the checklist's last row).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. Give one example each of code, content and configuration on the Pages & Co. site, and how each moves to the live site. **Answer:** the plugin (Git, release ZIP, deploy); the books (the database, once at launch, then created live); the supplier key (written on the server, never copied).
2. Name two translator-friendly rules for strings. **Answer:** whole sentences with placeholders; numbered placeholders with a translators comment; _n() for counts; _x() for ambiguous words; literal strings only.
3. Which two Reserve & Pick Up requirements do you expect to be the hardest, and which layer handles each? **Answer:** any honest pair; for example "nobody can see anyone else's holds" (the user ID in the SQL WHERE, "not found") and "works without JavaScript" (forms to admin-post.php under the block's store).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish Lesson 8.3's project first:** Steps 3 to 5 (Spanish, the accessibility fixes, version 1.1.0 with readme, ZIP and Plugin Check) and the Release note.
- **Then Lesson 8.4's project:** Steps 1 and 2, then 4 to 7 on the rehearsal server you started today, and the Deployment note. The final project deploys to that server.
- **Lesson exercises.** 8.3: Exercises 1 and 2 (3 is a stretch). 8.4: Exercises 1 and 2 (3 is a stretch).
- **Both Quick Quizzes** and **both journal entries**; both "Before the Next Lesson" lists.
- **The final project's plan:** read Lesson 9.1's Brief, From Requirements to Code, the Security and Privacy Checklist and How Your Work Is Assessed; write your requirements-to-layers table and three questions. Do not build yet.
- **Optional:** the labs not reached: `labs/module-16/l8_3-i18n-a11y-distribution/a11y_scan_starter.sh` (before and after your Step 4), `labs/module-16/l8_3-i18n-a11y-distribution/release_check_starter.sh` (after Step 5), `labs/module-16/l8_3-i18n-a11y-

`distribution/debug_seats_spanish.php`, `labs/module-16/l8_4-deployment-maintenance/restore_drill_starter.sh`, `labs/module-16/l8_4-deployment-maintenance/cron_audit_starter.php`, `labs/module-16/l8_4-deployment-maintenance/debug_launch_check.sh`.

BRING LIST FOR SESSION 17

- The site as Lesson 8.4 leaves it: `wordpress/wp-content` is a Git repository with a clean working copy and the tags release-1 and release-2 (`git status`, `git tag -n`), plugin and theme 1.1.1, `composer lint` exits 0.
- The rehearsal server `~/pagesco-live` launched, with plugin and theme 1.1.1, its new passwords in your password manager, `LOCK TABLES` granted, and one backup in `backups/` (Lesson 8.4, Steps 3 to 7). `docker compose up -d` in its folder before the session.
- The test environment works: `npx @wordpress/env start` in the plugin's folder, the twelve tests green (Node.js 24; the node:24 and composer:2 images pulled).
- An internet connection for npx and `npm ci` (Lesson 9.1's Milestones 5 and 8).
- Your plan for Reserve & Pick Up from tonight's homework (the requirements-to-layers table, the questions you still have), Lesson 9.1 open in a browser tab, pagesco-plan.txt, the journal and the workbook.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the 8.4 frame and Git; the final-project brief (at least 15 minutes); the 8.3 uninstall test.
- **Cut first (saves 3m):** the Plugin Check recordings (they are in the Answer Key).
- **Cut second (saves 4m):** the 8.4 pattern-cache step: tell it with the self-check's numbers.
- **Cut third (saves 8m):** the brief's first plan in pairs goes home with the reading.

RUNNING SHORT: WHAT TO ADD

- `restore_drill_solution.sh` on the projector: the drill read line by line.
- Lesson 8.4's Exercise 3 (the smoke test) live against the development site and a wrong address.
- Longer planning for the final project: pairs swap tables and review each other's against the security checklist.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 17: Final Project Kickoff and Studio I

How to run the first studio of the final project: turn Maria's brief for Reserve & Pick Up into data, rules and doors, meet the three most expensive mistakes on purpose, measure before building, then let learners build Milestones 1 to 4 (the plan, the holds table and settings, the rules and the email, the first doors) with real done-when outputs for every milestone and a check-in for every learner.

`1 lesson (first studio)` `3 hrs session` `Lesson 9.1 · Milestones 1–4` `Session 17 of 18 · Part 2`

HOW TO USE THIS GUIDE

- Session 17 is the first of two studios for the final project, Lesson 9.1's **Reserve & Pick Up**, the course's capstone. There is no new lesson content to present: the room turns Maria's brief into a design, then learners build Milestones 1 to 4 with you circulating. Session 18 finishes Milestones 7 to 9, holds the showcase and teaches Lesson 9.2.
- The lesson is 300 minutes for a learner working alone, 250 of them for the ten milestones. Where they happen: Milestones 1 to 3 and the start of 4 in this room; the rest of 4, then 5 (the two blocks) and 6 (live prices) at home; 7 (quality), 8 (release 1.2.0) and 9 (the code-only deploy) in the Session 18 studio; 10 (the write-up) at home. **Submission: one week after Session 18.** The rubric, the scoring guidance, the grading run and the reference numbers are in `final-assessment.html` at the kit's root.
- Learners met the brief at the end of Session 16 (the brief, the milestones table, the security and privacy checklist and the rubric were the reading). Today they need their site exactly as Lesson 8.4 left it: the wp-content Git repository with release-1 and release-2, plugin and theme 1.1.1, and the pagesco-live rehearsal server. The warm-up checks it.
- Every command and file below is the lesson's own, run on a site at the end of Lesson 8.4 (WordPress 7.1.3, PHP 8.5, MySQL 9.7, WP-CLI 2.12); the outputs are real and each one matches the lesson's own output card line for line. Each milestone has a "done when" with the output learners compare; the lesson hides a reference solution behind each one. The rule for the room, as the lesson says: try first, then open the reference and compare; different names are fine when the brief, the checklist and the checks are met.
- The kit's labs for this session (`labs/module-17/l9_1-final-project/`) are short practice on the same habits for the guided lab and for stuck learners, never the project. Teal blocks are hands-on, amber blocks are watch-outs.

THE #1 RISK OF SESSION 17: BUILDING ON THE WRONG SITE

Reserve & Pick Up is built on the site Lesson 8.4 left: Git in wp-content, release-2, the rehearsal server. A learner who skipped Lesson 8.4's Git steps or whose pagesco-live is broken can build Milestones 1 to 7 on their development site, but cannot release or deploy in Session 18 without it. **In the warm-up:** `git tag -n` in wordpress/wp-content (release-1 and release-2) and `docker compose ps` in pagesco-live, for everyone. **Fix now or plan:** a missing repository is five minutes (Lesson 8.4's `.gitignore`, `git init`, commit, tag release-2) and is done before Milestone 1; a broken pagesco-live is rebuilt at home with Lesson 8.4's launch steps and checked at the start of Session 18. Nobody starts Milestone 2 on a site that is not at 1.1.1.

BEFORE THE SESSION

- ☐ **At the end of Session 16** (or by email): learners read The Brief, Milestones and Timeline, From Requirements to Code, the Security and Privacy Checklist and How Your Work Is Assessed on `lesson_30.html`, and run `docker pull node:24` and `composer:2` at home if they have not since Lesson 8.4. Ask each to bring one question about the brief.
- ☐ **Your own reference site:** a copy of the course site at the end of Lesson 8.4 on the projector (the demo changes nothing that lasts except the query report file, which Milestone 1 keeps). Have `lesson_30.html` open at the brief, and `final-assessment.html` for the rubric questions.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-17` (9 results, about a minute and a half). With your own course project at the end of Lesson 8.4: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-17`.
- ☐ **Board, before learners arrive:** the counter drawing (shelf = data, house rules = rules, three doors: form, REST, block), the eight requirements as eight short lines, the milestone timeline (S17: 1 to 4 · home: 4 to 6 · S18: 7 to 9 · home: 10 · due one week after S18), and an empty check-in board with a column per milestone.
- ☐ **Print or share:** the workbook pages for Session 17 (milestone tracker, requirement map, the predict items), the rubric from `final-assessment.html` (one per learner: they score themselves at the end), sticky notes.

KNOWN PROJECT TRAPS

SYMPTOM	CAUSE	FIX
Every page: "There has been a critical error"; WP-CLI too	A line in `Plugin::feature_classes()` for a class whose file does not exist yet (or a typo in its namespace)	Read the end of debug.log (`Class "PagesCo\Books\..." not found`). Add each line together with its file; remove the line to get the site back.
The sed command adds nothing, or adds a line with spaces	The lesson's `sed ... a\` has three TABS after the backslash; a copy from some terminals turns them into spaces	Add the line in the editor instead; `grep -c '::class,'` must say 21, then 22, then 25.

SYMPTOM	CAUSE	FIX
`holds_tables 0` even after a page view	`maybe_upgrade()` is not hooked (the feature line is missing) or the version option already says 1 from an earlier try	`grep -c '::class,'`; `wp option get pagesco_books_holds_db_version`; delete the option and reload once.
holds-check.php: `pagesco_hold_duplicate` on the FIRST line	Holds from an earlier try are still in the table	`wp db query "TRUNCATE TABLE wp_pagesco_holds"` and run it again (Milestone 4 truncates too).
holds-check.php: `Uncaught Error: Call to a member function ... on false`	jordan or priya do not exist yet	Milestone 3's two `wp user create` lines first.
No email in Mailpit, or the wrong one	HoldMail's feature line missing; or another learner's newest message (shared Mailpit in some setups)	`grep -c '::class,'` says 22; Mailpit's search box: `to:jordan@example.com`.
`curl --user "jordan:$JORDAN"` answers 401 `rest_forbidden`	The shell variable is empty: a new terminal, or the `JORDAN=$(...)` line ran in another one	Create the application password again in this terminal (`echo $JORDAN` must print it).
Books > Holds missing for Maria	The menu needs `edit_others_posts`, or the HoldsPage line is missing	Maria is an editor: check the feature line, then the capability constant.
The forged POST returns 303 instead of 403	The handler skipped `check_admin_referer()`	That is a real security bug: the smoke test of Milestone 5 would flag it too.

SESSION 17 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Readiness at the start line: `git tag -n` shows release-1 and release-2, pagesco-live answers, the brief was read; three questions on Session 16.
0:10	20	**9.1** Frame: the brief, the layers, the rubric	Maria's eight requirements become data, rules and doors; the requirement map; the security and privacy checklist; how the 100 points are earned; the timeline to the submission.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:30	15	**9.1** Live demo: three mistakes before you build	A general capability (Common Mistake 2), a release file list without the new folder (Common Mistake 1), a WordPress global's name; then the query baseline of Milestone 1.
0:45	15	**9.1** Studio: Milestone 1, plan and baseline	The plan in pagesco-plan.txt, the query report and the three numbers; plan check-in per learner.
1:00	25	**9.1** Studio: Milestone 2, the data	HoldsTable, its line, Activator, DailySync, uninstall, two settings; done = the maybe_upgrade check shows six columns and version 1.
1:25	10	Break	Learners write their three baseline numbers and one design decision on a sticky note for the check-in board.
1:35	15	**9.1** Guided lab	`labs/module-17/l9_1-final-project/ownership_sql_starter.php`: ownership in the WHERE, the user from the login, one 404 for everyone else.
1:50	35	**9.1** Studio: Milestone 3, the rules and the email	Holds, HoldMail, two Subscribers, holds-check.php; done = the ten lines of the brief's rules and the email in Mailpit.
2:25	25	**9.1** Studio: Milestone 4 (start), the doors	HoldActions, HoldsEndpoint, HoldsPage; done in the room = the two routes and a visitor refused with 401; the rest of the checks go home.
2:50	10	Wrap-up	Exit ticket, the check-in board, homework (Milestones 4 to 6), bring list for Session 18.
Total	180		Ends at 3:00.

In the room. Lesson 9.1 is 300 minutes for a solo learner; the room has 150 after the warm-up, the break and the wrap-up. The frame and the demo take 35 (the brief, the requirement map, the checklist, the rubric, the three Common Mistakes and the baseline), the guided lab 15, and the studio 100: Milestones 1 to 3 completely (the lesson's estimate: 70 minutes) and the start of Milestone 4.

Goes home (about 2 to 3 hours). The rest of Milestone 4 (the REST checks with application passwords, the staff screen, the forged request, the truncate and the commit), Milestone 5 (the two blocks: about 45 minutes, and `npx @wordpress/create-block` needs the internet) and Milestone 6 (live prices, 15 minutes). The optional kit labs. The exercises, the lesson's quiz and the journal wait until after Session 18, when the feature is complete.

Protect, in this order: the readiness check (nobody builds on a site that is not at 1.1.1 with Git); the requirement map (learners leave knowing which layer handles each line of the brief); Milestone 2's done-when (the table created by `maybe_upgrade()`, the pattern every later milestone and the live deploy depend on); Milestone 3's

holds-check run. **Cut first:** the third demo step (the `$title` sniff: point at the lesson), then the guided lab (send `ownership_sql` home; Milestone 3 practices the same), then Milestone 4 in the room (it starts at home instead).

OPENING SEGMENT

0:00 Warm-up: Readiness at the Start Line — 10 min

TIMING

4m	Readiness: `git tag -n` and `git status --short | wc -l` in wordpress/wp-content, `docker compose ps` in pagesco-live
6m	Three questions on Session 16, then each learner's one question about the brief

SEQUENCE AND TALKING POINTS

- **Readiness (4m).** On the projector, then theirs. In wordpress/wp-content: `git tag -n` (release-1 "First launch" and release-2 "Plugin 1.1.1, theme 1.1.1") and `git status --short | wc -l` (0: a clean start, so the first milestone commit shows only the feature). In pagesco-live: `docker compose ps` (db, wordpress and cron running). Anyone missing a tag or with a dirty working copy fixes it before Milestone 1 (commit or stash what is there); a broken rehearsal server goes on the board for Session 18.
- **Three questions (3m), out loud:** (1) "An update replaced the plugin: did its activation hook run?" (No: Lesson 8.4. Today's Milestone 2 depends on it.) (2) "What does a release ZIP contain?" (Exactly the paths in package.json's files, built from a tag.) (3) "Code goes up, content comes down, configuration stays: which is the My holds page?" (Content: Milestone 9 creates it on the live site.)
- **Their questions about the brief (3m).** Collect them on the board and answer the ones the frame will not; the usual ones are "can I design it differently?" (yes: the rubric scores the brief, the checklist and the checks, not the reference's names) and "can I use WooCommerce?" (the lesson's FAQ: for payments yes, for this no).

LESSON PLAN

9.1 Final Project: Launch the Pages & Co. Website

160 min

OBJECTIVES (FROM THE LESSON PAGE)

- Turn a client's brief into data, rules, routes, blocks and screens, and decide where each rule is enforced
- Build a feature through every layer of the course: a custom table, settings, a service class with a custom hook, REST routes with real permission callbacks, form handlers, a staff screen, a dynamic and an interactive block, and a theme pattern with live data
- Make it secure and private by default: nonces and capabilities on every change, ownership in every query, escaping, and personal data kept no longer than needed
- Prove the feature with coding standards, PHPUnit tests, a smoke test from outside, accessibility checks and a performance measurement
- Release it from a tagged commit and deploy it as code only, then present your work against a rubric

TIMING IN THE ROOM

20m	Frame: the brief, data/rules/doors, the requirement map, the checklist, the rubric, the timeline
15m	Live demo: Common Mistakes 2, 1 and 3, then the Milestone 1 baseline
15m	Studio: Milestone 1
25m	Studio: Milestone 2
15m	Guided lab: `ownership_sql_starter.php`
35m	Studio: Milestone 3
25m	Studio: Milestone 4, the start

FRAME: FROM MARIA'S BRIEF TO LAYERS (20M)

Read the brief aloud, one requirement at a time, and underline on the board as the lesson says: **nouns are data** (customer, book, hold, three days, the counter's list), **verbs are actions** (reserve, cancel, collect, email), and each rule gets sorted by where WordPress lets you enforce it. Then the mental model, in the lesson's words:

Think of the counter at Pages & Co. The shelf behind it holds the books (the **data**). The house rules hang on the wall: three days, three books, no sold-out books (the **rules**). Customers can ask at the counter, phone, or use the website (the **doors**: a form, the REST API, a block). Whoever answers, they read the same house rules on the same wall. In code: one table class that only stores, one service class that only decides, and several

thin doors that only check who is asking and pass the question on. When a rule changes, you change it in one place, and every door follows.

Then the requirement map, the lesson's table, as a check: for each line, ask the room which layer handles it before you show the answer.

REQUIREMENT	HANDLED BY	MILESTONE
1. Reserve; three days; expired	`Holds::place()` computes `pickup_by` from the setting; `Holds::status()` computes "expired"	2, 3
2. Limits; no duplicates; not sold out; settings	`Holds::place()`; settings `hold_days` and `max_holds` on Settings > Pages & Co. Books	2, 3
3. My holds; cancel; only your own	the My Holds block; `HoldsTable::finish()` with the user ID in its `WHERE`; a "not found" for anyone else's hold	3, 4, 5
4. Staff list and "collected"	Books > Holds (`Admin\HoldsPage`), the collect handler, capability `edit_others_posts`	4
5. Email	the plugin's own action `pagesco_books_hold_placed`, and `Holds\HoldMail` listening to it	3
6. No JavaScript, screen readers, no reload	forms that post to `admin-post.php`; an Interactivity API store that sends the same form to the REST API; `role="status"` messages	4, 5, 7
7. Live prices on the home page	the `pagesco/book` binding source learns a book's slug; the theme's staff picks pattern binds its prices	6
8. Personal data; tested; safe deploys	the daily job deletes holds 30 days after their pickup date; uninstall drops the table; lint, tests, smoke test; a code-only deploy with a backup first	7, 8, 9

Then the two tables learners will be scored against: the **Security and Privacy Checklist** (the rubric's biggest area with Features: 20 points each) and the **rubric** itself. Read the checklist's "Proven by" column: several lines are proven from outside by the smoke test of Milestone 5, the rest by Milestone 3's checks, PHPUnit and code review.

THREAT	THE RULE IN RESERVE & PICK UP	PROVEN BY
Cross-site request forgery	Every form carries a nonce for its action *and its ID* (`pagesco_books_place_hold_30`); the REST API needs the `wp_rest` nonce with cookies; nothing changes on GET	Smoke test: a form without a nonce gets 403
Broken access	Customers need to be logged in (`read`); staff actions need `edit_others_posts`, checked in the handler and in the	Smoke test: visitors get 401, customers cannot

THREAT	THE RULE IN RESERVE & PICK UP	PROVEN BY
control	menu; every REST route has a `permission_callback` that returns a `WP_Error` with `rest_authorization_required_code()`	collect; Books > Holds refuses an author
Someone else's hold (IDOR)	The user ID comes from the login, never from a form; cancelling puts it in the SQL `WHERE`; another customer's hold is "not found", not "forbidden", so it does not even confirm that it exists	Milestone 3's checks, PHPUnit
Cross-site scripting	Escape late in every template (`esc_html` , `esc_attr` , `esc_url`); messages chosen from a fixed list by a code in the address, never text from the address; `data-wp-text` sets text, not HTML	Code review, `composer lint`
SQL injection	Every query through `$wpdb->prepare()` with `%i` , `%d` and `%s` , or `$wpdb->insert()` / `update()` with formats	Code review, `composer lint`
Open redirects	Handlers redirect with `wp_safe_redirect()` to addresses they build themselves	Code review
Abuse	At most `max_holds` waiting holds per customer; only logged-in customers, whose accounts the shop creates	Milestone 3's checks, PHPUnit
Personal data	Holds name customers: they are deleted 30 days after their pickup date by the daily job, dropped on uninstall, never sent to the supplier, and listed in the readme	Milestone 2 and 8, `readme.txt`

AREA	POINTS	MEETS (HALF)	EXCEEDS (FULL)
Features	20	Reserve, My holds, cancel, staff list and collect, the email and live prices all work in the browser	Every refusal has a clear message; expired holds show as expired; empty states are friendly; the settings change the rules without code
Data and rules	15	A table with keys created by `dbDelta()` ; the rules in one place	Indexes for the real queries; "expired" computed, not stored; the table created on update by `maybe_upgrade()` , not only on activation; a custom hook others can use

AREA	POINTS	MEETS (HALF)	EXCEEDS (FULL)
Security and privacy	20	Nonces, capabilities, escaping and prepared queries everywhere	Every line of the checklist, with ownership in SQL, "not found" for other people's holds, and old holds deleted on schedule
Blocks and experience	15	Both blocks render the right state for visitors, customers and sold-out books	Works without JavaScript; no reload with it; announced to screen readers; no duplicate IDs; a sensible editor preview
Code quality	10	`composer lint` is clean; the feature pattern of Lesson 5.1	Every standards exception documented with a reason; every string translatable with translator comments; a new POT
Testing	10	You tested every rule by hand	PHPUnit tests for the rules and the REST door pass in wp-env, and the smoke test passes on both sites
Release and deploy	5	A tagged release deployed to the rehearsal server	Versions bumped everywhere they live, the ZIP checked before upload, a backup before the deploy, the database untouched
Write-up and showcase	5	A write-up that says what you built	The write-up maps every requirement to code and names a decision you would defend; the showcase follows the checklist and fits five minutes

Score yourself, or swap with a classmate: each area earns 0 (missing), half (meets) or all of its points (exceeds). 80 or more out of 100 is a solid, portfolio-ready project. The reference solution is written to score full points; yours can get there with a different design. Finish with the timeline on the board, and the deal: "Each milestone ends with a check you can run. Commit after each group. Try first, then open the reference."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's three Common Mistakes, met on purpose before anyone builds, then Milestone 1's baseline. From the project folder with the `wp` shortcut; the Node and Composer steps in the plugin's folder (`wordpress/wp-content/plugins/pagesco-books`) as in Lessons 6.1 and 8.1. Every output below is real and equals the lesson's card.

1. **A general question instead of this one (3m).** "Before anyone writes a line: the bug that costs the most is silent." From the project folder, as Devon, about Maria's "Welcome" post (ID 15). Ask for a prediction first.

```
wp eval 'var_dump( current_user_can( "delete_posts" ), current_user_can( "dele
```

```
bool(true)
bool(false)
```

The general check says yes (Devon may delete HIS posts), the meta capability with the ID says no. A handler that asks the first question deletes anyone's post for anyone who may delete their own, with no error anywhere. "Holds are not posts, so WordPress has no meta capability for them: the owner goes into the query. That is Milestone 3."

2. **A release file list without the new folder (4m).** "The second costs you a live site." In the plugin's folder with Node (Lesson 6.1): a stand-in file in the folder Milestone 2 creates, then ask npm what a ZIP would contain. `npm pack --dry-run` writes nothing.

```
mkdir -p src/Holds
printf '<?php\n' > src/Holds/HoldsTable.php
npm pack --dry-run 2>&1 | grep -c ' src/'
npm pack --dry-run 2>&1 | grep -c ' src/Holds/'
rm -r src/Holds
```

```
24
0
```

24 PHP files from src/, 0 from src/Holds/: nothing warns, the ZIP installs, and the first request on the live site dies with "Class not found" (show the lesson's recorded log in Common Mistakes 1, read it from the bottom up). "Every new top-level folder in src/ goes into package.json's files in the same commit. Milestone 8 checks the ZIP before upload."

3. **A WordPress global's name (3m).** "The third is cheap to fix." The lesson's stand-in render.php keeps the title in `$title`; run the plugin's own standards on it (Composer, as in Lesson 8.1).

```
vendor/bin/phpcs src/blocks/demo/render.php; echo "exit status $?"
rm -r src/blocks/demo
```

```
FILE: src/blocks/demo/render.php
--------------------------------------------------------------------------------
FOUND 1 ERROR AFFECTING 1 LINE
--------------------------------------------------------------------------------
 10 | ERROR | Overriding WordPress globals is prohibited. Found assignment to
    |       | $title (WordPress.WP.GlobalVariablesOverride.Prohibited)
--------------------------------------------------------------------------------

...

exit status 1
```

`$title` is a WordPress global (wp-admin keeps the screen's title in it). In render.php the variable happens to be local, but the same line at the top level of a template would overwrite it. `$book_title` costs nothing: fix it when the fix is cheap, document an exception when it is not (Milestone 7).

4 **Measure before you build (5m).** "Lesson 8.2's rule: numbers first." Save the lesson's query report as wordpress/wp-content/mu-plugins/pagesco-query-report.php (it is Milestone 1; learners type it next), then:

```
wp config set SAVEQUERIES true --raw
curl -s -o /dev/null http://localhost:8080/
curl -s -o /dev/null http://localhost:8080/books/dune/
curl -s -o /dev/null http://localhost:8080/find-a-book/
tail -n 3 wp-content/debug.log
wp config delete SAVEQUERIES
```

```
Success: Added the constant 'SAVEQUERIES' to the 'wp-config.php' file with the
[26-Oct-2026 12:57:28 UTC] /: 22 queries; missing options: can_compress_script
[26-Oct-2026 12:57:28 UTC] /books/dune/: 25 queries; missing options: can_comp
[26-Oct-2026 12:57:28 UTC] /find-a-book/: 25 queries; missing options: can_cor
Success: Deleted the constant 'SAVEQUERIES' from the 'wp-config.php' file.
```

Write 22, 25 and 25 on the board: a visitor's view, nobody logged in. Milestone 7 measures again and the write-up compares. (A learner who opened wp-admin since Lesson 8.2 sees each number one lower: `can_compress_scripts` exists on their site.)

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Building the doors first.** A learner starts with the REST route or the block because that is what shows. Coach with the counter: "Which rule does this door enforce? Then it is in the wrong layer." The order of the milestones is the order of the layers. (Self-check Q12.)
- **Ownership in PHP, after the query.** `$hold = HoldsTable::get( $id ); if ( $hold->user_id !== $user ) ...` works, but tells a stranger that hold 12 exists (403 instead of 404) and is one

forgotten `if` away from the IDOR. Coach: "Put the owner in the WHERE and let a miss be not found." (Self-check Q5, the guided lab.)

- **Copying the reference without reading it.** The reference panels are complete; a learner can paste ten files in ten minutes and learn nothing. Coach: "Close the panel. Tell me what `waiting()` does with a 0." (Q10.) Pasting is allowed after trying; the showcase's "one decision you would defend" and the write-up are where it shows.

STUDIO: MILESTONE 1, PLAN AND BASELINE (15M)

Learners add the "Reserve & Pick Up" heading to pagesco-plan.txt (the eight requirements in their own words, the decisions they agree with and the ones they would change, the file list), then type the query report and take the three numbers. **Done when** pagesco-plan.txt has the plan and the three numbers, which on the reference were:

```
Success: Added the constant 'SAVEQUERIES' to the 'wp-config.php' file with the ra
[26-Oct-2026 12:57:28 UTC] /: 22 queries; missing options: can_compress_scripts,
[26-Oct-2026 12:57:28 UTC] /books/dune/: 25 queries; missing options: can_compres
[26-Oct-2026 12:57:28 UTC] /find-a-book/: 25 queries; missing options: can_compre
Success: Deleted the constant 'SAVEQUERIES' from the 'wp-config.php' file.
```

Plan check-in (go round, 1 minute each): ask for one decision they would make differently and why (good answers: a waitlist instead of the limit, a hold per copy, an expiry email). Write the decision on the check-in board: it is the seed of the showcase's "one decision you would defend".

STUDIO: MILESTONE 2, THE DATA (25M)

HoldsTable (a feature modeled on Lesson 5.5's EventsTable), its line in `Plugin::feature_classes()`, `HoldsTable::install()` in the Activator, the daily deletion in DailySync, the table and option in uninstall.php, two settings and their section. The line count after the feature line:

```
sed -i '/Abilities::class,/a\            Holds\\HoldsTable::class,' wp-content/plug
grep -c '::class,' wp-content/plugins/pagesco-books/src/Plugin.php
```

```
21
```

Done when the maybe_upgrade check shows no table, then six columns and version 1, and the settings section is there:

```
wp db query "SELECT COUNT(*) AS holds_tables FROM information_schema.TABLES WHERE
curl -s -o /dev/null http://localhost:8080/
wp db query "SELECT COLUMN_NAME, COLUMN_TYPE FROM information_schema.COLUMNS WHERE
wp option get pagesco_books_holds_db_version
```

```
holds_tables
0

COLUMN_NAME COLUMN_TYPE
id  bigint unsigned
book_id bigint unsigned
user_id bigint unsigned
status  varchar(20)
pickup_by   date
created_at  datetime

1
```

```
wp eval 'var_dump( PagesCo\Books\Settings::get( "hold_days" ), PagesCo\Books\Setti
```

```
int(3)
int(3)
```

The settings page's section headings (as admin, Settings > Pages & Co. Books):

```
<h2 id="wp-settings-section-pagesco_books_notice-1">Store notice</h2>
<h2 id="wp-settings-section-pagesco_books_lists-2">Catalog and events</h2>
<h2 id="wp-settings-section-pagesco_books_holds-3">Reserve & Pick Up</h2>
<h2 id="wp-settings-section-pagesco_books_uninstall-4">Uninstall</h2>
```

- **Circulate for:** the schema's two spaces in `PRIMARY KEY (id)` (dbDelta needs them), the version option name, `delete_before()` in DailySync with `KEEP_DAYS`, and settings that are range-checked like Lesson 5.4's (1 to 14 days, 1 to 10 holds).
- **Ask:** "Why is the table created on this site although nobody activated anything?" (maybe_upgrade on init: Self-check Q4.) "Where would the table come from on the live site?" (The same, after the update in Milestone 9.)

GUIDED LAB (15M)

File: `labs/module-17/l9_1-final-project/ownership_sql_starter.php`; your reference is `labs/module-17/l9_1-final-project/ownership_sql_solution.php`. New practice on Milestone 3 and 4's most important habit, on wish lists instead of holds: a TEMPORARY table (gone when the script ends), `$wpdb->update()` with the owner in the WHERE, the user from `get_current_user_id()`, and one 404 for every refusal.

- **Run it as shipped (2m).** `cp labs/module-17/l9_1-final-project/ownership_sql_starter.php wordpress/` then `wp eval-file --use-include ownership_sql_starter.php`: every attempt prints TODO.

- **TODOs 1 to 3 in pairs (10m).** TODO 1 is the prepared SELECT (`%i` for the table, `%d` for the user); TODO 2 the update with three keys in the WHERE and `1 ===` on the result; TODO 3 the door. Circulate for the user ID taken from a parameter instead of the login, and for two different errors ("not yours" and "no such wish"): the lab wants one.
- **Expected when done:**

```
Devon closes wish 1          pagesco_wish_not_found (404)
Devon closes wish 3          closed
Maria closes wish 1          closed
Maria closes wish 1 again    pagesco_wish_not_found (404)
A visitor closes wish 2      pagesco_wish_not_found (404)
Maria's wishes, newest first:
  wish 2: The Hobbit, open
  wish 1: Dune, closed
```

- **Debrief (3m):** "Devon's attempt on wish 1 and the visitor's both say 404. What would 403 have told Devon?" (That wish 1 exists and is someone else's.) "Where in Reserve & Pick Up is this exact line?" (`HoldsTable::finish()` and `Holds::cancel()` .)
- **Fast finishers:** `labs/module-17/l9_1-final-project/rsvp_rules_starter.php` (one rules function with 401, 404 and 409, and a computed status), then `labs/module-17/l9_1-final-project/debug_trash_door.php` (Common Mistake 2 in a staff door, with two loud bugs).

STUDIO: MILESTONE 3, THE RULES AND THE EMAIL (35M)

Holds (a plain class: place, cancel, collect, waiting_hold and the static helpers), HoldMail on the plugin's own action, its feature line (22), then the two Subscribers the shop opens at the counter:

```
wp user create jordan jordan@example.com --role=subscriber --first_name=Jordan --l
wp user create priya priya@example.com --role=subscriber --first_name=Priya --last
```

```
Success: Created user 4.
Success: Created user 5.
```

Done when holds-check.php (the lesson's script in the WordPress root) breaks every rule once and prints exactly this, and the last email is in Mailpit:

```
wp eval-file holds-check.php
rm holds-check.php
```

```
Jordan reserves Dune                hold 1: waiting
Jordan reserves Dune again          pagesco_hold_duplicate
Jordan reserves The Great Gatsby    pagesco_hold_sold_out
Jordan reserves 1984                hold 2: waiting
Jordan reserves The Hobbit          hold 3: waiting
Jordan reserves Animal Farm         pagesco_hold_limit
Priya cancels hold 1                pagesco_hold_not_found
Jordan cancels hold 1               hold 1: cancelled
Jordan reserves Animal Farm         hold 4: waiting
A visitor reserves Dune             pagesco_hold_login_required
Animal Farm waits until: 3 days from today
```

```
From: WordPress <wordpress@example.com>
To: jordan@example.com
Subject: Your hold at Pages & Co.: Animal Farm

Hello Jordan,

We have put Animal Farm aside for you. Pick it up at the counter by:
...

Your holds: http://localhost:8080/my-holds/

See you soon at Pages & Co.
```

- **Read it with the room against the brief:** duplicate, sold out (The Great Gatsby, Lesson 5.2's data), the limit at the fourth waiting book, Priya's "not found" for Jordan's hold, a cancel that frees a place, the visitor. Every successful hold sent an email.
- **Circulate for:** `pickup_by` with `wp_date()` (Q9), the action fired AFTER the insert with the hold and the book, `_n()` and a translators comment on the limit message, and `HoldMail` using the raw title (`$book->post_title`), not `get_the_title()`, in a plain-text email.
- **Stuck learners:** `rsvp_rules_starter.php` is the same shape in thirty lines; do it, then come back.

STUDIO: MILESTONE 4, THE START (25M)

The doors: HoldActions (three admin-post.php handlers, a nonce per action and ID, a 303 back), HoldsEndpoint (GET and POST /holds, DELETE /holds/<id>, one permission callback), HoldsPage (Books > Holds for staff). Three feature lines (25). **Done in the room** = the two routes and a visitor turned away:

```
wp eval 'echo implode( "\n", preg_grep( "#^/pagesco/v1/holds#", array_keys( rest_g
```

```
/pagesco/v1/holds
/pagesco/v1/holds/(?P<id>\d+)
```

```
curl -s -H "Content-Type: application/json" -d '{"book":32}' http://localhost:8080
```

```
{
  "code": "rest_forbidden",
  "message": "Log in to reserve books.",
  "data": {
    "status": 401
  }
}
```

Goes home with the reference outputs to compare: Jordan refused by the limit (409), Priya's 201, Jordan's DELETE of Priya's hold 5 answered as not found:

```
{
  "code": "pagesco_hold_not_found",
  "message": "You have no waiting hold with that ID.",
  "data": {
    "status": 404
  }
}
```

the staff screen for Maria and not for Devon, the forged "Mark collected" without its nonce:

```
HTTP/1.1 403 Forbidden

<div class="wp-die-message">The link you followed has expired.</div>
```

then the truncate, the two `application-password delete` lines and the first commit (`git log --format=%s -1` prints the milestone message). Remind learners that `$JORDAN` and `$PRIYA` live only in the terminal that created them.

CHECK-IN BOARD

Tick each learner's milestones during the studio; it tells you who needs a 15-minute check-in before Session 18 and who may help others next time.

LEARNER	M1 PLAN + 3 NUMBERS	M2 SIX COLUMNS, VERSION 1	M3 HOLDS-CHECK: 10 LINES	M4 ROUTES + 401	NOTE FOR SESSION 18

CHECKS FOR UNDERSTANDING

Ask these out loud during the studio:

- Which layer enforces "at most three books"? **Answer:** the rules, in `Holds::place()`; every door calls it (the lesson's quiz Q2).
- Why does `cancel()` answer "not found" for another customer's hold? **Answer:** so a stranger cannot even learn that the hold exists; the owner is in the WHERE.
- Who creates the holds table on the live site in Milestone 9? **Answer:** `HoldsTable::maybe_upgrade()` on the first request that loads the new code.
- Where do the user's ID and the hold's ID come from in the cancel handler? **Answer:** the user from the login (`get_current_user_id()`), the hold from the form, checked by the nonce for its action and that ID.

Workbook self-check, items Q1 to Q12 (learners do them at the end of the studio or at home).

Answers: **Q1** B) A hold is one customer, one book, a status and a date: typed columns with indexes answer "this customer's holds" and "what is waiting" directly, and old rows can be deleted cleanly; **Q2** D) Nowhere: `Holds::status()` computes it from `pickup_by` and today's date every time a hold is shown; **Q3** `20` (twenty features); `25` after Milestone 4 (five new features, one line each); **Q4** A) On the next request that loads WordPress: `maybe_upgrade()` runs on init, finds no version option and installs it; **Q5** C) Cancel ANY customer's waiting hold by sending its ID; **Q6** B) The request is well formed and the customer is allowed to ask; the shop's current state says no (Conflict); **Q7** `bool(true)` then `bool(false)`: the meta capability with the ID, `delete_post`, 15; **Q8** D) The rules class knows nothing about email: others (an SMS, a staff notice in Exercise 2) attach to the same moment without changing the rules; **Q9** A) `wp_date( 'Y-m-d', time() + $days * DAY_IN_SECONDS )`; **Q10** `WHERE ( 0 = 0 OR user_id = 0 )`: always true, so the user filter is switched off (all customers); with 4 it becomes `( 4 = 0 OR user_id = 4 )`; **Q11** C) 403 with `The link you followed has expired.`; **Q12** the data (a table class that only stores), the rules (one class, Holds, that only decides), the doors (REST routes, forms, blocks that only check who is asking and pass the question on). Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a learner who is behind after Milestone 2 pairs with a neighbor for Milestone 3 (one keyboard, the behind learner reads each rule aloud before it is typed) and types Milestone 4 at home from the reference with the lesson open. Book a 15-minute check-in before Session 18 for anyone not past Milestone 3. Learners who joined Part 2 without Part 1: the project needs no theme work until Milestone 5's template line.

Stretch: the kit's `own_action` folder lab and `debug_trash_door.php`; then the lesson's Going Further idea, the waitlist, as a plan only (Milestone 1's format: rules, doors, personal data and for how long), to defend in the showcase.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

On the slide; learners answer in the workbook's Exit Ticket box or on a sticky note. Sort the notes before Session 18: question 2's "not yet" pile gets a word in the Session 18 warm-up, because the smoke test and the PHPUnit test both depend on it.

1. Name the three layers of Reserve & Pick Up and one class in each. **Answer:** data (HoldsTable), rules (Holds), doors (HoldActions, HoldsEndpoint, the blocks; HoldsPage for staff).
2. Priya sends `DELETE /pagesco/v1/holds/5` for Jordan's hold. What does she get, and which line of code decides it? **Answer:** 404 `pagesco_hold_not_found`; `HoldsTable::finish()` with her user ID in the WHERE (called by `Holds::cancel()`).
3. Which milestone is the next one you will finish, and what is its done-when? **Answer:** their own (Milestone 4: the REST checks and the forged request answered 403; or Milestone 5: the four states of the Reserve block and the smoke test's eight ok lines).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish Milestone 4** with the lesson's checks (the application passwords, the staff screen as Maria and Devon, the forged request), the truncate, revoking the test passwords and the commit.
- **Milestone 5:** the Reserve for Pickup and My Holds blocks (scaffold with `npx @wordpress/create-block`: needs the internet), the template line, the My holds page, the four states, and holds-smoke.sh: eight `ok` lines on your development site. Try the JavaScript path in a private window.
- **Milestone 6:** live prices in the staff picks (the binding learns a slug), with and without the plugin; commit.
- **Optional:** the kit labs `labs/module-17/l9_1-final-project/rsvp_rules_starter.php`, the `own_action` folder lab and `labs/module-17/l9_1-final-project/debug_trash_door.php`.
- **Not yet:** the lesson's exercises, quiz and journal come after Session 18, when the feature is complete.
- **Before Session 18 (it needs them):** `npx @wordpress/env start` once in the plugin's folder (wp-env from Lesson 8.1; the first start downloads images: do it at home), then `npx @wordpress/env stop`; pagesco-live starts with `docker compose up -d`.

BRING LIST FOR SESSION 18

- Your site at Milestone 6 (committed), holds-smoke.sh in the WordPress root, wp-env started once at home, pagesco-live working.
- pagesco-plan.txt with the plan and the baseline numbers, the workbook, and a first idea for the showcase's "one decision you would defend".

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the readiness check; the requirement map; Milestone 2's done-when; Milestone 3's holds-check run.

- **Cut first (saves 3m):** the third demo step (the `$title` sniff); point at Common Mistakes 3 on the page.
- **Cut second (saves 15m):** the guided lab: `ownership_sql` goes home, Milestone 3 practices the same habit.
- **Cut third (saves 25m):** Milestone 4 in the room; it starts at home with the lesson open.

RUNNING SHORT: WHAT TO ADD

- Milestone 4's application-password checks together: Jordan's 409, Priya's 201, Jordan's DELETE of hold 5. Ask for a prediction of each status first.
- A design review in threes: each learner explains their HoldsTable and one rule; the others look for the owner in every query.
- Start Milestone 5's scaffold for everyone on a good connection (`npx @wordpress/create-block ...`), so the download is done before home.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 18: Final Project Studio II, Showcase and Next Steps

How to run the course's last session: finish Reserve & Pick Up in the studio (standards, tests and numbers, release 1.2.0 with its ZIP check, the code-only deploy to the rehearsal server), hold a five-minute showcase for every learner, then teach Lesson 9.2 (headless and CORS, what WordPress 7.1 ships for AI, an ability answered for a model, a portfolio-ready repository) and close the course.

`2 lessons (studio + 9.2)` `3 hrs session` `Lessons 9.1–9.2`

`Session 18 of 18 · Part 2 · course close`

HOW TO USE THIS GUIDE

- Session 18 is the course's last. Its first half finishes the final project (Lesson 9.1, 300 minutes for a solo learner): the studio does Milestones 7 (quality), 8 (release 1.2.0) and 9 (the code-only deploy), then every learner shows the feature in a five-minute **showcase**. Its second half teaches Lesson 9.2 (110 minutes): headless, AI in core, abilities as tools, contributing, careers.
- Learners arrive with Milestones 4 to 6 done at home (Session 17's homework): both blocks, holds-smoke.sh with eight `ok` lines, live prices, committed. The warm-up checks that on every machine. Milestone 10, the write-up, goes home; **the submission is due one week after today** (the lesson's submission checklist, in the workbook). The rubric, the scoring guidance, the grading run and the reference numbers for scoring are in `final-assessment.html` at the kit's root.
- Every command below is the lessons' own, run in page order: 9.1 from a site at the end of Lesson 8.4, 9.2 from the end of 9.1 (WordPress 7.1.3, PHP 8.5, MySQL 9.7, WP-CLI 2.12); every output equals the lesson's card. Two kinds of step cannot run that way, in the lesson either, and are QUOTED from the lesson's recorded cards and labelled: the PHPUnit run in wp-env, and Milestone 9 on the learners' rehearsal server.
- The kit's labs: `labs/module-18/l9_1-final-project/` (version places, the release file list, a smoke test, a release-check handout) for stuck or early learners in the studio, `labs/module-18/l9_2-next-steps/` (the 9.2 guided lab and practice). Teal blocks are hands-on, amber blocks are watch-outs.

THE #1 RISK OF SESSION 18: THE STUDIO EATS THE SHOWCASE AND LESSON 9.2

Milestones 7 to 9 are 75 minutes in the lesson's estimate and the studio has 65; wp-env's first start and `npm ci` can take minutes on venue Wi-Fi. **Protect the showcase:** at 1:15 everyone stops, wherever they are; a learner who has not deployed shows the development site and the smoke test there, and deploys at home (the rehearsal-server proof goes into the submission). **Before:** wp-env started once at home (Session 17's bring list), the node:24 and composer:2 images pulled. **In the room:** a learner whose wp-env does not start records the rest of Milestone 7 and runs PHPUnit at home; nobody debugs Docker for ten minutes during the studio.

BEFORE THE SESSION

- ☐ **Two days before:** remind learners to finish Milestones 4 to 6 (holds-smoke.sh: eight ok lines, exit status 0), to run `npx @wordpress/env start` once in the plugin's folder and stop it, and to start pagesco-live once (`docker compose up -d` in it).
- ☐ **Your projector site** at the end of Lesson 9.1 for the 9.2 demo (or a copy you bring to that state), and your own Lesson 8.4 rehearsal server if you want to show Milestone 9 live. Tabs: `lesson_30.html` at Milestone 7, `lesson_31.html`, Mailpit, Books > Holds.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-18` (14 results, about two minutes). On your own project: `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-18`.
- ☐ **Showcase groups of three** on paper, mixing strong and weaker projects; the showcase sheet below, one per group, and the rubric from `final-assessment.html`.
- ☐ **Course close:** the evaluation form (link or paper), the submission checklist printed, certificates of completion if your program issues them (after the final project is scored).

KNOWN TRAPS OF MILESTONES 7 TO 9

SYMPTOM	CAUSE	FIX
`composer lint` exit 1 with 11 warnings in HoldsTable.php	Expected: the custom table's direct queries	Milestone 7's documented exception in phpcs.xml.dist, then exit status 0.
wp-env: `Error: Could not connect to Docker` or a port in use	Docker not running, or another wp-env on 8888/8889	Start Docker; `npx @wordpress/env stop` in the other project. Not in the room for more than five minutes: PHPUnit at home.
PHPUnit: 12 tests instead of 17	tests/HoldsTest.php not saved in the plugin's tests/, or a class name typo	`ls tests/`; the class must extend the test case like Lesson 8.1's.
`unzip -l ... | grep -c 'src/Holds/'` prints 0	"src/Holds" missing from package.json's files (Common	Add it, commit, tag again (delete and recreate the local tag: it was never pushed),

SYMPTOM	CAUSE	FIX
	Mistake 1)	rebuild.
`Composer could not detect the root package` in release/	Expected: the git-archive copy has no .git	Nothing to fix (Lesson 8.4).
Live site: critical error after the deploy	A ZIP without a folder its class map names	`wp --skip-plugins plugin install /deploy/pagesco-books-1.1.1.zip --force`, then rebuild the release (Common Mistake 1's rollback).
Live smoke test: `FAILED shoptest can log in`	`$SMOKE_PASS` empty: another shell, or reset-password not run	Run the reset-password line again in the same container shell.
Live smoke test: `FAILED the form reserves the book`	That customer already has the book waiting (a duplicate), or the book is sold out	Another book, or cancel the hold first; on your dev site the reference's own jordan has Dune waiting after the lesson.
No email on the live site	By design: the rehearsal server has no mail server	wp_mail() returns false and the hold is placed anyway (HoldMail's comment); a real host needs SMTP.

SESSION 18 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Rebuild check: the Reserve block's four states and holds-smoke.sh's eight ok lines on every development site, live prices; two questions on Session 17.
0:10	30	**9.1** Studio: Milestone 7, quality	composer lint and the documented exception, HoldsTest in wp-env (17 tests), the speed numbers against the baseline, the accessibility pass.
0:40	15	**9.1** Studio: Milestone 8, release 1.2.0	Five version places, the POT, commit and tag release-3, the release build, the ZIP check (4 files in src/Holds) before anything leaves the machine.
0:55	20	**9.1** Studio: Milestone 9, the code-only deploy	Keep 1.1.1 for a rollback, back up, install both ZIPs, watch maybe_upgrade create the table, create My holds and shoptest on live, the smoke test: eight ok.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:15	10	Break	Learners open the tabs for their showcase in order (the lesson's checklist) and write their "one decision" on a card.
1:25	30	**9.1** Showcase	Groups of three: five minutes each with the lesson's showcase checklist, two minutes of questions; you rotate with the scoring line.
1:55	10	**9.2** Frame and mental model	Where you are now; headless as the stockroom and its shop windows; AI in core as plumbing; contributing and careers in one minute each.
2:05	15	**9.2** Live demo	CORS from another origin, what 7.1 really ships for AI, the Connectors list, a model's function call answered for Maria and refused for Devon.
2:20	15	**9.2** Guided lab	`labs/module-18/l9_2-next-steps/cors_headers_starter.sh`: what a front end on another origin may read, and a protected route it can read but not use.
2:35	15	**9.2** Start the lesson project	Step 4 (no secret in the history, the README committed) and the first lines of step 5 (next-plan.txt); steps 1 to 3 go home.
2:50	10	Wrap-up and course close	Final submission checklist, exit ticket, the last journal entry, course evaluation, what comes next.
Total	180		Ends at 3:00.

In the room. The lessons estimate the remaining Milestones 7 to 10 at 90 minutes and Lesson 9.2 at 110: far more than 180 with a showcase. The studio takes 65 minutes for Milestones 7 to 9, the showcase 30, Lesson 9.2 55 (frame, demo, guided lab, the start of its project).

Goes home (about 2 to 3 hours, the last homework). Whatever is left of Milestones 7 to 9, Milestone 10 (RESERVE-AND-PICKUP.md) and the **submission within one week**; Lesson 9.1's exercises, quiz and journal; Lesson 9.2's project steps 1 to 3 (the storefront, the model's view, the release watch) and the rest of step 5, its exercises, quiz and the course's last journal entry (it answers the two questions of the first one).

Protect, in this order: the showcase for every learner; Milestone 8's ZIP check before anything is uploaded; the code-only deploy rule (never the development database on the live site); the 9.2 demo's model call for Maria and Devon (the permission callback still decides). **Cut first:** the 9.2 guided lab (it goes home), then the Connectors step of the demo, then the accessibility walk of Milestone 7 (at home, with axe or Lighthouse).

OPENING SEGMENT

0:00 Warm-up: the Rebuild Check — 10 min

TIMING

6m	Every development site: the smoke test from the WordPress root; the four states in the browser if it fails
4m	Two questions on Session 17, then the plan for the studio

SEQUENCE AND TALKING POINTS

- **The smoke test on every machine (6m).** `sh holds-smoke.sh http://localhost:8080 jordan jordan-pass dune; echo "exit status $?"` in the WP-CLI container. Done = eight `ok` lines, exit status 0, exactly as the reference:

```
ok      visitor sees the login link
ok      REST refuses a visitor (status 401)
ok      jordan can log in
ok      a form without a nonce is refused (status 403)
ok      the form reserves the book
ok      My holds shows the waiting hold
ok      the customer cancels the hold
ok      a customer cannot mark holds collected (status 403)
exit status 0
```

- A `FAILED the form reserves the book` on a re-run usually means Jordan already has Dune waiting: use another book or cancel it first (the test leaves a cancelled hold, so a clean run repeats). Any other FAILED line is a real bug; note the learner for the first studio round.
- **Two questions (2m):** "Who creates the holds table on the live site?" (maybe_upgrade on the first request: today's Milestone 9.) "Priya deletes Jordan's hold 5: what does she get?" (404, ownership in the WHERE: the exit ticket's question 2.)
- **The plan (2m):** the studio stops at 1:15 for the break, wherever anyone is; the showcase starts at 1:25.

9.1 Final Project: Launch the Pages & Co. Website

95 min

OBJECTIVES (FROM THE LESSON PAGE)

- Turn a client's brief into data, rules, routes, blocks and screens, and decide where each rule is enforced
- Build a feature through every layer of the course: a custom table, settings, a service class with a custom hook, REST routes with real permission callbacks, form handlers, a staff screen, a dynamic and an interactive block, and a theme pattern with live data
- Make it secure and private by default: nonces and capabilities on every change, ownership in every query, escaping, and personal data kept no longer than needed
- Prove the feature with coding standards, PHPUnit tests, a smoke test from outside, accessibility checks and a performance measurement
- Release it from a tagged commit and deploy it as code only, then present your work against a rubric

TIMING IN THE ROOM

30m	Studio: Milestone 7, standards, tests, speed, accessibility
15m	Studio: Milestone 8, release 1.2.0
20m	Studio: Milestone 9, the code-only deploy
30m	Showcase, in groups of three

STUDIO: MILESTONE 7, QUALITY (30M)

"The feature works. Now prove it the four ways Module 8 taught, and fix what the proof finds." **Standards** first, as a summary, in the plugin's folder with Composer:

```
composer lint -- --report=summary; echo "exit status $?"
```

```
PHP CODE SNIFFER REPORT SUMMARY
----------------------------------------------------------------------
FILE                                                  ERRORS  WARNINGS
----------------------------------------------------------------------
src/Holds/HoldsTable.php                              0       11
----------------------------------------------------------------------
A TOTAL OF 0 ERRORS AND 11 WARNINGS WERE FOUND IN 1 FILE
----------------------------------------------------------------------

...

Script phpcs handling the lint event returned with error code 1
exit status 1
```

11 warnings, all in HoldsTable.php: the custom table's direct queries and their missing cache. The lesson's answer is a documented exception in phpcs.xml.dist (Self-check Q1); then:

```
composer lint; echo "exit status $?"
```

```
exit status 0
```

Tests: tests/HoldsTest.php (five tests: the rules and the REST door) and the whole suite in wp-env, on the learner's computer. Recorded in wp-env (the lesson's card; your time and memory differ):

```
Installing...
Running as single site... To run multisite, use -c tests/phpunit/multisite.xml
Not running ajax tests. To execute these, use --group ajax.
Not running ms-files tests. To execute these, use --group ms-files.
Not running external-http tests. To execute these, use --group external-http.
PHPUnit 9.6.38 by Sebastian Bergmann and contributors.

Book Shortcode (PagesCo\Books\Tests\BookShortcode)
 ✔ Renders a published book
 ✔ Prints nothing for drafts and pages
 ✔ Escapes the author

Books Endpoint (PagesCo\Books\Tests\BooksEndpoint)
 ✔ Filters by genre and sorts by price
 ✔ Rejects an unknown genre
 ✔ Every book matches the schema

Daily Sync (PagesCo\Books\Tests\DailySync)
 ✔ Schedules one daily event at three
 ✔ Run removes only past events

Holds (PagesCo\Books\Tests\Holds)
 ✔ Place keeps the book for the set days and sends an email
 ✔ Refuses duplicates sold out books and too many holds
 ✔ An expired hold is computed and frees the book
 ✔ Only the customer can cancel their hold
 ✔ The rest route needs a logged in customer

Supplier Client (PagesCo\Books\Tests\SupplierClient)
 ✔ Reads availability by isbn
 ✔ Turns an error status into a wp error
 ✔ Rejects an answer without items
 ✔ Passes on a network failure

Time: 00:00.214, Memory: 46.50 MB

OK (17 tests, 47 assertions)
```

Then the proof that the tests catch the dangerous bug: remove `$user_id` from the `finish()` call in `Holds::cancel()`, run only that test (recorded):

```
PHPUnit 9.6.38 by Sebastian Bergmann and contributors.

F                                                                   1 / 1 (100%)

Time: 00:00.041, Memory: 44.50 MB

There was 1 failure:

1) PagesCo\Books\Tests\HoldsTest::test_only_the_customer_can_cancel_their_hold
Failed asserting that stdClass Object (...) is an instance of class "WP_Error".

/wordpress-phpunit/includes/abstract-testcase.php:879
/var/www/html/wp-content/plugins/pagesco-books/tests/HoldsTest.php:117

FAILURES!
Tests: 1, Assertions: 1, Failures: 1.
```

Put `$user_id` back, 17 tests OK, `npx @wordpress/env stop`. **Speed:** the same three pages as Milestone 1's baseline:

```
Success: Added the constant 'SAVEQUERIES' to the 'wp-config.php' file with the ra
[26-Oct-2026 13:00:04 UTC] /: 31 queries; missing options: can_compress_scripts,
[26-Oct-2026 13:00:04 UTC] /books/dune/: 25 queries; missing options: can_compres
[26-Oct-2026 13:00:04 UTC] /find-a-book/: 25 queries; missing options: can_compre
Success: Deleted the constant 'SAVEQUERIES' from the 'wp-config.php' file.
```

Front page 31 against 22: three queries per staff pick for the live prices, a decision for the write-up (Q5). The book page is unchanged at 25. **Accessibility:** two holds for Jordan, then the names of My holds' buttons:

```
<button type="submit" class="wp-element-button" aria-label="Cancel the hold on 19
<button type="submit" class="wp-element-button" aria-label="Cancel the hold on Du
```

- **Circulate for:** a lint exception for the whole plugin instead of one file (Q1); the test file not found by PHPUnit; numbers measured logged in (they must be a visitor's, like the baseline).
- **Stuck or waiting for wp-env:** the kit's `debug_release_check.php` (Milestone 8's version check, with a silent bug).

STUDIO: MILESTONE 8, RELEASE 1.2.0 (15M)

Every place a version lives (the plugin header and constant, package.json with "src/Holds" in files, readme.txt's Stable tag, changelog and upgrade notice, the theme's style.css), the new POT, then commit, tag and build exactly like Lesson 8.4's releases. The POT:

```
Plugin file detected.
Success: POT file successfully generated.
191
2
```

The commit and tag (the diffstat of the release commit):

```
 33 files changed, 1648 insertions(+), 50 deletions(-)
```

The clean build in the git-archive copy:

```
Composer could not detect the root package (pagesco/pagesco-books) version, defau
Installing dependencies from lock file
Verifying lock file contents can be installed on current platform.
Nothing to install, update or remove
Generating optimized autoload files
```

Done when the ZIP check prints, before anything leaves the machine:

```
mv release/pagesco-books/pagesco-books.zip release/
rm -r release/pagesco-books
unzip -l release/pagesco-books.zip | grep -c 'src/Holds/'
unzip -l release/pagesco-books.zip | grep -c 'build/blocks/reserve/'
unzip -q release/pagesco-books.zip -d /tmp/check
php -r 'require "/tmp/check/pagesco-books/vendor/autoload.php"; var_dump( class_ex
rm -r /tmp/check
unzip -p release/pagesco.zip pagesco/style.css | grep '^Version'
ls release
```

```
4
8
bool(true)
bool(true)
Version: 1.2.0
pagesco-books.zip  pagesco.zip
```

- 4 files from src/Holds and 8 from build/blocks/reserve, and the ZIP's own autoloader finds the new classes with nothing around it: the check that would have caught Common Mistake 1.
- **Fast finishers or stuck learners:** `labs/module-18/l9_1-final-project/version_audit_starter.php` (all five version places, read the way the tools read them) and `labs/module-18/l9_1-final-project/files_list_starter.php` (which src/ folders a ZIP would leave out, without Node).

STUDIO: MILESTONE 9, THE CODE-ONLY DEPLOY (20M)

On the learners' rehearsal server (pagesco-live from Lesson 8.4), which no runner can reach: the outputs below are the lesson's recorded cards. "Code only: the rehearsal server's database is the live one now. It is not Thursday evening. Go." Upload, keeping 1.1.1 for a rollback (the deploy folder):

```
holds-smoke.sh
pagesco-1.1.1.zip
pagesco-1.2.0.zip
pagesco-books-1.1.1.zip
pagesco-books-1.2.0.zip
pagesco-live.sql
pagesco-site.php
```

Back up first:

```
Success: Exported to '/backups/pagesco-before-1.2.0.sql'.
-rw-r--r--    1 1000     1000        109925 Oct 26 07:43 /backups/pagesco-before-
```

Deploy the code; the table appears one command later (Q3):

```
Unpacking the package...
Installing the plugin...
Removing the old version of the plugin...
Plugin updated successfully.
Success: Installed 1 of 1 plugins.
Unpacking the package...
Installing the theme...
Removing the old version of the theme...
Theme updated successfully.
Success: Installed 1 of 1 themes.
Tables_in_pagesco (wp_pagesco_holds)
wp_pagesco_holds

1
+---------------+--------+---------+
| name          | status | version |
+---------------+--------+---------+
| pagesco-books | active | 1.2.0   |
+---------------+--------+---------+
+---------+--------+---------+
| name    | status | version |
+---------+--------+---------+
| pagesco | active | 1.2.0   |
+---------+--------+---------+
```

My holds is content, so the live site does not have it:

```
404
```

Create the page and the shop's smoke-test customer on live:

```
57
200
4
+------------+--------------+
| user_login | display_name |
+------------+--------------+
| shoptest   | Shop Test    |
+------------+--------------+
```

Done when the smoke test passes on the production-configured site (recorded):

```
ok      visitor sees the login link
ok      REST refuses a visitor (status 401)
ok      shoptest can log in
ok      a form without a nonce is refused (status 403)
ok      the form reserves the book
ok      My holds shows the waiting hold
ok      the customer cancels the hold
ok      a customer cannot mark holds collected (status 403)
exit status 0
```

- The email is not delivered there (no mail server: by design, the hold is placed anyway). Then `rm -r release` on the development site and the history:

```
0
Pages & Co. Books 1.2.0 and theme 1.2.0: Reserve & Pick Up
Reserve & Pick Up blocks, live prices in the staff picks
Reserve & Pick Up: holds table, settings, rules, email, REST routes, form handler
Pages & Co. Books 1.1.1 and theme 1.1.1: HTTPS for the supplier, Update URI
Pages & Co. release 1: theme 1.1.0, Pages & Co. Books 1.1.0, site plugin 1.2.0
release-1       First launch
release-2       Plugin 1.1.1, theme 1.1.1
release-3       Plugin 1.2.0, theme 1.2.0: Reserve & Pick Up
```

- **A smoke test of their own:** `labs/module-18/l9_1-final-project/smoke_rest_starter.sh` (status codes, a REST header, a redirect) takes an address, so learners can run it on the rehearsal server too.

STUDIO TRACKER

LEARNER	M5/M6 AT HOME: SMOKE 8 OK	M7 LINT 0, 17 TESTS, NUMBERS	M8 RELEASE-3, ZIP CHECK	M9 LIVE SMOKE 8 OK	SHOWCASE DONE

SHOWCASE (30M)

Groups of three, at one laptop each in turn: five minutes per learner with a timer, two minutes of questions, then the next. You rotate between the groups and fill the showcase line of the rubric (Write-up and showcase, 5 points: the showcase follows the checklist and fits five minutes; the write-up comes with the submission). The lesson's checklist, on the slide and in the workbook:

1. **The problem** (30 seconds): the sticky notes, in one sentence each from the brief.
2. **A visitor** opens a book: "Log in to reserve". Then **a customer** logs in and reserves it: no reload, the message is announced. Show the email in Mailpit.
3. **The rules**: reserve the same book again, then a sold-out one, then one too many. Change "Holds per customer" in the settings and try again.
4. **My holds**: the list, a cancel. Then turn JavaScript off in the browser and reserve again: it still works.
5. **The counter**: Books > Holds as Maria, "Mark collected".
6. **The home page**: change Dune's price in the editor, reload the home page.
7. **The proof**: `composer lint`, the PHPUnit summary, the smoke test's output on the rehearsal server, `git tag -n`.
8. **One decision you would defend** (why a custom table, why "not found" instead of "forbidden", why expired is computed) and **one thing you would do next**.

Learners who have not deployed show the development site and say what is left; the rehearsal server's smoke test goes into the submission. Peers note one strength and one question per showcase on the workbook page; the question goes into the presenter's write-up ("the one question they asked that you could not answer yet", Lesson 9.1's Before the Next Lesson).

LEARNER	CHECKLIST STEPS SHOWN (OF 8)	THE DECISION THEY DEFENDED	FIT 5 MINUTES?	ONE STRENGTH FOR THE FEEDBACK

LEARNER	CHECKLIST STEPS SHOWN (OF 8)	THE DECISION THEY DEFENDED	FIT 5 MINUTES?	ONE STRENGTH FOR THE FEEDBACK

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Fixing the proof instead of the code.** A failing smoke test line "fixed" by changing the test's grep, a lint warning silenced for the whole plugin. Coach: "What does this line of the checklist protect? Make the code do that."
- **Importing the development database to make the live site match.** The 404 on My holds tempts everyone. Coach: "Code goes up, content comes down, configuration stays. Which is a page?" (Q4.)
- **A showcase that is a code tour.** Ten minutes of scrolling through classes. Coach with the checklist: the problem in one sentence, then the browser, then the proof, then ONE decision.

CHECKS FOR UNDERSTANDING

Ask during the studio:

- Why does the theme get a new version although its PHP did not change? **Answer:** its template and pattern changed, and the live pattern cache only notices a new Version.
- What would have caught a ZIP without src/Holds? **Answer:** listing the ZIP and loading a new class with its own autoloader before upload (the lesson's quiz Q5).
- Which data does a deploy change on the live site? **Answer:** none but what the new code creates itself (the holds table); the page and the test customer are created by hand on live.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** C) Documents ONE exception in phpcs.xml.dist, for that file, with its reason: a custom table has no WordPress API, and holds are per user and change with every click, so a cache would mostly be invalidated before it is read; **Q2** `1.2.0 1.2.0`: the theme's template and pattern changed, and without development mode the live site caches pattern files until the theme's Version changes (Lesson 8.4); **Q3** A) `wp plugin install` ran with the OLD plugin in memory, and `wp db query` does not start WordPress: no new code has run init yet; **Q4** D) Create the page ON the live site (it is content: it lives in the database), with `wp post create`, as the readme's upgrade notice says; **Q5** three per staff pick (get_page_by_path() for the slug, then the book and its meta) for the live prices; indexed primary-key lookups under a millisecond each, the page has no per-user content so a page cache serves it, and an object cache keeps them; **Q6** B) A screen reader would otherwise read "Cancel, Cancel, Cancel" with no way to tell which hold each one cancels. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a learner without a working rehearsal server releases (Milestone 8) and runs the smoke test of their development site in the showcase; the deploy is done at home with Lesson 8.4's launch steps first. A learner far behind shows Milestones 1 to 6 and gets a written plan with a check-in date inside the submission week.

Stretch: the lesson's Exercises 1 to 3 (collect over REST, the shop's email through the action, the personal data exporter) as must-use plugins; the waitlist of Going Further as a plan in the write-up's "Next".

9.2 Next Steps: Headless, AI in Core & Your Career

55 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what "headless" WordPress means, when it is worth its costs, and build a small front end on your own REST routes that pages through results, handles HTML fields correctly and works across origins
- Describe what WordPress 7.0 and 7.1 ship for AI (the AI client, wp_supports_ai(), the Connectors screen) and what they do not (a model, a provider, an AI feature), and write code that degrades gracefully when no provider is set up
- Show how an ability such as pagesco/check-stock becomes a tool an AI model can call, and why its permission callback still decides
- Find your way into contributing to WordPress: the Make teams, Trac and GitHub, Learn WordPress, meetups and WordCamps
- Choose a career direction, turn the Pages & Co. project into a portfolio piece without leaking secrets, and keep up with WordPress releases

TIMING IN THE ROOM

10m	Frame: where you are now, headless, AI as plumbing, contributing and careers
15m	Live demo: CORS, AI in core, Connectors, a model's function call for Maria and Devon
15m	Guided lab: `cors_headers_starter.sh`
15m	Start the lesson project: step 4 and the start of step 5

MENTAL MODEL TO SAY

"A traditional WordPress site is a shop where the stockroom and the shop floor are one building: the theme is the shop floor. Headless keeps the stockroom, WordPress with the stock, the rules and the staff entrance, and sends goods out through a loading door, the API, to shop windows anywhere: a website built with another framework, a phone app, a kiosk." Then the trade, on the board: what the theme gave for free (templates, the Site Editor, previews, plugins' front ends, one deployment) has to be rebuilt in front. "It pays for apps and many front ends; it costs a second deployment and every preview."

Then AI in core in one sentence: "WordPress 7 ships plumbing: a client plugins can use, a screen for a provider's key, a switch to turn it off. No model, no provider, no AI feature." Then contributing (make.wordpress.org, a team, one meeting, evidence in every report) and careers (agency, freelance, in-house, a plugin business, headless) in one minute each: the lesson has the tables, the learners read them at home.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

From the project folder on a site at the end of Lesson 9.1; the outputs are real and equal the lesson's cards.

1. **Where you are now (2m).** "Thirty lessons ago this was a dashboard." From the project folder, on a site at the end of Lesson 9.1:

```
wp core version
wp theme list --status=active --fields=name,version
wp plugin list --status=active,must-use --fields=name,status,version
wp eval 'echo count( rest_get_server()->get_routes( "pagesco/v1" ) ), " routes
```

```
7.1.3
+---------+---------+
| name    | version |
+---------+---------+
| pagesco | 1.2.0   |
+---------+---------+
+---------------+----------+---------+
| name          | status   | version |
+---------------+----------+---------+
| pagesco-books | active   | 1.2.0   |
| mailpit-smtp  | must-use |         |
| pagesco-site  | must-use | 1.2.0   |
+---------------+----------+---------+
6 routes under pagesco/v1
```

Read it as a list of everything they built: the theme 1.2.0, the plugin 1.2.0, 6 routes of their own under pagesco/v1. "Two of those are today's starting points: the routes, and the ability."

2. **Your API, from another origin (4m).** "A front end on another host asks the shop's API the way a browser on port 3000 would." In the WP-CLI container:

```
curl -s -I -H 'Origin: http://localhost:3000' http://localhost:8080/wp-json/pa
```

```
HTTP/1.1 200 OK
Access-Control-Expose-Headers: X-WP-Total, X-WP-TotalPages, Link
Access-Control-Allow-Headers: Authorization, X-WP-Nonce, Content-Disposition,
X-WP-Total: 11
X-WP-TotalPages: 2
Access-Control-Allow-Origin: http://localhost:3000
Access-Control-Allow-Methods: OPTIONS, GET, POST, PUT, PATCH, DELETE
Access-Control-Allow-Credentials: true
```

WordPress answers CORS by itself and repeats the page's origin: any origin may read public data, and the paging headers are exposed. "Who may DO something is still the permission callback's job, not CORS's."

3 **What WordPress 7.1 really ships for AI (4m).** "You may have read that WordPress 7 has AI built in." Ask the site:

```
wp eval '
var_dump( wp_supports_ai() );
$prompt = wp_ai_client_prompt( "Write one sentence about Dune by Frank Herbert
var_dump( $prompt->is_supported_for_text_generation() );
$text = $prompt->generate_text();
var_dump( is_wp_error( $text ) );
echo $text->get_error_code(), ": ", $text->get_error_message(), "\n";
print_r( $text->get_error_data() );
'
```

```
bool(true)
bool(false)
bool(true)
prompt_invalid_argument: No models found that support text_generation for this
Array
(
    [status] => 400
    [exception_class] => WordPress\AiClient\Common\Exception\InvalidArgumentEx
)
```

Allowed, a prompt builder, and no model: generate_text() returns a WP_Error instead of throwing. Then search core for a caller:

```
grep -rl 'wp_ai_client_prompt *(' wp-admin wp-includes
```

```
wp-includes/ai-client.php
```

One file, the definition. Then the Connectors list (three AI providers, none active, plus Akismet):

```
wp eval 'foreach ( wp_get_connectors() as $id => $c ) { printf( "%-9s %-17s %-
```

```
anthropic Anthropic         ai_provider    ai-provider-for-anthropic/plugin.ph
google    Google            ai_provider    ai-provider-for-google/plugin.php,
openai    OpenAI            ai_provider    ai-provider-for-openai/plugin.php,
akismet   Akismet Anti-spam spam_filtering akismet/akismet.php, not active
```

4 **A model's function call, answered by WordPress (5m).** "The model half needs a provider; the WordPress half does not, and it is yours." Lesson 9.2's model-check.php (project step 2) prints what a model is told about pagesco/check-stock and sends the call a model would send, through core's resolver. Save it in the WordPress root, then:

```
wp eval-file model-check.php --user=maria
```

```
Tool: wpab__pagesco__check-stock
Description: Says whether the shop has a book in stock, how many copies the su
Input schema: {"type":"object","properties":{"slug":{"type":"string","descript
Answer for maria: {"title":"The Great Gatsby","in_stock":false,"supplier_avai
```

```
wp eval-file model-check.php --user=devon | tail -n 1
```

```
Answer for devon: {"error":"Ability \"pagesco\/check-stock\" does not have nec
```

"A model can only ever do what the logged-in person may do, and only with the abilities the plugin listed for that prompt." `rm model-check.php`.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Treating CORS as security.** "I will restrict the origin and the API is safe." Coach: "curl sends no Origin and ignores CORS. Who refuses a visitor on /holds?" (The permission callback: the guided lab's last line.)
- **A generate_text() result printed as text.** The lesson's Common Mistake 2: a WP_Error into esc_html() is a fatal error on every book page. Coach: "What does it return when no model exists? On every fresh site."
- **Reading only the first page.** Six books, no error. Coach: "What does X-WP-TotalPages say?" (The kit's debug_headless_client.php.)

GUIDED LAB

File: `labs/module-18/l9_2-next-steps/cors_headers_starter.sh`; your reference is `labs/module-18/l9_2-next-steps/cors_headers_solution.sh`. It asks the shop's API the way a browser on another origin would and prints, header by header, what that page may read, then a protected route: readable, refused.

- **Run it as shipped (2m).** `cp labs/module-18/l9_2-next-steps/cors_headers_starter.sh wordpress/` then `docker compose run --rm wpcli sh cors_headers_starter.sh`.
- **TODOs 1 to 4 (10m).** The same grep and cut on another header each time; TODO 4 reads the status line with `curl -i`. Circulate for the carriage returns (`tr -d '\r'`), which make a value look right and compare wrong.
- **Expected when done:**

```
== GET /pagesco/v1/books, from http://localhost:3000 ==
Allow-Origin: http://localhost:3000
Allow-Credentials: true
Expose-Headers: X-WP-Total, X-WP-TotalPages, Link
Books: 11 on 2 pages
holds as a visitor: 401, Allow-Origin: http://localhost:3000
```

- **Debrief (3m):** "The holds route answered 401 and still sent Allow-Origin. Why is that fine?" (The page may read the refusal; it cannot get in.)
- **Fast finishers:** `labs/module-18/l9_2-next-steps/ai_readiness_starter.php` (five honest lines for a client meeting about AI, and a helper that never breaks the page), then `labs/module-18/l9_2-next-steps/debug_headless_client.php`.

START THE LESSON PROJECT

The lesson project, **Pages & Co., What Comes Next**, is five short steps. In the room: **step 4** (prove that no secret was ever committed, then the README the repository's front page needs), because it makes the final project's repository ready to share, and the first lines of **step 5**.

```
for secret in sandbox-7f3a9c maria-pass devon-pass jordan-pass 'sk-ant-'; do
  printf '%-15s %s commit(s)\n' "$secret" "$(git log --all -S "$secret" --oneline
done
git ls-files | grep -E '(^|/)(\.env|wp-config\.php)$|\.sql$' | wc -l
```

```
sandbox-7f3a9c  0 commit(s)
maria-pass      0 commit(s)
devon-pass      0 commit(s)
jordan-pass     0 commit(s)
sk-ant-         0 commit(s)
0
```

The README is hidden by line 3 of .gitignore until one line lets it in:

```
.gitignore:3:/* README.md
```

After the one line and the commit:

```
0
README for the portfolio
Pages & Co. Books 1.2.0 and theme 1.2.0: Reserve & Pick Up
110
```

- **Done in the room** = zero everywhere, the README committed (110 tracked files), no tag (it changes no code: not a release).
- **Step 5 (5m):** next-plan.txt, opened next to pagesco-plan.txt: the project from their first journal entry, its site map and the first three features with the lesson to reread for each.
- **Goes home:** steps 1 to 3 (the storefront, the model's view of the ability, the release-watch routine) and the rest of step 5. The kit's `labs/module-18/l9_2-next-steps/publish_scan_starter.sh` is step 4's partner on disk.

CHECKS FOR UNDERSTANDING

Ask during the demo and the lab:

- Should Pages & Co.'s next redesign be headless? **Answer:** probably not: one website, staff who edit in the block editor; headless pays for several front ends or an app.
- Where does a client's API key for an AI provider go? **Answer:** an environment variable or constant on the server (the lesson's quiz Q3), never in the front end or the plugin.
- A model asks to run pagesco/check-stock for Devon. What decides? **Answer:** the ability's permission callback, run as Devon: it refuses.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** `Access-Control-Allow-Origin: https://shop.example`; no: WordPress echoes any origin, so a page anywhere may READ the answers, but the permission callbacks still refuse (a visitor gets 401 from /holds); **Q8** C) Know which fields are HTML: decode the stored-escaped name once to text (DOMParser), then put it on the page with textContent; **Q9** A) Plumbing: the AI client (wp_ai_client_prompt()), the Connectors screen and a switch to turn AI off; no model, no provider and no core feature; **Q10** `wp-includes/ai-client.php`: only the file that defines the function; no core screen, block or feature calls the AI client; **Q11** D) `wpab__pagesco__check-stock`; **Q12** a secret that was committed once and deleted later is still in the history, and anyone who clones the repository can read it; -S lists every commit that added or removed the string, on every branch. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners still finishing the project spend the guided lab on Milestone 9 or 10 instead; the lab and the 9.2 project wait for after the submission.

Stretch: the lesson's Exercise 4 (a second ability as a tool) and the Going Further static build; the kit's `ai_readiness` lab.

CLOSING

FINAL SUBMISSION (ONE WEEK FROM TODAY)

The lesson's submission checklist, read aloud and in the workbook:

- `git tag -n` in `wordpress/wp-content` lists `release-3`, and `git status` is clean
- `pagesco-books.zip` (1.2.0) and `pagesco.zip` (1.2.0) are kept with your release notes
- The output of `composer lint`, the PHPUnit run and both smoke tests is saved in your write-up or next to it
- The rehearsal server runs 1.2.0, has a My holds page, and its backup from before the deploy is in `backups/`
- `RESERVE-AND-PICKUP.md` maps every requirement to the code, lists your decisions and next steps
- If you share the project publicly (a portfolio, a Git host), share the code and screenshots only: never `.env`, database dumps or passwords

Agree how it arrives (a link to a private repository you can read, or a ZIP of the wp-content repository's `git archive release-3` plus the two release ZIPs and the write-up). The grading run and scoring sheet are in `final-assessment.html`.

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. Which of your checks proves that another customer's hold cannot be cancelled, and which one cannot prove it? **Answer:** HoldsTest's test_only_the_customer (and holds-check.php) proves it; the smoke test cannot: it plays one customer.
2. Code goes up, content stays: name one thing of Reserve & Pick Up for each. **Answer:** up: the plugin and theme 1.2.0 (the ZIPs); created on live: the My holds page and the shoptest account (and the holds table, by the code itself).
3. What will you build next with WordPress, and which lesson will you reread first? **Answer:** their own (next-plan.txt).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish the final project:** whatever is left of Milestones 7 to 9, Milestone 10 (RESERVE-AND-PICKUP.md, the lesson's outline) and the submission checklist; due one week from today.
- **Lesson 9.1:** the exercises (collect over REST, tell the shop, the personal data exporter), the Quick Quiz, the journal prompt (compare the site you planned in Lesson 1.1 with the one you launched).
- **Lesson 9.2:** project steps 1 to 3 and the rest of step 5; Exercises 1 to 3 (Exercise 4 is a stretch); the Quick Quiz; the last journal entry, which answers the first one's two questions again.
- **Optional:** `labs/module-18/l9_2-next-steps/ai_readiness_starter.php`, `labs/module-18/l9_2-next-steps/publish_scan_starter.sh`, `labs/module-18/l9_2-next-steps/debug_headless_client.php`; for the release, `labs/module-18/l9_1-final-project/version_audit_starter.php` and `labs/module-18/l9_1-final-project/smoke_rest_starter.sh`.

- **Before you stop:** `npx @wordpress/env stop`, `docker compose stop` in pagesco-dev and pagesco-live (both keep their data).

COURSE CLOSE (IN THE LAST 10 MINUTES)

- **Look back (2m):** the first exit ticket of Session 1 ("where does your own code go?") against today's feature. Name three things the room can do now that it could not in Session 1.
- **Evaluation (3m):** the course evaluation, before anyone leaves.
- **What comes next (2m):** the lesson's Looking Ahead: the glossary, cheat sheet, common errors and troubleshooting pages as the reference; React + WordPress Mastery for headless; the Freelance Launchpad for client work; one meetup and one Make team. Remind them that results and certificates follow the scoring of the final project.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the showcase for every learner; the ZIP check; the code-only deploy rule; the model call for Maria and Devon.
- **Cut first (saves 15m):** the 9.2 guided lab; it goes home.
- **Cut second (saves 5m):** the Connectors step of the demo; the lesson's card shows it.
- **Cut third (saves 5m):** Milestone 7's accessibility walk; with axe at home, results in the write-up.

RUNNING SHORT: WHAT TO ADD

- Common Mistake 1's rollback live on your rehearsal server: a ZIP without src/Holds, the critical error, `wp --skip-plugins plugin install ... --force`.
- Lesson 9.2's storefront served from a second origin (`npx serve -l 3000 wordpress/storefront`) with the browser's Network panel open.
- A five-minute career round: each learner names the path from the lesson's table that fits them and the one piece of their project they would show first.

Complete Facilitator Guide · all 18 sessions · v1.0 · © 2026 Ray de la Paz